

从 SLAM 到空间智能：传统几何、3DGS 与具身智能地图

导论

第0章 从 SLAM 到 Spatial AI

0.1 本书回答的核心问题

1986年，Smith 和 Cheeseman 在 IEEE 机器人与自动化汇刊上发表论文，首次提出"同时定位与建图" (Simultaneous Localization and Mapping, SLAM) 的概念框架。近四十年过去，SLAM 从一个概率估计问题出发，经历了视觉传感器崛起、深度学习渗透、再到今天与具身智能交汇的三次范式转移。

本书只有一个核心问题：当机器人不再满足于"知道自己在哪里"，而是要拥有可定位、可理解、可查询、可规划、可交互、可长期更新的空间记忆时，**SLAM 必须变成什么**？

旧目标与新目标的张力

传统 SLAM 的定义精确而狭窄：给定传感器观测序列，估计传感器运动轨迹，同时重建环境地图。数学上建模为联合后验估计 $p(\mathbf{x}_{0:T}, \mathbf{m} \mid \mathbf{z}_{1:T}, \mathbf{u}_{1:T})$ ，其中 $\mathbf{x}$ 是位姿， $\mathbf{m}$ 是地图， $\mathbf{z}$ 是观测， $\mathbf{u}$ 是控制输入。从 EKF-SLAM 到 ORB-SLAM3，再到 FAST-LIO2，几代研究者在这个框架内将精度和效率推向了极致。

但这个框架正在撞上天花板。一个例子说明问题：

一个配备顶级视觉-惯性 SLAM 系统的机器人，能在陌生公寓中实时追踪位姿到厘米精度，构建数百万三维点的点云。但如果你问它"冰箱在哪里""客厅和卧室的通道有多宽""我上周放在沙发上的书还在吗"——它一无所知。

这揭示了一个根本性断层。传统 SLAM 回答几何存在性问题 (**geometry exists**)，空间智能需要回答语义可用性问题 (**semantics matters**)。几何精度是必要条件，但不是充分条件。

维度	传统 SLAM 目标	Spatial AI 目标
定位	六自由度位姿估计	全局一致、语义锚定的长期定位
地图表示	点云、体素、网格	神经场、3DGS、场景图、可查询记忆
语义理解	无，或封闭类别标签	开放词汇、自然语言查询、对象关系
时间跨度	单次会话（session）	终身学习、跨天/跨月更新
交互能力	被动观测	主动规划、推理、执行
目标用户	算法开发者	机器人系统、终端用户、大模型

上表不是功能扩充清单，而是揭示了问题本质的转变。传统 SLAM 输出**数据结构**（点云、位姿图），新目标要求的是**可操作的数字孪生**——它能被大模型查询，被规划器调用，被用户用自然语言交互。地图不再是算法的终点，而是更大智能系统的起点。

为什么是现在？

三个技术变量在2022-2024年间交汇，使范式转移从可能变为必然。

第一，神经辐射场和 3D 高斯泼溅重新定义了地图表示。2020年 Mildenhall 等人的 NeRF（ECCV）证明多层感知机可以编码场景几何与外观；2023年 Kerbl 等人的 3DGS（SIGGRAPH）证明这个编码可以做到实时渲染、实时更新（arXiv）。地图变成可微分的神经表示——意味着地图可以端到端地与学习系统耦合。

第二，视觉-语言模型和大型语言模型打开了开放词汇理解的大门。CLIP（ICML, 2021）、GPT-4V、LLaVA 让机器能关联“像素”与“语义”。当这些能力与 3D 地图结合，机器人不再受限于预定义类别列表，可以对“那个红色的靠窗的椅子”进行推理。

第三，具身智能提出了最终的使用场景。从室内导航（VLN, CVPR 2018）到机器人操作（RT-2, arXiv 2023），具身智能需要统一的空间媒介来连接感知、推理与行动。SLAM 的语义地图恰好可以成为这个媒介。

这三个变量交汇，催生了本书讨论的**空间智能 SLAM**——不再是独立算法，而是具身智能系统核心组件的新范式。

SLAM 与具身智能的关系

具身智能（Embodied AI）强调智能体通过与物理环境的交互来学习和推理。在这个框架中，空间地图的角色发生了质变：它不再只是定位的副产品，而是智能体与环境交互的**记忆接口**。

一个具身智能体完成“帮我把桌上的杯子拿到厨房”这个任务，需要以下能力链条：(1) 理解“桌子”“杯子”“厨房”的空间位置；(2) 规划从当前位置到桌子的路径；(3) 在接近杯子时进行精细操作；(4) 将杯子运输到厨房并放置。这个链条的每一步都依赖空间表示：第(1)步需要语义地图，第(2)步需要占据地图和路径规划器，第(3)步需要局部几何细节，第(4)步需要全局一致的空间坐标系。传统 SLAM 只提供了第(4)步所需的几何框架，而空间智能 SLAM 的目标是为整个链条提供统一的空间记忆基础。

这是本书贯穿始终的视角：**SLAM 的终点不是地图，而是让机器人能在地图之上思考、计划和行动。**

0.2 SLAM 的三次范式转移

第一阶段：几何 SLAM（1986–2015）

几何 SLAM 的核心假设：地图由几何基元构成，问题是一个状态估计问题。

早期 EKF-SLAM 将环境和位姿建模为联合高斯分布，用扩展卡尔曼滤波在线更新。其局限很快暴露：计算复杂度随特征点数量平方增长，线性化误差导致一致性丢失。FastSLAM 用粒子滤波突破瓶颈，将复杂度降至对数级别，但粒子退化限制了大规模应用。

视觉 SLAM 的突破来自稀疏特征点法。PTAM (ISMAR, 2007) 首次将追踪与建图解耦到两个并行线程，证明单目相机可以在桌面规模实时运行。ORB-SLAM (IEEE T-RO, 2015) 将框架工程化到极致：多线程架构（追踪、局部建图、回环检测）、ORB 特征、Covisibility 图、本质图优化，成为后续数年的事实标准（CVF 开放获取）。同期，LSD-SLAM (ECCV, 2014) 和 DSO (ECCV, 2017) 探索直接法，绕过特征提取直接优化光度误差，在半稠密重建上取得进展。

LiDAR SLAM 走了不同路径。LOAM (RSS, 2014) 利用激光雷达的几何精度优势，通过点面特征匹配和因子图优化，实现了室外大场景的厘米级定位。后续系列 (LeGO-LOAM, LIO-SAM, FAST-LIO) 在 IMU 融合、计算效率、鲁棒性上持续推进，使 LiDAR SLAM 成为自动驾驶和测绘领域的主流方案。

这一阶段的共同特征：

- **表示**：显式几何基元（点、线、面、位姿图节点）
- **优化**：非线性最小二乘 / 因子图优化 / BA (Bundle Adjustment)
- **输出**：稀疏或半稠密点云，附带相机/传感器轨迹
- **语义**：无，或离线后处理
- **关键瓶颈**：对纹理缺失、动态物体、光照变化的脆弱性；地图仅包含几何，无法支撑高层任务

几何 SLAM 的遗产不可低估。ORB-SLAM3 的代码至今仍被无数项目 fork，GTSAM、G2O 等优化库是整个领域的基础设施。但几何 SLAM 的边界也很清晰：它解决的是“定位和建图”问题本身，而不是“有了地图之后能做什么”。

第二阶段：学习增强 SLAM（2016–2022）

学习增强 SLAM 的核心假设：深度学习的先验可以替代或增强传统手工设计的模块。

深度估计网络的成熟是起点。MonoDepth (CVPR, 2017) 证明单目深度估计可以训练到足够精度，催生了深度-位姿联合估计框架：SfM-Learner (CVPR, 2017)、DF-VO (ICRA, 2020) 等系统用神经网络预测深度和相对位姿，将 SLAM 部分模块替换为学习组件。

特征匹配是被深度学习攻克另一个堡垒。SuperPoint (CVPRW, 2018) 学习检测和描述特征点，SuperGlue (CVPR, 2020) 用图神经网络匹配特征对，LoFTR (CVPR, 2021) 用 Transformer 做稠密匹配，将弱纹理区域匹配率提升数倍。这些进展直接增强了传统 SLAM 前端——SuperPoint + SuperGlue 成为许多现代系统的标配（CVF 开放获取）。

更具颠覆性的是端到端学习 SLAM。DROID-SLAM (NeurIPS, 2021) 将光流估计、对应关系学习和位姿优化封装在一个可微分架构中，在 TUM、ETH 等多个基准上超越了传统方法 (arXiv)。TartanVO (RSS, 2021) 证明纯学习 VO 可以泛化到未见环境。NeuralRecon (CVPR, 2021) 和 Atlas (CVPR, 2021) 展示了实时 MVS 与 SLAM 结合的可能。

神经隐式表示的引入是这一阶段最重要的遗产。iMAP (ICCV, 2021) 首次将 NeRF 与 SLAM 结合，用单个 MLP 编码场景几何，同时优化网络权重和相机位姿。NICE-SLAM (CVPR, 2022) 引入分层特征网格，将大场景表示的可扩展性提升一个数量级 (arXiv)。Co-SLAM 和 ESLAM 进一步优化了表示效率和重建精度。这些工作证明了一个核心洞察：**地图可以是一个可学习的函数，而不是静态的数据结构。**

这一阶段的关键局限在于：语义理解仍然封闭——依赖 COCO、ADE20K 等预定义类别；神经表示的实时性、编辑性和长期更新能力仍然不足。NeRF 的 MLP 推理速度慢，且无法直接编辑——要改变场景中某个物体的位置，必须重新训练整个网络。这些局限正是第三阶段要突破的。

第三阶段：空间智能 SLAM (2023-至今)

空间智能 SLAM 的核心假设：**地图是具身智能系统的空间记忆接口，需要支持查询、推理、规划和长期更新。**

这一阶段的标志是 3D Gaussian Splatting (3DGS) 的引入。Kerbl 等人的 3DGS (SIGGRAPH, 2023) 用数百万个三维高斯椭球替代了 NeRF 的 MLP，实现了 1080p 分辨率的实时渲染 (arXiv)。更重要的是，3DGS 的显式表示使添加、删除、编辑高斯变得直接可行——要移动一个物体，只需移动对应的高斯集合。这种可编辑性是 NeRF 无法比拟的。SplataM (arXiv, 2023)、Gaussian Splatting SLAM (arXiv, 2023)、Photo-SLAM (CVPR, 2024) 等系统迅速证明 3DGS 可以胜任 SLAM 的地图表示。

开放词汇语义理解是第二个关键变量。VLMaps (CVPR, 2023) 将 CLIP 特征投射到 3D 网格上，支持自然语言查询空间位置 (arXiv)。ConceptGraphs (CVPR, 2024) 构建了基于 3D 对象节点的场景图，每个节点附带 CLIP 特征，支持组合式查询 ("A left of B")。OpenScene (CVPR, 2023) 和 LERF (ICCV, 2023) 在 NeRF/3DGS 上实现开放词汇查询，将像素-语言对齐推到了新高度 (CVF开放获取)。

第三个变量是 LLM 与 SLAM 的融合。SayPlan (arXiv, 2023) 用 LLM 进行高层空间规划，LLM-Grounder (arXiv, 2024) 用 LLM 解析自然语言查询并在 3D 场景图中定位。3D-Mem (arXiv, 2024) 和 GSMem (arXiv, 2024) 将 3DGS 地图组织为 LLM 可访问的记忆结构，实现了跨会话的空间问答。

第四个变量是基础模型重塑视觉几何。DUST3R (CVPR, 2024) 用 Transformer 直接从图像对预测点图 (point map)，跳过了传统特征匹配和位姿估计流程。MASt3R-SLAM (CVPR, 2025) 将其扩展为完整 SLAM 系统。VGGT (CVPR, 2025) 和 FoundationSLAM (CVPR, 2025) 展示了"大一统"视觉几何模型的可能性——一个网络处理深度、位姿、点云多个任务 (arXiv)。

这一阶段的共同特征：

- **表示**：3DGS、场景图、神经-显式混合表示、LLM 可访问的记忆结构
- **优化**：可微分渲染 + 语言-视觉对齐 + 因子图融合
- **输出**：可查询、可编辑、可语义检索的空间记忆
- **语义**：开放词汇、自然语言接口、对象关系推理
- **关键瓶颈**：长期自主性尚未验证；计算资源需求高；仿真到现实的迁移仍在早期

三次转移的对比与递进关系

维度	几何 SLAM	学习增强 SLAM	空间智能 SLAM
时间	1986–2015	2016–2022	2023–至今
核心表示	点、线、面、位姿图	NeRF、特征网格、学习描述子	3DGS、场景图、LLM 记忆
前端	手工特征 (SIFT/ORB)	学习特征 (SuperPoint/LoFTR)	基础模型 (DUST3R/VGGT)
后端	EKF/粒子滤波/BA	可微分 BA + 网络训练	可微分渲染 + 因子图 + LLM 推理
语义	无	封闭类别 (COCO/ADE20K)	开放词汇 (CLIP/LLM)
地图用途	定位与路径规划	新视角合成 + 稠密重建	查询、推理、规划、交互
代表系统	ORB-SLAM, LOAM, DSO	DROID-SLAM, NICE-SLAM, NeuralRecon	SplataM, ConceptGraphs, GSMem
核心瓶颈	纹理依赖、无语义	封闭语义、实时性、编辑困难	长期自主、计算成本、Sim2Real

上表展示了三次转移的递进关系：每一阶段不是对前一阶段的否定，而是在保留成果的基础上扩展能力边界。几何 SLAM 建立了状态估计和优化的数学基础；学习增强 SLAM 证明深度学习可以提升每个模块的性能；空间智能 SLAM 将地图从数据结构转变为可操作的记忆系统。

这种递进关系有直接的工程体现。最先进的 3DGS-SLAM 系统（如第11章将讨论的 Gaussian Splatting SLAM）仍然使用因子图优化来计算相机位姿——这是几何 SLAM 的核心技术；它们使用 SuperPoint 或 DUST3R 来做前端特征匹配——这是学习增强 SLAM 的成果；最终它们输出一个可被语言查询的 3DGS 地图——这是空间智能 SLAM 的特征。三个阶段的技术在同一个系统中并存，而不是互相替代。理解这一点，是避免被技术营销话语误导的关键。

0.3 本书不做什么

不做源码逐行讲解

ORB-SLAM3 的 GitHub 仓库有超过 5 万行 C++ 代码。逐行讲解会占据整本书篇幅，且三个月后某个提交就可能让讲解过时。本书的做法是：讲清楚核心算法的数据流和关键决策点，读者带着这些理解去读源码，效率远高于被牵着鼻子走。

不做数学百科

李群 $SE(3)$ 、舒尔补、Levenberg-Marquardt 阻尼因子——这些是 SLAM 的数学基础，但已有成熟教材覆盖（Sola 等人的《A Micro Lie Theory》、Barfoot 的《State Estimation for Robotics》）。本书只引入解决问题所必需的最小数学工具，每个公式都指向具体的设计决策或工程权衡，不为了完备性而堆砌推导。

不做论文平均主义

SLAM 领域每年产生数百篇论文。本书的选择标准：一篇论文必须满足以下至少一个条件才会被精读——(a) 定义了一个子领域的起点；(b) 解决了长期存在的瓶颈；(c) 展示了可工程化的完整系统；(d) 对后续研究产生了路径依赖性的影响。满足条件的论文按精读模板展开，不满足条件的仅在需要对比时被引用。

不做传感器全覆盖

事件相机 SLAM、毫米波雷达 SLAM、声学 SLAM——这些是重要的研究方向，但本书聚焦于视觉（单目/双目/RGB-D）和激光雷达两条主线，因为它们是当前学术界和工业界最活跃、最成熟的技术路线。事件相机和雷达传感器在需要时会被提及，但不会展开为独立章节。

不保证代码可运行

本书提供伪代码和关键算法流程，但不提供可直接编译运行的源码。技术迭代速度太快，可运行的代码库应该通过配套 GitHub 仓库维护，而不是印在纸上。

0.4 本书怎么读

六篇结构总览

本书分为六篇，对应 SLAM 从传统到前沿的完整光谱。

第一篇：传统 SLAM 最小必要知识（第1-6章）

如果你是 SLAM 新手，从这里开始。第1章建立几何基础——针孔模型、对极几何、PnP、BA，以及这些模块如何组合成完整的视觉 SLAM 系统。第2章深入点特征（ORB、SuperPoint）、线特征（LSD、PL-VIO）和面特征，讨论为什么稀疏、半稠密、稠密重建需要不同的前端设计。第3章覆盖视觉 SLAM 代表系统：ORB-SLAM 系列、直接法（DSO）、视觉-惯性融合（VINS-Mono, OKVIS）。第4章转向 LiDAR SLAM：LOAM 系列、基于 ICP 的配准、语义 LiDAR SLAM。第5章讨论 VIO 和紧耦合融合的理论基础。第6章介绍 metric-semantic SLAM——如何将封闭类别语义融入传统 SLAM 框架。

这一篇的目标不是把你变成传统 SLAM 专家，而是建立三个基础：(1) 理解 SLAM 的问题分解方式（前端-后端-回环）；(2) 熟悉关键数据结构和优化工具；(3) 识别传统方法的边界——这些边界正是后续章节要突破的地方。

第二篇：学习增强 SLAM（第7-9章）

第7章讨论深度估计和特征学习如何替代或增强 SLAM 前端。SuperPoint、SuperGlue、LoFTR 的原理和对传统系统的增益是重点。第8章精读 DROID-SLAM（NeurIPS, 2021）——端到端可微 SLAM 的代表作，分析其迭代对应场（Iterative Correspondence Field）的设计和泛化能力。第9章覆盖神经隐式 SLAM：iMAP、NICE-SLAM、Co-SLAM，讨论 NeRF 作为地图表示的优劣，以及为什么这个方向在 3DGS 出现后发生了根本性转折。

第三篇：3DGS-SLAM 主战场（第10-14章）

这是本书的核心篇章。第10章从原理层面讲透 3D Gaussian Splatting：高斯基元的参数化、可微分光栅化、自适应密度控制（split/clone/prune）。第11章覆盖早期 3DGS-SLAM 系统：SplaTAM、Gaussian Splatting SLAM、Photo-SLAM，分析它们如何将 3DGS 嵌入 SLAM 的 tracking-mapping 循环。第12章讨论工程化问题：高斯数量控制、内存优化、动态场景处理、多传感器融合。第13章进入语义 3DGS：LangSplat、SAGA、OpenGaussian——将语言特征嵌入高斯基元，实现开放词汇查询。第14章展望 2026 年 3DGS-SLAM 的前沿方向：实时大场景、终身学习、与扩散模型的结合。

第四篇：视觉几何基础模型（第15–18章）

基础模型正在改变视觉几何的游戏规则。第15章精读 DUST3R（CVPR, 2024）：用 Transformer 直接从图像对预测点图和置信度，统一了深度估计和位姿估计两个传统上分离的问题。第16章讨论 MAST3R-SLAM（CVPR, 2025）如何将 DUST3R 扩展为在线 SLAM 系统。第17章分析 VGGT（CVPR, 2025）和 FoundationSLAM（CVPR, 2025）——"大一统"视觉几何模型的技术路线和设计权衡。第18章讨论基础模型时代 SLAM 的新范式：模块化的传统系统 vs. 端到端基础模型，各自的适用边界在哪里。

第五篇：开放词汇语义地图与具身智能（第19–23章）

SLAM 的终极价值在于支撑具身智能。第19章精读 VLMaps（CVPR, 2023）和 ConceptGraphs（CVPR, 2024）——从 2D 语言特征到 3D 场景图的构建方法。第20章深入 OpenScene、LERF、LangSplat 等开放词汇 3D 理解系统。第21章讨论记忆系统：3D-Mem 和 GSMem 如何将 3DGS 地图组织为 LLM 可访问的结构化记忆，支持跨会话空间问答。第22章覆盖具身导航：从经典 VLN 到基于 LLM 的推理导航（SayPlan, NavGPT）。第23章讨论 VLA 模型与空间地图的耦合——RT 系列、OpenVLA 如何利用（或不利用）空间表示。

第六篇：2026 之后（第24章）

最后一章是对未来的结构化展望。第24章讨论 3DGS-SLAM 的未解决问题（长期自主、多智能体协作、极端环境泛化），以及世界模型（World Models）、扩散模型、物理仿真与 SLAM 的融合趋势。

阅读路径建议

读者背景	推荐阅读路径
SLAM 零基础	第1章 → 第3章 → 第10章 → 第11章 → 第19章
有传统 SLAM 经验，想了解深度学习	第7章 → 第8章 → 第9章 → 第10章 → 第13章
有 NeRF/3DGS 背景，想了解 SLAM	第1章（快速浏览）→ 第3章 → 第10–11章 → 第15–16章
做具身智能/机器人，想了解地图表示	第1章（快速浏览）→ 第10章 → 第19–23章
追求前沿全景	第0章（本章）→ 第8章 → 第11章 → 第15章 → 第19章 → 第24章

上表是起点而非限制。SLAM 领域的知识图谱高度互联：传统 BA 的数学工具在第11章的 3DGS-SLAM 中仍是核心优化手段；第9章的 NeRF-SLAM 与第10章的 3DGS 之间存在直接演进关系；第19章的场景图构建依赖于第6章的 metric-semantic SLAM 基础。鼓励读者在不同篇章间跳跃阅读，遇到前置知识不足时回到对应章节补充。

对于时间有限的读者，还有一个"最小路径"：只读第0章（建立全景认知）→ 第10章（理解 3DGS 原理）→ 第11章（理解 3DGS-SLAM 系统）→ 第19章（理解语义地图）→ 第21章（理解记忆系统）→ 第24章（理解未来方向）。这条路径约覆盖全书三分之一篇幅，但能建立起从传统 SLAM 到空间智能的核心认知框架。

论文精读模板

本书对关键论文采用统一模板精读，包含十个问题：

1. 论文试图解决什么痛点？
2. 核心假设是什么？
3. 输入输出是什么？
4. 地图表示是什么？
5. Tracking 怎么做？
6. Mapping 怎么做？
7. 优化目标是什么？
8. 相比前作真正推进了什么？
9. 没有解决什么？
10. 对后续研究的影响是什么？

前三个问题建立问题意识，中间四个问题拆解技术细节，后三个问题评估影响和边界。读完一篇论文的精读后，你应该能用十分钟向同事讲清楚这篇工作的核心价值，而不是罗列一堆公式。

一个提醒

SLAM 是工程密集型领域。读十篇论文不如跑通一个系统、看一次失败案例。本书尽可能提供工程视角的分析：为什么某个设计在实际中会失效，哪些参数在调参时需要特别注意，哪些论文的实验结果需要带着质疑的眼光审视。理论理解必须通过实践检验，这是本书贯穿始终的方法论立场。

另一个实用的阅读建议：每读完一章，尝试回答"这一章的核心瓶颈是什么""如果用这一章的技术做一个实际系统，最大的坑会在哪里"。这两个问题能帮你从"知道有什么论文"过渡到"知道什么能用"。

本章一句话总结

SLAM 正从"几何估计问题"进化为"空间记忆系统"——本书的目标是在这个进化过程中，为你提供从数学基础到前沿实践的完整技术图景。

第一篇：传统 SLAM 的最小必要知识

第1章 SLAM 问题的本质：估计、约束与地图

SLAM（Simultaneous Localization and Mapping）表面上是两个任务——估计自身轨迹、构建环境地图。但三十年的研究反复证明：**SLAM 的本质是一个联合估计问题**，其核心难点不是算法实现，而是如何在误差累积、数据关联不确定和计算资源受限的三重夹击下，维持对“我在哪里、周围环境是什么”的一致信念。

本章不推导公式，不罗列算法。我们要回答三个问题：SLAM 究竟在估计什么？传感器观测提供了什么约束？地图表示如何决定了系统的上限？理解这三点，后续二十章的所有技术细节才有根基。

1.1 SLAM 的一句话定义

SLAM 是在未知环境中，利用传感器观测同时估计自身轨迹和环境地图。

这句话每一个词都有代价。

"未知环境"意味着没有先验地图可以依赖——GPS 信号可能被高楼遮挡，预下载的地图可能过时，机器人必须从零开始理解空间。"同时"是难点所在：轨迹估计依赖地图，地图构建又依赖轨迹，两者耦合在一起，误差会相互传染。"传感器观测"决定了你能看到什么、看不到什么，也决定了误差的形态。最后，"轨迹"和"地图"的定义本身就在不断演化——从稀疏特征点到神经辐射场，SLAM 社区对"地图"的理解已经翻了五代。

1.1.1 与纯定位、纯重建的区别

纯定位（如基于先验地图的 GNSS/INS 融合）只估计机器人状态，地图是固定的。纯重建（如多视角立体视觉 MVS）只构建地图，相机位姿要么事先标定好，要么在离线阶段一次性求解。SLAM 比两者都困难，因为它没有冷启动的特权——第一帧图像进来时，既没有位姿也没有地图，系统必须在模糊中逐步建立确定性。

问题类型	已知输入	求解目标	核心难点
纯定位	先验地图	机器人位姿	观测与地图的匹配
纯重建	已知相机位姿	三维结构	遮挡、纹理缺失、深度歧义
SLAM	无	位姿+地图联合估计	耦合误差、冷启动、实时性

上表的关键信息是：SLAM 把定位和重建的难点打包在了一起。纯定位中一个匹配错误只会影响当前位姿；SLAM 中一个匹配错误会通过地图污染后续所有帧。这是误差累积的根源。

1.2 状态、观测与约束

SLAM 系统内部只处理三个对象：**状态**（需要估计的东西）、**观测**（传感器给的东西）、**约束**（观测与状态之间的数学关系）。所有算法，无论叫滤波还是优化，本质上都是在约束条件下对状态进行估计。

1.2.1 状态：系统在追踪什么

状态向量通常包含两部分：机器人自身参数和环境参数。

机器人状态取决于传感器配置。最简配置是单目相机，状态只有六自由度位姿 $\mathbf{T} \in SE(3)$ （三维旋转+平移）。加上 IMU 后，状态扩展到速度 $\mathbf{v} \in \mathbb{R}^3$ 和 IMU 零偏 $\mathbf{b} = [\mathbf{b}_g^\top, \mathbf{b}_a^\top]^\top$ （陀螺仪零偏和加速度计零偏），共 15 维。加上轮式编码器后，还可能包含轮径缩放因子、外参标定等。

地图状态的维度取决于表示方式。稀疏特征点 SLAM（如 ORB-SLAM）每张地图点只有三维坐标 $\mathbf{X} \in \mathbb{R}^3$ ，整张地图几百到几千个点。稠密重建（如 DTAM、BundleFusion）地图状态是百万量级的深度值或体素占用概率。隐式表示（如 NeRF、3D Gaussian Splatting）地图参数是神经网络权重或高斯参数，维度从百万到上亿不等。

状态的维度直接决定了优化的代价。稀疏 SLAM 可以在 CPU 上实时运行；稠密或隐式表示通常需要 GPU，且优化周期从毫秒降到秒级。

1.2.2 观测：传感器提供了什么

SLAM 系统常见的传感器及其观测如下：

传感器	观测内容	频率	信息类型	典型误差源
单目相机	二维图像	30Hz	外观、纹理	光照变化、运动模糊、遮挡
RGB-D 相机	图像+深度图	30Hz	外观+几何	深度缺失、多径干扰、边缘失真
激光雷达	三维点云	10Hz	几何结构	点云稀疏、运动畸变、雨雾衰减
IMU	角速度+加速度	200-1000Hz	运动学	零偏漂移、尺度因子误差、温度敏感
轮式编码器	左右轮速	50Hz	里程计	打滑、轮径变化、地面不平
GNSS	全局位置	1-10Hz	绝对坐标	多径、信号遮挡、电离层延迟

多传感器融合的核心直觉是：**不同传感器的误差形态互补**。相机在高纹理区域提供精确的几何约束，但在快速运动或低光照下失效；IMU 在短时间内积分精确，但长时间漂移严重；GNSS 提供绝对位置锚点，但精度和可用性受限。把它们组合在一起，本质上是让各种误差在优化中相互抵消。

1.2.3 约束：观测如何绑定状态

约束是 SLAM 的数学骨架。五种最常见的约束类型：

运动约束 (Motion Constraint)。来自 IMU 或轮速的积分模型。以 IMU 为例，两个时刻 i 和 j 之间的相对位姿、速度和位置可以通过 IMU 测量值积分得到，形成一个预积分因子：

$$\mathbf{r}_{\Delta \mathbf{R}_{ij}} = \log \left(\left(\Delta \tilde{\mathbf{R}}_{ij} \right)^\top \mathbf{R}_i^\top \mathbf{R}_j \right), \quad \mathbf{r}_{\Delta \mathbf{v}_{ij}} = \mathbf{R}_i^\top (\mathbf{v}_j - \mathbf{v}_i - \mathbf{g} \Delta t_{ij}) - \Delta \tilde{\mathbf{v}}_{ij}$$

预积分的核心洞察是：IMU 测量值可以在局部参考系中预先积分，避免每次线性化点更新后重新积分（Forster et al., TRO 2017）。这使得 IMU 因子可以以 $O(1)$ 代价插入因子图，而不是 $O(n)$ 。

重投影误差（Reprojection Error）。来自视觉观测。三维地图点 $\mathbf{X}_k$ 通过当前位姿 $\mathbf{T}_i$ 投影到图像平面，应与实际检测到的特征点 $\mathbf{z}_{ik}$ 重合：

$$\mathbf{r}_{\text{repr}} = \mathbf{z}_{ik} - \pi(\mathbf{T}_i \mathbf{X}_k)$$

其中 $\pi(\cdot)$ 是相机投影模型。重投影误差是视觉 SLAM 最基本的约束单元。它把三维地图状态与二维图像观测绑定在一起。

ICP 误差（Iterative Closest Point）。来自激光雷达或 RGB-D 的深度配准。当前帧的点云 $\mathcal{P}_i$ 与参考点云（或局部地图） $\mathcal{P}_{\text{ref}}$ 通过最近邻匹配建立对应关系，最小化点到平面或点到点的距离：

$$E_{\text{ICP}} = \sum_{(\mathbf{p}, \mathbf{q}) \in \mathcal{C}} (\mathbf{n}_q^\top (\mathbf{T}_i \mathbf{p} - \mathbf{q}))^2$$

ICP 的致命弱点是：它依赖一个足够好的初始位姿猜测。如果初猜偏离太远，最近邻匹配会陷入错误对应，优化收敛到错误解。

回环约束（Loop Closure Constraint）。当机器人回到曾经到过的地方，当前帧与历史帧之间建立的相对位姿约束。回环约束是唯一能够抑制**全局漂移**的手段。没有回环，SLAM 系统的轨迹误差随时间（或距离）线性增长；有了回环，误差可以被重新分配，全局一致性得以恢复。

语义约束（Semantic Constraint）。较新的方向。如果系统检测到"这是同一个门框"或"两张图都是办公桌"，语义匹配可以建立跨时序、跨视角的约束。语义约束的优势在于对视点和光照变化更鲁棒；劣势在于语义分割本身的误差会引入新的污染。

这五种约束在因子图（Factor Graph）中统一表示为节点（状态变量）和因子（约束）。图优化的本质就是调整状态变量，让所有因子的残差平方和最小：

$$\mathbf{X}^* = \arg \min_{\mathbf{X}} \sum_k \|\mathbf{r}_k(\mathbf{X})\|_{\Sigma_k}^2$$

其中 Σ_k 是协方差矩阵，对不同约束按置信度加权。

1.3 SLAM 为什么难

SLAM 不是"视觉+优化"的简单拼接。三十年来，四个结构性难点反复出现，每个都触及问题本质。

1.3.1 误差会累积

这是 SLAM 的宿命性问题。

纯视觉里程计（VO）每帧只估计相对运动，新帧的位姿建立在旧帧估计之上。每帧都有微小误差——特征点检测精度亚像素级、深度估计有噪声、匹配有歧义。误差逐帧传播，轨迹像脱缰的野马逐渐偏离真实路径。

定量来看，单目 VO 的位置漂移率通常在 1% ~ 5% 之间（即每行走 100 米，误差 1 到 5 米）。双目或 RGB-D 因具有绝对尺度，漂移可降至 0.1% ~ 1%。带 IMU 的视觉惯性里程计（VIO）通过高频惯性测量约束，短期漂移更低，但长时间运行后仍然会缓慢发散。

误差累积的数学根源在于：**SLAM 的观测模型是局部的**。一帧图像只能看到前方几十米的范围，无法直接约束全局位置。只有回环检测能够提供全局锚点——当机器人回到起点，起点和终点的坐标本应是同一个，这个约束把漂移"拉"了回来。

1.3.2 数据关联会出错

数据关联（Data Association）回答的问题是：当前看到的東西，和之前看到的東西是同一个吗？

在视觉 SLAM 中，数据关联是特征匹配。两帧图像的特征描述子（ORB、SIFT、SuperPoint）相似，就认为对应同一个三维点。但描述子相似不等于几何一致——重复纹理（一排相同的窗户）、光照剧变（开灯/关灯）、视角大幅变化（正面看 vs 侧面看）都会让匹配出错。

在激光 SLAM 中，数据关联是点云配准。ICP 假设最近邻点就是正确对应，但在走廊、隧道等几何退化场景中，多组不同的位姿变换都可能产生相近的配准误差，系统无法区分。

错误的关联比没有关联更可怕。一个错误的回环约束会把整个地图拉扯变形，引发灾难性后果。因此鲁棒的 SLAM 系统都内置了外点剔除机制（RANSAC、M-estimator、EM 估计内点概率），但保守的外点剔除又会漏掉正确的回环——这是一个经典的精度-召回权衡。

1.3.3 地图表示决定系统上限

SLAM 社区花了二十年才明白：**地图不是副产品，它是系统的核心设计决策**。

稀疏特征点地图（如 PTAM、ORB-SLAM）只保留图像中显眼的角点或斑块，地图轻量、定位精确，但地图本身对人类或下游任务毫无用处——你看不到墙在哪里、门在哪里。

稠密点云地图（如 KinectFusion、BundleFusion）保留了表面的完整几何，支持碰撞检测和路径规划，但存储开销大、更新代价高，且没有语义信息——系统知道"这里有一面墙"，但不知道"这是一面玻璃墙"。

隐式场地图（NeRF、3DGS）提供了连续的表面表示和新视角合成能力，但训练慢、渲染慢，且难以与传统机器人模块（碰撞检测、运动规划）对接。

地图表示的选择，决定了 SLAM 系统能做什么、不能做什么。从本书的视角看，**SLAM 的历史就是地图表示不断升级的历程**——从稀疏点到点云，到 **TSDF**，到神经网络，再到面向具身智能的语义场和场景图。

1.3.4 实时性与全局一致性冲突

SLAM 系统有两个互相矛盾的目标。

实时性要求每帧处理时间在 33ms 以内（30FPS），否则会产生延迟累积，机器人可能已经撞到墙上了，系统还在处理三帧之前的图像。实时性意味着只能做局部优化——只优化最近几帧的位姿和地图。

全局一致性要求所有历史帧和地图点协同优化，消除漂移。全局优化（如 BA，Bundle Adjustment）的计算复杂度随关键帧数量至少呈线性增长，大规模场景下几秒钟都完成不了一次。

所有实用的 SLAM 系统都在走钢丝：用一个轻量前端（跟踪新帧，毫秒级）保证实时性，用一个后台线程（局部/全局 BA、回环检测、位姿图优化）逐步修复全局一致性。前后端的分离是工程妥协，也是误差在系统内流动的通道。

难点	根源	典型应对	代价
误差累积	观测局部性，无全局锚点	回环检测	需要重访，延迟修复
数据关联错误	感知歧义、外观变化	鲁棒核、外点剔除	可能漏掉正确回环
地图表示受限	精度与效率不可兼得	分层混合表示	系统复杂度剧增
实时性 vs 全局一致性	计算资源有限	前后端分离	全局修正有延迟

1.4 地图的五种形态

本节是全书的伏笔。地图表示的选择决定了 SLAM 系统的定位精度、重建质量、存储效率和下游可用性。下面五种形态按出现时间排列，每一种都对应着一个研究范式。

1.4.1 稀疏点地图：定位的最低代价解

稀疏点地图只保留图像中最有辨识度的特征点。PTAM (Klein & Murray, ISMAR 2007) 和 ORB-SLAM (Mur-Artal et al., T-RO 2015) 是这一范式的巅峰。

核心思想：用 FAST 角点或 ORB 描述子提取图像特征，通过多视角三角化估计特征点的三维坐标，后续帧通过匹配这些特征点来定位。

输入：单目/双目/RGB-D 图像序列。

输出：相机轨迹 + 稀疏三维点云。

地图表示：三维点坐标 $\mathbf{X}_k \in \mathbb{R}^3$ + 描述子 $\mathbf{d}_k \in \mathbb{R}^{128/256}$ 。

稀疏点地图的优点是极致轻量——一张桌面级场景的地图只有几千个点，匹配速度快，定位精度高（ORB-SLAM3 在 EuRoC 数据集上达到厘米级）。缺点是地图本身没有几何完整性——系统知道某个角点在空间中的位置，但不知道角点周围是什么。你没法用稀疏点地图做碰撞检测，也没法问“前面有没有障碍物”。

稀疏点地图的另一个局限是：**它对纹理有依赖**。面对白墙、走廊、光滑桌面等低纹理场景，特征提取失败，系统直接丢失。这是稀疏特征点 SLAM 的结构性缺陷。

1.4.2 稠密点云/网格：几何的完整表达

稠密表示追求“看到什么就重建什么”。KinectFusion (Newcombe et al., ISMAR 2011) 开创了实时稠密重建的先河，后续 BundleFusion (Dai et al., TOG 2017) 在精度和鲁棒性上大幅提升。

核心思想：每一帧深度图通过 ICP 与全局模型对齐，深度值融合到截断符号距离函数 (TSDF) 体素网格中，最后通过 Marching Cubes 提取三角网格。

输入：RGB-D 图像序列。

输出：相机轨迹 + 稠密三角网格 / 点云。

地图表示：TSDF 体素网格 $D(\mathbf{x}) : \mathbb{R}^3 \rightarrow \mathbb{R}$ ，表示每个体素到最近表面的有符号距离。

TSDF 的融合公式简洁有效。对于新观测到的深度值 $z(\mathbf{u})$ （像素 $\mathbf{u}$ 对应的深度），沿光线方向找到对应的体素，更新其 SDF 值和权重：

$$D_{\text{new}}(\mathbf{x}) = \frac{W(\mathbf{x})D(\mathbf{x}) + w_{\text{new}}(\mathbf{x})d_{\text{new}}(\mathbf{x})}{W(\mathbf{x}) + w_{\text{new}}(\mathbf{x})}, \quad W_{\text{new}}(\mathbf{x}) = W(\mathbf{x}) + w_{\text{new}}(\mathbf{x})$$

其中 $d_{\text{new}}(\mathbf{x}) = z(\mathbf{u}) - \|\mathbf{x} - \mathbf{c}\|$ 是当前观测的 SDF 值， w_{new} 是该观测的置信度（通常与深度成反比）。

稠密点云/网格适合机器人任务：碰撞检测、表面分析、3D 打印。但它的存储和计算代价高——一个房间级别的 TSDF 网格可能占用数百 MB GPU 内存。更重要的是，它没有语义。系统知道这里有一个表面，但不知道它是地板、墙壁还是人。你没法问“哪里有椅子”，因为地图里没有“椅子”这个概念。

1.4.3 TSDF/Surfel：重建专用的高效表示

TSDF 和 Surfel（表面元素）是稠密重建的两种主流底层表示。

TSDF 如前所述，用体素网格存储有符号距离。优势是融合简单、表面提取成熟（Marching Cubes）；劣势是内存开销与场景体积成正比——空房间和满房间占用同样多内存。

Surfel 表示用一组有向圆盘来近似表面。每个 surfel 包含位置 $\mathbf{p}$ 、法线 $\mathbf{n}$ 、半径 r 和颜色/外观描述。

ElasticFusion (Whelan et al., RSS 2015) 是 Surfel SLAM 的代表作。

核心思想：不用全局体素网格，直接用点云级别的 surfel 集合表示表面。新的深度帧通过 ICP 配准到 surfel 地图，新 surfel 动态添加，旧 surfel 通过局部变形图（deformation graph）来修正回环引起的几何扭曲。

Surfel 表示的优势是**非均匀分辨率**——平坦区域用少量大 surfel，复杂区域用密集团 surfel，内存效率更高。劣势是大规模场景下 surfel 数量膨胀，且回环修改变形图的计算代价随场景规模增长。

TSDF/Surfel 的共同局限是：它们为重建而生，不为理解而生。地图里没有对象、没有语义、没有关系。机器人看到一张桌子，地图里只是一堆点或 surfel，系统不知道“这是一张桌子”。

1.4.4 NeRF/隐式场：连续、紧凑，但昂贵

神经辐射场（NeRF, Mildenhall et al., ECCV 2020）引发了三维表示的革命。NeRF 不用显式存储点或体素，而是用一个神经网络来表示整个场景。

核心思想：场景被建模为一个连续函数 $F_\theta : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$ ，输入是空间位置 $\mathbf{x} \in \mathbb{R}^3$ 和观察方向 $\mathbf{d} \in \mathbb{S}^2$ ，输出是颜色 $\mathbf{c} \in \mathbb{R}^3$ 和体密度 $\sigma \in \mathbb{R}_+$ 。渲染一张图像通过体渲染积分实现：

$$\hat{C}(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt, \quad T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$$

其中 $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ 是从相机光心出发的光线。

NeRF 对 SLAM 的启发在于：地图可以是一个连续、可微的函数。这打破了“离散点/体素”的范式——不再需要 Marching Cubes 提取表面，不再受限于体素分辨率。NICE-SLAM (Zhu et al., CVPR 2022) 和 Co-SLAM (Wang et al., CVPR 2023) 将 NeRF 与 SLAM 结合，实现了稠密神经重建。

但 NeRF-SLAM 有三道坎：

1. **训练慢**。NeRF 需要逐场景优化网络权重，一次完整训练需要数小时到数天。NICE-SLAM 通过分层特征网格加速，但实时性仍有差距。

2. **渲染慢。**每张图像需要沿每条光线进行数百次网络前向传播，实时渲染困难。
3. **难以编辑和查询。**神经网络是一个黑盒，你无法直接问"距离我 1 米内的障碍物在哪里"，需要采样查询，效率低下。

隐式场的核心贡献是证明了"连续表示"的可行性。但它的训练代价和不可解释性，限制了在实时机器人系统中的应用。

1.4.5 3DGS/语义场/场景图：具身智能的新地图接口

三维高斯散射 (3D Gaussian Splatting, 3DGS, Kerbl et al., SIGGRAPH 2023) 用一组三维高斯分布来表示场景，每个高斯包含位置 μ 、协方差 Σ 、不透明度和球谐系数 (SH) 表示的颜色。

核心思想：不用神经网络，直接用显式的高斯原语集合表示场景。渲染时，高斯被投影到图像平面并通过光栅化 (splatting) 混合，避免了 NeRF 中昂贵的光线采样。

3DGS 对 SLAM 的意义是三重突破：

1. **渲染快。**光栅化比体渲染快 1-2 个数量级，SplataTAM (Keetha et al., CVPR 2024) 和 Gaussian Splatting SLAM (Yugay et al., CVPR 2024) 实现了接近实率的重建。
2. **优化快。**高斯参数可以直接通过梯度下降优化，不需要训练神经网络。新增观测可以通过分裂、克隆、剪枝高斯来动态更新地图。
3. **显式+连续兼得。**高斯集合是显式的（可以直接读写），但场景的连续表面通过高斯混合隐式表达。

语义场 (Semantic Field) 在 3DGS/NeRF 的基础上增加语义输出：每个高体素/高斯不仅输出颜色，还输出语义类别概率。系统知道"这里不仅是红色的，而且是椅子"。OpenFusion (Rosu et al., ICRA 2024) 和 SAM3D 系列工作正在推动这一方向。

场景图 (Scene Graph) 是更高层次的地图表示。它不直接存储几何，而是存储**对象级别的关系图**：节点是检测到的对象（桌子、椅子、人），边是空间关系（"左边"、"上面"、"邻近"）。Hydra (Hughes et al., RSS 2022) 和 ConceptGraphs (Gu et al., CVPR 2024) 是这一范式的代表。

场景图对具身智能至关重要。当一个人说"去厨房拿桌子上的杯子"，机器人不需要知道杯子的精确三维坐标，它需要知道"厨房"和"桌子"的关系，以及"杯子"在"桌子"上。场景图天然支持这种符号级别的推理。

地图形态	代表工作	核心优势	核心局限	适用任务
稀疏点地图	ORB-SLAM3	轻量、实时、定位精确	无几何/语义信息	定位、AR
稠密点云/ TSDF	KinectFusion, BundleFusion	完整几何、碰撞检测	内存大、无语义、难更新	重建、导航
Surfel 地图	ElasticFusion	非均匀分辨率、局部变形	规模膨胀、回环复杂	室内重建
NeRF/隐式场	NICE-SLAM, Co-SLAM	连续、紧凑、新视角合成	训练慢、渲染慢、黑盒	可视化、编辑
3DGS/语义场/场景图	SplaTAM, ConceptGraphs	实时、显式、可查询、可推理	表示仍在演化	具身智能、语言交互

上表揭示了一个清晰的演进脉络：**地图从"给人看的几何"走向"给机器理解的环境模型"**。稀疏点只能定位，稠密几何可以导航，隐式场可以合成新视角，而语义场和场景图才能让机器人听懂"把那个红色盒子放到左边柜子上"这样的指令。

1.5 SLAM 的系统骨架

尽管具体实现千差万别，SLAM 系统的模块结构高度统一。理解这个骨架，后续章节讨论任何具体算法时都能迅速定位。

1.5.1 前端：感知与局部估计

前端负责处理原始传感器数据，提取信息，并给出初始的位姿和地图估计。

视觉前端：检测特征点（ORB、SuperPoint）或直接法像素匹配（DSO），通过最小化光度误差或重投影误差估计帧间运动，输出相对位姿。

惯性前端：IMU 预积分，输出两帧之间的相对旋转、速度和位置约束。

激光前端：点云配准（ICP、NDT、特征匹配），输出帧间位姿变换。

前端的共同特征是**局部性**——只关心相邻帧之间的关系，不关心全局一致性。前端的运行频率最高（通常每帧都要处理），是系统的实时瓶颈。

1.5.2 后端：优化与全局一致性

后端接收前端输出的约束（因子），构建并求解优化问题。

局部 BA：只优化最近 N 个关键帧和它们看到的地图点。规模可控，可以每帧或每隔几帧运行一次，消除短期漂移。

全局 BA：优化所有关键帧和所有地图点。计算代价高，通常在回环检测后触发，或在后台线程中以较低频率运行。

位姿图优化：当地图规模很大时，全局 BA 的变量数爆炸。位姿图优化只优化关键帧位姿，忽略地图点，把地图点的约束转化为帧间相对位姿约束。这是大尺度 SLAM 的标配。

后端的本质是一个**稀疏最小二乘求解器**。无论用 Levenberg-Marquardt、Dog-Leg 还是共轭梯度，核心都是在海森矩阵（或信息矩阵）的稀疏结构上做高效求解。第 2 章会深入推导。

1.5.3 回环检测：全局一致性的最后防线

回环检测模块独立运行，持续将当前帧（或关键帧）与历史帧比较，判断"这里是不是来过"。

传统方法用词袋模型（Bag of Words, DBoW2/DBoW3）将图像描述为视觉词汇的直方图，通过向量相似度判断回环。深度学习方法用网络提取全局描述子（NetVLAD、CosPlace），在特征空间中检索最近邻。

检测到回环后，系统执行两个操作：

1. **几何验证**：用 RANSAC 和位姿估计验证回环的几何一致性，剔除误匹配。
2. **全局修正**：将回环约束插入位姿图或因子图，触发全局优化，将漂移"拉平"。

回环检测是 SLAM 系统中**最不容出错**的模块。一个假阳性回环可以摧毁整张地图；一个假阴性回环则让漂移继续累积。精度-召回的权衡是回环检测的核心 tension。

1.5.4 地图管理：表示的选择与更新

地图管理模块负责地图的存储、查询、融合和更新。不同表示（稀疏点、TSDF、NeRF、3DGS）对应完全不同的地图管理策略。

地图管理的一个关键决策是**关键帧策略**。不是所有帧都值得保留——视觉变化不大或视点重复的帧被丢弃，只有信息增益足够大的帧被选为关键帧插入地图。关键帧选择影响地图的冗余度、覆盖范围和计算效率。

1.6 与具身智能的关系

SLAM 正在从一个独立的机器人子模块，演变为具身智能的**空间感知基座**。这个转变的本质是：SLAM 的输出不再是"轨迹和点云"，而是**机器人在环境中行动所需的空间理解**。

传统 SLAM 的终点是"把地图建出来"。具身智能要求 SLAM 回答更深层的问题：

- **导航**："从这里到厨房的最短路径是什么？"——需要拓扑地图或占据栅格。
- **操作**："杯子在桌子的哪个位置？"——需要对对象级别的语义定位和 6-DoF 位姿估计。
- **交互**："请描述你周围的环境"——需要场景图和自然语言接口。
- **长期自主**："三个月前那张桌子还在这里吗？"——需要动态地图更新和终身学习能力。

这些需求推动了地图表示的升级（第 1.4 节的五种形态），也推动了 SLAM 与基础模型的融合。视觉-语言模型（VLM）为 SLAM 提供语义理解能力；SLAM 为 VLM 提供三维空间上下文。两者结合，才能支撑真正的具身智能。

本书的后半部分（第 14-24 章）将专门讨论这一融合。本章只想传递一个信息：**SLAM 的问题框架（估计+约束+地图表示）是具身智能空间能力的理论根基**。无论地图是稀疏点还是语义场，无论优化是滤波还是深度学习，SLAM 的本质问题没有变：在不确定性的海洋中，用有限的观测推断自我与环境的状态。

1.7 代表论文精读

1.7.1 ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM

作者：Carlos Campos, et al. | 来源：IEEE T-RO 2021 (arXiv)

1. 论文试图解决什么痛点？

单目 SLAM 无法恢复绝对尺度；视觉惯性 SLAM 在快速运动或低纹理场景下丢失后难以恢复；现有系统无法在长时间运行中管理多个独立地图。

2. 核心假设是什么？

- 场景中存在足够的纹理特征可供 ORB 提取和匹配。
- IMU 可用且已标定，提供高频运动约束和绝对尺度。
- 回环检测可以建立当前地图与历史地图之间的连接。

3. 输入输出是什么？

输入：单目/双目/RGB-D 图像序列 + 可选 IMU 数据。

输出：六自由度位姿轨迹 + 稀疏三维特征点地图 + 多地图合并结果。

4. 地图表示是什么？

稀疏三维点云 + ORB 描述子。每个地图点存储三维坐标、平均视角方向、最佳描述子、可见关键帧集合。

5. Tracking 怎么做？

三阶段跟踪：初始帧间匹配估计运动；局部地图跟踪通过投影匹配 refinement；如果跟踪丢失，启动重定位模块（DBoW2 词汇树检索 + EPnP + BA）。

6. Mapping 怎么做？

局部 BA 优化最近的关键帧和地图点；全局 BA 在后台线程运行；回环检测通过词袋模型匹配候选回环帧，Sim3 位姿对齐后执行位姿图优化和全局 BA。

7. 优化目标是什么？

视觉重投影误差 + IMU 预积分误差 + 多地图联合重投影误差，统一在因子图中优化。

8. 相比前作真正推进了什么？

- 提出 Atlas 多地图框架：丢失后不重置，而是创建新地图，后续回环时合并。
- 视觉惯性紧耦合：IMU 预积分与视觉 BA 统一优化，尺度可观测。
- 支持最大惯性 SLAM（纯 IMU 初始化后即可启动）。

9. 没有解决什么？

- 仍然是稀疏点地图，没有语义、没有稠密几何。
- 对动态场景敏感，没有内置动态物体剔除机制。
- 回环检测依赖 DBoW2，对大幅视角变化鲁棒性有限。

10. 对后续研究的影响是什么？

ORB-SLAM3 是稀疏视觉 SLAM 的工程巅峰。它证明了在 CPU 上实现高精度多传感器 SLAM 是可行的，也为后续工作提供了基准架构。其多地图管理思想影响了动态 SLAM 和长期自主导航研究。

1.7.2 NICE-SLAM: Neural Implicit Scalable Encoding for SLAM

作者：Zihan Zhu, et al. | 来源：CVPR 2022 (CVF 开放获取)

1. 论文试图解决什么痛点？

NeRF 用于 SLAM 时，逐场景训练慢、无法实时；纯坐标 MLP 难以表达高频几何细节；单一全局特征网格无法同时处理多尺度细节。

2. 核心假设是什么？

- RGB-D 输入可用（提供深度和颜色监督）。
- 场景几何可以在分层特征网格中多尺度表达。
- 局部优化可以在有限的关键帧窗口内实现准实时性能。

3. 输入输出是什么？

输入：RGB-D 图像序列。

输出：相机轨迹 + 连续隐式场景表示（可渲染任意视角、可提取网格）。

4. 地图表示是什么？

分层特征网格（Mid-level + Low-level + High-level）+ 一个浅层解码 MLP。不同层级负责不同尺度的几何和外观细节。

5. Tracking 怎么做？

通过渲染当前帧的深度和颜色图，与观测值比较，最小化光度误差和深度误差来优化相机位姿：

$$E_{\text{track}} = \lambda_d \|\hat{D} - D\|^2 + \lambda_p \|\hat{I} - I\|^2$$

6. Mapping 怎么做？

关键帧窗口内的局部 BA：固定位姿，优化特征网格参数和解码 MLP 权重。通过随机采样像素进行优化，而非全图优化。

7. 优化目标是什么？

深度重建误差 + 颜色光度误差 + 特征网格正则化，联合优化。

8. 相比前作真正推进了什么？

- 分层编码替代单一 MLP，几何细节表达能力大幅提升。
- 引入了局部关键帧窗口机制，使大规模场景 SLAM 可行。
- 在 Replica 和 ScanNet 数据集上首次实现了接近 NeRF 质量的实时 SLAM 重建。

9. 没有解决什么？

- 实时性仍受限于 GPU 计算，无法达到 30FPS。
- 依赖 RGB-D 输入，纯单目方案失败。
- 没有回环检测，长时间运行仍有漂移。
- 隐式表示难以直接用于碰撞检测和运动规划。

10. 对后续研究的影响是什么？

NICE-SLAM 开创了"分层神经编码 + SLAM"的范式，直接启发了 Co-SLAM（引入坐标编码）、Point-SLAM（用神经点替代网格）等工作。它证明了隐式表示在 SLAM 中的可行性，也暴露了隐式 SLAM 在实时性和功能性上的根本局限——这正是 3DGS SLAM 崛起的背景。

1.7.3 SplatAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM

作者：Nikhil Keetha, et al. | 来源：CVPR 2024 (CVF 开放获取)

1. 论文试图解决什么痛点？

NeRF-SLAM 渲染慢、训练慢、难以增量更新。3D Gaussian Splatting 提供了快速渲染和显式表示的能力，但尚未与 SLAM 系统有效结合。

2. 核心假设是什么？

- RGB-D 输入可用。
- 场景可以用一组三维高斯充分近似。
- 高斯光栅化的可微性允许通过梯度下降进行端到端位姿和地图优化。

3. 输入输出是什么？

输入：RGB-D 图像序列。

输出：相机轨迹 + 三维高斯地图（可实时渲染新视角、可提取网格）。

4. 地图表示是什么？

三维高斯集合 $\mathcal{G} = \{\mu_i, \Sigma_i, \alpha_i, \mathbf{SH}_i\}$ ，每个高斯包含位置、协方差、不透明度和球谐颜色系数。

5. Tracking 怎么做？

通过可微高斯光栅化渲染当前视角的深度图和 silhouette，与 RGB-D 观测比较，优化位姿：

$$E_{\text{track}} = \lambda_{\text{rgb}} \|\hat{I} - I\|_1 + \lambda_{\text{depth}} \|\hat{D} - D\|_1 + \lambda_{\text{sil}} \text{BCE}(\hat{S}, S)$$

6. Mapping 怎么做？

每帧观测通过以下方式更新高斯地图：

- 深度不连续区域生成新高斯（增加几何细节）。
- 已有高斯通过梯度更新位置、形状和外观。
- 低贡献度高斯被剪枝，控制地图规模。

7. 优化目标是什么？

光度一致性 + 深度一致性 + Silhouette 一致性 + 高斯正则化（控制高斯数量和尺寸）。

8. 相比前作真正推进了什么？

- 首次将 3DGS 完整集成到 SLAM 闭环中，渲染速度比 NeRF-SLAM 快 1-2 个数量级。
- 显式高斯表示允许增量更新——新增和删除高素是局部操作，不像 NeRF 需要重新训练网络。
- RGB 和深度同时用于 tracking 和 mapping，精度超过当时的 NeRF-SLAM 方法。

9. 没有解决什么？

- 依赖 RGB-D 输入，单目 3DGS SLAM 仍在研究中。
- 没有显式的回环检测和全局优化机制，长时间运行漂移问题未解决。
- 高斯数量随场景增长，大规模场景下的内存管理尚未解决。
- 没有语义信息，无法支持高层理解和交互。

**10. 对后续研究的影响是什么？

SplaTAM 是 3DGS-SLAM 浪潮的标志性工作。它证明了 3DGS 不仅可以替代 NeRF 用于新视角合成，还可以作为 SLAM 的核心地图表示。后续 Gaussian Splatting SLAM (Yugay et al., 2024)、GS-SLAM (Yan et al., 2024) 等工作在此基础上进一步提升了精度和鲁棒性。SplaTAM 也代表了地图表示从隐式神经网络回归显式原语的趋势——这种趋势与具身智能对“可查询、可编辑、可推理”地图的需求高度一致。

1.8 本章总结

SLAM 是在未知环境中同时估计自身轨迹和环境地图的联合估计问题。它的核心不是某个算法，而是三个永恒的张力：**误差在局部观测中累积，数据关联在感知歧义中出错，地图表示在精度与效率之间取舍**。传感器提供运动约束、重投影约束、回环约束和语义约束，这些约束在因子图中博弈，最终输出对自我和环境状态的信念。

地图表示决定了 SLAM 系统的上限。从稀疏点到稠密点云，到 TSDF/Surfel，到 NeRF 隐式场，再到 3DGS 和场景图——每一次升级都在回答同一个问题：**地图不只是给定位用的，它是机器理解空间、与人类交互、在环境中行动的根基**。本书后续章节将沿着这条线索展开：先建立数学骨架，再深入每一种传感器和算法，最终走向具身智能的空间感知未来。

第2章 概率滤波到图优化：传统 SLAM 的数学骨架

第1章把 SLAM 描述为一个状态估计问题：机器人从带噪声的观测中推断自身位姿和环境结构。估计问题需要数学工具。本章沿着 SLAM 数学骨架的演进主线——从卡尔曼滤波到粒子滤波，再到图优化——梳理出一套贯穿全书的统一语言。

这条演进路线不是简单的技术替代，而是认知框架的跃迁。EKF-SLAM 把"估计"理解为在线更新一个概率分布；粒子滤波把"不确定性"表示为多个假设的并行追踪；图优化则把"一致性"表达为对全局约束的联合求解。理解这三条路线的内在逻辑，是阅读后续所有视觉/激光/多传感器 SLAM 系统的前提。

2.1 EKF-SLAM：历史起点

SLAM 问题的早期解法几乎全部建立在贝叶斯滤波框架上，核心是扩展卡尔曼滤波（Extended Kalman Filter, EKF）。EKF-SLAM 的思想极其直观：把机器人的位姿和环境中的所有路标点的坐标拼成一个**状态向量**，用一个多元高斯分布描述这个向量的不确定性——均值代表最优估计，协方差矩阵代表估计的置信度。

设状态向量为 $\mathbf{x} = [\mathbf{x}_r^\top, \mathbf{m}_1^\top, \dots, \mathbf{m}_N^\top]^\top$ ，其中 $\mathbf{x}_r$ 是机器人位姿， $\mathbf{m}_i$ 是第 i 个路标点。EKF 维护的协方差矩阵 $\mathbf{P}$ 的维度为 $(3 + 2N) \times (3 + 2N)$ （2D 情形）或 $(6 + 3N) \times (6 + 3N)$ （3D 情形）。每一步运动更新和观测更新都遵循标准的 EKF 递推公式：预测时根据运动模型推进状态，更新时根据观测残差和卡尔曼增益校正状态。

特征数量 N	状态维度	协方差矩阵元素数	单次更新运算量
10	23	529	$\sim 10^4$
100	203	41,209	$\sim 10^6$
500	1,003	$\sim 10^6$	$\sim 10^8$
1,000	2,003	$\sim 4 \times 10^6$	$\sim 10^9$

协方差矩阵的元素数随路标数量呈二次增长。更关键的是，观测更新需要对一个与状态维度同规模的矩阵求逆，计算复杂度达到 $O(N^3)$ 。当路标超过几百个时，EKF-SLAM 就失去了实时性。上表中 1,000 个特征点的场景意味着协方差矩阵有 400 万元素，单次更新需约 10^9 次浮点运算——这即便在当代嵌入式平台上也是沉重负担。

除了计算瓶颈，EKF-SLAM 还面临一个更深层的结构缺陷：**线性化误差**。运动模型和观测模型本质上是非线性的，EKF 用一阶泰勒展开将其局部线性化。线性化点选择在当前估计值处，而当系统远离线性化点（例如机器人快速转弯、观测角度大幅变化）时，线性化误差会导致协方差估计过于乐观，最终滤波器发散。这种发散不是工程调参能彻底解决的，它根植于 EKF 的单点线性化机制。

数据关联错误（Data Association Error）是另一个致命弱点。EKF-SLAM 依赖最近邻等启发式方法将当前观测与已有路标匹配。一旦把观测 A 错误地关联到路标 B ，协方差矩阵会把这个错误信息传播到整个状态，后续滤波步骤很难自行恢复。在特征密集或外观相似的环境中，这种错误几乎是必然的。

EKF-SLAM 在学术史上意义重大——它首次给出了 SLAM 问题的完整概率解法，证明了 SLAM 的可行性。但作为工程方案，它已经被后来者全面超越。

2.2 粒子滤波与 FastSLAM

粒子滤波（Particle Filter）用一种完全不同的方式表达不确定性。EKF 用一个高斯分布的均值和协方差刻画整个后验；粒子滤波则用一组加权的**粒子**（样本）直接近似后验分布。每个粒子代表一个完整的状态假设，权重反映该假设与观测的吻合程度。

粒子滤波的核心操作循环是：传播（按运动模型采样新粒子）、权重更新（按观测似然调整权重）、重采样（按权重复制/淘汰粒子）。这种非参数表示天生可以描述多模态分布和强非线性系统，这是 EKF 无法做到的。

FastSLAM（Montemerlo et al., 2002）是粒子滤波在 SLAM 中最成功的应用。它的核心洞察来自对 SLAM 后验的概率分解。给定机器人的完整轨迹 $\mathbf{x}_{0:t}$ ，所有路标点的估计彼此条件独立：

$$p(\mathbf{x}_{0:t}, \mathbf{m}_1, \dots, \mathbf{m}_N \mid \mathbf{z}_{1:t}) = p(\mathbf{x}_{0:t} \mid \mathbf{z}_{1:t}) \prod_{i=1}^N p(\mathbf{m}_i \mid \mathbf{x}_{0:t}, \mathbf{z}_{1:t}) \tag{1}$$

这个分解是精确的，不需要近似（arXiv）。FastSLAM 据此将 SLAM 拆分为两部分：用粒子滤波估计机器人轨迹 $p(\mathbf{x}_{0:t} \mid \mathbf{z}_{1:t})$ ，每个粒子携带 N 个独立的 EKF，分别估计 N 个路标点在给定该粒子轨迹下的位置。

这种**Rao-Blackwellized** 分解把时间复杂度从 EKF 的 $O(N^3)$ 降到了 $O(M \log N)$ ，其中 M 是粒子数， N 是路标数。对数因子来自树状数据结构对路标查找的加速。2002 年的实验表明，FastSLAM 可以处理比 EKF-SLAM 多两个数量级的路标点（Stanford）。

方法	轨迹表示	地图表示	复杂度	非线性处理	主要缺陷
EKF-SLAM	单点估计	联合高斯	$O(N^3)$	一阶线性化	线性化误差、计算膨胀
FastSLAM	粒子集	条件独立 EKF	$O(M \log N)$	粒子采样	粒子退化、高维失效

粒子退化（Particle Degeneracy）是 FastSLAM 的致命弱点。重采样过程中，权重低的粒子被剔除，高权重粒子被复制。经过若干步后，有效粒子数急剧下降，绝大部分粒子共享同一条祖先轨迹，轨迹空间的多样性丧失。FastSLAM 2.0 通过把观测信息融入提议分布来缓解这一问题，但无法根除。在 3D 空间中，粒子滤波需要的粒子数随状态维度指数增长——这是维度灾难，不是算法补丁能解决的。

FastSLAM 的历史贡献在于证明了 SLAM 后验的可分解性，以及非高斯不确定性在 SLAM 中的表达可能性。但它没能成为现代 SLAM 的主流框架，根本原因是：粒子集不适用于大规模增量优化，且无法自然地与回环检测等全局约束机制协同工作。

2.3 图优化：现代 SLAM 的核心形式

从 2007 年开始，SLAM 的数学骨架经历了一次根本性范式转移：从**滤波**（filtering）转向**平滑**（smoothing）。滤波方法只维护当前时刻的状态估计，历史观测被"吸收"进当前状态后就被丢弃。平滑方法则保留全部历史状态（或一个滑动窗口内的状态），把所有观测当作对整条轨迹的约束，通过全局优化求解。

这个转变的动机来自 SLAM 的本质需求：**一致性**。当机器人经过长距离运动回到起点时，回环检测（Loop Closure）发现"当前位置就是曾经到过的地方"。这条约束跨越了数百甚至数千个时间步，EKF 和粒子滤波都无法高效地将其融入估计——它们已经丢掉了旧时刻的状态。图优化天然解决这个问题：回环约束只是一条连接两个历史节点的额外边，优化器会自动将误差沿图传播到所有受影响的位姿。

2.3.1 因子图的构成

现代 SLAM 的后端几乎全部被建模为**因子图**（Factor Graph）。因子图是一种二分图，包含两类节点：

变量节点（Variable Nodes）代表需要估计的量：

- 机器人位姿 $\mathbf{T}_i \in SE(3)$
- 三维路标点 $\mathbf{p}_j \in \mathbb{R}^3$
- IMU 速度 $\mathbf{v}_k \in \mathbb{R}^3$
- IMU bias $\mathbf{b}_k \in \mathbb{R}^6$

因子节点（Factor Nodes）编码概率约束，每个因子对应一个误差函数 $e(\cdot)$ 和一个信息矩阵 Σ^{-1} （即协方差的逆，表达该约束的置信度）：

因子类型	连接变量	误差定义	来源
先验因子	单个位姿	与先验估计的偏差	初始化、GPS
里程计因子	相邻位姿	相对位姿测量残差	视觉里程计、激光里程计
重投影因子	位姿 + 路标点	图像投影位置残差	特征点匹配
IMU 预积分因子	相邻位姿+速度+bias	预积分残差	IMU 测量值累积
回环因子	非相邻位姿	回环检测的相对位姿残差	场景重识别

2.3.2 优化目标

因子图优化的目标是找到所有变量节点的一组赋值，使所有因子产生的**加权误差平方和**最小：

$$\mathbf{X}^* = \arg \min_{\mathbf{X}} \sum_k \|\mathbf{e}_k(\mathbf{X}_k)\|_{\Sigma_k}^2 \tag{2}$$

其中 $\mathbf{X}_k$ 是与第 k 个因子关联的变量子集， $\|\cdot\|_{\Sigma}^2 = \mathbf{e}^\top \Sigma^{-1} \mathbf{e}$ 是马氏距离（Mahalanobis distance），信息矩阵 Σ^{-1} 自动对高置信度约束赋予更大权重。

这个目标函数是一个**非线性最小二乘**问题。求解通常采用 Gauss-Newton 或 Levenberg-Marquardt 迭代法：在当前估计点处线性化每个误差函数，求解一次线性最小二乘得到增量，更新变量，重复直到收敛。

2.3.3 稀疏性：图优化高效的根基

图优化之所以能在实时 SLAM 中运行，关键不在于优化算法本身，而在于 SLAM 问题固有的**稀疏性**。一个路标点只被一小部分相机位姿观测到——不是全部位姿都观测到全部路标点。在因子图中，这种稀疏性表现为：大部分变量节点之间没有直接连接，Hessian 矩阵（二阶信息矩阵）是稀疏的。

以 Visual SLAM 为例，若轨迹包含 1,000 个关键帧和 50,000 个路标点，状态变量总数超过 15 万，对应的 Hessian 矩阵在理论上是一个 150000×150000 的稠密矩阵。但由于稀疏性，实际非零元素占比不足 0.01%。利用稀疏线性代数求解器（如稀疏 Cholesky 分解），优化时间从不可行降到几十毫秒。

稀疏性还带来了模块化的系统架构。新的传感器接入只需增加对应的因子类型，不需要改动优化器的核心结构。视觉因子、IMU 因子、激光因子、GPS 因子、回环因子可以在同一个图中和平共处——这是滤波方法无法企及的灵活性。

2.3.4 从批处理到增量优化

最早的图优化 SLAM 是**批处理**（batch）模式：收集全部数据，构建完整因子图，一次性优化。这种"离线后处理"模式精度最高，但不满足在线 SLAM 的实时需求。

iSAM（Kaess et al., ICRA 2008）引入了增量 QR 矩阵分解：新测量到来时，只更新 QR 分解中变化剧烈的部分，而非从头求解。iSAM2（Kaess et al., TRO 2012）进一步提出 **Bayes Tree** 数据结构，实现了对受新测量影响的子图的局部重新线性化，把增量优化的效率提升了一个数量级。今天 GTSAM 库中的增量优化器就是 iSAM2 的延续（CVF开放获取）。

优化模式	处理方式	延迟	适用场景
批处理 BA	全图优化	高	离线重建、SfM
局部/滑动窗口	局部子图优化	中	实时视觉-惯性 SLAM
增量式 (iSAM2)	Bayes Tree 局部分解	低	在线图优化 SLAM
双线程（ORB-SLAM）	局部BA+全局BA异步	低	实时关键帧式 SLAM

上表展示了优化模式的演进。现代 SLAM 系统普遍采用"局部+全局"的双层策略：前端线程用滑动窗口局部 BA 保证跟踪实时性，后端线程在检测到回环时触发全局位姿图优化修正累积漂移。ORB-SLAM3、VINS-Mono 等系统的成功，很大程度上归功于这种分层优化策略。

2.3.5 代表性求解器

SLAM 社区发展了几套成熟的图优化后端库：

g2o（Kümmerle et al., ICRA 2011）是 Freiburg 大学开发的通用图优化框架。设计简洁、易于扩展，支持 Gauss-Newton、Levenberg-Marquardt 等多种求解器，是视觉 SLAM 研究中使用最广泛的优化库。ORB-SLAM 系列默认使用 g2o 作为后端。

Ceres Solver（Google）是一个通用的非线性最小二乘库，不是专门为 SLAM 设计，但凭借出色的自动微分（Auto Diff）和数值稳定性，成为视觉-惯性 SLAM 系统的首选后端。OKVIS、VINS-Mono 都基于 Ceres 构建。

GTSAM (Georgia Tech) 以因子图的数学完备性著称，内置 iSAM2 增量求解器，在因子图的稀疏性利用上最为深入。适合需要精确不确定度估计和增量推理的研究型系统。

SE-Sync (Rosen et al., TRO 2019) 是一种基于半定规划的求解器，能够为位姿图优化提供**全局最优性证书**。当数据关联噪声大、前端性能差时，SE-Sync 比传统局部优化器更鲁棒，但计算开销更高。

2.3.6 关键论文精读：iSAM2

iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree (Kaess et al., TRO 2012)

痛点：图优化 SLAM 中，每次新测量到来都重新求解全图是不现实的。需要一种只更新受新测量影响的部分子图的方法。

核心假设：因子图具有良好的稀疏性和局部性——新测量只与少量历史变量直接相关。

输入输出：输入为增量到达的因子（位姿约束、观测约束等）；输出为所有变量节点的 MAP 估计和边缘协方差。

方法结构：将因子图转换为 Bayes Tree 数据结构。Bayes Tree 编码了变量消除 (variable elimination) 产生的条件独立性结构。新因子加入时，只需重新消除 Bayes Tree 中受影响的路径 (clique path)，将变化限制在局部子树内。

相比前作推进了什么：iSAM (2008) 使用增量 QR 分解，但需要周期性重新排序以保持稀疏性。iSAM2 用 Bayes Tree 消除了这种"批量重启"需求，真正做到了纯增量式求解。

没有解决什么：当回环约束连接了时间跨度很大的两个位姿时，clique path 可能很长，局部更新退化为近乎全局的重新求解。此外，重线性化的阈值是经验参数，调参不当影响收敛效率。

对后续研究的影响：iSAM2 确立了增量图优化的范式，GTSAM 库至今仍是学术研究和高级 SLAM 系统的标准后端之一。其 Bayes Tree 思想也为后续概率推理库（如 MiniSAM）提供了架构模板。

2.4 Bundle Adjustment

Bundle Adjustment (光束法平差，简称 BA) 是图优化在视觉 SLAM 中最核心的表现形式。理解 BA 不需要复杂的公式，抓住一个直觉就够了：**同时优化相机位姿和三维点位置，使所有三维点到图像上的投影与实际观测点尽可能重合。**

"Bundle"一词来自摄影测量学，指从相机光心出发、穿过图像点、射向三维空间的光线束。"Adjustment"即调整相机和三维点的参数，让这些光线束在空间中正确交汇。

2.4.1 重投影误差

BA 的误差函数是**重投影误差** (Reprojection Error)。给定一个三维点 $\mathbf{P}_j$ 和一个相机位姿 $\mathbf{T}_i$ ，将该点投影到相机坐标系：

$$\mathbf{p}_{ij}^{\text{proj}} = \pi(\mathbf{T}_i^{-1} \mathbf{P}_j) \quad (3)$$

其中 $\pi(\cdot)$ 是相机投影模型（含内参）。重投影误差定义为投影位置与观测位置 $\mathbf{z}_{ij}$ 的像素距离：

$$\mathbf{e}_{ij} = \mathbf{z}_{ij} - \pi(\mathbf{T}_i^{-1} \mathbf{P}_j) \quad (4)$$

BA 的目标就是最小化所有观测对的重投影误差之和：

$$\min_{\{\mathbf{T}_i\}, \{\mathbf{P}_j\}} \sum_{i,j \in \mathcal{V}} \|\mathbf{e}_{ij}\|_{\Sigma_{ij}}^2 \quad (5)$$

其中 $\mathcal{V}$ 是可见性集合——即哪些点被哪些相机观测到。这个可见性集合是稀疏的：一个路标点只出现在少量关键帧中，一个关键帧也只观测到少量路标点。

2.4.2 联合优化的力量

BA 的核心价值在于**联合优化**。想象一条包含 100 个关键帧的轨迹，如果逐帧独立估计位姿，第 1 帧的误差会逐帧传播，到第 100 帧时漂移巨大。BA 把 100 个位姿和所有三维点放进同一个优化问题，任意两帧之间的约束（包括回环）都会通过因子图传播到全局，把整条轨迹"拉"到一致的状态。

这种联合性也有计算代价。BA 的 Hessian 矩阵具有特殊的块结构：

$$\mathbf{H} = \begin{bmatrix} \mathbf{H}_{cc} & \mathbf{H}_{cp} \\ \mathbf{H}_{cp}^\top & \mathbf{H}_{pp} \end{bmatrix} \quad (6)$$

$\mathbf{H}_{cc}$ 连接所有相机位姿， $\mathbf{H}_{pp}$ 连接所有三维点， $\mathbf{H}_{cp}$ 是相机-点的交叉块。利用 **Schur 补**（Schur Complement），可以先消去三维点变量，求解一个只含相机位姿的"缩减系统"，再通过回代求出三维点更新。由于 $\mathbf{H}_{pp}$ 是块对角的（每个三维点只与观测到它的相机相连），求逆代价很低。这个技巧使 BA 的求解效率提升了数个数量级，是现代 BA 求解器（如 g2o、Ceres）的标准步骤（CVF 开放获取）。

2.4.3 局部 BA 与全局 BA

实时 SLAM 系统中，全局 BA（优化所有关键帧和所有地图点）无法在每帧运行。工程上的标准做法是分层：

局部 BA 只优化当前帧及其共视关键帧（通常 5-10 帧）观测到的地图点。计算量小，可以每帧运行，保证跟踪精度。

全局 BA 在检测到回环后触发，优化所有关键帧和地图点，消除长期漂移。通常在一个独立的后台线程中异步执行，不阻塞前端跟踪。

位姿图优化（Pose Graph Optimization）是全局 BA 的轻量替代。通过 Schur 补将所有三维点边缘化掉，只剩下关键帧之间的相对位姿约束。位姿图比完整 BA 快得多，但精度略低。ORB-SLAM 的全局优化就采用位姿图 + 可选的全 BA 两步策略。

2.4.4 关键论文精读：PTAM

Parallel Tracking and Mapping for Small AR Workspaces (Klein & Murray, ISMAR 2007)

虽然 PTAM 不是第一个使用 BA 的系统，但它是将 BA 引入实时视觉 SLAM 的里程碑工作。

痛点：2007 年前的视觉 SLAM 系统（如 MonoSLAM）使用 EKF，受限于线性化误差和计算瓶颈，地图只能维持几十个点。BA 精度高但被认为无法实时。

核心假设：跟踪（Tracking）和建图（Mapping）可以分离到两个线程中。跟踪线程对每帧做轻量级位姿估计；建图线程在后台用 BA 持续优化地图。

输入输出：输入为单目相机视频流；输出为相机位姿和稀疏三维点云。

地图表示：稀疏点云地图，地图点由 FAST 角点特征三角化得到。

Tracking：对新帧提取 FAST 角点，与已有地图点匹配，通过 EPnP 算法估计相机位姿。这是一个纯跟踪过程，不参与地图更新。

Mapping：后台线程对所有关键帧和地图点执行 BA（通过 Levenberg-Marquardt）。关键帧选择策略保证地图中只保留"信息量大"的帧。

优化目标：最小化重投影误差，联合优化关键帧位姿和地图点位置。

相比前作推进了什么：首次证明了 BA 可以在消费级硬件上（2007 年的 Core 2 Duo）实时运行，前提是分离跟踪和建图。这一架构设计（双线程 + BA）成为此后十年视觉 SLAM 的标准模板。

没有解决什么：PTAM 的地图规模仍然有限，没有回环检测机制，长期漂移无法消除。此外，初始化需要用户配合平移运动来触发三角化。

对后续研究的影响：PTAM 开创了"关键帧 + BA + 双线程"的架构范式。ORB-SLAM 直接继承了这一设计，并加入了回环检测和全局优化，将其推向实用化。

2.5 李群李代数：只讲够用部分

SLAM 中优化的核心变量是相机位姿，而位姿属于一个特殊的数学空间——**欧氏变换群 $SE(3)$** 。如果直接用一个 6 维向量（3 维平移 + 3 维旋转参数）表示位姿并在欧氏空间上做梯度下降，很快就会遇到麻烦：旋转矩阵必须是正交的（ $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$ ），用普通向量加法更新会破坏这个约束。

2.5.1 为什么需要流形

$SE(3)$ 不是一个向量空间，而是一个**流形（Manifold）**。流形上不能做无限制的加法：把一个旋转矩阵加一个向量，结果不再是旋转矩阵。

流形可以在局部用切空间（Tangent Space）来近似。切空间是一个向量空间，可以在上面自由地做线性运算。关键操作是：**在切空间中求增量**，然后通过**指数映射（Exponential Map）**将增量"投射"回流形上更新位姿。这个过程不需要维护任何约束——指数映射的输出天然合法的 $SE(3)$ 元素。

李群李代数理论为这套操作提供了严格的数学基础。在 SLAM 中，只需要掌握三个核心概念：

概念	数学对象	含义	在 SLAM 中的角色
$SE(3)$	李群	4x4 变换矩阵	位姿的存储形式
$\mathfrak{se}(3)$	李代数	6 维向量（3维旋转+3维平移）	优化的增量空间
$\exp(\cdot)$	指数映射	$\mathfrak{se}(3) \rightarrow SE(3)$	将优化增量转换为位姿更新

2.5.2 扰动模型

图优化和 BA 的求解器（Gauss-Newton、Levenberg-Marquardt）都依赖线性化。在流形上的线性化通过在李代数切空间中的扰动实现。

对位姿 $\mathbf{T} \in SE(3)$ 施加扰动 $\xi \in \mathfrak{se}(3)$:

$$\mathbf{T}(\xi) = \exp(\xi^\wedge) \cdot \mathbf{T} \quad (7)$$

其中 $\xi^\wedge$ 是李代数向量到反对称矩阵的映射。误差函数对扰动 ξ 求导，而不是对 $\mathbf{T}$ 直接求导。这避免了在矩阵元素上求导带来的约束违反问题。

优化的迭代更新变为:

$$\mathbf{T}_{k+1} = \exp(\delta\xi^\wedge) \cdot \mathbf{T}_k \quad (8)$$

其中 $\delta\xi$ 是在切空间中求解的增量。所有线性代数操作（矩阵求逆、Cholesky 分解等）都在切空间中进行，最后通过指数映射回到 $SE(3)$ 。

旋转部分 $SO(3)$ 有类似的结构。 $SO(3)$ 是 3×3 旋转矩阵的集合，其李代数 $\mathfrak{so}(3)$ 是 3 维反对称矩阵构成的向量空间，通过指数映射 $\exp: \mathfrak{so}(3) \rightarrow SO(3)$ 与旋转矩阵相连。

2.5.3 为什么要用 $SE(3)$ 而不是四元数或其他表示

旋转有多种参数化方式：欧拉角、旋转矩阵、四元数、轴角。欧拉角有万向锁奇异性，不应作为优化变量。四元数无奇异且高效，但有一个单位模约束 ($q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1$)，需要额外处理。 $SE(3)$ 的李代数表示是唯一无约束、无奇异、几何意义清晰的优化参数化。

$SE(3)$ 上的扰动模型有一个微妙之处：左扰动还是右扰动。左扰动 $\exp(\xi^\wedge) \cdot \mathbf{T}$ 将增量施加在世界坐标系；右扰动 $\mathbf{T} \cdot \exp(\xi^\wedge)$ 将增量施加在局部坐标系。两种选择都可以，只要 Jacobian 推导与之一致即可。大部分 SLAM 库（如 g2o 的 `VertexSE3Expmap`）采用左扰动。

2.5.4 代码层面的直觉

在 C++ 实现中，图优化库（g2o、GTSAM、Ceres）内部全部使用李代数表示增量。以 Ceres 为例，用户自定义的位姿参数块存储为四元数 + 平移（7维），但 Ceres 的 `LocalParameterization` 接口会自动处理切空间中的增量更新——用户只需定义误差函数，不必手动实现指数映射。g2o 的 `VertexSE3Expmap` 则直接内部维护 $\mathfrak{se}(3)$ 的增量形式。

理解李群李代数的要点不是记住指数映射的级数展开式，而是建立这个直觉：位姿优化不在位姿本身所在的空间进行，而是在其切空间中进行；切空间的增量通过指数映射回流形完成更新。这个框架保证了旋转约束始终满足，且导数计算具有清晰的几何意义。

2.6 滤波、粒子与图：三条路线的对比

维度	EKF-SLAM	FastSLAM	图优化 SLAM
时间模型	递归滤波	递归粒子滤波	平滑（全历史）
不确定性	单高斯	粒子集（多模态）	优化后的协方差
计算瓶颈	协方差更新 $O(N^3)$	粒子退化	稀疏线性求解
回环处理	无法自然支持	无法自然支持	天然支持
传感器融合	需统一协方差	每个粒子独立	因子即插即用
地图规模	< 1,000 路标	10k+ 路标（2D）	无上限
现代地位	教学/小型嵌入式	2D 激光 SLAM	绝对主流

图优化成为现代 SLAM 的核心，不是因为前两种方法在数学上"错误"，而是因为 SLAM 的工程需求发生了根本变化：

回环检测的普及改变了游戏。当回环约束成为 SLAM 系统的标配，滤波方法"只维护当前状态"的设计哲学就与全局一致性需求产生了根本冲突。图优化保留历史节点的设计，让回环只是一条额外的边。

多传感器融合的需求加速了转变。现代 SLAM 系统通常融合相机、IMU、激光、GPS、轮速计等多种传感器。图优化的因子可以独立建模每种传感器的约束，然后统一求解。EKF 需要把所有传感器塞入同一个状态向量和同一个协方差矩阵，工程耦合度极高。

计算能力的提升消除了图优化最后的障碍。2010 年前的嵌入式平台无法运行 BA 规模的稀疏线性求解；今天手机芯片可以轻松处理数千个关键帧的图优化。当计算不再是瓶颈时，精度更高的图优化自然取代了近似更强的滤波方法。

2.7 与具身智能的关系

本章讨论的是 SLAM 的数学骨架，看似与具身智能没有直接关联。但这条从滤波到图优化的演进路线，深刻影响了具身智能系统的空间推理能力。

首先，图优化的**因子即插即用**特性，是具身智能系统融合多模态感知的基础。一个具身智能体可能同时携带相机、IMU、触觉传感器、关节编码器——每种传感器的约束都可以建模为因子图中的边。GTSAM 和 Ceres 的后端优化能力，是这些异构信息实现融合统一的数学基础设施。

其次，BA 的**联合优化**思想超越了视觉 SLAM。在神经隐式表示（NeRF、3DGS）中，场景参数和相机位姿的联合优化本质上仍是 BA 的扩展形式。从 PTAM 的稀疏点到 NeRF 的辐射场，"重投影误差最小化"的核心逻辑一脉相承。

最后，李群李代数的**流形优化**框架是机器人学的基础语言。不仅 SLAM 的位姿优化需要它，机械臂的运动规划、无人机姿态控制、人形机器人全身动力学优化——所有涉及旋转和平移的问题都在 $SE(3)$ 上展开。具身智能中的"空间智能"，其数学根基就是流形上的概率推理与优化。

本章一句话总结

SLAM 的数学骨架从卡尔曼滤波的在线递推，跃迁到图优化的全局联合求解，核心驱动力是回环检测对全局一致性的需求与多传感器融合对模块化约束的需求；因子图、Bundle Adjustment 和李群李代数共同构成了现代 SLAM 的三根数学支柱。

第3章 视觉 SLAM：特征法、直接法与稠密法

第2章搭建了 SLAM 的数学骨架——状态估计、非线性最小二乘、图优化。但相机看到的不是抽象的状态变量，而是像素。本章回答一个核心问题：**从像素到相机位姿和三维地图，有哪几条技术路线？** 三条：特征法、直接法、稠密法。它们使用像素信息的方式截然不同，各自定义了视觉 SLAM 的三个时代。

特征法先提取图像中的显著点（角点），再用描述子匹配建立帧间对应关系，最终通过重投影误差优化位姿和地图点。这是传统 SLAM 最成熟的范式，工程化程度最高。直接法跳过特征提取，直接比较像素亮度值，最小化光度误差，理论上利用了图像中全部几何信息。稠密法则追求逐像素的深度估计，输出可供交互的三维表面模型。本章逐一拆解每条路线的核心思想、关键瓶颈与代表系统，最后以 ORB-SLAM3 作为传统视觉 SLAM 的工程化巅峰之作进行总结。

3.1 特征法

3.1.1 核心流程

特征法 SLAM 将问题分解为五个连续步骤：

- 1. 特征检测。** 在图像中寻找角点或 blob。ORB（Oriented FAST and Rotated BRIEF）是视觉 SLAM 的事实标准：FAST 检测角点，BRIEF 生成二进制描述子，整体计算速度极快且具备旋转不变性。ORB-SLAM 系列将 ORB 特征用于跟踪、建图、重定位和回环检测三个环节，实现"一种特征，全系统通用"(arXiv)。
- 2. 描述子匹配。** 每帧提取数百至数千个特征点及其描述子。相邻帧通过描述子距离（汉明距离）建立对应关系。匹配质量直接决定后续位姿估计的精度。
- 3. 位姿估计（PnP）。** 已知三维地图点及其在二维图像上的投影，通过 Perspective-n-Point（PnP）求解相机位姿。常用 EPnP 或 RANSAC + PnP 的组合剔除误匹配。如果地图尚未建立（初始化阶段），则通过八点法或本质矩阵分解获取初始位姿。
- 4. 三角化（Triangulation）。** 多视角观察同一特征点后，通过三角化计算其三维坐标。新三角化的点加入地图，供后续帧定位使用。
- 5. 捆绑调整（Bundle Adjustment, BA）。** 同时优化相机位姿和三维点坐标，最小化重投影误差：

$$e_{i,j} = x_{i,j} - \pi(T_{iw}, X_{w,j})$$

其中 $x_{i,j}$ 是第 j 个地图点在第 i 帧的观测像素坐标， T_{iw} 是第 i 帧的相机位姿， $X_{w,j}$ 是地图点的世界坐标， π 为投影函数。优化目标为所有观测误差的加权和：

$$\min_{T,X} \sum_{i,j} \rho_h(e_{i,j}^T \Omega_{i,j}^{-1} e_{i,j})$$

ρ_h 为 Huber 鲁棒核函数， $\Omega_{i,j}$ 为信息矩阵。BA 是特征法精度的根本保证，也是计算开销最大的环节。PTAM 率先将 BA 引入实时 SLAM，证明了精确的稀疏重建可以在多核 CPU 上实时运行。

3.1.2 回环检测

长时间运行后，位姿估计的误差会累积。回环检测通过判断相机是否回到了曾经到过的地方来消除累积漂移。词袋模型（Bag of Words, BoW）是经典方案：将图像描述为视觉单词的集合，通过 DBoW2/DBoW3 库快速计算图像相似度。检测到回环后，执行位姿图优化或全局 BA，将闭环约束传播到整个地图。

回环检测的核心挑战是“感知混叠”——视觉上相似但实际位置不同的场景会导致假阳性闭环。DBoW2 通过时间一致性检验（要求连续三帧匹配到同一区域）来过滤误检，但这牺牲了大量召回率。ORB-SLAM3 的回环检测对此进行了关键改进。

3.1.3 关键瓶颈

特征法的问题在于**信息丢弃**。一张图像数百万像素，特征法只使用其中几百到几千个角点。在弱纹理环境（白墙、走廊）中，角点稀少，系统直接失效。此外，特征提取和匹配占据大量计算时间——ORB-SLAM3 的 Tracking 线程约 30-40% 的时间花在特征计算上。特征匹配失败后的鲁棒性处理（RANSAC、外点剔除）进一步增加了系统复杂度。更重要的是，特征法天生只能构建稀疏地图，无法直接用于碰撞检测或表面交互——这对机器人应用是一个根本性限制。

3.2 直接法

3.2.1 本质区别

直接法的核心洞察：**角点不是上帝指定的。任何具有足够亮度梯度的像素都携带几何信息。**

特征法先做特征检测再做位姿估计，是两步法。直接法跳过特征检测，直接利用原始像素值，将位姿估计和对应关系求解融为一体。其优化目标是**最小化光度误差**（photometric error），而非重投影误差：

$$r_k = I_j[p'(T_i, T_j, d, c)] - I_i[p]$$

即同一三维点在不同帧中的投影像素亮度应一致——这是**亮度恒常假设**（brightness constancy assumption）。 I_i, I_j 为两帧图像， p 为参考帧像素位置， p' 为根据当前位姿估计投影到目标帧的像素位置， d 为深度， c 为相机内参。

3.2.2 直接法的分类

按使用像素的范围，直接法分为三级：

类型	使用像素	代表系统	特点
稠密	全部像素	DTAM	计算量巨大，需 GPU
半稠密	梯度显著的像素	LSD-SLAM	平衡精度与效率
稀疏	稀疏采样像素	DSO	实时性强，精度高

稠密直接法使用所有像素，最接近理想的信息利用，但实时性需要 GPU 支持。半稠密法仅使用沿边缘和梯度丰富区域的像素，在 CPU 上即可实时运行。稀疏直接法则进一步减少像素数量，将直接法的核心思想与稀疏优化的效率结合。

3.2.3 光度标定

直接法对成像过程极度敏感。自动曝光、自动增益、伽马校正、卷帘快门都会破坏亮度恒常假设。DSO 的作者提出光度标定流程：离线估计相机响应函数、曝光时间和镜头衰减（vignetting），将原始像素值转换为辐射度值（irradiance），大幅提升跟踪精度(TUM CV 讲义)。

光度标定的核心公式为像素响应模型：

$$I(x) = G(t \cdot V(x) \cdot R(x))$$

$I(x)$ 为观测像素值， G 为非线性响应函数， t 为曝光时间， $V(x)$ 为镜头衰减（vignetting）， $R(x)$ 为真实辐射度。通过多幅不同曝光图像拟合 G 和 V ，可将 $I(x)$ 反转为 $R(x)$ ，使亮度恒常假设在不同帧间成立。

3.2.4 失败模式

直接法有三类典型失败场景：

- 1. **光照突变**：亮度恒常假设被打破，光度误差失去意义。
- 2. **大基线运动**：帧间位移过大，直接图像对齐陷入局部极小值，需要良好的初始化。
- 3. **卷帘快门**：逐行曝光导致图像各行采集时刻不同，直接法难以建模。特征法对滚动快门相对鲁棒，因为特征位置决策基于局部信息，不易被行延迟污染。

特征法与直接法的对比如下表：

维度	特征法	直接法
误差类型	重投影误差	光度误差
信息利用	稀疏角点	全部或部分像素
地图密度	稀疏	半稠密至稠密
运行速度	较快	较慢（但并行性好）
外点处理	灵活（RANSAC 等）	困难（需在线加权）
初始化要求	低	高（需良好初始值）
光照鲁棒性	较好	较差
弱纹理场景	差（角点稀少）	较好（利用边缘信息）

特征法胜在鲁棒性和工程成熟度，直接法胜在信息利用率和地图密度。现代系统（如 SVO）已尝试融合两种思想：用直接法做初始位姿估计，再用特征法精化和闭环。

3.3 半稠密与稠密

3.3.1 从稀疏到稠密的谱系

视觉 SLAM 的地图表示沿一条连续谱分布：稀疏点云（几百个点）、半稠密深度图（梯度区域的深度估计）、稠密表面模型（房间级乃至城市级的完整三维表面）。

DTAM (ICCV 2011) 是稠密 SLAM 的先驱。它将场景表示为关键帧上的稠密代价体 (cost volume)，通过多视角光度一致性优化逐像素深度，并利用稠密模型渲染预测图像进行帧到模型的跟踪。DTAM 的代价函数为：

$$C_r(u, d) = \frac{1}{|I(r)|} \sum_{m \in I(r)} \|I_r(u) - I_m(\pi(KT_{mr}\pi^{-1}(u, d)))\|_1$$

即通过将参考帧 r 的像素 u （深度为 d ）投影到多个邻近视图 m 中，计算光度差异的平均值。DTAM 展示了稠密地图可显著提升跟踪鲁棒性，但需要 GPU 实时运行。

KinectFusion (ISMAR 2011) 利用 RGB-D 相机改变了稠密 SLAM 的格局。它将场景表示为截断符号距离函数 (TSDF) —— 三维空间中的每个体素存储到最近表面的距离值。新帧的深度图通过加权平均融合进 TSDF 体积，相机位姿通过 ICP（迭代最近点）与稠密模型对齐。TSDF 融合公式为：

$$F_k(p) = \frac{W_{k-1}(p)F_{k-1}(p) + W_{R_k}(p)F_{R_k}(p)}{W_{k-1}(p) + W_{R_k}(p)}$$

$F_k(p)$ 为体素 p 处的 TSDF 值， W 为权重。KinectFusion 的局限是固定体积的立方体网格，无法处理大场景；后续 Voxel Hashing、InfiniTAM 等通过哈希稀疏体素解决了可扩展性问题。

3.3.2 ElasticFusion：Surfel 地图与无位姿图的闭环

ElasticFusion (RSS 在线会议记录) 是稠密 RGB-D SLAM 的代表作, 其核心创新在于**完全放弃位姿图优化**, 转而通过非刚性表面变形实现全局一致性。

地图表示: Surfel。 ElasticFusion 不用点云、不用体素, 而是用 surfel (surface element, 面元) 表示场景。每个 surfel 是一个带位置、法向、半径、颜色和时间戳的小圆盘。Surfel 集合天然表示表面, 比点云更结构化, 比体素更节省内存——只存储有表面的地方。

帧到模型跟踪。 不同于帧到帧配准, ElasticFusion 将当前 RGB-D 帧与已构建的 surfel 模型配准, 同时最小化光度误差 (颜色一致性) 和几何误差 (ICP 点对面距离)。帧到模型跟踪比帧到帧更稳定, 因为模型融合了大量历史观测, 噪声更低。

窗口化 surfel 融合。 新帧的 surfel 被融合进全局模型。时间上久远的 surfel 被标记为"非活跃" (inactive), 近期观测的标记为"活跃" (active)。系统维护一个局部时间窗口, 优先保证局部一致性。

闭环机制。 ElasticFusion 有两层闭环:

- **局部闭环:** 检测当前帧与邻近 surfel 模型的几何一致性, 通过 deformation graph 进行小范围非刚性变形。
- **全局闭环:** 基于 Randomized Ferns 的外观数据库检测远处回环, 触发更大范围的表面变形。

Deformation Graph 是 ElasticFusion 的核心数据结构。它是一组稀疏采样的节点, 每个节点影响周围的 surfel。闭环检测后, 优化 deformation graph 的节点位移, 然后将变形传播到所有 surfel。这种方式直接在地图层面消除漂移, 无需维护相机位姿的历史图。

ElasticFusion 精读

1. **痛点:** 传统稠密 SLAM 依赖位姿图优化保证全局一致性, 但位姿图无法纠正早期融合到地图中的错误表面——融合后帧被丢弃, 错误不可恢复。
2. **核心假设:** RGB-D 相机观测的表面可以用 surfel 集合近似, 且闭环误差可通过非刚性表面变形在地图层面直接修正, 无需位姿图。
3. **输入:** RGB-D 视频流; **输出:** 全局一致的稠密 surfel 表面模型。
4. **地图表示:** Surfel 集合——带位置、法向、半径、颜色、时间戳和置信度的面元。
5. **Tracking:** 帧到模型的联合光度-几何配准 (RGB 直接法 + Depth ICP)。
6. **Mapping:** 窗口化 surfel 融合 + deformation graph 驱动的非刚性变形。
7. **优化目标:** 同时优化相机位姿和 surfel 模型, 局部和全局闭环均通过 deformation graph 传播约束, 保持"尽可能刚性" (as-rigid-as-possible) 的变形特性。
8. **推进了什么:** 首次实现了无位姿图、在线闭环的实时稠密 SLAM, 证明了地图层面变形比位姿图更适合稠密重建; 在 ICL-NUIM 数据集上将重建误差降至 0.7-2.8 cm 范围。
9. **未解决:** 房间尺度限制, 大场景下 surfel 数量爆炸; 删除历史帧导致错误融合的表面信息不可恢复; 不支持纯视觉输入 (必须 RGB-D)。
10. **后续影响:** 启发了 BundleFusion、BAD-SLAM 等后续稠密系统, surfel 表示至今仍被 3D Gaussian Splatting 等新兴方法借鉴。

3.4 ORB-SLAM 系列: 传统视觉 SLAM 的巅峰

3.4.1 从 ORB-SLAM 到 ORB-SLAM3

ORB-SLAM (T-RO 2015) 是特征法 SLAM 的里程碑。ORB-SLAM2 扩展了双目和 RGB-D 支持。ORB-SLAM3 (arXiv) 将能力边界推至视觉惯性、多地图、多会话——它是传统（非学习）视觉 SLAM 中功能最完整的开源系统，也是工程化 SLAM 的标杆。

ORB-SLAM 精读

1. **痛点**: PTAM 开创了并行跟踪与建图，但缺乏回环检测、遮挡处理、自动初始化，且尺度漂移严重。
2. **核心假设**: ORB 特征足够快、足够有区分度，可统一用于跟踪、建图和回环三个任务。
3. **输入**: 单目/双目/RGB-D 图像序列；**输出**: 相机轨迹 + 稀疏三维点云地图。
4. **地图表示**: 稀疏三维点（地图点）+ 关键帧位姿图 + 共视图（covisibility graph）。
5. **Tracking**: 每帧提取 ORB 特征，与局部地图匹配，通过 Motion-only BA 优化位姿；跟踪丢失时启用基于 DBoW2 的重定位。
6. **Mapping**: 关键帧插入、冗余剔除、Local BA 优化局部窗口内的位姿和地图点。
7. **优化目标**: 重投影误差最小化，分三个粒度——Tracking（仅位姿）、Local BA（位姿+点）、Global BA / Pose Graph（全局一致性）。
8. **推进了什么**: 首次将 ORB 特征全系统统一使用，引入共视图管理大规模地图，多线程架构成为后续 SLAM 系统的标准模板。
9. **未解决**: 纯单目模式尺度不可观；对弱纹理、快速运动敏感。
10. **后续影响**: ORB-SLAM 的开源代码成为视觉 SLAM 研究的基础设施，数千篇后续论文以其为 baseline。

3.4.2 ORB-SLAM3 的系统架构

ORB-SLAM3 的核心架构沿用多线程设计，但每个模块都大幅增强：

Tracking 线程。处理每帧图像，提取 ORB 特征，最小化重投影误差估计位姿。视觉惯性模式下，加入 IMU 预积分残差联合优化。跟踪丢失时，在 Atlas 的全部地图中尝试重定位；若失败，将当前地图标记为非活跃并创建新地图。

Local Mapping 线程。管理关键帧和地图点的插入与剔除，执行 Local Visual 或 Visual-Inertial BA。IMU 初始化在此线程完成：先进行纯视觉 BA 获得无尺度轨迹，再通过 MAP 估计恢复尺度、重力方向、IMU 偏置和速度，通常在 2 秒内将尺度误差收敛到 5% 以下，15 秒后可达约 1%(arXiv)。

Loop and Map Merging 线程。检测活跃地图与 Atlas 中其他地图的共享区域。若闭环在同一地图内，执行闭环校正；若在不同地图间，执行无缝地图合并。这是首个完整的多地图闭环系统。

3.4.3 Atlas 多地图系统

Atlas 是 ORB-SLAM3 最显著的创新。它维护一组离散地图：一个活跃地图用于实时定位，其余为非活跃地图。当跟踪丢失时，系统不崩溃——而是创建新地图继续建图。当通过高召回率 DBoW2 检测到曾经到过的地方时，地图被重新激活或合并。

传统 SLAM 跟踪丢失即意味着系统死亡。Atlas 让 SLAM 具备了“记忆”：多次进入同一栋建筑的不同时间、不同路径构建的地图可以被自动合并为一个统一模型。这对于长期运行的服务机器人、自动驾驶高精地图更新等应用具有核心价值。

3.4.4 改进的回环检测

传统 DBoW2 要求连续三帧匹配到同一区域才进行几何验证，精度高但召回率低。ORB-SLAM3 的回环检测先对候选关键帧进行几何一致性检验（通过 Sim(3) 或 SE(3) 直接对齐），再与三个共视关键帧验证局部一致性。召回率显著提高，数据关联更稠密，地图精度随之提升。代价是略高的计算开销，但 Loop 线程独立运行，不影响实时 Tracking。

3.4.5 抽象相机模型

ORB-SLAM3 将相机模型抽象化，只需提供投影、反投影和雅可比函数即可接入新模型。内置支持针孔模型和鱼眼模型，为广角、全景相机提供了原生支持。这种抽象避免了相机模型变更时重写整个 SLAM 系统的工程负担。

3.4.6 ORB-SLAM3 精读

1. **痛点**：视觉 SLAM 长期面临四个割裂——单目与多目割裂、视觉与视觉惯性割裂、单地图与多会话割裂、针孔与广角相机割裂。
2. **核心假设**：最大后验估计（MAP）可统一处理视觉和惯性信息；多地图表示可无缝支持多会话 SLAM。
3. **输入**：单目/双目/RGB-D 图像，可选 IMU 数据；支持针孔和鱼眼模型。**输出**：相机轨迹 + 稀疏/半稠密地图 + Atlas 多地图结构。
4. **地图表示**：Atlas——多个离散地图的集合，每个地图包含关键帧、地图点、共视图和生成树。
5. **Tracking**：特征匹配 + 重投影误差 + IMU 预积分残差；丢失时在 Atlas 全局重定位。
6. **Mapping**：Local BA + IMU 参数初始化与精化；关键帧和地图点冗余管理。
7. **优化目标**：视觉重投影误差与 IMU 误差的联合 MAP 估计，分局部 BA、闭环校正、全局 BA 三个层次。
8. **推进了什么**：首次将单目/双目/RGB-D/视觉惯性/多地图/多会话/多相机模型统一在一个系统中，EuRoC 数据集双目惯性模式绝对轨迹误差约 3.6 厘米，TUM-VI 手持序列平均精度达 9 毫米。
9. **未解决**：仍依赖特征点，弱纹理场景性能下降；回环检测依赖词袋模型，对视觉相似但几何不同的场景可能产生假阳性；计算开销随地图规模增长。
10. **后续影响**：ORB-SLAM3 的开源实现成为视觉 SLAM 研究和商业产品的标准基线，其多地图架构启发了后续多会话 SLAM 和终身建图研究。

3.4.7 LSD-SLAM 与 DSO 的补充位置

LSD-SLAM (ECCV 2014) 将直接法推向半稠密建图。它跟踪图像中所有梯度显著的像素（主要是边缘），构建半稠密深度地图。采用关键帧架构，与 PTAM 类似地分离跟踪和建图线程，加入回环检测和全局优化，支持单目和双目输入，在 CPU 上以约 30 Hz 实时运行。LSD-SLAM 证明了直接法可在 CPU 上实时运行并生成半稠密地图，但其将位姿和结构分开优化的策略（位姿图优化与深度估计分离）不如联合优化精确。

DSO (T-PAMI 2017) 将直接法与稀疏优化结合，开创了“直接稀疏”范式。它在滑动窗口内（通常 7 个关键帧）联合优化光度误差，同时估计相机位姿、逆深度、光度参数（曝光时间、增益）和相机内参：

$$E_{i,j,k} = \sum_{p \in N_{p_k}} \omega_p \left\| (I_j[p'] - b_j) - \frac{t_j e^{a_j}}{t_i e^{a_i}} (I_i[p] - b_i) \right\|_{\gamma}$$

i, j 为帧索引, k 为点索引, N_{p_k} 为以 p_k 为中心的 8 像素邻域 ("residual pattern"), t 为曝光时间, a, b 为光度参数, ω_p 为基于图像梯度的权重, γ 为 Huber 范数。DSO 使用 Schur 补进行边缘化, First Estimates Jacobian 避免线性化点不一致。默认使用 2000 个点、7 个关键帧的窗口, 在 TUM-Mono VO 数据集上展现出优异精度(TUM 讲义)。

DSO 精读

1. **痛点**: 直接法要么太稠密 (DTAM, 需 GPU), 要么太粗糙 (LSD-SLAM, 位姿与结构分离优化)。
2. **核心假设**: 稀疏采样的像素包含足够几何信息, 窗口内联合光度优化可同时达到精度和效率。
3. **输入**: 单目图像序列 (经光度标定)。**输出**: 相机轨迹 + 稀疏逆深度点云。
4. **地图表示**: 滑动窗口内的关键帧位姿 + 各关键帧上 host 的点 (逆深度参数化)。
5. **Tracking**: 新帧与最近关键帧的直接图像对齐 (多尺度金字塔 + 常速运动模型初始化)。
6. **Mapping**: 窗口内光度 BA, 联合优化位姿、逆深度、光度参数和内参; 老帧通过边缘化退出窗口。
7. **优化目标**: 上述光度误差 $E_{i,j,k}$ 的加权和, 带边缘化先验。边缘化策略保留最新的两帧, 剔除可见点少于 5% 的旧帧, 若窗口超容则按空间分布距离评分剔除。
8. **推进了什么**: 证明了稀疏直接法可超越特征法精度, 同时保持 CPU 实时性; 光度标定显著提升性能; 完整的光度误差 BA 框架。
9. **未解决**: 无闭环, 本质上是视觉里程计而非完整 SLAM; 对光照变化和卷帘快门敏感; 需要精确的光度标定。
10. **后续影响**: DSO 的窗口优化框架启发了大量后续视觉惯性系统 (VI-DSO)、学习增强系统 (CNN-DSO、Deep-DSO), 以及现代多传感器融合方案。

3.5 三条路线的比较与适用场景

维度	特征法（ORB-SLAM3）	半稠密直接法（LSD-SLAM）	稀疏直接法（DSO）	稠密法（ElasticFusion）
传感器	单目/双目/RGB-D/IMU	单目/双目	单目	RGB-D
地图表示	稀疏点云	半稠密深度图	稀疏点云	稠密 surfel 表面
误差类型	重投影误差	光度误差	光度误差	光度+几何误差
闭环方式	位姿图+全局 BA	位姿图	无	Deformation graph
实时性	是（30-40 FPS）	是（约 30Hz）	是	是（需 GPU）
弱纹理	差	较好	较好	好（有深度）
光照变化	较好	差	差	一般
主要用途	通用定位	半稠密建图	高精度 VO	稠密表面重建

特征法是通用场景的首选，工程成熟度最高，适合需要长期稳定运行的系统。直接法在弱纹理或需要半稠密地图的场景有优势，但对光照敏感。稠密法面向三维重建和 AR/VR 应用，输出可直接用于渲染和交互。选择哪条路线取决于传感器类型、应用场景和计算资源。一个务实的趋势是：特征法负责回环检测和全局一致性（DBoW2 + BA），直接法负责局部跟踪的精度和密度——这种混合范式正在成为现代 SLAM 系统的主流架构。

3.6 与具身智能的关系

视觉 SLAM 是具身智能的空间感知底座。机器人要抓取物体、导航到目标、避开障碍，都需要知道自己的位置和周围的三维结构。

特征法 SLAM（如 ORB-SLAM3）提供了轻量级的自定位能力，适合计算资源有限的机器人平台。其稀疏地图虽不直接可用于碰撞检测，但可与占据栅格或体素地图融合，为路径规划提供输入。ORB-SLAM3 的多地图能力对终身学习的机器人尤为重要——机器人在生命周期中会反复访问同一环境，能够存储、调用和合并历史地图是持续适应的基础。

直接法和稠密法生成的半稠密或稠密地图更接近具身智能的需求。ElasticFusion 的 surfel 地图可直接用于碰撞避免和表面交互。但稠密地图的存储和更新代价是实际部署的瓶颈——机器人运行数小时，surfel 数量可能达到数百万。如何在保持地图质量的同时控制规模，是将稠密 SLAM 部署到真实机器人的核心挑战。

更具前瞻性的视角是：本章讨论的所有传统方法都在为后续章节中的神经隐式表示奠定基础。NeRF 的可微渲染借鉴了直接法的光度误差思想，3D Gaussian Splatting 的显式表示继承了 surfel 的高效渲染特性。传统 SLAM 的几何直觉与深度学习的信息利用能力的结合，正在定义空间智能的下一个阶段。

3.7 一句话总结

特征法用角点定义了视觉 SLAM 的第一个十年，直接法证明了跳过角点也能精确估计运动，稠密法将 SLAM 的输出从点云升级为可交互的表面模型；ORB-SLAM3 站在特征法的顶峰，将多传感器、多地图、多会话融为一炉，标志着传统工程化 SLAM 的最高成就——而新的篇章，正从神经网络与这些经典系统的交汇处开启。

第4章 视觉惯性 SLAM 与多传感器融合

单目视觉 SLAM 无法恢复度量尺度，在快速运动、光照突变、纹理缺失时跟踪丢失。IMU（惯性测量单元）以 100-1000 Hz 输出角速度和加速度，恰好填补这些盲区。本章讨论视觉与 IMU 的融合方式——松耦合与紧耦合——并以 VINS-Mono 为典型，拆解 IMU 预积分、滑动窗口优化、鲁棒初始化等核心模块。最后简要扩展到多相机、多 IMU、轮速计与 GPS 的工程融合实践。

4.1 为什么 IMU 对 SLAM 重要？

相机擅长空间约束：一张图像包含大量环境几何信息，通过特征匹配或光度对齐可以建立帧间相对位姿约束。但相机有三大软肋。

运动模糊与失锁。高速运动或低纹理场景下，视觉跟踪容易中断，单帧图像又无法提供速度或加速度信息。纯视觉系统一旦丢失跟踪，只能依赖重定位或回环恢复，期间轨迹完全断裂。

尺度不可观。单目相机只有 bearing（视线方向）测量，缺少 depth 信息，尺度永远不可恢复（第3章已详述）。双目或 RGB-D 虽然可以解决，但增加了硬件体积与功耗。

计算延迟。视觉前端需要提取特征、匹配、做几何校验，典型帧率 10-30 Hz，远不能满足无人机、机械臂等需要 100 Hz 以上反馈的控制回路。

IMU 恰好在这三点上与相机互补。加速度计和陀螺仪以千赫兹级频率输出，不受光照、纹理、动态物体的影响；通过积分加速度可以恢复度量尺度；IMU 的短期预测精度高，可以填补视觉跟踪丢失的空白窗口。一句话：**相机擅长空间约束，IMU 擅长短时运动约束，二者互补。**

IMU 也有硬伤。加速度计同时测量重力和运动加速度，仅靠积分无法区分；陀螺仪和加速度计都存在时变 bias 和噪声，长时间积分后位姿漂移严重。因此 IMU 适合"短时间精准、长时间漂移"，相机适合"长时间空间一致、受环境影响大"。融合的目标是各取所长：用 IMU 填补视觉盲区，用视觉校正 IMU 漂移。

从可观性分析角度看，单目 VINS 有四个不可观方向：三维平移（3 DOF）和绕重力方向的旋转（yaw，1 DOF）。尺度、roll、pitch 在加入 IMU 后均可观——前提是系统存在足够的加速度激励（平移加速度而非纯旋转）。这意味着 VINS 不能在静止状态下完成初始化，必须依赖一段运动过程来唤醒。

4.2 松耦合与紧耦合

视觉与 IMU 的融合架构分为两大类：松耦合（loosely-coupled）与紧耦合（tightly-coupled）。这两个概念是理解所有 VINS 系统的钥匙。

4.2.1 松耦合：先视觉估计，再融合位姿

松耦合将视觉和 IMU 视为两个独立模块。视觉前端单独输出位姿估计（如 ORB-SLAM、PTAM 或纯视觉 VO），IMU 单独做积分 propagation；然后用一个外部滤波器（通常是 EKF）将两个位姿结果融合。

视觉模块输出 T_{cw} 和不确定性，IMU 模块输出积分后的位姿预测。EKF 以 IMU 积分作为状态传播，以视觉位姿作为观测量进行更新。整个架构像两条流水线在末端汇合。

松耦合的优点是模块化高、实现简单、计算量低。视觉和 IMU 可以独立开发、独立调试，一个模块失效不影响另一个运行。缺点是信息损失严重：视觉前端丢弃了特征点级别的几何信息（重投影误差、特征深度、共视关系），只输出一个六自由度位姿和协方差。如果视觉位姿本身有误差或失效，IMU 无法利用特征级别的约束来修正——融合发生在“结果层”而非“数据层”。

4.2.2 紧耦合：视觉观测与 IMU 预积分一起优化

紧耦合将相机原始特征观测和 IMU 原始测量放在同一个优化框架中联合求解。视觉约束体现为特征点的重投影误差，惯性约束体现为 IMU 预积分残差，两者构成一个统一的非线性最小二乘问题，同时优化位姿、速度、IMU bias 和特征点三维位置。

关键区别在于：紧耦合中 IMU bias、相机-IMU 外参、甚至时间偏移都可以与其他状态一起在线标定；松耦合中这些参数通常离线标定好、固定不变。

4.2.3 对比分析

维度	松耦合	紧耦合
融合层级	位姿结果层	原始测量层
视觉信息利用	仅六自由度位姿	特征重投影误差、深度、共视
IMU bias 估计	通常离线固定	在线联合优化
相机-IMU 外参	通常离线标定	可在线标定
计算量	低，适合嵌入式	高，需滑动窗口控制规模
精度	中，信息有损失	高，信息利用充分
代表性系统	MSCKF 早期版本、松耦合 GPS/INS	OKVIS、VINS-Mono、ORB-SLAM3

紧耦合在精度上显著优于松耦合，尤其在视觉退化场景（纹理少、光照差、快速运动）。代价是计算复杂度大幅上升：一个滑动窗口内可能同时优化十几个关键帧位姿、上百个特征点、速度、bias 等多个状态变量。工程上必须通过滑动窗口（sliding window）和边缘化（marginalization）来控制问题规模。优化方法的选择也是分水岭：EKF 类（MSCKF、ROVIO）通过卡尔曼更新维持线性近似，计算高效但精度略低；图优化类（OKVIS、VINS-Mono）通过非线性最小二乘迭代优化，允许重复线性化（relinearization），精度更高但计算量大。近年的趋势是图优化占据主流，因为 IMU 预积分理论解决了高频 IMU 数据与低频视觉优化之间的衔接问题。

4.3 VINS-Mono

VINS-Mono 是 2017 年由香港科技大学 Qin、Li、Shen 开源的单目视觉惯性 SLAM 系统 (arXiv)。它被认为是视觉惯性 SLAM 领域的工程标杆——不是因为某一项技术首创，而是因为它把初始化、预积分、滑窗优化、回环检测、重定位、位姿图优化等模块完整拼接为一个可靠系统，并开源了 PC 和 iOS 双版本。

4.3.1 系统架构

VINS-Mono 采用五线程并行架构：

1. **测量预处理**：提取图像 FAST 角点，用 KLT 光流跟踪；IMU 数据在相邻图像帧间做预积分。
2. **初始化**：从静止未知状态启动，联合估计尺度、重力方向、速度、陀螺仪 bias、三维特征点位置。
3. **视觉惯性里程计 (VIO)**：核心模块，基于滑动窗口的紧耦合非线性优化。
4. **回环检测与重定位**：基于 DBoW2 检测回环，紧耦合重定位将历史地图特征与当前 VIO 联合优化。
5. **位姿图优化**：4-DOF 全局优化 (x, y, z, yaw)，校正长期漂移。

五个模块以不同频率并行运行。VIO 输出高频位姿，回环和位姿图优化在后台异步校正漂移。

4.3.2 IMU 预积分

IMU 预积分是紧耦合优化的基石，由 Lupton & Sukkarieh (2012) 首创，Forster et al. (2015, 2017) 改进为 SO(3) 流形上的形式 (arXiv)。

核心问题：在优化过程中，每当窗口中某个关键帧位姿被调整，两帧之间所有 IMU 原始测量都需要重新积分，计算量不可接受。预积分的解决思路是——将参考系从世界坐标系切换到局部 IMU 坐标系 b_k ，使得积分结果仅与 IMU bias 有关，与世界坐标系中的位姿、速度无关。

给定两帧之间的 IMU 原始测量（加速度 $\hat{\mathbf{a}}_t$ 、角速度 $\hat{\boldsymbol{\omega}}_t$ ），预积分得到三个相对量：

$$\boldsymbol{\alpha}_{b_{k+1}}^{b_k} = \iint \mathbf{R}_t^{b_k} (\hat{\mathbf{a}}_t - \mathbf{b}_a - \mathbf{n}_a) dt^2 \quad (9)$$

$$\boldsymbol{\beta}_{b_{k+1}}^{b_k} = \int \mathbf{R}_t^{b_k} (\hat{\mathbf{a}}_t - \mathbf{b}_a - \mathbf{n}_a) dt \quad (10)$$

$$\boldsymbol{\gamma}_{b_{k+1}}^{b_k} = \int \frac{1}{2} \boldsymbol{\Omega} (\hat{\boldsymbol{\omega}}_t - \mathbf{b}_\omega - \mathbf{n}_\omega) \boldsymbol{\gamma}_t^{b_k} dt \quad (11)$$

α 为相对位移, β 为相对速度, γ 为相对旋转 (四元数表示)。这三个量完全由 IMU 测量值在局部坐标系下积分得到, 与世界坐标系中的位姿无关。当 bias 估计发生变化时, VINS-Mono 使用一阶线性近似进行后验修正, 避免完全重新积分。

预积分的协方差通过连续时间误差状态传播推导。在图优化中, 预积分项作为一个二元因子 (binary factor) 连接相邻两个状态节点, 残差定义为预测值与优化当前估计之间的偏差。

4.3.3 滑动窗口优化

滑动窗口优化是 VINS-Mono 的核心。VIO 维护一个有限长度的关键帧窗口 (典型 10 帧左右), 窗口内的所有状态变量被联合优化。

状态向量包含:

- 每个关键帧的位姿 $\mathbf{P}_{b_k}^w$ 、旋转 $\mathbf{q}_{b_k}^w$ 、速度 $\mathbf{v}_{b_k}^w$
- 加速度计 bias $\mathbf{b}_a$ 、陀螺仪 bias $\mathbf{b}_\omega$
- 窗口内所有特征点的三维逆深度 λ
- 可选: 相机-IMU 外参 $\mathbf{P}_c^b$ 、 $\mathbf{q}_c^b$

优化目标函数由三部分组成:

$$\min_{\mathcal{X}} \left\{ \sum_{(i,j) \in \mathcal{C}} \rho(\mathbf{r}_c(\hat{\mathbf{z}}_l^{c_j}, \mathcal{X})) + \sum_{k \in \mathcal{B}} \|\mathbf{r}_B(\hat{\mathbf{z}}_{b_{k+1}}^{b_k}, \mathcal{X})\|_{\Sigma_{b_{k+1}}^{b_k}}^2 + \sum_{(l,v) \in \mathcal{O}} \rho(\mathbf{r}_o(\hat{\mathbf{z}}_l^{c_v}, \mathcal{X})) \right\} \quad (12)$$

三项分别为: 视觉重投影残差 (带有 Huber 鲁棒核 ρ)、IMU 预积分残差、以及回环重定位残差。优化使用 Ceres Solver 求解。

滑动窗口的直觉: 为什么不把所有历史帧都放入优化? 因为关键帧数量随时间线性增长, 优化问题规模将无界膨胀, 无法在实时约束下求解。滑动窗口的策略是——只保留最近 N 个关键帧的状态在优化问题中, 将窗口外最老的一帧通过边缘化 (marginalization) 压缩为一个先验因子 (prior factor), 保留其对窗口内状态的约束信息但不保留其本身。这相当于在放弃旧状态的同时, 把旧测量"凝结"成一个对当前窗口的惩罚项。

边缘化的数学操作是舒尔补 (Schur complement): 将旧状态从信息矩阵中消去, 得到约化后的信息矩阵和向量。这一步必须小心处理线性化点 (linearization point) ——如果边缘化后线性化点发生变化, 需要正确处理以避免信息矩阵不一致 (inconsistency)。VINS-Mono 采用一阶偏导数修正来解决这个问题。

关键帧选择策略: VINS-Mono 使用两个准则。一是视差 (parallax): 当前帧与上一个关键帧之间的平均特征视差超过阈值时插入新关键帧; 同时用陀螺仪积分补偿旋转分量, 避免纯旋转导致的错误三角化。二是跟踪质量: 当跟踪到的特征点数量低于阈值时强制插入关键帧, 防止特征丢失。

4.3.4 初始化

VINS-Mono 的初始化是其工程亮点之一。系统从完全未知的状态启动, 没有先验的尺度、速度、重力方向或 IMU bias。初始化过程分为三步:

第一步: 视觉结构恢复。用纯视觉运动恢复结构 (SfM) 处理前若干帧 (滑动窗口), 得到位姿的相对比例和特征点的三维位置。注意此时尺度是任意的, 重力方向也未对齐。

第二步：惯性对齐。将视觉 SfM 的结果与 IMU 预积分对齐。求解四个量：尺度因子 s 、重力方向 $\mathbf{g}$ 、陀螺仪 bias $\mathbf{b}_\omega$ 、每帧速度 $\mathbf{v}$ 。这是一个线性最小二乘问题，利用 IMU 预积分与视觉位姿之间的约束建立方程组。重力方向 g 的模值已知 (9.8 m/s^2)，只需要估计其方向。这一步同时校正陀螺仪 bias——如果不校正，预积分的旋转会与视觉 SfM 产生系统性偏差。

第三步：非线性优化精化。将前两步的结果作为初值，进行一次完整的视觉惯性联合优化 (full BA)，精化所有状态。初始化完成后，系统切换到正常的 VIO 滑动窗口优化模式。

初始化模块同时也是失效恢复模块：当 VIO 检测到跟踪丢失（特征丢失过多、优化发散），系统自动重新进入初始化流程。

4.3.5 回环与重定位

VINS-Mono 的回环检测使用 DBoW2 词袋模型（与 ORB-SLAM 相同），在关键帧上提取 BRIEF 描述子建立视觉词汇表。检测到候选回环后，进行几何验证 (PnP + RANSAC)，剔除误匹配。

重定位采用紧耦合方式：将回环匹配到的历史地图特征点作为额外观测量，直接加入当前 VIO 的滑动窗口优化中。这与 ORB-SLAM 的松耦合重定位不同——VINS-Mono 的回环特征与当前帧特征在同一个优化问题中被联合求解，信息利用更充分。

由于 IMU 提供了 roll 和 pitch 的可观性，VINS-Mono 的漂移仅存在于 4 DOF（三维平移 + yaw）。因此全局优化采用 4-DOF 位姿图优化而非 7-DOF（ORB-SLAM 单目需要优化尺度）。这减少了优化变量，提升了效率。

4.4 代表论文精读

OKVIS：因子图 VIO 的代表

OKVIS (Open Keyframe-based Visual-Inertial SLAM) 由 ETH Zurich 的 Leutenegger 等人于 2015 年发表在 IJRR。它是图优化 VIO 的先驱之一。

痛点：视觉和惯性信息如何在高频 IMU 和低频图像之间有效融合？当时的方案多用 EKF，但 EKF 对非线性测量只做一次性线性化，误差累积。**核心假设：**关键帧策略可以将问题规模限制在可管理范围内；非线性优化允许重复线性化，精度优于 EKF。**输入：**双目或单目图像 + IMU 测量。**输出：**关键帧位姿、速度、IMU bias、稀疏三维地图点。

OKVIS 的前端使用多尺度 Harris 角点检测 + BRISK 描述子，后端在滑动窗口上做非线性优化，代价函数由重投影误差和 IMU 预积分误差加权组成。旧关键帧通过边缘化移出窗口。OKVIS 用 Google Ceres 求解器做优化。

相比 MSCKF 等滤波方法，OKVIS 的贡献在于证明了图优化 VIO 可以实时运行。它未解决的关键问题是：缺少回环检测和全局优化模块，长期漂移无校正机制；且针对单目 VIO 的优化不足，stereo 配置下表现明显更好。

ROVIO：滤波式 VIO

ROVIO (Robust Visual Inertial Odometry) 由 ETH Zurich 的 Bloesch 等人于 2015 年发表在 IROS, 后续扩展论文于 2017 年发表在 IJRR。

痛点: 如何在计算资源极度受限的平台上 (如微型无人机) 实现鲁棒 VIO? **核心假设:** 直接法 (像素光度误差) 比特征法更信息高效; EKF 的迭代扩展形式 (IEKF) 可以缩小与图优化的精度差距。**输入:** 单目图像 + IMU 测量。**输出:** 六自由度位姿、速度、IMU bias。

ROVIO 前端提取 FAST 角点, 在每个角点周围提取多分辨率图像块 (multi-level patch) 作为直接法跟踪的模板。IMU 预测用于对图像块做运动补偿和 warp。EKF 更新步骤使用图像块的光度误差作为创新项 (innovation), 而非传统的重投影误差。

ROVIO 的独特之处在于将特征点和直接法混合: 用 FAST 角点确定跟踪位置, 用图像块光度误差做精细对齐。三维特征点用机器人中心 bearing vector + 距离参数化。作为纯滤波方法, ROVIO 计算效率极高, 适合嵌入式平台。但它不构建地图、不做回环检测, 本质上是一个视觉惯性里程计 (VIO) 而非完整 SLAM 系统。

VINS-Mono: 工程完整度高的滑窗优化系统

VINS-Mono 由香港科技大学 Qin、Li、Shen 于 2018 年发表在 IEEE Transactions on Robotics (arXiv)。

痛点: 现有 VIO 系统多为里程计, 缺少回环检测和全局优化; 初始化过程脆弱; 工程部署门槛高。**核心假设:** 一个完整的单目 VINS 必须包含初始化、VIO、重定位、全局优化四个环节; 滑动窗口优化可以在精度和实时性之间取得平衡。**输入:** 单目图像 + IMU 测量。**输出:** 全局一致的六自由度位姿、稀疏三维地图、尺度恢复的轨迹。

地图表示: 稀疏三维点云, 特征点用逆深度参数化。**Tracking:** 基于滑动窗口的紧耦合非线性优化, 维护 10 帧左右的关键帧窗口, 使用 IMU 预积分连接帧间运动约束。**Mapping:** 特征点通过多帧三角化得到三维位置, 加入优化问题。**优化目标:** 视觉重投影误差 + IMU 预积分残差 + 回环重定位残差, 使用 Huber 鲁棒核, Ceres 求解。

相比前作的推进: 1) 提出鲁棒的在线初始化流程, 无需静止启动; 2) IMU 预积分带 bias 修正; 3) 紧耦合回环重定位; 4) 4-DOF 位姿图优化; 5) 完整的开源系统 (PC + iOS), 工程可复现性极高。

没有解决的问题: VINS-Mono 依赖特征点法, 在弱纹理、高动态场景下仍易丢失跟踪; 单目深度估计在远距离特征上精度差; 滑动窗口的边缘化引入线性化误差积累, 长时间运行后一致性下降; 系统不做稠密重建, 地图仅为稀疏点云。

对后续研究的影响: VINS-Mono 的开源代码成为视觉惯性 SLAM 的事实标准基准。后续 VINS-Fusion 扩展到双目、GPS、轮速计等多传感器; OpenVINS、Basalt 等系统在其思想上改进; 大量无人机和 AR 应用直接基于 VINS-Mono 开发。

ORB-SLAM3: 视觉惯性、多地图的工程化总结

ORB-SLAM3 由西班牙 Zaragoza 大学的 Campos 等人于 2021 年发表在 IEEE Transactions on Robotics (arXiv)。

痛点：之前的 SLAM 系统要么只支持单一传感器配置，要么无法在跟踪丢失后无缝恢复和重用历史地图。**核心假设：**最大后验（MAP）估计应贯穿整个系统——包括 IMU 初始化、局部 BA、回环检测和地图合并；多地图系统可以在丢失后启动新地图，重访时自动合并。**输入：**单目/双目/RGB-D 图像 + IMU（可选）。**输出：**多地图结构、全局一致的位姿、稀疏特征地图。

地图表示：Atlas 多地图结构，包含一个 Active Map（当前跟踪）和多个 Non-active Map（历史地图）。每个地图内维护共视关系图和生成树。**Tracking：**视觉惯性模式下，IMU 初始化被建模为一个 MAP 估计问题而非传统分离式初始化；局部 BA 在滑动窗口中联合优化视觉和惯性项。**Mapping：**关键帧插入、地图点生成与剔除、共视关键帧局部 BA。**回环检测：**改进的 DBoW2 词袋模型，提升了召回率；检测到的回环触发地图合并（intra-map loop）或多地图融合（inter-map loop）。

相比前作的推进：1) 首个支持单目/双目/RGB-D、纯视觉/视觉惯性/多地图的统一 SLAM 库；2) IMU 初始化完全基于 MAP 估计，比 VINS-Mono 的分离式初始化更统一；3) 多地图系统允许系统在丢失后"复活"而非重启；4) 在所有传感器配置下均开源，精度提升 2-10 倍。

没有解决的问题：ORB-SLAM3 仍是基于稀疏特征的方法，不做稠密重建；对动态物体鲁棒性有限；多地图合并依赖回环检测的召回率，在特征稀疏的大场景中召回不足。

4.5 多传感器融合：多相机、轮速、GPS

视觉惯性融合是底线，不是终点。真实机器人系统往往携带多相机、轮速计（wheel encoder）、GPS、气压计、甚至 LiDAR。本节简述多传感器融合的工程实践。

多相机。双目相机提供天然的深度测量，消除了尺度不可观问题，在纹理稀疏区域比单目更稳定。OKVIS 的 stereo 配置比 monocular 表现好得多；VINS-Fusion 扩展了双目支持。多相机（如环视相机阵列）提供 360° 视野，部分相机被遮挡时系统仍可从其他视角获取信息。工程上多相机主要增加了外参标定和计算负载的挑战。

轮速计。地面机器人的轮速计提供一维速度测量（前进方向），可作为额外速度约束加入优化。VINS on Wheels (Wu et al., 2017, ICRA) 分析了轮速计的可观性贡献：在平面运动中，轮速计可以帮助观测 yaw 方向的漂移，提升平面轨迹精度。

GPS。全球定位系统提供低频（1-10 Hz）绝对位置测量。松耦合 GPS/INS 将 GPS 位置与 VIO 位姿用 EKF 融合；紧耦合方案将 GPS 伪距（pseudorange）作为原始观测量直接加入图优化。紧耦合的优势在于：即使可见卫星少于 4 颗（无法解出完整 GPS 位置），系统仍可从单颗卫星的测量中获得约束。VINS-Fusion 提供了 GPS 紧耦合的示例实现。

通用融合框架。Qin & Shen 提出的 "A General Optimization-based Framework for Local Odometry Estimation with Multiple Sensors" (arXiv, 2019) 将 VINS-Mono 的思想推广到任意传感器组合：每个传感器提供一个残差项，所有残差在同一个滑动窗口优化中被联合最小化。统一框架的好处是传感器可以即插即用——添加新传感器只需定义其残差函数和雅可比矩阵。

传感器	信息类型	频率	优势	局限
单目相机	空间几何	10~60 Hz	轻量、信息丰富	无尺度、易失锁
IMU	运动动力学	100~1000 Hz	高频、补盲区	偏置漂移
双目相机	深度+空间	10~30 Hz	有尺度、更稳定	体积大、计算高
轮速计	平面速度	50~200 Hz	平面约束精准	仅地面、打滑失效
GPS	绝对位置	1~10 Hz	全局无漂移	室内/遮挡失效

传感器越多，系统的鲁棒性越高——当单个传感器失效时，其他传感器可以接管。但传感器组合并非简单叠加：外参标定、时间同步、异构噪声建模、故障检测与隔离，这些工程问题往往决定了系统的实际表现。

4.6 与具身智能的关系

视觉惯性 SLAM 是具身智能的"前庭系统"。具身智能体（机器人、无人机、自动驾驶车辆）需要知道自己的位姿和运动状态，才能做出与环境交互的决策。纯视觉 SLAM 在动态、弱纹理、光照变化的真实环境中不够鲁棒；加入 IMU 后的 VINS 提供了更可靠的状态估计基础。

在具身智能场景中，SLAM 的输出不仅是位姿。VIO 估计的速度和加速度信息可以直接用于运动控制；IMU bias 的在线估计反映了传感器的健康状态；特征点的三维分布隐含了场景的稀疏几何结构。这些信息都可以作为下游策略网络或规划器的输入。

当前 VINS 系统在具身智能应用中的主要瓶颈有三。一是实时性与精度的权衡：高精度图优化消耗大量 CPU 资源，留给策略网络的计算预算有限。二是动态环境：室内有人走动、室外有车辆通行的场景中，静态特征假设频繁被打破。三是长时间自主运行：现有系统数小时后仍会积累不可忽略的漂移，需要周期性的回环或外部定位校正。解决这些问题需要将 VINS 与语义理解（区分动态/静态物体）、全局定位（如视觉地图匹配）、以及学习与优化结合的方法（第22章）融合，这是视觉惯性 SLAM 向空间智能演进的方向。

一句话总结：相机提供空间几何，IMU 提供运动约束，预积分连接二者，滑动窗口控制计算，紧耦合释放精度上限——这就是视觉惯性 SLAM 的核心逻辑。

第5章 LiDAR SLAM：从 LOAM 到语义激光建图

第4章讲视觉惯性SLAM，核心传感器是相机和IMU。本章换一个传感器主角——激光雷达（LiDAR）。LiDAR SLAM 不是本书后续技术演进的主轴，但它是自动驾驶和室外机器人不可绕过的基础。理解 LiDAR SLAM 的优势、天花板和在具身智能中的真实位置，对全书的地图观至关重要。

本章聚焦三个问题：LiDAR 相比相机最根本的优势在哪里？LOAM 的双线程架构为什么成为 LiDAR SLAM 的范式起点？LiDAR-Inertial 融合如何将精度推向厘米级？最后我们会明确 LiDAR SLAM 在具身智能中的角色——它不是主角，但它是高精度几何骨架的不可替代的提供者。

5.1 LiDAR SLAM 的优势

LiDAR 直接测量三维空间中的点坐标。每个返回点携带的信息很单纯：该方向上的距离值。这种直接性带来了四个视觉传感器无法比拟的优势。

尺度天然准确。 单目视觉SLAM的尺度是观测不到的，必须从运动视差中推断，因此尺度漂移是结构性问题。LiDAR 返回的距离值以米为单位，每一帧点云的尺度都是确定的。系统不需要像单目视觉那样小心翼翼地维护尺度一致性，也不用担心尺度因子随时间漂移。

几何结构强。 图像像素携带颜色信息，但深度需要三角化或多视图几何间接求解。LiDAR 点云本身就是三维几何采样。墙面、地面、障碍物边缘在点云中直接以空间结构的形式呈现，不需要经过特征匹配、对极几何或深度估计的层层近似。这使得 LiDAR SLAM 在几何结构丰富的环境（城市街道、室内走廊、隧道）中极其稳定。

不依赖光照。 LiDAR 是主动传感器，自带激光发射器。白天黑夜、逆光阴影、光照突变——这些让视觉SLAM头痛的场景对 LiDAR 几乎没有影响。自动驾驶车辆必须在夜间可靠运行，这是 LiDAR 成为标配的核心原因之一。

测距精度高。 机械式 LiDAR（如 Velodyne HDL-64E）的测距精度可达厘米级，固态 LiDAR（如 Livox Mid-360）在百米量程上仍保持分米级精度。相比之下，RGB-D 相机的有效深度范围通常不超过10米，室外强光下深度传感器直接失效。

特性	单目相机	双目相机	RGB-D	LiDAR
尺度确定性	不可观，需初始化	可观但基线受限	精确	精确
有效深度范围	数米至百米（三角化）	数米至数十米	0.5-10m	50-200m+
光照鲁棒性	差	差	极差（室外）	优
几何直接性	间接（像素→深度）	间接（视差→深度）	直接	直接
适用场景	室内/室外通用	室外受限	室内为主	室外/大场景为主

上表的结论直白：LiDAR 在室外大场景、长距离、全天候条件下没有替代品。室内小场景 RGB-D 或纯视觉 suffice；自动驾驶和室外机器人的主传感器必须是 LiDAR。

但 LiDAR 也有硬边界。点云稀疏——机械 LiDAR 每帧约10万点，分配到200米范围内，远处物体只有几个返回点。没有颜色信息——点云不含纹理，无法做基于外观的闭环检测或语义分割（除非融合相机）。雨雾天气激光衰减严重，点云质量急剧下降。最后，LiDAR 成本高、体积大、功耗高，虽然固态 LiDAR 在2020年后价格大幅下降，但与摄像头相比仍不在一个量级。

5.2 LOAM 的关键思想

LiDAR SLAM 的奠基之作是 LOAM (LiDAR Odometry and Mapping)，由 Zhang 和 Singh 于2014年发表在 RSS (Robotics: Science and Systems) 上。LOAM 的核心贡献不是提出某种新的配准算法，而是将 **LiDAR SLAM** 解耦为两个并行线程：高频低精度的里程计 (**Odometry**) 和低频高精度的建图 (**Mapping**)，在单核 CPU 上实现了实时低漂移建图。

5.2.1 多线程架构的直觉

LiDAR 的帧率通常为10Hz。每一帧点云到来时，如果直接与全局地图做配准，计算代价极高：全局地图可能包含数百万点，最近邻搜索的复杂度随地图规模线性增长。但如果只与上一帧做配准，速度快了，漂移却会像滚雪球一样累积——因为没有全局一致性约束。

LOAM 的洞察是：用两阶段设计兼顾实时性和低漂移。

- **Odometry 线程**：当前帧与上一帧做特征配准，输出10Hz的位姿估计。计算量小，实时性好，但每一帧都叠加前一帧的误差，单独看 drift 严重。
- **Mapping 线程**：当前帧与累积的局部地图做配准，输出1Hz的精修位姿。地图由历史关键帧维护，配准对象是数千帧点云的融合结果，全局一致性远优于帧间配准。

两个线程的结果在后台融合：odometry 提供高频位姿，mapping 提供低频校正。最终系统输出的是介于两者之间的一个平滑轨迹——既保持了高帧率，又抑制了长期漂移。

5.2.2 边缘特征与平面特征

LOAM 不在原始点上做配准，而是提取两类几何特征：**边缘特征** (edge features) 和**平面特征** (planar features)。

提取方法基于局部曲率。对每条扫描线上的每个点，计算其邻域点的散布程度。曲率大的点标记为边缘特征（对应墙角、杆状物、轮廓线），曲率小的点标记为平面特征（对应墙面、地面、天花板）。

这种提取有两个好处。第一，特征数量只有原始点的1%到5%，配准速度提升两个数量级。第二，边缘和平面的几何语义明确——边缘约束位姿的五个自由度（沿边缘方向不可观），平面约束三个自由度（沿法线方向完全约束），配准时能自然处理不同的信息贡献。

5.2.3 Odometry：帧间特征匹配

Odometry 线程取当前帧的边缘特征和平面特征，与上一帧的特征集合做匹配。边缘特征匹配的目标是找到最近的两点构成的直线，最小化点到直线的距离；平面特征匹配的目标是找到最近的三点构成的平面，最小化点到平面的距离。

优化变量是当前帧相对于上一帧的六自由度位姿变换 $\mathbf{T} \in SE(3)$ 。目标函数是所有特征点到对应几何元素（线/面）的距离平方和：

$$E_{\text{odom}} = \sum_{\mathbf{p} \in \mathcal{E}} d_{\text{line}}^2(\mathbf{T} \cdot \mathbf{p}, \mathcal{L}) + \sum_{\mathbf{p} \in \mathcal{P}} d_{\text{plane}}^2(\mathbf{T} \cdot \mathbf{p}, \mathcal{F})$$

其中 $\mathcal{E}$ 和 $\mathcal{P}$ 分别是边缘和平面特征点集， $\mathcal{L}$ 是对应的直线， $\mathcal{F}$ 是对应的平面。这个优化用 Levenberg-Marquardt 求解，迭代3-10次收敛，耗时约10-20ms。

5.2.4 Mapping：帧到地图配准

Mapping 线程维护一个局部地图，由过去数秒内的关键帧特征累积而成。当新一帧到来时，不是与单帧做匹配，而是与这个局部地图做配准。

地图中的特征密度远高于单帧。边缘在地图中表现为更长的线段，平面表现为更大的面片。配准目标函数与 odometry 类似，但对应关系搜索的范围更大、匹配质量更高。由于 map 线程只需 1-2Hz 运行，它有更多时间做精细优化。

两个线程的结果通过一个简单但有效的方式融合：odometry 给出高频位姿增量 $\mathbf{T}_{odom}$ ，mapping 给出低频位姿校正 $\mathbf{T}_{map}$ 。最终输出位姿是两者的插值——odometry 保证平滑性，mapping 保证全局一致性。

5.2.5 LOAM 论文精读

论文试图解决什么痛点？ 2014年之前，LiDAR SLAM 要么精度高但不能实时（离线 ICP 配准全局地图），要么能实时但漂移严重（帧间 ICP 里程计）。LOAM 要解决的核心矛盾是**实时性与低漂移之间的张力**。

核心假设是什么？ 环境包含足够的几何结构（边缘和平面）以支持特征提取和配准；传感器运动不过于剧烈，使得帧间配准的初始位姿猜测在收敛域内。

输入输出是什么？ 输入是机械旋转 LiDAR 的点云序列（如 Velodyne HDL-64E，10Hz）；输出是六自由度位姿轨迹和全局一致的三维点云地图。

地图表示是什么？ 以边缘线特征集合和平面面片集合的形式维护地图，不是原始点云存储，而是按体素栅格组织特征。

Tracking 怎么做？ 双线程：odometry 线程做帧间特征匹配（10Hz），mapping 线程做帧到地图配准（1-2Hz）。

Mapping 怎么做？ 累积历史关键帧的特征到局部地图，新帧的特征与地图特征做最近邻匹配，优化位姿使特征点到对应线/面的距离最小。

优化目标是什么？ 最小化边缘特征点到对应直线的距离 + 平面特征点到对应平面的距离，Levenberg-Marquardt 迭代求解。

相比前作真正推进了什么？ 首次在单核 CPU 上实现了实时低漂移 LiDAR SLAM。双线程解耦架构成为后续所有 LiDAR SLAM 系统的范式基础。

没有解决什么？ 没有闭环检测——纯 odometry + mapping 结构，大闭环误差无法消除。没有处理退化场景——长走廊、空旷区域的几何结构不足以约束六自由度位姿。没有语义理解——地图是纯几何的，不包含任何语义信息。运动畸变补偿粗糙——假设一帧扫描期间传感器匀速运动。

对后续研究的影响是什么？ LOAM 的双线程架构直接催生了 LeGO-LOAM、LIO-SAM、FAST-LIO 等后续系统。“边缘+平面”的特征提取范式至今仍是 LiDAR SLAM 的主流设计。LOAM 的开源代码成为自动驾驶和机器人领域的事实标准基线。

5.3 ICP、NDT 与点云配准

LiDAR SLAM 的核心计算单元是点云配准（Point Cloud Registration）：给定两个点云集合 $\mathcal{P}$ （源点云）和 $\mathcal{Q}$ （目标点云），寻找刚体变换 $\mathbf{T} \in SE(3)$ 使得 $\mathcal{P}$ 与 $\mathcal{Q}$ 最佳对齐。本节只讲目标函数的直觉，不涉及求解细节。

5.3.1 ICP：迭代最近点

ICP（Iterative Closest Point）由 Besl 和 McKay 于1992年提出，是最经典的点云配准算法。其核心思想朴素而有效：给定初始位姿猜测，对源点云中每一点，在目标点云中找到最近邻点，建立对应关系；然后优化位姿使得所有对应点对的距离最小；用新位姿更新对应关系，重复迭代直到收敛。

ICP 的目标函数有多种形式。**点到点 ICP**（Point-to-Point）最小化对应点之间的欧氏距离：

$$E_{\text{point}} = \sum_{(\mathbf{p}, \mathbf{q}) \in \mathcal{C}} \|\mathbf{R}\mathbf{p} + \mathbf{t} - \mathbf{q}\|^2$$

其中 $\mathcal{C}$ 是对应点集合， $\mathbf{T} = (\mathbf{R}, \mathbf{t})$ 是旋转和平移。

点到平面 ICP（Point-to-Plane）是 LiDAR SLAM 中更常用的变体。它不最小化到对应点的距离，而是最小化到对应点切平面的距离：

$$E_{\text{plane}} = \sum_{(\mathbf{p}, \mathbf{q}) \in \mathcal{C}} (\mathbf{n}_q^\top (\mathbf{R}\mathbf{p} + \mathbf{t} - \mathbf{q}))^2$$

其中 $\mathbf{n}_q$ 是 $\mathbf{q}$ 处的局部法向量。点到平面 ICP 的优势在于：它允许源点沿着目标表面的切线方向"滑动"，而不是 rigidly 锁到某个对应点上。在 LiDAR SLAM 中，同一面墙的两帧扫描点不可能完全重合（激光从不同角度采样），点到平面 ICP 对这种采样差异更鲁棒。

ICP 的致命弱点是**对初猜的依赖性**。如果初始位姿偏离真实值太远，最近邻对应会大量错误，优化收敛到错误解。这是所有基于对应关系的方法的结构性限制。

5.3.2 NDT：正态分布变换

NDT（Normal Distributions Transform）由 Biber 和 Straßer 于2003年提出，走了一条完全不同的路线：不把目标点云当作点的集合，而是把它编码为局部概率分布。

具体做法：将目标点云所在的三维空间划分为规则体素网格（cells）。对每个包含点的 cell，用该 cell 内所有点拟合一个正态分布 $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ ，其中 $\boldsymbol{\mu}$ 是均值（代表该 cell 的表面中心位置）， $\boldsymbol{\Sigma}$ 是协方差矩阵（代表表面的方向和平整度）。

配准的目标变成了：**找到位姿 $\mathbf{T}$ ，使得源点云中的每个点在变换后落在目标分布上的概率最大。**

$$E_{\text{NDT}} = - \sum_{\mathbf{p} \in \mathcal{P}} \log \phi(\mathbf{T} \cdot \mathbf{p}; \boldsymbol{\mu}_{c(\mathbf{p})}, \boldsymbol{\Sigma}_{c(\mathbf{p})})$$

其中 $\phi(\cdot)$ 是正态分布概率密度函数， $c(\mathbf{p})$ 是点 $\mathbf{p}$ 变换后落入的 cell。最大化概率等价于最小化负对数似然。

NDT 相比 ICP 有两个优势。第一，**没有显式对应关系搜索**——每个点直接落入某个 cell 的正态分布，不需要找最近邻。这对大规模地图尤其重要，因为 kd-tree 最近邻搜索的复杂度随地图规模增长。第二，**对初猜更宽容**——概率分布提供了梯度信息，即使对应关系不完美，分布的平滑性仍能引导优化向正确方向收敛。

特性	ICP（点到点）	ICP（点到平面）	NDT
对应方式	最近邻点对点	最近邻点对面	空间 cell 概率分布
目标函数	对应点欧氏距离	点到切平面距离	负对数似然
初猜敏感度	高	中高	中等
地图规模可扩展性	差（kd-tree 搜索慢）	差	优（cell 查找 $O(1)$ ）
适用场景	精修对齐	LiDAR SLAM 主流	大规模地图、粗糙对齐

LOAM 的 odometry 线程本质上是点到线 + 点到平面的特征级 ICP，而不是在原始点上做全量 ICP。这是它能达到实时性的关键。FAST-LIO2 则采用了增量式 kd-tree 来加速最近邻搜索，实现了原始点级的直接配准，精度更高。

5.4 LiDAR-Inertial SLAM

纯 LiDAR SLAM 有一个结构性缺陷：帧率受限。机械式 LiDAR 通常 10Hz，固态 LiDAR 最高也不过 20-50Hz。高频运动（车辆颠簸、无人机快速机动）在两次扫描之间的位姿变化不可忽略，产生运动畸变（motion distortion）。IMU 的频率通常是 200-1000Hz，刚好填补这个空白。

LiDAR-Inertial SLAM 的核心思路是：**IMU 提供高频位姿积分和运动畸变补偿，LiDAR 提供低频几何校正。**两者融合后，系统兼具高带宽和低漂移。

5.4.1 LIO-SAM

LIO-SAM (Shan et al., IROS 2020) 在 LOAM 的基础上引入 IMU，并采用因子图优化框架替代 LOAM 的独立双线程结构。

系统结构：前端提取 LOAM 风格的边缘和平面特征，IMU 预积分提供高频位姿预测和运动去畸变。后端维护一个滑窗因子图，包含三种因子：IMU 预积分因子、LiDAR 里程计因子和闭环因子。因子图用 iSAM2 增量求解。

LIO-SAM 的关键创新是把 **LiDAR odometry 当作一个因子插入全局因子图**，而不是像 LOAM 那样做两个独立线程的松耦合。IMU 因子约束相邻帧之间的运动学一致性，LiDAR 因子约束点云配准的几何一致性，闭环因子约束全局一致性。所有因子联合优化，消除了 LOAM 中双线程结果不一致的问题。

LIO-SAM 的另一个实用贡献是引入了 **GPS 因子** 作为可选项。当 GPS 信号可用时，其绝对位置测量可以作为额外的全局锚点插入因子图，极大抑制长期漂移。

5.4.2 FAST-LIO 与 FAST-LIO2

FAST-LIO (Xu and Zhang, RA-L 2021) 走了一条更激进的技术路线：**迭代卡尔曼滤波 + 直接点云配准。**

与 LIO-SAM 的因子图优化不同，FAST-LIO 采用紧密耦合的迭代扩展卡尔曼滤波（**Iterated EKF**）作为后端。IMU 积分作为状态预测步骤，LiDAR 点云配准作为观测更新步骤。EKF 的状态向量包含位姿、速度、IMU 零偏和重力方向。

FAST-LIO 的核心洞察是：**把 LiDAR 点云配准中的每个有效点当作一个独立的观测，塞入 EKF 的更新方程。**传统 EKF 只处理一次观测更新；FAST-LIO 的"迭代"在于它对同一批点云观测做多次线性化-更新循环，类似于高斯-牛顿迭代，收敛更快、精度更高。

FAST-LIO2 (Xu et al., T-RO 2022) 在此基础上做了两个关键升级：

第一，**直接法配准**。LOAM 和 LIO-SAM 都在提取的边缘/平面特征上做配准，丢弃了绝大部分原始点。FAST-LIO2 直接在原始点上做配准——每个返回点都参与位姿估计。这要求更快的最近邻搜索，于是引入了**增量式 kd-tree (ikd-Tree)**，支持点级的插入、删除和最近邻查询，将地图维护的复杂度从 $O(N)$ 降到接近 $O(\log N)$ 。

第二，**去除特征提取**。FAST-LIO2 完全跳过 LOAM 风格的特征提取步骤，所有原始点直接参与优化。这简化了系统架构，同时提高了精度——因为不再丢失几何信息。

系统	融合框架	配准方式	地图表示	特征提取	典型精度
LOAM	双线程松耦合	特征级点到线/面	特征集合	边缘+平面	0.5–1% 漂移
LIO-SAM	因子图紧耦合	特征级点到线/面	点云+因子图	边缘+平面	0.3–0.5% 漂移
FAST-LIO	迭代EKF紧耦合	特征级直接法	点云	边缘+平面	0.2–0.4% 漂移
FAST-LIO2	迭代EKF紧耦合	原始点直接法	ikd-Tree	无（全点）	0.1–0.2% 漂移

上表的关键信息：从 LOAM 到 FAST-LIO2，LiDAR SLAM 的精度提升了一个数量级。核心驱动力不是配准算法的革命性变化，而是**融合框架的紧耦合程度不断加深**——从双线程松耦合到因子图联合优化，再到迭代 EKF 的一步预测-更新。FAST-LIO2 通过原始点直接配准 + 增量 kd-tree，把 LiDAR-Inertial SLAM 的精度推向了厘米级。

5.4.3 退化问题

LiDAR-Inertial SLAM 有一个共同的软肋：**几何退化**（Degeneracy）。当环境中没有足够的几何结构来约束六自由度位姿时，配准优化会在某些方向上欠约束。

典型退化场景：长直走廊——墙面平行，沿走廊方向的平移不可观；开阔平地——地面平坦，垂直方向的位移和俯仰/横滚角的约束弱；隧道——圆柱对称结构，沿隧道轴线的旋转和平移约束不足。

退化不是理论上的边角案例，是工程中的高频问题。解决方案通常有三类：检测退化方向并在优化中降低其权重（DegenLO）；利用 IMU 的可观性补偿 LiDAR 的退化方向（FAST-LIO 的 IMU 先验天然有这个作用）；融合视觉信息提供纹理约束（LVI-SAM 等 LiDAR-Visual-Inertial 系统）。

5.5 LiDAR SLAM 在具身智能中的位置

LiDAR SLAM 不是具身智能中地图技术的主角，但它有三个不可替代的角色。

高精度几何骨架。 视觉方法（无论是传统特征法还是神经辐射场）在几何精度上始终受限于像素分辨率和三角化误差。LiDAR SLAM 提供厘米级的几何真值，是室外场景中所有语义理解、路径规划和物理交互的底层坐标框架。自动驾驶车辆上的语义地图、高精地图（HD Map）的底图，本质上都是 LiDAR SLAM 输出的几何骨架上叠加语义层。

室外长期地图。 室内具身智能可以用视觉+神经隐式表示（如 NeRF、3DGS）做稠密场景理解。但室外大场景（城市街区、园区、野外）的几何范围动辄数平方公里，神经表示的存储和计算代价不可接受。LiDAR SLAM 输出的稀疏点云或体素地图，配合 kd-tree 或哈希索引，是唯一能在室外大规模部署的地图形态。

语义/语言地图的空间基座。 第19–22章会讨论开放词汇语义地图和具身导航。这些系统的输入通常是视觉-语言模型（VLM）输出的语义标签或自然语言描述。但语义标签需要锚定在三维空间坐标上才有导航价值。LiDAR SLAM 提供这个锚定框架——它的位姿精度和几何一致性确保语义标签不会漂移到错误的位置。VLMaps、ConceptGraphs 等系统都依赖 LiDAR（或 RGB-D）的点云作为语义锚定的空间基座。

角色	功能	不可替代性	代表系统
几何骨架	厘米级坐标框架	视觉方法精度不足	LOAM、FAST-LIO2
室外长期地图	平方公里级地图维护	神经表示存储代价太高	LIO-SAM、LeGO-LOAM
语义空间基座	语义标签的三维锚定	纯视觉语义地图漂移大	VLMaps、ConceptGraphs

LiDAR SLAM 的天花板也很清晰。它不能提供颜色或纹理信息——语义理解需要视觉。它在室内小场景不经济——一个扫地机器人不可能装 Velodyne。它的地图是纯几何的——没有场景理解能力。这些限制正是后续章节（第6章 Metric-Semantic SLAM、第7章深度学习进入 SLAM）要突破的方向。

LiDAR SLAM 的价值不在前沿，而在基础。它是室外具身智能的"地基建图器"——不 flashy，但所有上层应用都依赖它的输出作为空间参考系。

本章一句话总结： LOAM 的双线程架构定义了 LiDAR SLAM 的范式，LiDAR-Inertial 融合将精度推向厘米级，而 LiDAR SLAM 在具身智能中的终极角色不是主角，是所有上层语义理解和物理交互赖以锚定的高精度几何骨架。

第6章 Metric-Semantic SLAM：从几何地图到语义地图

机器人需要知道"墙在哪里"才能不撞到墙。但如果要执行任务，它还需要知道：墙边的那个物体是桌子还是椅子？门能不能让机器人通过？杯子在桌子上还是柜子里？这个区域有没有人？这些问题超出了纯几何SLAM的能力边界。Metric-Semantic SLAM试图在重建三维几何的同时，给每个几何元素贴上语义标签，让地图从"可导航"进化为"可理解"。

本章回顾这一领域的演进，精读代表性系统Kimera，并分析传统语义SLAM的根本瓶颈。这一瓶颈不是某篇论文的失误，而是整个传统范式的结构性限制——它恰恰解释了为什么从第7章开始，深度学习不再只是SLAM的外围工具，而是必须进入核心。

6.1 为什么仅有几何不够？

纯几何SLAM的回答方式是："前方2.3米处有一团点云，法向量朝右，可以作为导航支撑面。"机器人听到的却是："一团圆乎乎的东西。"它不知道那是沙发——可以坐上去；也不知道那是玻璃门——不能撞上去。几何信息足以让机器人不摔倒，但不足以让它完成任何有意义的任务。

举几个具体场景。

导航与通过性判断。走廊尽头有一扇门。几何SLAM告诉机器人："opening，宽0.9米，高2.1米。"机器人本体宽0.85米——数值上过得去。但门是开着的还是关着的？把手在哪一侧？这是推还是拉？没有语义，机器人只能对着门框发呆。知道"这是门"还不够，还需要知道"门的状态"——这些属性信息完全不在几何表示中。

操作与抓取规划。机械臂需要抓取桌上的杯子。几何地图里，杯子是一簇点云，桌面是另一簇点云。但哪一簇是"杯子"、哪一簇是"书本"？哪个可以被夹持、哪个应该避开？语义标签直接决定抓取策略。更进一步，即使识别出了"杯子"，抓取姿态的选择也需要语义知识——从杯口上方伸入还是从把手侧面夹持？纯几何无从回答。

人机交互。人对机器人说："去厨房把桌上的红色杯子拿来。"这句话里，"厨房"（地点）、"桌"（家具）、"红色杯子"（物体+属性）全部是语义概念。纯几何地图根本无法解析这条指令。即使机器人通过语音解析知道了要找"杯子"，如果地图里没有语义标签，它也无法将语言概念"杯子"与三维空间中的任何几何结构关联起来。

动态环境推理。几何SLAM把移动的人也当作一堆点云，纳入地图优化。结果：地图被拖变形，定位漂移。如果系统知道"这是人"，就可以将其排除在静态地图之外，保证定位稳定性。区分"动态但可忽视的人"和"动态且需要跟踪的推车"，需要语义理解。

任务规划与常识推理。语义地图支持超出几何的任务规划。机器人想知道"哪里可以坐"——它需要在地图中查询"座椅"类别。想知道"哪里可能有人"——需要知道"沙发""办公桌""餐厅"这些地点概念与"人"的共现关系。想知道"这个区域适合抓取吗"——需要综合判断空间可达性、物体可操作性、障碍物分布。这些查询的输入是语义概念，输出是行动决策，几何地图在其中只能扮演辅助角色。

这些场景指向同一个结论：**几何是机器人感知的基础层，语义才是与任务、语言、推理对接的接口层。**Metric-Semantic SLAM的目标就是同时输出几何和语义——不只是给点云上色的视觉装饰，而是让地图成为连接感知和行动的桥梁。

6.2 早期语义SLAM：四条路线

2010年代中后期，随着深度学习在图像分类和分割上的突破，SLAM社区开始系统性地探索将语义信息注入三维重建。早期工作沿着四条路线展开，每条路线解决一个子问题，但也都留下了未解的缺口。

6.2.1 语义分割 + 几何融合

最直接的思路：用CNN给RGB图像做语义分割，再把分割结果投射到三维地图上。McCormac等人提出的**SemanticFusion** (ICRA 2017) 是这条路线的代表。它背靠ElasticFusion的surfel重建，将2D分割标签通过反投影传播到3D surfel上，并用贝叶斯概率更新在多帧之间累积语义置信度。最终每个surfel携带一个语义标签分布，而不是硬标签。(ICRA)

SemanticFusion的创新在于概率融合：第 t 帧观测到的语义标签 l_t 通过贝叶斯更新融合到已有的标签分布上：

$$P(L = l | l_{1:t}) \propto P(l_t | L = l) \cdot P(L = l | l_{1:t-1})$$

其中 L 是surfel的真实标签， $P(l_t | L = l)$ 来自2D分割网络的类别概率输出。多帧累积后，语义估计比单帧更稳定。

这条路线的问题在于语义分割和几何重建是两个脱节的管道。分割网络不知道三维结构，重建模块不理解语义边界。当分割出错时——比如把"沙发"错分成"椅子"，或把镜面反射区域错分成"窗户"——三维地图会一本正经地记录错误标签。更严重的是，系统没有能力检测和纠正这类错误。

6.2.2 对象级地图

另一条思路跳过逐像素分割，直接在三维空间中检测、跟踪和重建物体。Salas-Moreno等人的**SLAM++** (CVPR 2013) 是开创性工作。它假设场景由已知类别的物体组成（椅子、桌子、显示器等），用预训练的3D对象模型库在RGB-D帧中检测物体实例，以物体位姿作为地图的"路标"，纳入位姿图优化。(CVPR开放获取)

SLAM++的优势在于压缩：一张桌子不再需要百万个点，而是6自由度位姿 + 一个模型引用。地图规模与场景复杂度解耦。但依赖预定义3D CAD模型库极大地限制了适用场景——你不可能为世界上每一个物体都建一个模型。

后续工作试图放松这一约束。**Fusion++** (McCormac et al., 3DV 2018) 不再依赖CAD模型，而是为每个检测到的物体维护一个独立的TSDF子体积，在线重建物体几何。**MaskFusion** (Runz et al., ISMAR 2018) 引入实例分割（Mask R-CNN），能够同时跟踪和重建多个运动物体，处理动态场景。**Mid-Fusion** (Xu et al., ICRA 2019) 用octree管理多实例对象重建，提升了可扩展性。(3DV / ISMAR / ICRA)

对象级路线的根本问题是数据关联：当物体被遮挡后重新出现时，如何确认它是同一个实例？传统方法依赖几何相似性和空间连续性，在复杂场景中经常出错。另外，对象级地图忽略了没有明确边界的"stuff"类——墙、地板、天花板——而这些恰恰占据了室内场景的大部分表面积。

6.2.3 语义辅助定位

第三条路线不关注地图长什么样，而是用语义信息提升定位精度。**VSO (Visual Semantic Odometry)** (Lianos et al., ECCV 2018) 将语义分割得到的物体轮廓作为高层特征，辅助几何特征匹配。语义标签的一致性可以作为回环检测的额外约束，减少感知混叠（perceptual aliasing）——两个几何上完全相同的走廊无法区分，但如果语义标注不同（"走廊A有灭火器，走廊B没有"），回环检测就更可靠。(ECCV)

VSO的核心洞察是：语义信息不仅丰富地图，还能约束位姿估计。当纹理贫乏导致几何特征匹配失败时，语义轮廓可以提供额外的对应关系。然而VSO只解决定位问题，不输出语义地图，无法支持后续的任务规划和交互。

6.2.4 语义点云与语义网格

第四条路线聚焦于稠密三维表示的语义标注。在TSDF或occupancy grid上逐体素赋予语义标签，然后用marching cubes提取带语义颜色的三维网格。**PanopticFusion** (Narita et al., IROS 2019) 是这条路线的集大成者，它同时处理"stuff"（不可数背景类：墙、地板）和"things"（可数前景物体：杯子、椅子），实现全景分割（panoptic segmentation）级别的三维语义重建。(IROS)

PanopticFusion使用空间哈希（spatial hashing）组织TSDF体素，支持大规模场景。它还构建了一个全连接条件随机场（CRF）对体素语义进行正则化，保证标注的空间一致性。这套pipeline在ScanNet v2上达到了与离线3D CNN方法接近的精度，但完全在线运行。

系统	年份	传感器	地图表示	语义层级	核心贡献
SLAM++	2013	RGB-D	对象位姿	对象级	以3D CAD模型为路标的对象级SLAM
SemanticFusion	2017	RGB-D	surfel云	像素级	2D语义向3D surfel的贝叶斯概率传播
VSO	2018	单目	点云	像素级	用语义轮廓辅助位姿估计
MaskFusion	2018	RGB-D	surfel云	实例级	多运动物体的实时识别、跟踪与重建
Fusion++	2018	RGB-D	TSDF子体积	对象级	无CAD模型的体素对象重建
PanopticFusion	2019	RGB-D	TSDF体素	全景级	stuff+things的统一体积分割

上表梳理了2013–2019年间代表性系统的技术路线。三个趋势值得注意。第一，传感器从RGB-D为主逐步扩展，但RGB-D仍是绝对主流——语义SLAM对稠密深度的依赖始终存在。第二，语义层级从像素级向实例级、对象级演进，但全景级（同时处理stuff和things）直到2019年才出现。第三，地图表示从稀疏点云向稠密surfel、TSDF体素发展，越来越重量级。这些系统奠定了metric-semantic SLAM的技术基础，但缺乏一个统一的模块化框架将它们整合在一起。Kimera正是为填补这个空白而生。

6.3 Kimera：模块化Metric-Semantic SLAM

Kimera由MIT的Rosinol等人于2020年提出 (ICRA 2020)，是早期metric-semantic SLAM中少有的开源、模块化、CPU实时运行的完整系统。它将VIO、回环检测与位姿图优化、轻量mesh重建和稠密语义重建整合进统一架构。(ICRA / arXiv)

6.3.1 论文精读

试图解决什么痛点？ 2019年之前，metric-semantic SLAM的组件分散在不同代码库中：VIO系统

（OKVIS、VINS-Mono、ROVIO）只做状态估计；几何重建系统（ElasticFusion、Voxblox）不做语义；语义系统（SemanticFusion、PanopticFusion）依赖RGB-D且不做全局回环优化。研究者要做一个完整的语义SLAM demo，得把三四个不兼容的代码库缝在一起，调试代价极高。Kimera的目标是把VIO、SLAM、mesh重建和语义标注装进一个统一框架，在CPU上实时跑通。

核心假设是什么？ (1) 机器人配备双目相机和IMU；(2) 2D语义分割标签可以从外部网络（如DeepLab、Mask R-CNN）获取；(3) 场景以静态为主，动态物体不纳入语义重建；(4) 语义类别是预定义的封闭集。

输入输出是什么？ 输入：双目图像流 + IMU数据 + 2D语义分割标签（每张像素一个类别ID）。输出四项：(a) IMU率（200Hz）的实时位姿估计；(b) 全局一致的关键帧轨迹（通过回环优化）；(c) 轻量级局部3D mesh（用于避障）；(d) 全局语义标注3D mesh。

地图表示是什么？ Kimera使用多种地图表示，各司其职。局部mesh用三角网格（Delaunay三角化 + 反投影），顶点3D位置来自VIO后端的路标点估计，提供低延迟碰撞检测。全局几何重建用TSDF体素网格（Voxblox风格的bundled raycasting更新），支持大规模场景。语义信息以体素级别的标签概率分布存储，贝叶斯融合多帧观测后用marching cubes提取带语义颜色的三角网格。

Tracking怎么做？ Kimera-VIO模块负责tracking，分前后端。前端：Shi-Tomasi角点检测 + Lukas-Kanade光流跟踪 + 双目匹配，5点RANSAC做单目几何校验，3点RANSAC做双目校验。前端只输出特征对应关系。后端：基于GTSAM/iSAM2的固定滞后平滑器（典型窗口5-10秒），融合IMU预积分测量和“无结构”视觉因子——3D路标点通过DLT（直接线性变换）三角化后直接边缘化（marginalize），不进入状态向量，保持状态维度可控。

无结构视觉因子的核心优势是避免了维护大量3D路标点。传统VIO需要同时优化相机位姿和数百上千个3D点，状态维度随路标数量线性增长。Kimera-VIO在每个关键帧用DLT快速三角化特征点，计算重投影残差后直接将3D点从因子图中边缘化，只留下位姿、速度、IMU偏置在状态向量中。

Mapping怎么做？ Kimera将mapping拆成三个并行子模块：

Kimera-Mesher（快车道）：单帧mesh通过Delaunay三角化成功跟踪的2D特征点，再反投影到3D空间生成。多帧mesh融合固定滞后窗口内的单帧mesh：新增窗口中出现的顶点和三角面，更新已有顶点的3D位置（根据VIO后端最新估计），删除窗口外的旧顶点。整个流程约10ms，专门用于机器人附近的快速避障。如果mesh中检测到平面，还会向VIO后端注入正则化因子，形成VIO与meshing的紧耦合。

Kimera-RPGO（后台）：基于DBoW2的词袋模型检测回环候选，几何校验（单目/双目RANSAC）筛选空间一致的候选。但几何校验后的回环仍可能包含外点（perceptual aliasing：两个完全相同的房间在不同楼层）。Kimera使用增量一致性测量集最大化（PCM）剔除外点：维护一个一致性矩阵记录回环之间的空间兼容性，用最大团算法找出最大一致回环集合，只将这些回环加入位姿图优化。PCM让回环检测对DBoW2的阈值参数不敏感。

Kimera-Semantics（慢车道）：以Kimera-RPGO优化后的全局一致位姿为输入，用密集立体匹配生成深度图，结合2D语义标签，通过bundled raycasting更新TSDF体素和语义标签概率。关键设计：只在TSDF截断距离（truncation distance）内传播语义标签，减少计算量；每个体素维护一个类别概率向量，贝叶斯更新多帧观测；最终用marching cubes提取语义网格。

优化目标是什么？ Kimera-VIO的后端优化最大后验（MAP）估计，因子图包含IMU预积分因子和无结构视觉重投影因子：

$$\mathbf{X}^* = \arg \min_{\mathbf{X}} \sum_i \|\mathbf{r}_{\text{IMU}}^i\|_{\Sigma_{\text{IMU}}}^2 + \sum_j \|\mathbf{r}_{\text{vis}}^j\|_{\Sigma_{\text{vis}}}^2$$

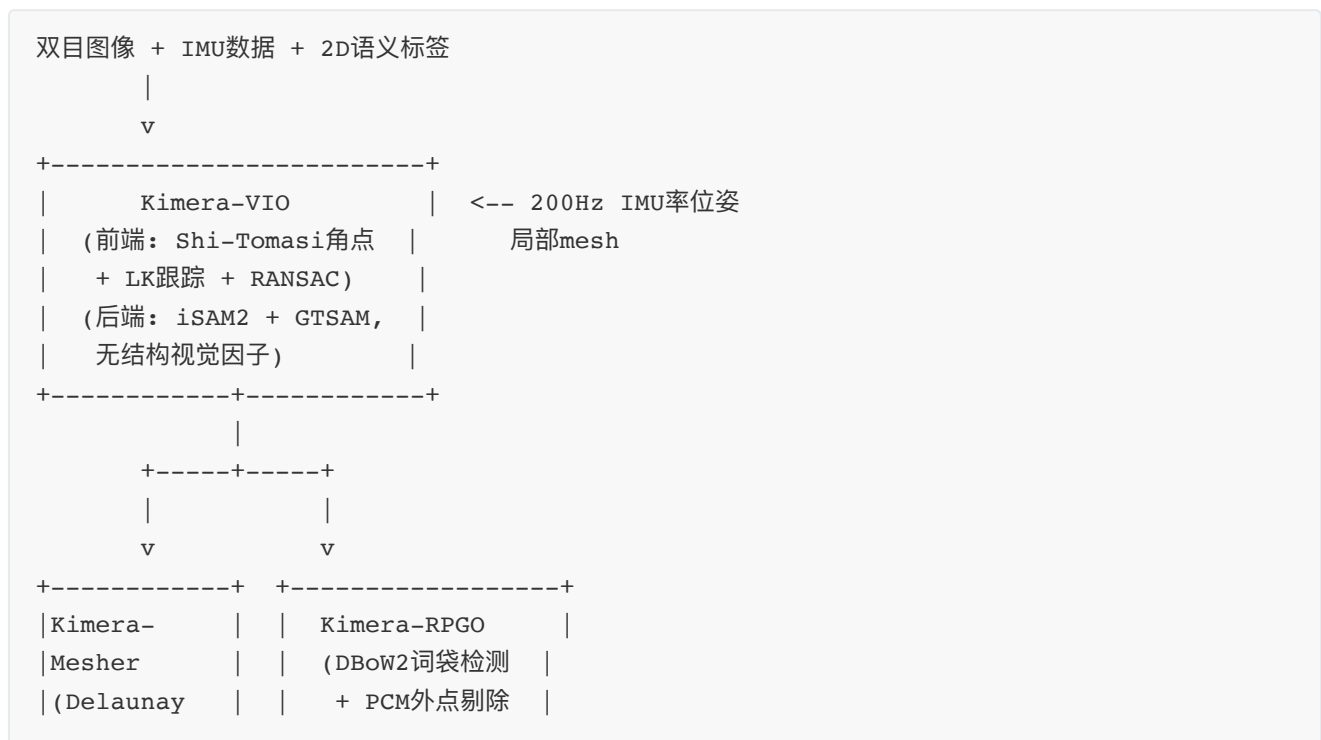
其中 $\mathbf{X}$ 包含位姿、速度、IMU偏置， $\mathbf{r}_{\text{IMU}}$ 是预积分残差， $\mathbf{r}_{\text{vis}}$ 是重投影残差。Kimera-RPGO在PCM筛选后的回环集合上运行Gauss-Newton位姿图优化。Kimera-Semantics不参与位姿优化，只做纯几何重建与语义融合。

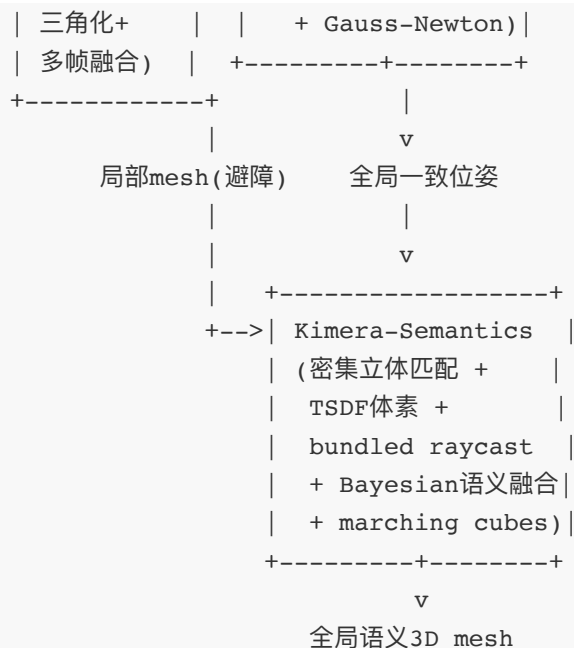
相比前作真正推进了什么？ 四个层面的推进。传感器层面：大多数语义SLAM依赖RGB-D（需要主动深度投射），Kimera只用双目+IMU即可工作，适用范围更广（室外阳光下RGB-D失效，双目不受影响）。架构层面：四个模块可独立运行或组合，研究者可以只拿VIO做实验，也可以跑完整语义SLAM——这种模块化设计在当时独一无二。实时性层面：全部在CPU上实时运行，不依赖GPU，嵌入式平台友好。鲁棒性层面：PCM外点剔除让回环检测对参数不敏感；VIO前端的多层校验（退化点过滤+外点剔除）提升了跟踪稳定性。

没有解决什么？ Kimera的语义来自外部2D分割网络，而这些网络在封闭类别上训练（如ADE20K的150类或Cityscapes的19类）。系统无法识别训练集中没有的物体，也无法用语言描述去查询地图。此外，Kimera-Semantics的TSDF重建只在静态场景下有效，动态物体被视为噪声而非独立跟踪目标。最重要的是，语义与几何之间的耦合很浅：语义只是“贴”在几何上的一层装饰，不参与位姿优化，也不影响重建策略。这些限制不是Kimera独有的，而是整个传统语义SLAM的共性瓶颈。

对后续研究的影响是什么？ Kimera的开源释放（<https://github.com/MIT-SPARK/Kimera>）降低了metric-semantic SLAM的研究门槛，成为后续工作的基准平台。更重要的是，它证明了模块化架构的可行性——不同研究者可以替换其中某一个模块（比如把VIO换成学习式的，把语义分割换成开放词汇的），而不必重写整个系统。Kimera的后续扩展包括Kimera-Multi（分布式多机器人metric-semantic SLAM）、Hydra（实时3D动态场景图构建）和Khronos（动态时空metric-semantic SLAM），形成了一个活跃的研究生态。

6.3.2 系统架构图





这个架构的核心思想是**分层与分流**。VIO和Mesher走快车道（10-20ms），保证低延迟避障和状态估计；Semantics走慢车道（100ms），保证全局重建质量；RPGO在后台维护全局一致性。不同任务按需取用不同层级的输出——避障用局部mesh，导航用优化后位姿，人机交互用全局语义mesh。

6.3.3 实验性能与定位

Kimera在EuRoC MAV数据集上进行了全面评估。位姿估计方面，Kimera-VIO的固定滞后平滑在V1_01_easy上ATE（绝对平移误差）为0.11m，加入Kimera-RPGO回环后降至0.08m，与VINS-Mono+回环（0.06m）和OKVIS（0.16m）处于同一水平。RPGO中的PCM模块效果显著：当DBoW2阈值 α 从0.001放宽到10（产生更多回环候选，外点比例更高）时，不带PCM的PGO误差从0.05m暴涨到1.59m，而Kimera-RPGO保持在0.05m左右——鲁棒性提升超过30倍。

几何重建方面，Kimera-Semantics的全局mesh在V1_01上完整度RMSE为0.364m，比Kimera-Mesher的多帧mesh（0.482m）精确约24%，但计算耗时约为后者的100倍。这精确验证了“快慢分流”的设计动机：避障用快速粗糙mesh，可视化与交互用慢速精确mesh。

时间开销方面，Kimera-VIO约20ms/帧（50Hz），Mesher约10ms，Semantics约100ms。全部满足实时性。

6.3.4 优点、失败模式与适用场景

Kimera的强项是模块化和工程成熟度。四个模块解耦，可独立测试和替换；全部C++实现，配有CI测试、Jupyter可视化工具和benchmarking脚本。这让它成为学术研究和小型机器人demo的理想起点。

失败模式有三类。第一，语义依赖外部2D分割网络，分割出错（反光、遮挡、弱光）会导致3D语义标注系统性错误，且系统无法自知。第二，TSDF语义重建假设场景静态，动态物体会留下语义拖影。第三，语义标签是封闭集，无法处理训练集外的类别——这是本章最后一节要深入分析的系统性瓶颈。

适用场景：室内静态环境、已知类别集合、CPU-only平台、需要快速部署metric-semantic SLAM原型的研究和应用。

6.4 传统语义SLAM的瓶颈

传统语义SLAM——从SLAM++到Kimera——共享同一个结构性缺陷：**封闭类别、静态标签、离线训练**。这三个约束把语义地图牢牢锁死在预定义类别清单上。

封闭类别意味着系统只能识别训练数据中出现过的类别。ADE20K有150类，NYUv2有40类，ScanNet有20类，Cityscapes有19类。现实世界中，这个杯子可能是"mug"，那个碗可能是"ceramic bowl"，但传统系统只能用训练集中的标签去硬套。更关键的是，用户无法用自然语言去查询地图。"帮我找一个能放水杯的表面"——这句话里没有出现在任何封闭类别清单中。"表面"不是一个训练类别，"能放"更不是。

静态标签意味着语义一旦赋予就不再改变。但场景是动态的：椅子被移动了、桌上新放了一本书、房间被重新装修。传统系统没有机制去检测语义变化并更新地图。PanopticFusion尝试用CRF正则化，但只是在空间维度上平滑，不具备时间维度上的更新能力。

离线训练意味着语义分割网络在部署前就已经"固化"。它不能在新环境中学习新概念，不能被终端用户自定义类别，也不能随着使用经验改进。你部署一个机器人在博物馆里，但分割网络从来没见过"雕塑"这个类别——整个系统的语义层对这个概念视而不见。

瓶颈维度	具体表现	影响
封闭类别	仅支持预定义类别清单	无法识别新物体，无法响应任意语言查询
静态标签	语义标注一旦赋予不更新	动态场景中地图与真实世界脱节
离线训练	分割网络部署前固化	无法适应新环境，无法在线学习新概念
浅层耦合	语义仅"贴"在几何上	语义不参与位姿优化和重建策略
人工标注依赖	训练数据需逐像素标注	标注成本高，类别扩展困难

这五个约束相互关联、共同作用，构成了传统语义SLAM无法突破的天花板。封闭类别和离线训练是数据层面的限制：系统只能"认识"训练时见过的东西。静态标签是时间层面的限制：系统无法适应变化的世界。浅层耦合是架构层面的限制：语义没有真正融入SLAM的优化核心。人工标注依赖则是经济层面的限制：每增加一个新类别，都需要expensive的像素级标注数据。

这些约束共同指向一个根本性问题：**传统语义SLAM建立的是"工程师的地图"——精心设计、预先定义、仅供机器使用；但具身智能需要的是"用户的地图"——开放词汇、动态更新、能用语言查询、与任务紧耦合。**这个差距不是某个系统的缺陷，而是整个传统范式的结构性局限。

与具身智能的关系

具身智能的核心命题是：机器人通过在环境中行动来学习和推理。行动需要目标，目标来自任务，任务通常由人类用语言下达。“把桌上的书放回书架”——这个指令需要地图同时支持：(a) 语言理解（“书”“书架”“桌上”是语义概念）；(b) 空间推理（找到书的位置，规划到书架的路径）；(c) 物理交互判断（书的重量、书架的可达性）。传统语义SLAM只能覆盖(b)的一部分——它知道“桌子”在哪里，但不知道“书”和“书架”的关系，更无法理解“放回”这个动作的含义。

传统语义SLAM的瓶颈恰恰是具身智能需要突破的关键。开放词汇（让系统理解任意语言描述）、动态更新（让地图随环境变化）、任务驱动（让语义与行动规划耦合）——这三项需求构成了从第7章开始学习增强SLAM的核心动机。当深度学习不再只是给SLAM提供2D标签的外围工具，而是改造SLAM的核心表示和优化机制时，metric-semantic SLAM才真正走向“可理解”和“可行动”的地图。传统SLAM的“封闭类别”瓶颈，正是开放词汇地图要解决的问题。

第二篇：学习增强 SLAM 与神经地图

第7章 深度学习进入 SLAM：不是替代，而是补强

第6章揭示了传统SLAM的结构性瓶颈：handcrafted 特征在弱纹理、光照剧变、大视角差异下频繁失效，几何假设在动态环境中被破坏，优化器对错误关联缺乏识别能力。这些问题的共同点是——它们发生在SLAM流程的特定环节，而非整个系统。

深度学习恰好具备“精准替换单个模块”的能力。一个卷积网络可以只做特征提取，一个注意力网络可以只做匹配，一个深度估计网络可以只补尺度。这种“即插即用”的特性，让深度学习不必推翻SLAM的几何框架，而是逐个加固最脆弱的环节。

本章回答三个问题：深度学习可以替代或增强SLAM的哪些环节？代表性的学习模块（特征、匹配、深度）如何工作？以及——最重要但最容易被忽视的——学习方法引入了哪些新的风险？

7.1 学习什么：SLAM pipeline 的八个可学习节点

传统SLAM系统可以拆解为一条相对固定的流水线。深度学习侵入这条流水线的每一个节点，形成了“学习增强型SLAM”（learning-augmented SLAM）的完整版图。

SLAM 环节	传统方法	学习方法代表	学习目标
特征点检测	FAST、Harris、DoG	SuperPoint (CVPR Workshop 2018)	提升重复性和可匹配性
特征描述	SIFT、ORB、BRIEF	SuperPoint描述子、D2-Net	增强光照/视角不变性
特征匹配	最近邻+比率测试、RANSAC	SuperGlue (CVPR 2020)、LightGlue	降低误匹配率，处理遮挡
光流估计	Lucas-Kanade、恒速模型	PWC-Net、RAFT	为直接法提供鲁棒对应场
深度估计	双目三角化、结构光	MiDaS、DPT、ZoeDepth	单目恢复绝对尺度
语义分割	无	SegNet、Mask R-CNN	滤除动态物体，提供语义地图
动态物体检测	几何一致性检验	Mask R-CNN、YOLO + 掩膜	减少外点污染
回环检测	DBoW2/3词袋模型	NetVLAD、DINOV2全局描述子	克服感知混叠，跨季节识别

上表勾勒出的全景是：深度学习并非要端到端地取代整个SLAM系统，而是在每个环节提供更高质量的信息。特征点检测从手工算子变为数据驱动响应函数；描述子从固定编码变为自适应编码；匹配从贪婪最近邻变为全局最优分配；深度估计从双目约束变为单目推理；回环检测从局部特征统计变为全局嵌入向量。

这种模块化替换的优势在于可组合性。SuperPoint可以接入ORB-SLAM的后端，NetVLAD可以与PTAM的位姿图耦合，单目深度网络可以为任何视觉里程计提供尺度先验。每个学习模块都是独立的"能力补丁"，系统设计师可以根据场景需求选择性装配。

7.2 学习型特征与匹配：SuperPoint、SuperGlue、LightGlue

SLAM前端最脆弱的环节是数据关联（data association）：在两张图像中找到对应于同一个3D点的2D位置。传统方法遵循"检测→描述→匹配"三段式流水线，每个环节都有独立的手工设计。深度学习对这整个流水线进行了重新设计。

SuperPoint：检测与描述的统一

SuperPoint (CVPR Workshop 2018) 的核心洞察是：特征检测和描述可以共享同一个编码器，在一个前向传播中联合完成。

核心假设：兴趣点检测虽然是语义上"不良定义"的（不像人脸关键点有明确的语义对应），但可以通过自监督的方式从合成数据学到真实图像上的泛化能力。

训练分为两步。第一步，在一个名为 Synthetic Shapes 的合成数据集上预训练一个基础检测器 MagicPoint。该数据集包含简单几何图形（立方体、三角形、椭圆等），角点位置无歧义，因此可以精确标注。MagicPoint 学到的是"角点响应"的基本模式。

第二步，将 MagicPoint 泛化到真实图像。SuperPoint 提出 **Homographic Adaptation**：对同一张真实图像施加多个随机单应变换（旋转、缩放、平移、透视变换），用 MagicPoint 分别在原图和变换后的图像上检测角点，再将检测结果通过逆单应变回原图坐标系叠加。多次变换的检测结果叠加后，形成一张"热力图"作为伪真值（pseudo-ground-truth）。这种方法不需要人工标注，就能在 MS-COCO 等大规模真实图像数据集上生成训练信号。

最终的网络是全卷积架构：共享编码器提取特征，然后分两个头——一个输出兴趣点概率图（detector head），一个输出描述子张量（descriptor head）。一次前向传播，同时得到稀疏特征点和密集描述子。在 Titan X GPU 上，480×640 图像的处理速度为 70 FPS。

SuperPoint 对 SLAM 的直接贡献是**可重复性**（repeatability）：在视角变化、光照变化下，同一个3D点被检测到的概率显著高于 FAST 和 Harris。这意味着更长的特征追踪寿命和更稳定的位姿估计。

SuperGlue：匹配作为最优传输

SuperPoint 解决了"检测+描述"的问题，但匹配仍然是暴力的最近邻搜索。SuperGlue (CVPR 2020) 的革命性在于：**把匹配本身也变成学习问题**。

核心洞察：特征匹配不是独立的逐点决策，而是一个全局分配问题（assignment problem）。一张图中的每个特征点应该与另一张图中的哪个特征点匹配，需要同时考虑所有特征点的空间关系和视觉相似性。

SuperGlue 将两幅图像的特征点集合视为图的两个节点集。每个节点携带视觉描述子和2D位置信息。网络通过图神经网络（GNN）在节点间传递信息：自注意力（self-attention）在同一张图的节点间聚合空间上下文，交叉注意力（cross-attention）在两张图的节点间比较视觉相似性。经过多轮消息传递后，网络预测一个代价矩阵，然后通过可微分的 Sinkhorn 算法求解最优传输问题，输出部分分配矩阵——即匹配结果。

这种设计的优雅之处在于它自然处理遮挡和非重复特征：分配矩阵允许"不匹配"（dustbin），对无法找到对应点的特征直接标记为未匹配，避免了暴力最近邻的强制匹配错误。

SuperGlue 在 SLAM 中的效果可以用数字说话：在 MegaDepth 数据集的两视角位姿估计任务上，相比 SuperPoint + 最近邻匹配，SuperGlue 将内点数（inliers）提升了 2-3 倍，同时显著降低了误匹配率。更多的内点意味着更稳定的位姿估计，更少的跟踪丢失。

LightGlue：效率与自适应

SuperGlue 的代价是高计算开销。GNN 的消息传递和 Sinkhorn 迭代在大规模特征集上运行缓慢。

LightGlue 的出发点是：**匹配不需要对所有特征对一视同仁，大多数特征对可以在早期被排除**。

LightGlue 采用自适应深度和自适应宽度的 Transformer 架构。在匹配的早期层，网络判断哪些特征对"明显不可能匹配"，将其从后续计算中剪枝。同时，网络动态决定需要多少层 Transformer：简单图像对可以提前终止，困难图像对则增加计算深度。这种"先易后难"的策略使 LightGlue 在保持 SuperGlue 级别精度的同时，运行速度快了 2-3 倍。

在 SL-SLAM 等系统中，SuperPoint + LightGlue 已完全替代 ORB 特征和暴力匹配，在 KITTI、EuRoC 等标准数据集上实现了更低的绝对轨迹误差（ATE）。

三者关系与系统价值

SuperPoint、SuperGlue、LightGlue 构成了一个完整的学习型前端流水线：检测→描述→匹配。它们对 SLAM 的价值不是让某个指标好看，而是解决了传统方法在三种场景下的系统性失效：

弱纹理场景：白色墙面、光滑桌面、天空——传统角点检测器几乎无响应，SuperPoint 仍能输出有意义的特征点。

光照剧变：从室外进入室内、从白天到夜晚——SIFT/ORB 描述子的梯度统计在曝光变化下剧烈变化，学习描述子通过大规模数据训练获得了光照不变性。

大视角变化：回环检测时，机器人从相反方向接近已访问区域，视角差异可达 180 度——SuperGlue 的全局注意力机制可以建立传统方法无法发现的远距离对应关系。

这三种场景恰好覆盖了传统SLAM最容易丢失跟踪、漏检回环的工况。

7.3 深度估计与单目尺度：从不可观到可推理

单目视觉SLAM的根本性缺陷是**尺度不可观测性**（scale unobservability）。从单张图像的2D投影反推3D结构时，深度存在一个全局的尺度歧义——所有深度乘以同一个缩放因子都产生相同的图像。这导致单目SLAM的轨迹和地图只能恢复到相似变换（similarity transform），而非刚体变换（rigid transform）。尺度漂移随之产生：初始化时假设的尺度在后续帧中逐渐偏离真实值。

传统解法依赖额外的传感器（IMU、双目、RGB-D）或先验假设（相机高度固定、平面地面）。但这些方案增加硬件成本或限制应用场景。

单目深度网络的基本原理

深度学习提供了一条不同的路径：训练一个网络从单张 RGB 图像直接预测绝对深度图。这一思路的成立基于一个关键观察——图像中包含了丰富的深度线索：纹理梯度、大气透视、物体相对大小、遮挡关系、阴影等。人类可以从单眼视觉感知深度，神经网络同样可以学习这些统计规律。

代表性工作包括：

- **MiDaS** (arXiv 2019): 在多个异构数据集上混合训练，学习一个对域偏移鲁棒的相对深度估计器。MiDaS 的创新在于用尺度不变损失函数（scale-invariant loss）处理不同数据集的深度尺度不一致问题。
- **DPT** (ICCV 2021): 使用 Vision Transformer 作为编码器，通过多头注意力聚合多尺度特征，在 NYUv2 和 KITTI 上达到了当时最优的精度。
- **ZoeDepth** (CVPR 2023): 首次实现了"相对深度→绝对深度"的跨数据集迁移，通过引入度量分箱（metric bins）模块，将相对深度预测转换为绝对米制深度。

这些网络通常采用编码器-解码器架构。编码器（ResNet、Swin Transformer、DINOv2 等）提取多尺度图像特征，解码器通过上采样和跳跃连接恢复空间分辨率，输出与输入图像同分辨率的深度图。训练监督信号可以是激光雷达点云投影的稀疏深度（KITTI）、RGB-D 传感器的稠密深度（NYUv2），或双目立体匹配的几何一致性（自监督）。

深度网络如何补SLAM的尺度短板

将单目深度网络接入SLAM有多种方式，按集成深度递增：

深度初始化（最浅层集成）：在新特征点被三角化时，用网络预测的深度替代三角化结果作为初始值。这加速了新地图点的收敛，但不改变SLAM的几何优化流程。CNN-SVO (RSS 2018) 是最早的代表：用深度预测初始化 SVO 中特征点的深度均值和方差，显著减少了深度滤波器的收敛时间。

尺度对齐（中层集成）：每帧运行深度网络，将预测的深度图与SLAM的稀疏地图点进行尺度对齐，然后传播尺度因子。Yin 等人的方法使用期望最大化（EM）算法从匹配点估计单目 VO 的尺度因子。这种方法不改变SLAM的内部结构，只在输出层做尺度校正。

深度作为先验（深层集成）：将深度预测作为先验项纳入 BA 的优化目标。D3VO (ECCV 2020) 将深度网络的输出与几何估计的深度通过不确定性加权进行融合，深度、位姿、不确定性三层信息在统一的优化框架中联合估计。

端到端深度VO（最深层集成）：一些系统直接学习从图像序列到相机位姿的映射，完全绕开传统几何优化。DeepVO (ICRA 2017) 和 ESP-VO (ECCV 2017) 用卷积网络提取视觉特征，用 LSTM 建模时序依赖，直接回归 6-DoF 位姿。

需要强调的是，浅层集成往往比深层集成更可靠。深度网络提供的是**先验信息**（prior），而非**几何真理**。将其作为优化器的初始化或约束，比直接替代几何推理更容易引入系统性偏差。

精度与代价的权衡

单目深度网络的精度已相当可观。ZoeDepth 在 KITTI 上的绝对相对误差（AbsRel）约为 0.05，意味着预测深度与真实深度的偏差约为 5%。这一精度足以满足自动驾驶场景下的尺度恢复需求。

但代价不可忽视。MiDaS 在桌面 GPU 上的运行速度约为 10-20 FPS，DPT 更慢。这对实时SLAM系统构成挑战。解决方向包括模型蒸馏（将大网络压缩为小网络）、TensorRT 加速、以及仅在关键帧上运行深度网络（普通帧仍用几何三角化）。

另一个限制是深度网络的预测是"单次"的——不考虑时序一致性。同一序列中相邻帧的深度预测可能在细节上不一致，需要SLAM后端的时序约束来平滑。

7.4 学习方法的风险：四个必须正视的问题

深度学习进入SLAM带来了显著的性能提升，但也引入了传统方法不曾面临的新风险。这些风险不是理论上的，而是在实际部署中反复出现的系统性问题。设计学习增强型SLAM系统时，以下四个风险必须被正视。

7.4.1 泛化风险：训练域之外的失效

传统手工特征（SIFT、ORB）基于数学定义，其行为是可预测的——ORB 在所有图像上都按照同样的规则工作。学习特征则不同：SuperPoint 的行为取决于它在训练时见过什么。如果训练数据以室内场景为主，它在室外广阔场景上的特征分布可能偏移；如果训练以白天图像为主，它在夜晚图像上的检测响应可能衰减。

泛化风险在几个维度上表现：

场景类型域偏移：SuperPoint 和 SuperGlue 在 MegaDepth（室外建筑）和 ScanNet（室内场景）上训练。当部署到水下、森林、雪地、沙漠等训练数据中几乎不存在的场景时，特征检测的重复性和匹配的准确率都会显著下降。一篇对 LightGlue 泛化能力的系统研究 (arXiv 2026) 发现，当特征检测器从 SuperPoint 切换到 DeDoDe 或 SiLK 时，现成（off-the-shelf）的 LightGlue 模型性能下降 4%-14%，揭示了匹配器与检测器之间的隐式耦合关系。

相机域偏移：训练数据通常来自智能手机或标准相机（FOV ~50-70°）。当使用鱼眼相机、广角镜头或事件相机时，图像的畸变模式和统计分布完全不同，网络的特征提取行为不可预测。

外观域偏移：季节变化（夏天→冬天）、天气变化（晴天→雨天→雾天）、光照变化（白天→夜晚）都会导致域偏移。虽然 SuperPoint 的光照鲁棒性优于 SIFT，但当外观变化超过训练分布的覆盖范围时，性能下降不可避免。

应对策略包括域自适应（domain adaptation）——在目标场景的小规模数据上微调网络，以及多域训练——在多样化的数据集上联合训练以增加泛化覆盖。但这些方法都有代价：微调需要目标域的标注或计算资源，多域训练增加训练复杂度和潜在的精度折中。

7.4.2 不确定性表达不足：网络过度自信

传统SLAM的后端优化器（BA、位姿图优化）需要知道前端提供的信息有多可靠。ORB-SLAM 通过特征点的金字塔层级和跟踪寿命来估计不确定性。光度误差法通过图像梯度强度来加权残差。这些不确定性度量虽然粗糙，但存在。

深度学习方法在这方面存在系统性缺陷。大多数网络输出点估计（point estimate）——一个特征位置、一个描述子向量、一个深度值——但不附带可靠的不确定性度量。少数方法尝试输出不确定性（如 D3VO 的不确定性估计头），但这些"不确定性"通常是网络的自评估，缺乏校准（calibration）。一个网络可能在深度预测上犯了 50% 的相对误差，但仍输出很高的"置信度"。

这对SLAM后端是危险的。优化器假设所有观测按照其不确定性被正确加权。如果深度网络提供一个错误但高"置信度"的深度先验，优化器会将其当作强约束纳入，拉偏整个地图的尺度估计。

理想的学习模块应该输出**经过校准的不确定性**（calibrated uncertainty）——不确定性数值与实际误差之间存在统计一致性。贝叶斯神经网络、测试时数据增强（test-time augmentation）、集成方法（ensemble）是提升不确定性质量的几个方向，但都会增加计算开销。

7.4.3 训练数据偏置：数据集即世界观

深度网络的"世界观"完全由训练数据决定。这带来三个层面的偏置问题。

地理偏置：MegaDepth 的数据主要来自互联网上的旅游照片——欧洲建筑、美国地标、亚洲城市中心。这意味着网络在这些场景类型上表现最好，在农村、发展中地区、非旅游区域表现较差。如果一个SLAM系统用于全球范围的服务机器人，这种地理偏置会导致性能的不均匀分布。

场景结构偏置：室内训练数据（ScanNet、ARKitScenes）以人造环境为主——平直的墙壁、规整的家具、水平的地面。野外场景（森林、山地、洞穴）的结构模式完全不同。深度网络在室内学的"地面是平的"先验在野外不成立。

动态性偏置：大多数SLAM训练数据假设静态场景。当训练数据中动态物体（行人、车辆、宠物）的比例远低于真实世界时，网络缺乏正确处理运动物体的能力。语义分割网络可能在训练时见过的物体类别上表现良好，但遇到训练集中不存在的物体类型（如新型交通工具、穿着奇装异服的行人）时，动态掩膜可能失效。

这种偏置不是可以通过"增加数据量"简单解决的。它揭示了学习系统的根本性限制：网络永远只在训练数据的分布内表现可靠，而真实世界的分布是开放和不断演化的。

7.4.4 闭环系统中的错误放大：优化器是放大器

这是四个风险中最危险、最容易被忽视的。

SLAM的后端优化器是一个**错误放大器**。前端的一个小错误——比如一个错误的特征匹配、一个有偏的深度先验——在后端可能通过 BA 的链式求导被传播和放大。一个错误匹配导致一个错误的地图点，这个地图点参与多帧的位姿估计，将这些位姿拉偏，偏斜的位姿又影响更多地图点的三角化，形成恶性循环。

传统方法中，这种错误放大通过 RANSAC、鲁棒核函数（Huber、Cauchy）、外点筛选等机制被部分抑制。这些方法基于几何一致性检验——如果一个观测与多数观测不一致，它很可能是外点。

学习方法的输出不是几何观测，而是"黑盒预测"。当 SuperGlue 给出一个错误匹配时，没有简单的几何检验可以判断它是错的——匹配分数可能很高，网络"看起来很有信心"。当深度网络给出一个错误深度时，它与几何三角化结果的差异可能被解释为"场景非刚性"或"运动模糊"，而不是网络预测错误。

更隐蔽的问题是**系统性偏差**（systematic bias）。随机外点（如RANSAC处理的孤立错误匹配）对优化的影响有限，因为它们是零均值的。但学习方法的偏差往往是**结构性的**——深度网络倾向于低估远处物体的深度，特征匹配器在面对重复纹理时倾向于产生一致的虚假匹配。这种结构性偏差会逃避鲁棒核函数的检测，因为它们在统计上表现为"一致"的观测，而非孤立的异常值。

一个具体的场景：机器人在一条长走廊中行驶。深度网络因训练数据中缺乏"长走廊"的分布，系统性地高估了墙壁距离。这个有偏的深度先验被BA当作可靠观测使用，导致地图被"撑大"，尺度估计偏离真实值。由于偏差是系统性的（所有帧都有类似的高估），鲁棒核函数无法识别它。最终的轨迹误差可能远超纯几何方法。

风险应对的系统级思维

以上四个风险不是独立的，而是相互加强的。训练数据偏置导致泛化能力不足，不确定性估计不准使后端无法正确加权，而优化器的放大效应将这些前端的缺陷转化为系统级的精度崩溃。

应对这些风险需要系统级的设计思维，而非单纯改进单个网络：

冗余设计：不要完全依赖学习模块的输出。保留传统几何方法作为"校验器"——如果 SuperGlue 的匹配与 Lucas-Kanade 光流的结果差异过大，触发降级策略（降低该帧权重或增加不确定性）。

外生约束：用SLAM系统的其他信息源来约束学习模块。IMU 的重力方向可以检验深度网络的地面法线估计是否合理；闭环检测可以暴露深度尺度的系统性漂移。

在线适应：在部署过程中持续微调网络。continual learning 和在线域自适应是活跃的研究方向，核心挑战是在学习新域的同时不遗忘旧域的能力。

混合架构：将学习模块的输出视为"软约束"而非"硬观测"。在 BA 的目标函数中，学习深度先验应该用较低的权重（或较高的不确定性）参与，让几何观测占主导地位。

7.5 与具身智能的关系

具身智能（embodied AI）要求智能体在真实环境中移动、感知、交互。SLAM是这一能力的基础，而学习增强型SLAM为具身智能提供了传统方法无法提供的语义层次和空间理解。

学习模块为具身智能带来的独特价值包括：

语义-几何融合：传统SLAM输出几何地图（点云、网格），学习模块可以输出语义标签（这个点是地面、墙壁、还是家具）。语义信息使机器人能够理解空间的"功能"——哪里可以行走，哪里是障碍物，哪里是交互目标。

动态环境适应：真实世界中的环境不是静态的。人走动、门开关、家具被移动。学习模块（动态物体检测、场景变化检测）使SLAM能够区分"地图应该更新的变化"和"暂时的干扰"，这是长期自主运行的前提。

跨模态桥接：具身智能体往往配备多模态传感器——RGB相机、深度相机、IMU、触觉传感器、甚至语言指令。学习模块（特别是基于Transformer的架构）具备融合异构信息的能力，为跨模态SLAM提供了技术基础。

但这些优势必须与7.4节讨论的风险平衡。一个过度依赖学习模块的具身智能体，在面对训练分布之外的"开放世界"场景时，可能因为前端的系统性失效而丧失定位能力——这在自主导航中是不可接受的。因此，具身智能系统的设计原则是：**学习模块提供丰富的先验和语义理解，传统几何后端提供确定性的约束和安全边界。**

7.6 本章一句话总结

深度学习不是SLAM的替代者，而是其最薄弱环节的精准补强——前提是系统设计者清醒地认识学习方法的泛化边界、不确定性缺陷和在闭环优化中的错误放大效应。

第8章 DROID-SLAM：深度 SLAM 的重要分水岭

8.1 为什么 DROID-SLAM 必讲

SLAM 领域经历了三个阶段的演进。第一阶段以扩展卡尔曼滤波为代表，用概率递推估计位姿和路标点。第二阶段以图优化为核心，将 SLAM 建模为稀疏或非线性最小二乘问题，用 Bundle Adjustment（BA，光束法平差）统一求解。第三阶段——也就是 DROID-SLAM 所开启的阶段——将深度学习的表征能力与几何优化的可解释性熔于一炉，构建出端到端可微的 SLAM 系统。

DROID-SLAM 由 Teed 和 Deng 于 2021 年提出 (NeurIPS)。它并非深度学习进入 SLAM 的第一次尝试，却是**第一次**在精度、鲁棒性和泛化性三个维度上同时超越经典方法的深度 SLAM 系统。此前的工作（如 DeepV2D、BA-Net、TartanVO）要么只在小规模场景上验证，要么精度远不及 ORB-SLAM3 和 DSO 等经典系统。DROID-SLAM 改变了这一格局。

定量结果最能说明问题。在 EuRoC MAV 数据集的单目轨道上，DROID-SLAM 将平均绝对轨迹误差 (ATE) 降至 0.022m，在零失败的方法中比 ORB-SLAM3 降低 82%。在 TUM-RGBD 数据集上，它比 DeepFactors 降低 83% 的误差。在 ETH-3D RGB-D SLAM 排行榜上，它以 AUC 指标领先第二名 35%。在 TartanAir SLAM 竞赛中，单目轨道误差降低 62%，双目轨道降低 60%。这些数据表明，DROID-SLAM 不只是"深度学习方法中表现最好的"，而是"所有方法中表现最好的"。

更具说服力的是鲁棒性。在 ETH-3D 的 32 个 RGB-D 序列中，DROID-SLAM 成功跟踪 30 个，而次优方法仅成功 19 个。在 TartanAir、EuRoC 和 TUM-RGBD 上，它的失败率为零。这种鲁棒性源于其架构设计：网络不是一次性预测位姿，而是通过循环迭代不断修正估计，从粗到细地收敛到正确答案。

DROID-SLAM 的另一项突破是**模态泛化**。它仅在合成数据集 TartanAir 上以单目视频训练一次，测试时却能直接接受双目或 RGB-D 输入而无需重新训练，且性能随输入模态的丰富而提升。这意味着网络学到的不是特定传感器的映射，而是通用的几何推理能力。

从全书结构看，第 7 章讨论了深度学习进入 SLAM 各个模块的零散尝试，DROID-SLAM 是这些尝试的集大成者。第 9 章将讨论神经隐式 SLAM (iMAP、NICE-SLAM)，而 DROID-SLAM 正好是连接"学习的前端"与"神经表示的后端"的桥梁。不讲透 DROID-SLAM，就无法理解 2021 年之后 SLAM 领域的技术走向。

8.2 传统方法与 DROID-SLAM 的本质区别

8.2.1 传统 SLAM 的架构范式

经典 SLAM 系统（如 ORB-SLAM3、DSO、COLMAP）遵循一个清晰的模块化流水线：

前端负责数据关联。间接法 (Feature-based) 检测关键点 (ORB、SIFT)、计算描述子、做特征匹配，建立帧间的稀疏对应关系。直接法 (Direct) 跳过特征提取，在图像强度上定义光度误差，通过灰度一致性假设寻找对应。前端输出的对应关系带有噪声和离群点。

后端负责状态估计。将前端输出的对应关系输入优化器，最小化重投影误差 (间接法) 或光度误差 (直接法)，求解相机位姿和三维点云。BA 是后端的核——它把所有相机和三维点放入一个大规模非线性最小二乘问题中联合求解。

回环检测负责长期一致性。通过词袋模型 (BoW) 或全局描述子检测曾经到访过的地点，触发位姿图优化或全局 BA 以消除累积漂移。

这个范式的核心特征是：**几何优化是显式的、手工设计的、不可微的**。特征检测器 (FAST、Harris) 是手工设计的。匹配策略 (最近邻比值测试、RANSAC) 是手工设计的。BA 优化器 (g2o、GTSAM、g2o) 是显式求解的。整个系统不是端到端可训练的。

8.2.2 DROID-SLAM 的架构范式

DROID-SLAM 彻底颠覆了这一范式。它的核心架构由三个可微模块构成：

组件	传统 SLAM	DROID-SLAM
特征表示	手工描述子 (ORB/SIFT)	网络学习特征 (CNN 编码器)
数据关联	显式匹配 + RANSAC	4D 相关性体 + 软对应
状态更新	显式 BA (g2o/GTSAM)	可微 Dense BA Layer
迭代优化	不可微的外部求解器	循环网络端到端迭代
训练方式	无需训练, 参数手工调节	端到端训练, 梯度反向传播
传感器适配	重写 frontend/backend	同一模型, 输入即适配

上表揭示了本质区别。传统方法中，特征提取、匹配和优化是三个解耦的模块，各自独立设计。DROID-SLAM 将它们包裹在一个端到端可微的计算图中，每一层都可以从最终损失函数接收梯度信号。这种设计带来了三个直接好处：

第一，特征学习替代了手工描述子。 网络从数据中学到的特征表征比 ORB 更鲁棒，对光照变化、低纹理区域和运动模糊不那么敏感。

第二，软对应替代了硬匹配。 传统方法中，一个特征点要么匹配成功，要么被 RANSAC 剔除。DROID-SLAM 通过相关性体保留所有匹配可能性的连续分布，用置信度权重调制每个像素的贡献，避免了硬判决带来的信息损失。

第三，可微优化替代了外部求解器。 Dense BA Layer 不是调用外部库求解一个独立的优化问题，而是前向传播中的一个可微算子。梯度可以从位姿和深度输出一路反向传播到网络权重，使得整个系统能够端到端学习。

8.2.3 架构差异的工程含义

两种范式在工程实现上也有深刻差异。传统 SLAM 系统的可扩展性受限于手工组件的复杂度——增加一种传感器需要重写 frontend 和 backend。DROID-SLAM 的双目和 RGB-D 适配只需要在 DBA Layer 的优化目标中增加几个额外项，无需修改网络结构。传统方法中，ORB 特征在弱纹理区域（白墙、天空）束手无策；DROID-SLAM 的密集对应利用整幅图像的信息，在这些区域仍能保持跟踪。

但 DROID-SLAM 的范式也付出了代价：它需要 GPU 实时推理，显存占用超过 4GB；它的性能依赖于训练数据的分布覆盖，在训练时未见过的场景类型中可能退化；它的前端-backend 设计虽然精巧，但长期地图表示仍然相对薄弱。这些局限将在 8.4 节深入讨论。

8.3 DROID-SLAM 的三个关键技术

DROID-SLAM 的架构可以拆解为三个相互耦合的技术支柱：Correlation Volume（相关性体）、Recurrent Update（循环更新算子）和 Dense Bundle Adjustment Layer（密集光束法平差层）。三者缺一不可，共同构成了"学习到的对应关系 → 几何约束下的位姿-深度联合优化"的闭环。

8.3.1 Correlation Volume：从光流到多帧数据关联

DROID-SLAM 的数据关联机制直接继承自 RAFT (Recurrent All-Pairs Field Transforms)，但将其从两帧光流扩展到多帧 SLAM 场景。

特征提取。 输入图像经过两个并行的 CNN: feature network 和 context network。Feature network 输出特征图 $g_\theta(I) \in \mathbb{R}^{H \times W \times D}$ ，用于构建相关性体。Context network 输出的上下文特征注入 GRU，为更新操作提供先验信息。两个网络均由 6 个残差块和 3 个下采样层组成，输出分辨率为输入的 1/8。

4D 相关性体。 对帧图中的每条边 $(i, j) \in \mathcal{E}$ ，计算所有特征向量对的点积：

$$C_{u_1 v_1 u_2 v_2}^{ij} = \langle g_\theta(I_i)_{u_1 v_1}, g_\theta(I_j)_{u_2 v_2} \rangle$$

这是一个 4D 张量，尺寸为 $H \times W \times H \times W$ 。它编码了帧 i 中每个像素与帧 j 中所有像素的视觉相似度。不同于传统匹配中"一个点对应一个点"的硬判决，相关性体保留了所有可能对应的软分布。

相关性金字塔。 为捕获多尺度信息，对 4D 相关性体的后两个维度进行 4 层平均池化，形成金字塔结构。在查询时，从各层提取特征并拼接，使网络能在不同尺度上利用对应信息。

Lookup 操作。 给定当前位姿和深度估计，可以将帧 i 的像素投影到帧 j ，得到一组坐标。Lookup 操作在这些坐标为中心的邻域窗口内，从相关性体中索引特征值。这一操作是可微的，使用双线性插值。

相关性体的设计有几个关键优势：

- **密集覆盖：** 每个像素都参与，不依赖稀疏特征点的质量
- **可微索引：** 梯度可以从索引位置反向传播到坐标（即位姿和深度）
- **多尺度融合：** 金字塔结构处理大位移和小位移
- **预计算复用：** 特征图和相关性体可以在迭代中复用

8.3.2 Recurrent Update Operator: 循环迭代更新

这是 DROID-SLAM 的"控制中枢"。不同于 RAFT 只更新光流，DROID-SLAM 的更新算子同时输出位姿修正和深度修正。

对应场计算。 在每次迭代开始时，用当前的位姿 G 和深度 d 计算密集对应场：

$$p_{ij} = \Pi_c(G_{ij} \circ \Pi_c^{-1}(p_i, d_i))$$

其中 Π_c 是相机投影模型， Π_c^{-1} 是反投影函数， $G_{ij} = G_j \circ G_i^{-1}$ 是相对位姿。 p_{ij} 表示帧 i 中像素在帧 j 中的预测位置。

GRU 更新。 更新算子是一个 3×3 的卷积 GRU。输入包括：

- 相关性特征（从 correlation volume lookup 获得）
- 光流特征 ($p_{ij} - p_j$ ，编码运动信息)
- 前次 BA 残差（反馈信号）
- 上下文特征（context network 输出）
- 全局汇聚特征（空间平均的 hidden state，提供全局上下文）

GRU 不是直接预测位姿和深度更新，而是预测**流场修正** $r_{ij} \in \mathbb{R}^{H \times W \times 2}$ 和**置信度图** $w_{ij} \in \mathbb{R}_+^{H \times W \times 2}$ 。修正后的对应场为 $p_{ij}^* = r_{ij} + p_{ij}$ 。

这种设计的妙处在于：GRU 负责"感知哪里错了"，而几何优化负责"用几何约束修正错误"。网络不需要学会几何——它只需要学会识别可靠的对应关系并分配信度，真正的位姿和深度求解交给下面的 DBA Layer。

置信度学习的意义。 w_{ij} 是网络对每个像素对应可靠性的估计。遮挡区域、动态物体、反光表面会获得低置信度，在后续 BA 中被自动降权。这相当于一个内嵌的、学习的 outlier rejection 机制，替代了传统 SLAM 中的 RANSAC。

8.3.3 Dense Bundle Adjustment Layer：可微几何优化

DBA Layer 是 DROID-SLAM 最具原创性的贡献。它将传统 BA 的核心逻辑——最小化重投影误差——实现为一个可微神经网络层。

优化目标。 DBA Layer 的目标函数定义为：

$$E(G', d') = \sum_{(i,j) \in \mathcal{E}} \|p_{ij}^* - \Pi_c(G'_{ij} \circ \Pi_c^{-1}(p_i, d'_i))\|_{\Sigma_{ij}}^2$$

$$\Sigma_{ij} = \text{diag}(w_{ij})$$

这是加权重投影误差。 p_{ij}^* 是网络预测的目标对应位置， $\Pi_c(G'_{ij} \circ \Pi_c^{-1}(p_i, d'_i))$ 是用当前位姿和深度重投影后的位置，两者之间的加权马氏距离即为误差。

注意这个目标函数的简洁性：它优化的不是光度误差（如 DSO），而是几何误差。光度误差要求亮度恒定假设，在光照变化下脆弱；几何误差只要求空间一致性，对光照更鲁棒。

Gauss-Newton 求解。 DBA Layer 用一次 Gauss-Newton 迭代求解上述目标。线性化后的正规方程为：

$$\begin{bmatrix} B & E \\ E^T & C \end{bmatrix} \begin{bmatrix} \Delta \xi \\ \Delta d \end{bmatrix} = \begin{bmatrix} v \\ w \end{bmatrix}$$

由于每个误差项只涉及一个深度变量， C 是对角矩阵，可以用 Schur complement 高效求解：

$$\Delta \xi = (B - EC^{-1}E^T)^{-1}(v - EC^{-1}w)$$

$$\Delta d = C^{-1}(w - E^T \Delta \xi)$$

C^{-1} 的求解开销极小（逐元素取倒数）。位姿更新在 SE(3) 流形上进行： $G^{(k+1)} = \text{Exp}(\Delta \xi) \circ G^{(k)}$ 。深度更新直接相加： $d^{(k+1)} = d^{(k)} + \Delta d$ 。

可微性实现。 DBA Layer 在 PyTorch 中实现，利用自动微分引擎反向传播梯度。训练时使用 LieTorch 库在群元素的切空间中进行反向传播。推理时使用自定义 CUDA kernel，利用问题的块稀疏结构加速。

DBA 相比传统 BA 的区别。

特性	传统 BA	DBA Layer
优化变量	稀疏路标点	逐像素深度图
对应关系	硬匹配（一一对应）	软对应（加权）
置信度	二元（内点/外点）	连续权重图
求解器	外部库（g2o/GTSAM）	内嵌可微层
梯度流	不可反向传播	端到端可微
输入适配	需重写前端	统一优化目标

上表的关键在于最后两行。传统 BA 是 SLAM 系统的一个独立组件，网络无法“知道”BA 会如何处理它的输出。DBA Layer 让网络在训练时就经历完整的“预测对应 → 几何优化 → 观察误差”循环，因此学习到的特征和置信度天然适配后续的优化过程。

8.3.4 三者的协同：一个端到端的闭环

三个技术组件不是孤立运作的，它们构成一个迭代收敛的闭环：

1. 当前位姿 & 深度 → 计算密集对应场 p_{ij}
2. p_{ij} 索引 correlation volume → 提取匹配特征
3. 匹配特征 + 运动特征 + 反馈 → GRU 预测修正 r_{ij} 和置信度 w_{ij}
4. 修正后的对应 $p_{ij}^* +$ 置信度 w_{ij} → DBA Layer 求解 $\Delta\xi, \Delta d$
5. 更新位姿 $G^{(k+1)}$ 和深度 $d^{(k+1)}$
6. 返回步骤 1，迭代直至收敛

每次迭代中，GRU 的 hidden state 被更新，保留了历史信息。这使得系统能够逐步精细化估计，从粗略的初始猜测收敛到精确的解。训练时，这个循环展开 15 次迭代，损失函数施加在所有中间输出上（指数加权， $\gamma = 0.9$ ），迫使每一步都向正确答案靠近。

8.4 论文精读：DROID-SLAM

以下按本书统一的论文精读模板，逐条拆解 DROID-SLAM（NeurIPS 2021，arXiv:2108.10869）。

8.4.1 论文试图解决什么痛点？

深度学习方法在 SLAM 中的精度长期落后于经典方法。此前的端到端系统（DeepV2D、BA-Net、TartanVO）要么只在少量帧的小窗口上验证，要么在跨数据集泛化时表现糟糕。核心痛点是：如何将深度学习的表征优势与几何优化的精度优势结合，构建既鲁棒又精确的 SLAM 系统。

8.4.2 核心假设是什么？

DROID-SLAM 依赖三个核心假设：

1. 光度一致性隐含几何一致性：学习到的特征相关性可以替代手工特征匹配，作为数据关联的基础
2. 迭代优化可以端到端学习：循环网络能够学会逐步修正位姿和深度，类似经典 BA 的收敛过程
3. 几何约束是可微的：Gauss-Newton 迭代可以嵌入前向传播，梯度可以穿过优化层反向传播

8.4.3 输入输出是什么？

输入：单目、双目或 RGB-D 视频流。网络仅需图像帧 I_t ，相机内参已知。双目模式下左右目作为独立帧输入，相对位姿在 DBA 中固定。RGB-D 模式下深度传感器提供的深度作为优化目标中的额外约束项。

输出：

- 每帧相机位姿 $G_t \in \text{SE}(3)$
- 每帧像素级逆深度图 $d_t \in \mathbb{R}^{H \times W}$
- 帧间密集对应场 p_{ij}^*
- 对应置信度图 w_{ij}

8.4.4 地图表示是什么？

DROID-SLAM 维护一个帧图（frame graph）。节点是关键帧，边是共视关系。每个关键帧存储：相机位姿（SE(3)）、逆深度图（ $H \times W$ ）、特征图、上下文特征和 GRU hidden state。边由平均光流大小衡量共视程度，相邻关键帧及共视度最高的帧之间建立连接。

这不是传统意义上的“地图”（点云或体素栅格），而是一个以关键帧为中心的稀疏-密集混合表示。位姿是稀疏的（仅关键帧），深度是密集的（逐像素）。没有显式的全局点云或 mesh，重建结果通过关键帧的深度图和位姿按需合成。

8.4.5 Tracking 怎么做？

Tracking 就是前端线程的核心循环：

1. 新帧到达，提取特征图和上下文特征
2. 将其加入帧图，与最近的 3 个关键帧建立边
3. 用线性运动模型初始化新帧位姿
4. 运行若干次更新算子迭代（更新位姿和深度）
5. 用平均光流评估冗余度，若关键帧过多则删除最老或最冗余的帧

前端在实时约束下运行，迭代次数有限（通常 3-5 次）。位姿估计的频率等于视频帧率。

8.4.6 Mapping 怎么做？

Mapping 由后端线程负责，执行全局 Bundle Adjustment：

1. 用当前所有关键帧的光流矩阵重建帧图
2. 优先加入时间相邻边，再按光流大小加入共视边（Chebyshev 距离抑制邻近边）
3. 对整个帧图运行更新算子，通常包含数千帧和边
4. 使用 RAFT 的内存高效实现避免显存溢出

后端在独立线程中异步运行，不阻塞前端 tracking。推理时后端使用自定义 CUDA kernel 加速，关键利用问题的块稀疏结构做稀疏 Cholesky 分解。

非关键帧的位姿通过 motion-only BA 恢复：固定关键帧位姿和深度，只优化非关键帧位姿。

8.4.7 优化目标是什么？

训练损失由两部分组成：

Pose Loss：预测位姿与真实位姿的 SE(3) 距离

$$\mathcal{L}_{\text{pose}} = \sum_i \|\text{Log}_{SE(3)}(T_i^{-1} \cdot G_i)\|^2$$

Flow Loss：相邻帧间的光流差异（由预测深度和位姿诱导 vs 真实深度和位姿诱导）

总损失施加在所有迭代步骤的输出上，权重指数增长 ($\gamma = 0.9$)，早期迭代权重较小，后期迭代权重较大。这种课程式监督让网络先学会粗略估计，再逐步精细化。

8.4.8 相比前作真正推进了什么？

与两个最接近的前作——BA-Net 和 DeepV2D——相比，DROID-SLAM 有三项根本性推进：

vs BA-Net：BA-Net 的 BA 层不是“密集”的——它优化的是少量系数，用于线性组合一组预生成的深度基图。DROID-SLAM 直接优化逐像素深度，不受深度基的限制。BA-Net 优化光度重投影误差（特征空间），DROID-SLAM 优化几何误差（流空间），后者对光照变化更鲁棒。

vs DeepV2D：DeepV2D 在深度更新和位姿更新之间交替，没有真正的联合优化。DROID-SLAM 的 DBA Layer 同时求解所有位姿和深度，是真正的 Bundle Adjustment。此外，DeepV2D 只处理少量帧（2-12 帧），DROID-SLAM 可以处理任意长的序列。

vs RAFT：RAFT 只处理两帧光流。DROID-SLAM 将循环更新扩展到多帧 SLAM，引入了 DBA Layer 将流场修正映射为位姿和深度更新，并加入了全局上下文汇聚机制处理遮挡和动态物体。

8.4.9 没有解决什么？

DROID-SLAM 留下了五个未解决的问题：

1. **训练分布依赖**。仅在 TartanAir 合成数据上训练。虽然跨数据集泛化能力惊人（零样本在 EuRoC、ETH-3D、TUM-RGBD 上表现优异），但在与训练域差异极大的场景（如水下、夜景、极端天气）中性能可能退化。
2. **长期地图表示薄弱**。帧图本质上是稀疏的关键帧集合，没有显式的全局地图模型。与神经隐式表示（iMAP、NICE-SLAM）相比，它无法回答“空间某点是否被占据”这类查询，也无法做路径规划或碰撞检测。
3. **计算资源需求高**。需要 GPU 推理，显存 >4GB，推理速度约 12-15 FPS。这使得它在嵌入式平台（无人机、手机、AR 眼镜）上的部署受限。
4. **动态物体处理不足**。虽然置信度图可以在一定程度上抑制动态物体，但系统没有显式的运动分割或动态物体建模机制。在高度动态的环境中（人群、车辆），动态物体会干扰 tracking。
5. **回环检测相对简单**。依赖光流矩阵的共视判断，没有训练专用的视觉地点识别模块。在视角变化大、光照变化剧烈的回环场景中，可能漏检回环。

8.4.10 对后续研究的影响是什么？

DROID-SLAM 催生了三个研究方向：

效率优化。 DPVO (2023) 将密集对应降为稀疏 patch 对应，将 EuRoC 上的推理速度从 ~8 FPS 提升到 60 FPS，显存占用减半，同时保持相近的精度。DPV-SLAM (2024) 在 DPVO 基础上增加了回环检测，证明学习前端可以与经典后端高效协作。

与神经表示融合。 DROID-SLAM 的精确 tracking 为神经隐式 SLAM 提供了高质量的姿态初始化。后续工作（如 NICE-SLAM、Co-SLAM）将 DROID-SLAM 或类似的学习前端与 NeRF/隐式表示结合，实现了既精确又美观的重建。

对应学习范式的扩展。 DROID-SLAM 证明"学习对应 + 可微几何优化"是可行的范式。后续工作将其扩展到动态场景（DROID-W）、多传感器融合（DROID-VIO）、以及大尺度场景。

8.5 优点、失败模式与适用场景

8.5.1 核心优势

鲁棒性源于迭代。 DROID-SLAM 不是一次性预测答案，而是通过循环迭代逐步收敛。这赋予了它一种"自我修正"能力——初始估计可能很差，但多次迭代后能恢复正确轨迹。在 TUM-RGBD 的 fr1/desk 序列上，经典方法频繁失败，而 DROID-SLAM 的错误仅为 0.018m。

模态无缝切换。 同一模型从单目训练后，双目测试只需在 DBA 中固定左右目相对位姿；RGB-D 测试只需在优化目标中加入深度项。这种灵活性在传统 SLAM 中难以想象——ORB-SLAM3 的单目、双目、RGB-D 是三个独立的前后端组合。

密集输出的丰富性。 逐像素深度图为下游任务（避障、表面重建、语义标注）提供了比稀疏点云更丰富的信息。

8.5.2 失败模式

失败场景	原因	表现
训练域外场景	特征提取器未见过类似纹理/结构	特征失效，对应错误，BA 发散
极端动态环境	大部分像素被动态物体占据	置信度图失效，几何假设被破坏
快速光照变化	相关性体假设亮度局部恒定	流场估计错误
长序列无回环	帧图密度不够，全局约束不足	累积漂移
GPU 显存不足	高分辨率+长序列超出显存	无法运行或降采样损失精度

上表中，前两种失败模式与训练数据的覆盖范围直接相关。DROID-SLAM 的神经网络组件在分布内（in-distribution）数据上表现出色，但在分布外（out-of-distribution）数据上可能不如经典方法稳定——后者不依赖训练数据，只依赖几何假设。

8.5.3 适用场景

DROID-SLAM 最适合以下场景：

- 有 GPU 资源的室内/室外静态或轻度动态环境
- 需要单目、双目、RGB-D 灵活切换的应用
- 对鲁棒性要求高于计算效率的场景（如探索型机器人）
- 作为神经隐式 SLAM 的前端提供精确位姿

不适合的场景：

- 嵌入式平台（无 GPU 或显存 <4GB）
- 训练域外的极端环境
- 高度动态、遮挡严重的场景
- 需要实时全局一致地图（而不仅是轨迹）的应用

8.6 与具身智能的关系

DROID-SLAM 与具身智能的关系是“强前端，弱后端”。它为具身智能提供了精确的自身定位和密集的环境几何，但缺乏具身智能所需的完整空间表示。

从具身智能的视角看，一个完整的空间感知系统需要回答三个问题：我在哪？（定位）周围有什么？（建图）我该怎么走？（规划）。DROID-SLAM 精确回答了第一个问题，部分回答了第二个问题（深度图而非完整地图），但无法回答第三个问题——它的帧图表示不包含占据信息，不能直接用于路径规划或碰撞检测。

DROID-SLAM 与后续神经隐式 SLAM（第 9 章）的互补性恰恰在这里。DROID-SLAM 提供精确的位姿轨迹和密集深度，iMAP/NICE-SLAM 将这些信息整合到神经隐式场中，构建可用于规划、渲染和交互的完整空间表示。在第 10 章我们将看到，这种“学习前端 + 神经后端”的组合正在成为具身智能感知系统的标准架构。

另一个值得关注的方向是 DROID-SLAM 的可微性如何服务于端到端具身学习。由于整个系统是可微的，它可以被嵌入更大的端到端强化学习或模仿学习框架中。机器人可以学习不只是“如何移动”，而是“如何移动以获得更好的 SLAM 估计”——这种主动 SLAM（active SLAM）的能力，只有在前端可微的前提下才能实现。

8.7 一句话总结

DROID-SLAM 用可微的 Dense Bundle Adjustment Layer 替代了传统 SLAM 的手工几何优化器，证明“学习到的对应关系 + 循环迭代的几何优化”可以在精度和鲁棒性上同时超越经典方法，但它本质上是强 VO/SLAM 前端而非完整的空间智能系统，长期地图表示和训练分布依赖仍是通往具身智能需要跨越的障碍。

第9章 神经隐式 SLAM：iMAP、NICE-SLAM 与 NeRF-SLAM

9.1 神经隐式地图的基本思想

传统 SLAM 用一个离散的数据结构表示场景：点云是点的集合，体素网格是三维栅格，surfel 是小面片。这些表示有一个共同问题——分辨率在建图之初就被固定了。你想看更细的细节？只能在更密的网格上重新跑一遍。你想知道两个已知点之间的表面形状？只能插值猜测。

NeRF (Neural Radiance Field, 神经辐射场) (arXiv) 带来了一种完全不同的思路：用一个连续函数表示场景。输入三维坐标 $\mathbf{x} = (x, y, z)$ ，网络输出该点的几何属性（密度 σ 或 SDF 值）和外观属性（颜色 $\mathbf{c}$ ）。整个场景被压缩进神经网络的权重里。

这个思想的精髓可以用一句话概括：**场景即函数**。连续坐标进入，连续值出来。空间分辨率不再是预设参数，而是由网络表达能力决定。你问网络 (0.5, 1.2, 3.7) 处的占用概率，它能给你一个值；你问 (0.5001, 1.2001, 3.7001)，它也能给，而且几乎不受离散化粒度限制。

神经隐式表示的核心数学形式是坐标编码函数：

$$F_{\theta} : \mathbf{x} \mapsto (\sigma, \mathbf{c})$$

其中 θ 是网络参数。渲染图像时，从相机光心出发沿每条光线采样若干三维点，将这些点送进网络查询密度和颜色，再用体渲染 (volume rendering) 积分出像素颜色：

$$\hat{C} = \sum_{i=1}^N T_i \alpha_i c_i, \quad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

α_i 由采样点的密度和步长决定，相当于该点的不透明度贡献； T_i 是透射率，表示光线到达该点之前未被遮挡的概率。这个公式可微，意味着可以通过比较渲染图像和真实图像的像素差异，反向传播梯度去优化网络参数和相机位姿。

对于 SLAM 来说，这套框架有天然的吸引力。传统方法把 tracking（位姿估计）和 mapping（地图构建）割裂开来：tracking 用特征匹配或光度误差，mapping 用体素融合或网格优化。神经隐式表示把两者统一到同一个可微渲染框架里——位姿不对，渲染图就不对；地图不对，渲染图也不对。两者可以从同一个光度误差中联合优化。

但这套思想的落地并不容易。纯 MLP 表示的容量有限，渲染需要逐光线采样数十到数百个点，每个点都要过一次网络，计算开销巨大。把 NeRF 塞进实时 SLAM 系统，需要同时解决效率、扩展性和在线学习三个难题。

9.2 iMAP

论文精读

论文信息：iMAP: Implicit Mapping and Positioning in Real-Time, Edgar Sucar 等，帝国理工学院 Dyson Robotics Lab, ICCV 2021。(arXiv)

iMAP 试图解决什么痛点？ 2021 年之前，NeRF 已经被证明可以从多视角图像中重建逼真的三维场景，但它需要预先知道精确的相机位姿，且训练耗时数小时到数天。SLAM 系统虽然能实时估计位姿和建图，但输出的是离散的网格或点云，缺乏连续性和新视角合成能力。iMAP 要回答一个核心问题：能不能让 NeRF 在实时运行中边建图边定位，完全在线学习一个场景的隐式三维模型？

核心假设是什么？ 一个足够小的 MLP（多层感知机）在关键帧缓冲和增量式训练的加持下，可以在不遗忘已观测区域的前提下，在线学习整个场景的隐式表示，并且渲染速度足以支持实时 tracking。

输入输出是什么？ 输入是 RGB-D 视频流，每帧包含彩色图像和深度图。输出是：① 相机每帧的六自由度位姿；② 一个场景特定的隐式三维模型（MLP），能查询任意三维点的占据概率和颜色；③ 从任意视角渲染的深度图和彩色图。

地图表示是什么？ 单一 MLP，输入是三维坐标 (x, y, z) ，输出占据概率 $o \in [0, 1]$ 和颜色 (r, g, b) 。没有显示的特征网格或点云，整个场景全部编码在网络权重中。MLP 规模很小——只有 5 层、宽度 256，参数量约 0.3 MB。这种紧凑性是 iMAP 区别于后续方法的核心特征。

tracking 怎么做？ iMAP 采用双进程架构。tracking 进程以 10 Hz 运行：对当前帧，固定网络权重，通过可微渲染优化相机位姿。具体是从当前位姿假设出发，渲染深度图和彩色图，计算与真实 RGB-D 帧的像素级误差，反向传播梯度更新位姿。渲染时不是全图逐像素计算，而是基于信息增益动态采样 200 个像素——深度误差大的区域多采样，已经拟合好的区域少采样。

mapping 怎么做？ mapping 进程以 2 Hz 运行全局地图更新。维护一个关键帧集合（通常 5~10 帧），对它们做联合优化：随机选择一个关键帧，从中采样像素，沿光线采样三维点，送入 MLP 查询占据和颜色，用体渲染公式生成预测深度和颜色，最小化与真实 RGB-D 的损失。损失函数包含三项：深度 L1 损失、颜色 L2 损失、以及一个占据正则项鼓励表面附近的占据概率接近 1、远离表面的接近 0。

$$\mathcal{L} = \lambda_{rgb} \|\hat{C} - C\|_2 + \lambda_{depth} \|\hat{D} - D\|_1 + \lambda_{occ} \mathcal{L}_{occ}$$

关键帧选择策略基于信息增益：当一帧带来的深度不确定性降低足够大时，就将其纳入关键帧集合。老的关键帧如果与新帧共视度低，则被剔除。这种"回放式"策略是为了缓解单一 MLP 的灾难性遗忘——通过不断回访旧关键帧，让网络记住已经学过的区域。

优化目标是什么？ 联合优化网络权重 θ 和相机位姿 $\{\mathbf{T}_i\}$ ，使关键帧集合上的重渲染误差最小。注意这里不是逐帧做增量式更新，而是在关键帧缓冲上做随机采样式的反复训练，更接近小批量随机梯度下降。

相比前作真正推进了什么？ iMAP 是首个证明 MLP 可以作为实时 SLAM 系统唯一场景表示的工作。在此之前，NeRF 是离线重建工具，需要已知位姿；SLAM 是实时定位工具，输出离散地图。iMAP 把两者融合到了一个端到端可微框架里，开创了"神经隐式 SLAM"这一方向。

没有解决什么？ 三个致命问题。

第一，**灾难性遗忘**。单一 MLP 的容量有限，当相机 revisit 之前观测过的区域时，网络往往已经"遗忘"了那里的形状。信息引导采样和关键帧回访只能缓解，不能根除。根本原因是一个小 MLP 试图记住整个场景，新信息不可避免地覆盖旧记忆。

第二，**扩展性差**。MLP 的全局参数结构意味着场景越大，需要的网络容量越大，但增大网络会直接拖慢渲染速度。iMAP 只在桌面级小场景上做了演示，房间大小的场景就会导致细节丢失和跟踪漂移。

第三，**几何精度不高**。iMAP 的输出偏向平滑、过拟合的表面，缺乏尖锐边缘。这是因为 MLP 作为连续函数天然倾向于产生光滑的插值，难以表达墙面交界、家具棱角等高梯度几何。

对后续研究的影响是什么？ iMAP 证明了神经隐式表示用于实时 SLAM 的可行性，但它的失败模式也为后续工作指明了方向：如果单一 MLP 不行，那就用混合表示——把全局 MLP 换成局部特征网格 + 小解码器的组合。NICE-SLAM、Vox-Fusion、Co-SLAM 都走这条路。

系统架构与工程细节

iMAP 的工程实现值得细说。双进程架构中，tracking 和 mapping 并行运行，通过共享内存交换关键帧和位姿。MLP 只包含 0.3 MB 参数，渲染一帧 640×480 图像需要约 150 ms——远不能实时。iMAP 的 trick 是只渲染 200 个采样像素来估计位姿，而非整幅图像。这种稀疏渲染支撑了 10 Hz 的 tracking，但代价是 tracking 精度不如全图方法。

mapping 端的 2 Hz 全局更新意味着地图对新信息的响应有半秒延迟。相机快速运动时，这半秒内 tracking 用的还是旧地图，容易累积误差。

指标	iMAP	说明
地图表示	单一 MLP（5层×256维）	约 0.3 MB
Tracking 频率	10 Hz	稀疏像素渲染
Mapping 频率	2 Hz	关键帧集合上联合优化
适用场景	桌面级小场景	房间大小即出现退化
主要缺陷	灾难性遗忘、扩展性差	单一 MLP 的根本限制

iMAP 的教训非常明确：把全部场景信息压进一个 MLP 是优雅的，但不实用。真正要在 SLAM 中落地神经隐式表示，需要一种既能局部更新又保持连续性的表示方法。

9.3 NICE-SLAM

论文精读

论文信息：NICE-SLAM: Neural Implicit Scalable Encoding for SLAM, Zihan Zhu 等, ETH Zürich / 浙大等, CVPR 2022。(CVF开放获取)

NICE-SLAM 试图解决什么痛点？ iMAP 用单一 MLP 表示整个场景，导致灾难性遗忘和可扩展性不足。NICE-SLAM 的核心目标是：在保持神经隐式表示的连续性和预测能力的同时，让系统能扩展到较大室内场景，并缓解灾难性遗忘。

核心假设是什么？ 把场景的几何信息编码在多分辨率的三维特征网格里，用固定的小 MLP 做解码器，将局部特征解码为占据概率和颜色。特征网格可以局部更新，不需要像 iMAP 那样每次修改都影响全局。

输入输出是什么？ 输入是 RGB-D 视频流。输出是：① 相机位姿；② 场景的三维网格模型（可通过 Marching Cubes 提取）；③ 从任意视角渲染的彩色图和深度图。

地图表示是什么？ NICE-SLAM 采用**多层级局部特征编码**。场景几何由三个不同分辨率的体素特征网格表示——粗（coarse）、中（mid）、细（fine）——每个网格单元存储一个 32 维可学习特征向量。另外还有一个独立的外观特征网格负责颜色。

三个几何层级各有对应的预训练 MLP 解码器。输入一个三维点，先找到它在各层级中所在的体素，三线性插值得到特征，再分别送入对应解码器，输出三个层级的占据预测。细层级的预测是"残差"形式：它预测的是细粒度几何相对于中层表示的修正。这种残差结构让系统既能捕捉大尺度的房间结构（粗层级），又能重建物体表面的细节（细层级）。

$$O_{final} = O_{coarse} + O_{mid} + \Delta O_{fine}$$

外观表示与几何解耦。颜色由一个独立的高分辨率特征网格和解码器负责，输入包括三维坐标、法线方向和视点方向，可以建模视角相关的外观效果（如高光）。

tracking 怎么做？ 与 iMAP 类似，固定特征网格和 MLP 权重，通过可微渲染优化当前帧的位姿。但 NICE-SLAM 的渲染目标函数更精细：除了光度误差和深度误差，还引入了深度平滑项，在梯度大的区域给予更高权重。Tracking 同样使用稀疏像素采样以加速，但采样策略更简单——从深度有效区域均匀随机采样。

mapping 怎么做？ 这是 NICE-SLAM 与 iMAP 最根本的区别。Mapping 时，**只更新当前视野（frustum）内的特征网格单元**，而非像 iMAP 那样更新全局 MLP。这意味着新观测只会修改局部区域的特征，已观测但当前不可见的区域保持不变。这种局部更新机制从根本上缓解了灾难性遗忘。

Mapping 也采用关键帧结构。对每个关键帧，从其视锥体内采样光线，渲染深度和颜色，优化特征网格参数。粗、中、细三个层级的特征网格被联合优化，但细层级的更新率更低——只在确信有高质量观测的区域才更新细粒度特征，避免噪声区域产生伪影。

优化目标是什么？

$$\mathcal{L} = \lambda_{rgb} \|\hat{C} - C\|_2 + \lambda_{depth} \|\hat{D} - D\|_1 + \lambda_{smooth} \|\nabla \hat{D} \cdot \mathbf{n}\|_1$$

其中第三项是深度平滑项，鼓励深度图在表面法线方向上的梯度平滑。整体损失函数在几何和外观上是可分离的：几何特征网格通过深度和占据损失优化，外观特征网格通过颜色损失优化。

相比前作真正推进了什么？

第一，**可扩展性**。通过特征网格替代单一 MLP，NICE-SLAM 能处理比 iMAP 大数倍的室内场景（多房间公寓级别）。特征网格的内存消耗虽然随场景体积线性增长，但只分配有观测的区域，避免了全局密集网格的浪费。

第二，**局部更新**。只更新视锥体内的特征，意味着 revisit 旧区域时不需要像 iMAP 那样靠关键帧回访来"唤醒记忆"。这从根本上缓解了灾难性遗忘，也加快了 mapping 速度——每次迭代只优化一小部分参数。

第三，**多尺度几何**。粗-中-细三级残差表示让系统同时具备大尺度结构感和局部细节。粗层级捕捉墙壁、地板的宏观布局，细层级重建家具表面的纹理和棱角。

没有解决什么？

第一，**内存开销高**。特征网格的内存随场景体积线性增长，而且三个层级 + 外观网格需要同时驻留显存。对于大场景，显存消耗可达数 GB，远超 iMAP 的 0.3 MB。

第二，**预训练 MLP 的泛化限制**。解码器是预训练好然后固定的，它的泛化能力受限于训练数据的分布。遇到与训练集差异较大的场景（如室外、特殊材质），解码器可能产生系统性的几何偏差。

第三，**仍然没有闭环检测**。NICE-SLAM 是纯增量式系统，没有闭环检测和全局位姿图优化。长轨迹上的漂移会累积，而且一旦位姿出错，错误会被固化到特征网格中难以纠正。

第四，**实时性仍有差距**。虽然比 iMAP 快，但体渲染的开销依然很大。Mapping 频率约 2~5 Hz，tracking 约 10 Hz，勉强能算"准实时"，但离传统 SLAM 的 30 Hz 还有距离。

系统架构

NICE-SLAM 的系统流程可以概括为：前端接收 RGB-D 帧 → tracking 模块固定地图优化位姿 → mapping 模块在视锥体内更新特征网格 → 定期从隐式表示中提取网格表面。

模块	方法	频率
Tracking	可微渲染 + 位姿优化	~10 Hz
Mapping (coarse)	视锥体内粗网格特征更新	~2 Hz
Mapping (mid+ fine)	视锥体内中细网格联合更新	~2 Hz
Appearance	颜色网格独立优化	~2 Hz
网格提取	Marching Cubes	按需

NICE-SLAM 的 ATE（绝对轨迹误差）在 Replica 数据集上约为 iMAP 的 1/3~1/2，重建的网格完整性也显著优于 iMAP。但它的显存占用是 iMAP 的 10 倍以上。这是混合表示的固有代价：用空间换时间，用内存换容量。

9.4 NeRF-SLAM 的后续演进

NICE-SLAM 之后，神经隐式 SLAM 进入了快速发展期。各路线围绕"如何把神经隐式表示做得更快、更大、更鲁棒"展开竞争。

Vox-Fusion：体素与隐式表示的动态结合

Vox-Fusion (ISMAR 2022) 的核心思路是用八叉树（octree）动态管理体素分配。与 NICE-SLAM 预分配固定大小特征网格不同，Vox-Fusion 从一个小包围盒开始，当相机移动到新区域时，八叉树动态扩展，只有在有表面观测的位置分配新的体素特征。这种按需分配策略让系统能处理未知边界的场景——不需要预先知道房间有多大。

Vox-Fusion 的另一个特点是每个体素内部存储一个局部隐式表示，结合全局坐标输入 MLP 解码。这种"局部编码 + 全局解码"的混合结构，既保持了局部更新的灵活性，又借助全局 MLP 获得跨体素的连续性。实验表明，Vox-Fusion 在跟踪精度上略优于 NICE-SLAM，且内存使用更灵活——小场景只分配少量体素，大场景才扩展。

Co-SLAM：坐标编码与稀疏参数编码的联合

Co-SLAM (CVPR 2023) 采用了坐标编码（coordinate encoding）和稀疏参数编码（sparse parametric encoding）的联合策略。它使用一维哈希表存储多分辨率特征（借鉴 Instant-NGP 的多分辨率哈希编码），结合坐标基函数的三周期激活，实现了快速收敛和高保真重建。Co-SLAM 的跟踪速度达到 15~20 Hz，mapping 速度约 5 Hz，是当时最快的神经隐式 SLAM 之一。

ESLAM：特征平面替代特征网格

ESLAM (CVPR 2023) 把三维特征网格换成了多尺度轴对齐特征平面 (axis-aligned feature planes)，将三维表示压缩为二维平面的组合。这种从空间到平面的降维，大幅降低了显存占用（从 $O(N^3)$ 降到 $O(N^2)$ ），同时保持了足够的几何表达能力。ESLAM 用 TSDF（截断符号距离函数）替代占据概率，几何表示更加直接。

对比与总结

方法	表示方式	更新机制	场景规模	跟踪 Hz	主要优势
iMAP	单一 MLP	全局权重更新	桌面级	~10	紧凑、端到端
NICE-SLAM	多分辨率特征网格	视锥体局部更新	房间级	~10	多尺度、可扩展
Vox-Fusion	八叉树 + 体素特征	按需动态扩展	多层建筑	~10	未知边界、灵活内存
Co-SLAM	哈希编码 + 坐标编码	稀疏参数更新	大室内	~15	速度快、内存省
ESLAM	轴对齐特征平面	平面特征更新	大室内	~10	显存极低

上表清晰展示了神经隐式 SLAM 的演进脉络：从单一 MLP (iMAP) 到预分配网格 (NICE-SLAM)，再到动态结构 (Vox-Fusion)，最后到稀疏编码 (Co-SLAM、ESLAM)。每一代都在解决前一代的内存或效率瓶颈，但体渲染的计算开销始终是所有方法共同的天花板。

9.5 NeRF 为 SLAM 带来了什么

在讨论 NeRF-SLAM 的瓶颈之前，有必要先回答一个问题：NeRF 究竟给 SLAM 带来了哪些传统方法不具备的能力？归纳起来是三重贡献。

第一，连续可微的场景表示。

传统 SLAM 的地图是离散的：点云有分辨率限制，体素网格有尺寸限制，mesh 有三角面片粒度限制。NeRF 把场景表示为连续函数，理论上可以查询任意分辨率。更重要的是，这个表示是**可微的**——你可以通过比较渲染图像和真实图像的差异，反向传播梯度去优化地图和位姿。这种端到端的可微性，让 SLAM 系统第一次实现了几何、外观、位姿在同一个目标函数下的联合优化。传统方法的 tracking 和 mapping 通常是两个独立模块；神经隐式 SLAM 里，它们是同一个优化问题的不同变量。

第二，新视角合成能力。

传统 SLAM 建完图后，地图只能用于定位和路径规划。NeRF-SLAM 建好的隐式模型，可以从任意视角渲染逼真的彩色图和深度图。这意味着机器人建好一个房间的地图后，用户可以“飞”到任何角度查看场景——即使那个视角机器人从未实际到达过。这种能力对 AR/VR、远程巡检、数字孪生等应用至关重要。

第三，几何与外观的联合建模。

传统稠密 SLAM（如 KinectFusion）只建模几何（TSDF 或占据），颜色信息在建图后就丢掉了。NeRF-SLAM 同时建模几何（密度/SDF）和外观（颜色），而且外观可以与视角相关——同一点从不同角度看颜色可以不同，因为 MLP 接收视角方向作为输入。这让重建结果不仅几何准确，而且视觉逼真。

这三重贡献构成了神经隐式 SLAM 的价值基石。它们不是工程上的微调，而是**表示层面**的根本变革。即使后续方法（如 3D Gaussian Splatting）在具体实现上取代了 NeRF 的 MLP + 体渲染，这三种能力——连续可微性、新视角合成、几何外观联合优化——仍然是核心追求。

9.6 NeRF-SLAM 的瓶颈：优雅但不实用

NeRF-SLAM 的方向是对的，但 NeRF 的实现方式有问题。这个领域的所有方法都面临三个绕不过去的瓶颈。

瓶颈一：体渲染太慢。

这是最根本的问题。渲染一个像素需要从相机光心出发沿光线采样数十到数百个三维点，每个点都要查询一次网络（MLP 前向传播），然后做 alpha 合成。对于 640×480 的图像，这意味着每帧要执行数百万次 MLP 查询。在 GPU 上，这需要数百毫秒。

后续方法用各种手段加速：稀疏参数编码（Co-SLAM 的哈希网格）、特征平面（ESLAM）、预计算体素值（Plenoxel）。这些优化把 mapping 速度从 iMAP 的 2 Hz 提升到 5~10 Hz，但**每条光线仍需沿深度方向采样并积分**。这个过程是 $O(N \cdot H \cdot W)$ 的复杂度，其中 N 是每光线采样点数。相比之下，传统光栅化是 $O(P)$ ， P 是可见基元数量，与图像分辨率无关。

Instant-NGP（NVIDIA，2022）用多分辨率哈希编码把 NeRF 训练加速到了分钟级，但它仍然依赖光线行进的体渲染管线。在需要每帧都重新渲染的在线 SLAM 中，这种开销无法忽视。

瓶颈二：在线更新的困境。

NeRF 的训练需要大量迭代才能收敛。离线 NeRF 通常训练数万到数十万步；SLAM 中每帧只能做几次梯度更新。这种极端的数据稀缺性导致神经隐式 SLAM 的地图质量严重依赖关键帧缓冲和回放策略。iMAP 靠回访旧关键帧缓解遗忘，NICE-SLAM 靠局部特征网格减少更新范围，但两者都没有从根本上解决“在线学习”这个难题。

更严重的是，一旦位姿估计出错，错误的观测会被写进特征网格或网络权重，后续很难纠正。传统 SLAM 有误差可以被回环检测修正；神经隐式表示把误差固化在高维参数空间里，回环修改变得异常困难。

瓶颈三：显存与场景规模的矛盾。

iMAP 的 MLP 只有 0.3 MB，但它的容量只够桌面场景。NICE-SLAM 能做多房间场景，但显存占用暴增到数 GB。Co-SLAM 用哈希编码压缩，但特征表本身仍随场景体积增长。要表示城市级别的场景，这些方法要么牺牲精度降低网格分辨率，要么耗尽 GPU 显存。

根本矛盾在于：**神经隐式表示的表达能力来自可学习参数的数量，而参数数量直接决定内存占用和渲染开销**。你不可能同时拥有一个细节丰富、速度极快、内存极小的神经隐式地图。

一句话总结

NeRF 表达优雅——连续、可微、紧凑——但它的核心操作（光线行进 + 逐点网络查询）与实时 SLAM 的需求存在结构性矛盾。所有后续的加速手段（哈希编码、特征平面、稀疏参数）都是在缓解症状，而非根除病因。

这个瓶颈为下一章埋下了伏笔。2023 年，3D Gaussian Splatting (3DGS) 的出现彻底改变了局面：它用显式的三维高斯原语替代隐式的 MLP，用光栅化替代光线行进，用 alpha 合成替代体渲染积分。3DGS 在保持新视角合成质量的同时，把渲染速度提升了 1~2 个数量级。神经隐式 SLAM 花三年时间没解决的实时性问题，3DGS-SLAM 一上来就给了完全不同的答案。第 10 章，我们进入 3D Gaussian Splatting 的时代。

本章一句话总结

iMAP 证明 MLP 可以做实时 SLAM 地图，NICE-SLAM 证明局部多层次编码可以扩展它，但 NeRF 的体渲染从根本上限制了速度；3D Gaussian Splatting 正在取代这条路线。

第三篇：3D Gaussian Splatting SLAM：2024-2026 的主战场

第10章 3D Gaussian Splatting 为什么改变 SLAM？

第9章的结论是：NeRF-SLAM 受困于体渲染的 $O(N \cdot H \cdot W)$ 复杂度和在线更新困境。训练慢、渲染慢、地图难修改，这三座大山让 NeRF 更适合离线重建而非在线 SLAM。

2023 年 8 月，Kerbl 等人发表了 3D Gaussian Splatting (3DGS)。它用光栅化 (rasterization) 替代光线行进 (ray marching)，在保持 NeRF 级视觉质量的同时，将渲染速度提升了一到两个数量级。更关键的是，3DGS 是一种**显式表示**——场景由一组可独立操作的三维高斯椭球组成，每个高斯可被增删、移动、修改。这一特性让 3DGS 天然更适合 SLAM 的增量式建图需求。

一年之内，SplaTAM、MonoGS、Photo-SLAM、GS-SLAM 等系统相继出现，3DGS-SLAM 迅速成为视觉 SLAM 最活跃的研究方向 (arXiv)。本章回答三个问题：3DGS 到底是什么？它为什么比 NeRF 更适合 SLAM？以及——它为什么还不是“现成的 SLAM”？

10.1 3DGS 的一句话解释

3DGS 用一组三维高斯椭球表示场景。每个高斯携带以下可学习属性：

- **位置** $\mu \in \mathbb{R}^3$ ：高斯中心在世界坐标系中的三维坐标。
- **协方差矩阵** $\Sigma \in \mathbb{R}^{3 \times 3}$ ：描述高斯的形状和朝向。为保证正半定且可微优化，协方差分解为旋转矩阵 $\mathbf{R}$ （四元数参数化）和对角缩放矩阵 $\mathbf{S} = \text{diag}(s_x, s_y, s_z)$ ，即 $\Sigma = \mathbf{R} \mathbf{S} \mathbf{S}^\top \mathbf{R}^\top$ 。缩放因子 (s_x, s_y, s_z) 决定椭球的三个半轴长度——小的缩放产生尖锐的细节，大的缩放产生平滑的区域填充。
- **不透明度** $\alpha \in [0, 1]$ ：控制高斯对像素的贡献权重。 $\alpha = 0$ 时高斯完全透明， $\alpha = 1$ 时完全不透明。

- **球谐系数** (Spherical Harmonics, SH) $\mathbf{c} \in \mathbb{R}^{3(l+1)^2}$ ：编码视角相关的颜色， l 为 SH 阶数。 $l = 0$ 时退化为单一 RGB 颜色（与视角无关）， $l = 3$ 时可捕捉复杂的镜面反射和视角依赖效果。原始论文使用 $l = 3$ ，共 48 个系数。

渲染时，3D 高斯经历以下步骤投影到图像平面：

第一步：投影。给定世界到相机的变换矩阵 $\mathbf{W}$ 和投影变换的雅可比矩阵 $\mathbf{J}$ ，3D 高斯的协方差通过仿射近似投影到 2D (Zwicker 等人 2001)：

$$\Sigma^{2D} = \mathbf{JW} \Sigma \mathbf{W}^\top \mathbf{J}^\top$$

第二步：Tile 分配。屏幕划分为 16×16 像素的 tile，每个高斯根据投影后的 2D 位置和协方差确定其覆盖的 tile 集合。

第三步：深度排序。每个 tile 内的高斯按视图空间深度从前往后排序。

第四步： α -混合。对像素 $\mathbf{p}$ ，其颜色由覆盖该像素的所有高斯按深度顺序累积贡献：

$$C(\mathbf{p}) = \sum_{i \in \mathcal{N}} c_i \sigma_i \prod_{j=1}^{i-1} (1 - \sigma_j)$$

其中 $\sigma_i = \alpha_i \exp\left(-\frac{1}{2} \mathbf{x}'^\top \Sigma_i'^{-1} \mathbf{x}'\right)$ 是第 i 个高斯在该像素处的有效不透明度， $\mathbf{x}'$ 是像素在 2D 高斯局部坐标系中的偏移向量。当累积 α 达到饱和（接近 1）时，渲染器提前终止，避免处理被遮挡的高斯。

整个流程在 GPU 上通过 tile-based 光栅器实现，前向渲染和反向梯度传播都在毫秒级完成 (SIGGRAPH)。这与 NeRF 的“对每个像素沿光线采样 64-256 个点、每个点过一次 MLP”形成鲜明对比——3DGS 的渲染复杂度是 $O(G)$ ， G 为视锥内高斯数量，且投影后可被 tile 裁剪和 early-out 加速。这是 3DGS 渲染速度快一到两个数量级的根本原因。

10.2 它为什么适合 SLAM?

3DGS 与 NeRF 的差异不仅是“快”，更是表示范式的根本转变。以下四个维度决定 3DGS 更适合 SLAM。

10.2.1 显式表示 vs 隐式表示

NeRF 的场景知识存储在 MLP 权重中。查询一个点的颜色或密度，必须前向传播网络。地图没有“零件”——修改一个局部区域需要重新训练整个网络，不可避免地波及全局。这导致了第9章讨论的灾难性遗忘问题：iMAP 在新区域训练时旧区域质量下降，需要回放 (replay) 策略缓解。

3DGS 的场景知识存储在每个高斯的独立参数中。修改一个高斯只影响其空间邻域的渲染结果。新观测到来时，只需在可见区域插入新高斯，无需触碰已建好的远处地图。这种**局部性** (locality) 是增量式 SLAM 的核心需求——传统 SLAM 的点云、体素、surfel 表示都具备这一性质，3DGS 是唯一同时具备局部性和照片级渲染质量的表示。

10.2.2 渲染速度：光栅化 vs 光线行进

NeRF 的体渲染沿每条光线采样 N 个点（通常 $N=64\sim 256$ ），每个点执行一次 MLP 前向传播。渲染一张 $H \times W$ 的图像需要 $O(N \cdot H \cdot W)$ 次网络查询。即使使用哈希编码加速（Instant-NGP），光线行进的内存带宽和计算开销仍然巨大。Point-SLAM 用神经点云加速，渲染速度也只有约 8.7 FPS。NICE-SLAM 因分辨率限制渲染速度约 2.5 FPS。

3DGS 将 3D 高斯投影到 2D，通过 tile-based 光栅器直接绘制。投影操作是解析的，无需神经网络推理。Kerbl 等人的原始实现在 1080p 分辨率下达到 30-60 FPS 渲染，而训练时间从 NeRF 的数小时缩短到数分钟 (SIGGRAPH)。后续的 gsplat 等开源实现进一步优化了内存 footprint。GSSLAM (Gaussian Splatting SLAM) 在 Replica 上报告了 769 FPS 的渲染帧率——虽然不是系统帧率，但足以说明 3DGS 渲染管线的吞吐潜力。

对 SLAM 而言，渲染速度直接影响跟踪和建图的迭代效率。基于渲染损失的位姿优化需要反复渲染中间图像并比较 photometric loss——NeRF-SLAM 每帧只能做几次迭代，3DGS-SLAM 可以做数十次甚至上百次。GS-SLAM 每帧执行 30 次跟踪迭代和 60 次建图迭代仍能保持 4 FPS 系统速度，这是 NeRF-SLAM 无法企及的。

10.2.3 局部更新的自然性

3DGS 的原始论文引入了自适应密度控制（adaptive density control）机制。观测不足的区域通过**克隆**（clone，沿梯度方向复制高斯）和**分裂**（split，将大高斯沿最大梯度方向分裂为两个较小高斯）增加高斯数量；冗余区域通过**剪枝**（prune）删除低不透明度或 oversized 的高斯。透明高斯被直接移除，以释放表示能力给更需要的地方。这套机制天然可迁移到 SLAM 场景：

- **新区域发现**：相机移动到未观测区域时，从深度图反投影或立体匹配初始化新高斯。
- **细节增强**：反复观测的区域通过分裂细化为更小的高斯，提升重建精度。
- **噪声清理**：低不透明度、大协方差、长期不可见的高斯被周期性剪枝，保持地图紧凑。

NeRF-SLAM 没有对应的“局部重采样”机制。iMAP 用全局 MLP 更新，新区域的学习会干扰旧区域的权重。NICE-SLAM 的局部网格更新受限于预定义的分辨率和网格结构，无法在需要的地方自适应增加容量。3DGS 的自适应密度控制相当于“自动调节分辨率的体素网格”——细节丰富的区域自动加密，平坦区域保持稀疏。

10.2.4 与机器人pipeline的接口友好性

3DGS 的显式结构使其更容易与机器人系统的其他模块对接。下表对比 NeRF 与 3DGS 在关键接口上的差异：

接口需求	NeRF	3DGS
点云提取	需要 Marching Cubes 从密度场提取网格，计算量大	高斯中心 μ 直接构成点云，零额外开销
语义融合	需扩展网络输出语义通道，反向传播影响整个 MLP	直接关联每个高斯的语义标签，局部更新
对象操作	隐式场难以分割和移动单个对象	高斯可按空间聚类分组编辑，移动即改参数
运动规划	从密度场提取占用信息需体素查询	高斯不透明度直接提供占用信息，碰撞检测可解析计算
多传感器融合	MLP 输入维度固定，融合方式受限	各高斯独立，可附加任意属性（语义、特征、物理参数）

Tosi 等人 2024 年的综述明确指出：3DGS 的显式体积表示、快速渲染、丰富优化能力和直接梯度流使其成为 SLAM 的强大替代方案。3DGS 既保留了 NeRF 的可微渲染优势（可通过梯度下降优化场景表示），又恢复了传统几何表示的操作性（可查询、可编辑、可组合），这是它吸引 SLAM 研究者的根本原因 (arXiv)。

10.3 它为什么不天然就是 SLAM?

3DGS 是新视图合成 (NVS) 方法，不是 SLAM 方法。原始 3DGS 论文假设相机位姿由 COLMAP 预先计算，场景是静态的，所有图像一次性送入优化器。将 3DGS 变成 SLAM，需要补齐八个关键模块。每个模块都是一个独立的研究问题。

10.3.1 在线跟踪 (Online Tracking)

3DGS 原始论文假设相机位姿已知。SLAM 中，位姿是未知且需要实时估计的。基于 3DGS 的跟踪通过优化相机位姿 $\mathbf{T} \in SE(3)$ 使渲染图像与观测图像的 photometric loss 最小化：

$$\mathcal{L}_{\text{track}} = (1 - \lambda) \|\hat{I} - I\|_1 + \lambda(1 - \text{SSIM}(\hat{I}, I))$$

这要求：位姿优化必须快速收敛（每帧只有几毫秒预算）；对深度缺失（单目）或深度噪声（RGB-D）鲁棒；避免局部极小值，尤其是旋转歧义和低纹理区域。

MonoGS 和 Photo-SLAM 采用分析雅可比 (analytical Jacobian) 在 Lie 代数上直接优化位姿，收敛速度优于数值梯度。GS-SLAM 采用帧到模型 (frame-to-model) 策略，利用已建地图约束位姿。SpliTAM 引入 silhouette 损失辅助跟踪。但总体而言，纯基于渲染损失的跟踪精度仍落后于专用跟踪系统——DROID-SLAM 的 ATE 通常比 3DGS-SLAM 低一个数量级。这促使许多系统采用混合策略：外部跟踪器提供初始位姿，3DGS 渲染损失做精化。

10.3.2 新高斯生成 (Online Initialization)

离线 3DGS 从 COLMAP 点云初始化高斯，然后迭代优化。在线 SLAM 没有预先计算的稀疏点云——每帧需要决定：在哪里放置新高斯？用什么初始参数？

早期系统（如 Gaussian-SLAM）简单地将每帧深度图反投影为 3D 点并包裹为高斯。这种方式在深度噪声大或深度缺失时产生大量低质量高斯。RGS-SLAM 提出 one-shot dense initialization，从多视图对应关系（基于 MAST3R 等先验）一次性生成高质量高斯种子，跳过迭代密度控制阶段，高斯插入后立即参与建图。SplatMap 提出 SLAD（SLAM-Informed Adaptive Densification），利用 SLAM 的动态更新实时精化点云，从初始噪声点云逐步"生长"为高质量高斯地图。

10.3.3 地图增删（Map Growth Management）

高斯数量在 SLAM 过程中会无限制增长。Replica 数据集 2000 帧的序列可使高斯数量膨胀到 90 万以上，GPU 内存超过 24 GB（Feature 3DGS SLAM 的消融实验报告了精确数据）。没有有效管理，系统会在长序列上因内存耗尽而崩溃。

核心挑战在于：新增高斯的频率远高于删除高斯的频率。每次关键帧插入可能新增数百到数千高斯，但剪枝只在全局优化时执行。解决方案包括：冗余高斯检测与合并（将空间重叠、颜色相似的高斯合并为一个）；基于体素空间的高斯聚合（将同一体素内的高斯合并为代表性高斯）；可见性加权的周期性剪枝（删除长期不可见或低可见性的高斯）。实验表明，适当的管理策略可将高斯数量减少约 30-50% 同时保持渲染质量，内存从 >24 GB 降至 4-8 GB。Compact 3DGS 方法进一步通过几何码本和量化将存储降低五倍，但多为后处理优化。

10.3.4 回环一致性（Loop Closure）

这是 3DGS-SLAM 最突出的短板之一。大多数早期系统（MonoGS、Photo-SLAM、GS-SLAM）本质上是视觉里程计——没有回环检测，没有全局 Bundle Adjustment，轨迹漂移随时间累积。

实现回环的挑战在于：检测到回环后，如何将位姿修正传播到整个高斯地图？NeRF-SLAM（如 GO-SLAM）可以优化历史关键帧位姿并重新训练整个隐式地图，但这耗时巨大。3DGS 的显式结构反而让"地图更新"更复杂——每个高斯锚定在某个局部坐标系中，全局位姿调整需要联动更新所有关联高斯的位置。

GLC-SLAM 通过子地图（submap）划分和直接地图调整解决这一问题：将场景划分为锚定到全局关键帧的 3DGS 子地图，回环检测后执行位姿图优化，然后将修正结果传播到关联子地图。LoopSplat 提出 3DGS 配准方法实现高效的回环闭合和全局优化，避免昂贵的重新积分。但目前的闭环精度仍低于传统 SLAM 系统（ORB-SLAM3 的 DBoW2 + 位姿图优化），且计算开销更大。MAGIc-SLAM 探索多智能体场景下的分布式回环检测，利用基础视觉模型提升检测鲁棒性。

10.3.5 动态物体处理

现实场景包含移动的人、车、宠物。3DGS 的高斯是静态的——如果移动物体被建模为高斯，它们会在地图中留下"鬼影"（ghosting），污染跟踪和建图。更糟的是，动态高斯会吸收本应用于静态结构的表示容量。

动态物体处理需要三个步骤：检测动态区域（语义分割如 Mask R-CNN、多视图几何一致性检验）、从跟踪损失和建图中屏蔽动态高斯、在物体停止移动后重新集成。LCD-Splat 集成 Mask R-CNN 进行动态物体检测，结合改进的多视图几何约束，在 TUM RGB-D 和 Bonn 动态数据集上展示了鲁棒性。但语义分割的计算开销（在 LCD-Splat 中占每帧处理时间的 17%）使系统难以达到真正的实时性能。

10.3.6 内存控制（Memory Efficiency）

每个高斯存储：位置（3 float）、缩放（3 float）、旋转（4 float，四元数）、不透明度（1 float）、SH 系数（最高 48 float）。总计约 60 float/高斯，Adam 优化器状态下翻倍。100 万高斯约需 240 MB 参数存储，优化器状态翻倍到 480 MB，加上渲染中间变量，总 GPU 内存轻松超过数 GB。

相比 NeRF-SLAM（iMAP 的 MLP 只有数 MB，NICE-SLAM 的特征网格也只有数百 MB），3DGS-SLAM 的内存消耗是量级性的增长。GSSLAM 的高斯数量管理、Mini-Splat 的紧凑表示、Compact 3DGS 的后处理剪枝都是压缩方向，但在线内存控制仍是难题。

10.3.7 不确定性估计（Uncertainty Modeling）

SLAM 系统需要知道“什么是不确定的”——以便在信息不足时采取保守策略（如请求更多观测、降低运动速度）。传统 SLAM 有明确的不确定性模型：特征点位置有协方差矩阵，位姿估计有信息矩阵。

3DGS 的高斯参数没有不确定性度量。一个高斯的协方差 Σ 描述的是空间分布的形状（“这个高斯覆盖多大区域”），不是估计的不确定性（“这个位置估计有多可靠”）。两者在数学上完全不同。

MonoGS 尝试通过高斯的可见性分数和观测次数隐式编码可靠性。GLC-SLAM 显式建模高斯不确定性并引入不确定性最小化的关键帧选择策略。VB-GS（Variational Bayes Gaussian Splatting）从变分推断角度建模参数不确定性。但这些方法尚未形成统一框架，不确定性信息也未被下游任务（如规划、决策）有效利用。

10.3.8 长期维护（Long-term Operation）

机器人需要在数小时甚至数天的时间尺度上持续运行。这要求 SLAM 系统处理：光照变化（同一地点在不同时间的外观差异）、场景变化（家具移动、新物体出现、旧物体消失）、地图规模增长（城市级场景的高斯数量管理）、回环的时延检测（闭环可能在数小时后才被发现）。

3DGS-SLAM 在这些方向上的研究刚刚起步。Appearance embedding（外观嵌入）可以部分处理光照变化，但长期场景变化的高斯地图维护仍是一个几乎空白的领域。城市级 3DGS（如 GaussianOctree）通过层次化高斯管理大场景，但面向 SLAM 的增量式版本尚未成熟。

10.4 2026 综述给出的核心评价维度

2024-2026 年间，3DGS-SLAM 论文数量爆炸式增长，评价指标却各行其是——有的只在 Replica 上测试，有的只报渲染 PSNR，有的只关注跟踪 ATE。缺乏统一评价框架使得方法间难以公平比较。

Tosi 等人 2024 年的综述 "How NeRFs and 3D Gaussian Splatting are Reshaping SLAM" 以及后续 2026 年的 3DGS-SLAM 综述，将代表方法放在四个关键维度中比较：**渲染质量**、**跟踪精度**、**重建速度**、**内存消耗** (arXiv)。这四个维度应成为本书分析 3DGS-SLAM 的主框架。

10.4.1 渲染质量（Rendering Quality）

衡量高斯地图从新视角合成图像的视觉保真度。常用指标：

- **PSNR**（峰值信噪比，dB，越高越好）：衡量像素级重建精度。Replica 上领先方法可达 38-43 dB。
- **SSIM**（结构相似性，0-1，越高越好）：衡量感知层面的结构保留。Replica 上可达 0.97-0.99。
- **LPIPS**（学习感知图像块相似度，越低越好）：基于 AlexNet 特征的感知距离。Replica 上领先方法可

达 0.04-0.08。

下表汇总 Replica 数据集上代表性方法的渲染与跟踪表现（数据来自多论文综合 (CVF开放获取) (arXiv)）：

方法	表示	ATE↓ (cm)	PSNR↑ (dB)	SSIM↑	LPIPS↓	FPS
iMAP (ICCV'21)	NeRF MLP	2.58	—	—	—	<1
NICE-SLAM (CVPR'22)	神经隐式网格	1.06	—	—	—	~2.5
Point-SLAM (ICCV'23)	神经点云	0.52	29.43	0.935	0.235	~8.7
Co-SLAM (CVPR'23)	坐标+参数编码	1.00	—	—	—	~5
Gaussian-SLAM (arXiv'23)	3DGS	3.27	38.90	—	—	—
SplaTAM (CVPR'24)	3DGS	0.35	34.11	0.969	0.097	<1
GS-SLAM (CVPR'24)	3DGS	0.50	34.27	0.975	0.082	~4
MonoGS (CVPR'24)	3DGS	0.58	36.70	0.960	0.064	~3-5
Photo-SLAM (CVPR'24)	3DGS	0.44	34.13	0.970	0.099	>30
GSSLAM (CVPR'24)	3DGS	0.79	37.50	0.960	0.070	769
GS-ICP-SLAM	3DGS	0.16	38.86	0.970	0.049	>30

从表中可读出三个趋势。第一，3DGS-SLAM 的渲染质量（PSNR 34-39 dB）显著优于 NeRF-SLAM（Point-SLAM 29 dB），LPIPS 也普遍更低，说明 3DGS 在感知质量上更具优势。第二，3DGS-SLAM 的跟踪精度与传统 SLAM 仍有差距——GS-ICP-SLAM 通过 ICP 约束达到 0.16 cm ATE，而纯基于渲染损失的方法（MonoGS、GS-SLAM）在 0.5 cm 左右。第三，速度差异巨大：GSSLAM 通过轻量级设计达到 769 FPS，而 SplaTAM 因迭代优化策略复杂而低于 1 FPS。这说明 3DGS 的表示本身不决定速度，系统设计才是关键。

10.4.2 跟踪精度（Tracking Accuracy）

衡量相机位姿估计的准确性，通常以 ATE（Absolute Trajectory Error）的 RMSE 表示，单位 cm。

3DGS-SLAM 的跟踪策略可分为三类。**帧到模型**（Frame-to-Model）：渲染当前地图、与观测图像比较 photometric loss、优化位姿。MonoGS、GS-SLAM 采用此策略。优点是地图约束强，缺点是对初始位姿敏感、优化慢、易陷入局部极小。**帧到帧**（Frame-to-Frame）：利用外部跟踪器（DROID-SLAM、ORB-SLAM3）提供位姿，3DGS 仅负责建图。Splat-SLAM、Photo-SLAM 采用此策略。优点是跟踪鲁棒、速度快，缺点是无法利用高斯地图的渲染损失精化位姿。**混合策略**：渲染损失与几何约束联合优化。GS-ICP-SLAM 将渲染损失与 ICP 对齐结合，在 Replica 上达到 0.16 cm ATE，是目前 3DGS-SLAM 中的最优跟踪精度。

选择哪种策略取决于应用需求。AR 应用可能优先渲染质量，接受中等跟踪精度；机器人导航需要高精度跟踪，可能偏好混合策略；实时性要求高的场景可能需要帧到帧策略。

10.4.3 重建速度（Reconstruction Speed）

衡量系统处理每帧数据的吞吐量，单位为 FPS。注意区分两个速度：**系统 FPS**（整个 SLAM pipeline 的端到端帧率）和**渲染 FPS**（仅高斯渲染的速度）。一些论文只报渲染 FPS（如 GSSLAM 的 769 FPS），但这不代表系统能实时运行。

目前 3DGS-SLAM 的系统速度范围：高端（Photo-SLAM、GS-ICP-SLAM、内存高效型方法）> 30 FPS；中端（GS-SLAM、MonoGS）~3-5 FPS；低端（SplaTAM）< 1 FPS。速度的瓶颈通常不是渲染，而是高斯优化——每帧需要数十到数百次 Adam 迭代来更新高斯参数。RGS-SLAM 通过 one-shot 初始化减少建图迭代次数，显著提升 wall-clock 吞吐量。

10.4.4 内存消耗（Memory Consumption）

衡量 GPU 显存占用，单位 GB。3DGS-SLAM 的内存开销来自三方面：高斯参数存储（每个高斯约 60-70 float，Adam 状态翻倍，100 万高斯约 480 MB）、渲染中间变量（tile 排序、深度缓冲、梯度缓存）、附加特征（语义特征、外观嵌入等）。

Replica 数据集上的典型范围：无地图管理 > 24 GB；有剪枝和冗余控制 4-8 GB；紧凑型方法 2-4 GB。

Feature 3DGS SLAM 的消融实验明确显示：无地图管理时 Replica 2000 帧序列导致 912,595 个高斯，内存超过 24 GB；引入冗余剪枝后降至 281,927 个高斯（8.5 GB）；进一步引入 Top-K 选择后降至 215,857 个高斯（7.7 GB），渲染质量几乎无损失（PSNR 从 36.30 降至 35.94，仅 0.36 dB）。

10.4.5 四维度之间的张力

这四个维度并非独立，而是存在深刻的张力关系：更多的高斯和更高阶 SH 提升渲染质量但降低速度；更多优化迭代提升跟踪精度但降低帧率；大场景需要更多高斯使内存线性增长；跟踪精度和渲染质量的优化目标有时冲突（如 MonoGS 优先渲染但 ATE 0.58 cm，GS-ICP-SLAM 优先跟踪但 PSNR 达 38.86 说明联合优化的价值）。

好的 3DGS-SLAM 系统不是在一个维度上做到最好，而是在四个维度上找到适合目标应用的平衡点。嵌入式机器人平台可能需要牺牲渲染质量换取速度和内存；AR 应用优先渲染质量和跟踪精度；离线重建可以接受非实时速度换取最高质量。后续章节将按这四个维度逐一评价各代表方法。

10.5 与具身智能的关系

3DGS-SLAM 对具身智能的意义远超"更好的地图"。显式高斯表示使空间推理变得可操作。

语义-几何联合表示：每个高斯可附加语义标签、语言特征（CLIP embedding）、物理属性。Feature 3DGS SLAM 和 LangSplat 展示了在 3DGS 中联合重建几何场和语义场的可能性——机器人可以问"那个红色的物体在哪里？"并直接在地图中查询关联高斯。相比 NeRF 的语义扩展需要修改网络架构和重新训练，3DGS 的语义融合只是给每个高斯增加一个特征向量。

可交互的地图：高斯可被选择、移动、删除。这支持对象级别的操作——机器人可以识别出一组高斯构成"杯子"，然后在规划时将这组高斯作为刚体移动。神经隐式表示难以实现这种操作，因为"对象"在 MLP 权重中没有明确对应。

闭环感知-行动：3DGS 的可微渲染使"渲染-比较-优化"循环成为主动感知的工具。AG-SLAM（Active Gaussian Splatting SLAM）和 ActiveGAMER 利用渲染不确定性引导机器人移动到信息增益最大的视角，主动改善地图质量。传统 SLAM 的被动感知只是"走到哪看到哪"，主动感知则让机器人成为自己地图的"摄影师"。

实时需求：具身智能要求毫秒级响应。3DGS 的光栅化渲染比 NeRF 更适合这一需求，但目前的 3DGS-SLAM 系统速度仍不均匀——需要专门的工程优化（CUDA kernel 优化、混合精度训练、梯度裁剪）才能达到机器人部署标准。REACT3D 等工作已开始探索 3DGS-SLAM 的硬件加速。

长期自治的瓶颈：高斯数量增长、回环一致性、动态场景处理、光照变化适应等问题在机器人长期运行中会被放大。这些不是渲染问题，而是自主系统的生存问题。一个能在办公室运行 10 分钟的 3DGS-SLAM demo 和一个能在城市环境中运行 10 小时的机器人部署之间，隔着本章列出的八个关卡。

10.6 一句话总结

3D Gaussian Splatting 用显式高斯椭球替代隐式神经场，以光栅化速度克服了 NeRF 的渲染瓶颈，为 SLAM 提供了更快的优化、更灵活的局部更新和更友好的机器人接口；但高斯地图的在线初始化、回环一致性、动态物体处理、内存控制和长期维护仍是 3DGS-SLAM 需要攻克的八大关卡，而渲染质量、跟踪精度、重建速度和内存消耗四个维度的平衡将贯穿后续所有方法的评价。

第11章 早期 3DGS-SLAM：SplaTAM、MonoGS 与 Gaussian SLAM

第10章论证了 3D Gaussian Splatting（3DGS）作为场景表示的结构优势：显式参数化支持随机访问、可微光栅化带来实时渲染、 α -混合天生适合密度建模。这些特性为 SLAM 提供了全新的技术选项。2023 年底到 2024 年初，三个几乎同时出现的系统——SplaTAM、MonoGS 和 FlashSLAM——以不同的切入点验证了"地图能渲染"这一思想在 SLAM 中的工程可行性。它们共同定义了 3DGS-SLAM 的第一代范式。

这三个系统分工明确：SplaTAM 证明了 RGB-D 输入下 3DGS 可以同时做到精确跟踪和高保真建图；MonoGS 把问题推向更难的单目设定，展示了 3DGS 在无任何深度先验时的潜力；FlashSLAM 则指出纯梯度下降式跟踪的瓶颈，用特征匹配加速跟踪 pipeline。三者叠加，勾勒出早期 3DGS-SLAM 的技术版图。

11.1 SplaTAM：Splat, Track & Map 3D Gaussians

11.1.1 核心问题

Dense SLAM 需要同时估计相机轨迹和重建稠密三维场景。NeRF-SLAM 系列（如 NICE-SLAM、Point-SLAM）使用隐式或半隐式表示，渲染需要耗时的光线 march，地图扩展缺乏结构化的密度控制机制。SplaTAM 问了一个直接的问题：如果把场景表示为显式的 3D Gaussian 集合，是否能让 tracking、mapping 和 rendering 都变得更快、更可控？

答案是肯定的。SpliTAM 的地图是一张 3D Gaussian 参数表，渲染是光栅化而非光线求交，地图扩展通过"在需要的地方添加新的 Gaussian"完成。整个系统围绕可微渲染构建，把"渲染-比较-回传梯度"作为 tracking 和 mapping 的统一优化机制。

11.1.2 高斯地图表示

SpliTAM 对每个 Gaussian 做了简化：强制各向同性 (isotropic)，每个 Gaussian 仅由 8 个参数描述——位置 $\mu \in \mathbb{R}^3$ 、RGB 颜色 $c \in \mathbb{R}^3$ 、半径 $r \in \mathbb{R}^+$ 和不透明度 $o \in [0, 1]$ 。空间影响函数为：

$$f(x) = o \cdot \exp\left(-\frac{\|x - \mu\|^2}{2r^2}\right) \quad (13)$$

各向同性的牺牲是表达能力——无法描述细长的结构如电线、杆状物体。但收益也很显著：参数更少、光栅化更快、梯度传播更稳定。SpliTAM 的实验表明，对于室内 RGB-D SLAM，各向同性 Gaussian 足以重建出厘米级精度的几何。

11.1.3 Tracking：通过可微渲染优化位姿

SpliTAM 的 tracking 模块接收当前 RGB-D 帧，通过可微光栅化将当前高斯地图投影到相机坐标系，生成颜色图、深度图和轮廓图 (silhouette map)。优化目标为：

$$\mathcal{L}_{\text{track}} = \lambda_c \|C_{\text{render}} - C_{\text{obs}}\|_1 + \lambda_d \|D_{\text{render}} - D_{\text{obs}}\|_1 + \lambda_s \|S_{\text{render}} - S_{\text{obs}}\|_1 \quad (14)$$

其中轮廓图 S 是关键创新。SpliTAM 渲染一张二值化的 silhouette mask，标识"哪些像素被至少一个 opaque Gaussian 覆盖"。在跟踪时，loss 只施加在有可靠地图覆盖的像素上，避免新观测区域或空洞区域的干扰。这相当于给跟踪过程增加了一个"自信区域掩码"，大幅提升了在大运动和无纹理场景下的鲁棒性。

位姿优化在 $SE(3)$ 流形上进行，使用解析梯度通过光栅化反向传播。每帧跟踪通常需要 50-100 次迭代收敛。

11.1.4 Mapping：关键帧驱动地图更新

Mapping 线程维护一个关键帧窗口。当新关键帧被选中时，系统执行以下操作：

1. **高斯初始化**：将新关键帧的深度图反投影到 3D，在每个有效深度像素处初始化一个 Gaussian。
2. **联合优化**：固定相机位姿，在当前窗口内的所有关键帧上优化 Gaussian 参数（位置、颜色、半径、不透明度）。
3. **密化 (Densification)**：检查当前渲染误差大的区域，在这些区域添加新的 Gaussian。
4. **剪枝 (Pruning)**：移除透明度过低或对渲染贡献极小的高斯。

高斯参数通过 Adam 优化器更新。Mapping loss 与 tracking loss 形式相同，但只优化地图参数而不动位姿。

11.1.5 Silhouette 驱动的地图扩展

SplaTAM 的轮廓 mask 不仅用于 tracking，还用于控制地图扩展。当渲染的 silhouette 在某个区域为 0（即没有 Gaussian 覆盖该区域），而深度观测在该区域有有效读数时，系统将反投影这些新观测点并实例化为新的 Gaussian。这种“只在需要的地方生长”的策略避免了冗余 Gaussian 的堆积。

11.1.6 实验结果

SplaTAM 在 Replica、ScanNet、ScanNet++ 和 TUM-RGBD 四个数据集上进行了评估。在 Replica 上，SplaTAM 的 ATE RMSE 约为 0.3–0.6 cm，比 Point-SLAM 提升约 2 倍。新视角合成质量同样领先：PSNR 在 Replica office 场景达到约 35 dB，LPIPS 低至 0.07。渲染速度达到 400 FPS（分辨率 876×584 ），远超 NeRF-based SLAM 的个位数 FPS。

11.1.7 论文精读

论文试图解决什么痛点？

NeRF-SLAM 系列在 dense RGB-D SLAM 中占据主导，但隐式表示导致渲染慢、地图扩展缺乏结构化控制、无法显式判断“某区域是否已被建图”。SplaTAM 要做一个渲染快、地图扩展结构化、支持显式空间查询的 dense SLAM。

核心假设是什么？

RGB-D 输入提供足够的几何先验，使得各向同性 3D Gaussian 足以表达室内场景；可微光栅化提供的梯度足够精确，可以直接优化相机位姿。

输入输出是什么？

输入：RGB-D 图像序列，已知相机内参。输出：相机轨迹（SE(3) 位姿序列）、3D Gaussian 地图（位置和属性）、新视角合成（颜色和深度渲染）。

地图表示是什么？

各向同性 3D Gaussian 集合，每个 Gaussian 8 参数：位置 μ 、颜色 c 、半径 r 、不透明度 o 。

Tracking 怎么做？

当前帧位姿通过可微渲染与已有地图比较，最小化颜色、深度和轮廓的渲染误差，使用解析梯度在 SE(3) 上优化。轮廓 mask 限制 loss 只在有地图覆盖的可靠区域上计算。

Mapping 怎么做？

关键帧维护滑动窗口，窗口内固定位姿优化 Gaussian 参数（Adam）。新关键帧触发深度反投影初始化新的 Gaussian，之后通过渲染误差驱动的 densification 和透明度驱动的 pruning 控制地图规模。

优化目标是什么？

Tracking 和 mapping 共享相同的渲染误差 loss： ℓ_1 颜色误差 + ℓ_1 深度误差 + ℓ_1 轮廓误差，通过权重平衡。

相比前作真正推进了什么？

首次将 3DGS 引入 dense RGB-D SLAM，用显式 Gaussian 替代隐式/半隐式表示，渲染速度从几 FPS 提升到数百 FPS。轮廓 mask 同时服务于 tracking 鲁棒性和地图结构化扩展，是同时期工作中最完整的系统级设计。

没有解决什么？

各向同性 Gaussian 无法精确表达细长结构。纯梯度下降 tracking 在大运动和运动模糊下仍可能失败。没有回环检测，长序列会累积漂移。依赖 RGB-D 输入，无法处理纯 RGB 场景。

对后续研究的影响是什么？

SplaTAM 证明了 3DGS-SLAM 的基本可行性，定义了"tracking by rendering + mapping by differentiable splatting"的范式。后续工作在它的基础上引入各向异性 Gaussian、特征匹配辅助跟踪、以及回环闭合。

11.2 MonoGS / Gaussian Splatting SLAM：单目 Gaussian SLAM

11.2.1 核心问题

SplaTAM 依赖 RGB-D 输入，深度传感器提供了至关重要的几何尺度。但纯单目相机是最常见、最便宜的传感器，也是 SLAM 中最难处理的设定：没有绝对深度，尺度模糊，几何初始化困难。MonoGS 要做的是把 3DGS-SLAM 从 RGB-D 推进到单目。

MonoGS 是首个主要依赖 3D Gaussian Splatting 的单目 dense SLAM 系统，同时也支持 stereo 和 RGB-D 输入。它运行速度约为 3 FPS（单目输入），在当时已接近实时。

11.2.2 从 SfM 到 SLAM：在线高斯初始化

原始 3DGS (Kerbl et al., 2023, ACM TOG) 需要 COLMAP 提供的精确位姿作为先验，在离线状态下优化 Gaussian。MonoGS 必须在线地、从无到有地构建 Gaussian 地图，同时估计位姿。这是一个"先有鸡还是先有蛋"的问题：精确地图需要好位姿，好位姿又依赖好地图。

MonoGS 的解决方案分两步：

第一步：初始地图构建。 前两帧通过特征匹配 (SuperPoint + LightGlue) 和三角测量获得稀疏 3D 点，将这些点转换为初始 Gaussian。初始 Gaussian 具有较大的协方差（各向同性球体），半径设置为平均场景深度的 10%-20%，以表达初始深度的不确定性。

第二步：增量式 densification。 对于新关键帧，系统利用已有的多视图位姿估计深度图，在高图像梯度区域或深度不连续区域初始化新的 Gaussian。与 SplaTAM 直接反投影深度不同，MonoGS 的 densification 需要考虑深度的不确定性——新 Gaussian 的半径与其深度估计的不确定性成正比。深度不确定区域获得更大的初始半径，从而在后续优化中有更大的自由度调整。

11.2.3 Tracking：直接优化与解析 Jacobian

MonoGS 的 tracking 直接对当前帧位姿进行优化，loss 为光度误差：

$$\mathcal{L}_{\text{track}} = (1 - \lambda) \|C_{\text{render}} - C_{\text{obs}}\|_1 + \lambda(1 - \text{SSIM}(C_{\text{render}}, C_{\text{obs}})) \quad (15)$$

相比 SplaTAM，MonoGS 的 tracking 有两个关键差异：

1. **没有深度监督：**单目输入下，跟踪只能依赖光度一致性。这导致 tracking 的收敛 basin 比 RGB-D 版本

窄，对初始位姿更敏感。

2. **解析 Jacobian**：MonoGS 推导了位姿对渲染像素的解析 Jacobian（在 SE(3) 流形上），避免了数值微分的高昂开销。这是 MonoGS 能在 3 FPS 运行的关键。

11.2.4 几何正则化

单目 SLAM 面临严重的几何歧义：纹理缺乏区域可以收缩为薄片（plane collapse），细长物体会被拉成“面条”。MonoGS 引入两项正则化来对抗这些问题：

各向同性正则化（Isotropic Regularization）：惩罚过度拉伸的 Gaussian。Loss 项为：

$$\mathcal{L}_{\text{iso}} = \lambda_{\text{iso}} \sum_i \left\| \Sigma_i - \frac{\text{tr}(\Sigma_i)}{3} I \right\|_F \quad (16)$$

其中 Σ_i 是第 i 个 Gaussian 的协方差矩阵。此项强迫 Gaussian 接近球形，避免极端拉伸。

几何验证（Geometric Verification）：在 densification 时，通过多视图一致性检查新三角化点的可靠性。只有被至少两个视角以一致深度观测到的点才允许实例化为 Gaussian。

11.2.5 关键帧管理与窗口优化

MonoGS 维护一个局部关键帧窗口（通常 3-5 帧）。每个新关键帧加入窗口时，触发 densification 和一轮 joint optimization——同时优化窗口内所有关键帧的位姿和地图 Gaussian 参数。

关键帧选择策略基于视角变化：当当前帧与最后一个关键帧的平移或旋转超过阈值时，触发新关键帧。这保证了地图更新的均匀性。

11.2.6 实验结果

MonoGS 在 Replica 和 TUM-RGBD 上进行了评估（单目输入）。在 Replica 上，ATE RMSE 约为 1-2 cm，新视角合成 PSNR 达到 25 dB 以上。对于单目 SLAM，这是当时最好的结果。

MonoGS 的渲染质量尤其突出：透明物体（如玻璃杯）、细长结构（如电线）都被高保真重建。这是传统特征点法或甚至 NeRF-SLAM 难以企及的——显式 Gaussian 可以自然地表达半透明材质和复杂几何。

11.2.7 论文精读

论文试图解决什么痛点？

3DGS 在离线新视角合成中表现出色，但需要预先知道精确位姿。单目 SLAM 没有精确位姿，也没有深度图，如何在线地从零开始构建 Gaussian 地图？

核心假设是什么？

单目相机的连续帧间运动足够小，使得光度 tracking 可以收敛；多视图三角化提供的稀疏深度足以“引导”Gaussian 的增量初始化；几何正则化可以抑制单目重建固有的歧义。

输入输出是什么？

输入：单目 RGB 图像序列（也支持 stereo/RGB-D），已知内参。输出：相机轨迹（SE(3) 位姿，绝对尺度由 stereo 或 RGB-D 提供，单目下尺度不确定）、3D Gaussian 地图、新视角合成。

地图表示是什么？

各向异性 3D Gaussian，每个 Gaussian 由位置 μ 、协方差矩阵 Σ （通过缩放-旋转分解参数化）、颜色 c （球谐系数）和不透明度 α 描述。比 SplatAM 更完整的参数化，但增加了优化复杂度。

Tracking 怎么做？

固定 Gaussian 地图，通过可微光栅化渲染当前视角，最小化光度误差（L1 + SSIM），使用解析 Jacobian 在 SE(3) 上梯度下降优化位姿。

Mapping 怎么做？

关键帧窗口内的多帧联合优化，同时调整位姿和 Gaussian 参数。新关键帧触发深度估计和 densification，在深度不连续和高梯度区域插入新 Gaussian。周期性执行 opacity pruning 和 Gaussian merging 保持地图紧凑。

优化目标是什么？

Tracking: L1 颜色误差 + (1 - SSIM)。Mapping: 相同的光度误差 + 各向同性正则化 + 深度平滑项（RGB-D 模式下）。

相比前作真正推进了什么？

首次实现单目 3DGS-SLAM，突破了 3DGS 依赖离线 SfM 位姿的限制。解析 Jacobian 让 tracking 速度提升数个量级。几何正则化和验证机制使得单目下的 Gaussian densification 成为可能。

没有解决什么？

3 FPS 的速度不够实时。单目 tracking 的收敛 basin 窄，大运动下容易失败。没有回环检测。单目尺度模糊问题未根本解决。densification 策略仍较启发式。

对后续研究的影响是什么？

MonoGS 证明 3DGS 可以处理最困难的单目 SLAM 设定，启发了大量后续工作。它的解析 Jacobian 和几何正则化成为后续 3DGS-SLAM 的标准组件。它也揭示了速度瓶颈：每帧数十次可微渲染迭代太慢，需要更快的跟踪策略。

11.3 FlashSLAM：加速 RGB-D Gaussian SLAM

11.3.1 核心问题

SplatAM 和 MonoGS 的 tracking 都依赖梯度下降迭代——每帧需要 50–100 次可微渲染和反向传播。这两个场景下成为瓶颈：

1. **稀疏视角**：当帧率降低（如每隔 5–10 帧取一帧），相邻帧间的运动增大，梯度下降需要更多迭代才能收敛，甚至可能陷入局部最优。
2. **大相机运动**：快速旋转或平移时，渲染图像与观测差异巨大，光度 loss 的 basin of attraction 不足以拉回正确位姿。

FlashSLAM 的洞察是：tracking 和 mapping 应该解耦。Tracking 负责快速估计位姿，不需要可微渲染；Mapping 负责高保真 Gaussian 优化，可以较慢进行。

11.3.2 Tracking：特征匹配 + 点云注册

FlashSLAM 的 tracking 模块不使用可微渲染。它采用了一个预训练的特征匹配网络（LoFTR），在当前帧与参考关键帧之间提取稀疏但可靠的特征匹配点。然后将这些匹配点提升到 3D（使用深度图），通过点云注册（Point Cloud Registration，具体使用 Kabsch 算法）直接求解相对位姿。

具体流程为：

1. **特征匹配**：当前帧与最近的参考关键帧通过预训练 LoFTR 网络提取密集对应点，再用 RANSAC 剔除外点。
2. **3D 点云构建**：利用深度图将匹配点反投影为两组 3D 点云。
3. **点云注册**：通过 Kabsch 算法求解两组点云之间的刚性变换（SE(3)）。
4. **多帧验证**：将估计的位姿与多个参考关键帧交叉验证，取一致性最好的结果。

这一流程的核心优势在于速度：特征匹配 + 点云注册的耗时约为 80 ms，比 SplaTAM 的梯度下降 tracking 快 90%。更重要的是，特征匹配对大运动不敏感——只要两帧之间有可匹配的特征，即使运动很大也能获得合理的初始对应。相比之下，梯度下降 tracking 的 basin of attraction 受限于帧间运动幅度，大运动时渲染图像与观测差异巨大，loss landscape 可能不存在通往全局最优的梯度路径。

11.3.3 Mapping：高斯优化与深度噪声建模

FlashSLAM 的 mapping 线程与 SplaTAM 类似：使用可微渲染优化 Gaussian 参数。但 FlashSLAM 增加了一个关键组件——深度噪声建模。

消费级 RGB-D 相机（如 iPhone 的 LiDAR）的深度图存在显著的噪声，特别是在边缘和远距离区域。FlashSLAM 在 tracking 中对深度进行不确定性加权，在 mapping 中对深度 loss 施加鲁棒核函数（Huber loss）。这使得它在低质量深度输入下仍能保持稳定。

11.3.4 实验结果

FlashSLAM 在 Replica、TUM-RGBD、ScanNet 和 ScanNet++ 上进行了评估。在 Replica 上平均 ATE 为 0.55 cm，与 SplaTAM 相当。在稀疏设置下（每 5 帧取一帧），FlashSLAM 的跟踪精度比 SplaTAM 提升 92%。Tracking 时间稳定在 80 ms 以下。

FlashSLAM 还展示了在 iPhone 采集的真实数据上的鲁棒性，验证了消费级设备上的实用性。

11.3.5 评价

FlashSLAM 的价值在于揭示了早期 3DGS-SLAM 的一个关键设计抉择：tracking 是否必须与 rendering 绑定？FlashSLAM 的答案是否定的。通过特征匹配和点云注册解耦 tracking，系统在速度和鲁棒性上获得了显著提升。

但这种解耦也有代价：特征匹配在低纹理区域可能失败（如 FlashSLAM 在 Replica office4 上的跟踪误差增大）；tracking 不再直接优化渲染质量，可能导致 tracking 和 mapping 之间的小不一致。后续工作（如第 12 章将讨论的）尝试在两者之间找到更好的平衡。

11.4 系统模式总结：第一代 3DGS-SLAM 的共性结构

SplaTAM、MonoGS 和 FlashSLAM 虽然侧重点不同，但共同定义了 3DGS-SLAM 的第一代系统架构。本节提炼五个核心模式，它们构成后续章节的技术基础。

11.4.1 Tracking by Rendering

这三个系统都验证了"通过渲染来跟踪"的核心思想：将当前地图渲染到相机视角，与观测图像比较，通过渲染误差反向传播到位姿参数。

系统	Tracking 监督信号	优化方法	平均耗时/帧
SplaTAM	颜色 + 深度 + 轮廓	梯度下降 (SE(3))	~2.7 s
MonoGS	颜色 (L1 + SSIM)	梯度下降 + 解析 Jacobian	~0.33 s
FlashSLAM	无渲染 (特征匹配+点云注册)	Kabsch/ICP	~0.08 s

分析： 纯梯度下降 tracking (SplaTAM) 最通用但最慢，每帧需要数十到上百次渲染迭代。解析 Jacobian (MonoGS) 将速度提升约 8 倍，但仍受限渲染开销。FlashSLAM 彻底绕过渲染做 tracking，速度最快，但依赖特征匹配的质量。这个 trade-off 成为后续 3DGS-SLAM 研究的主线：如何在保持渲染质量的同时加速 tracking。

11.4.2 Mapping by Differentiable Splatting

Mapping 是三个系统共同使用可微渲染的环节。固定相机位姿后，通过渲染误差优化 Gaussian 参数（位置、协方差、颜色、不透明度）。

可微 splatting 相比 NeRF 的光线 march 有两个结构性优势：

1. **速度快：** 光栅化复杂度与像素数线性相关，而非与采样点数相关。渲染速度从 NeRF 的几 FPS 提升到 3DGS 的数百 FPS。
2. **梯度精确：** 每个像素对可见 Gaussian 参数的梯度有闭式表达，避免了体渲染中采样噪声带来的梯度方差。

Mapping loss 通常包括颜色项 $\mathcal{L}_{\text{color}}$ 、深度项 $\mathcal{L}_{\text{depth}}$ (RGB-D 模式) 和正则化项 $\mathcal{L}_{\text{reg}}$ ：

$$\mathcal{L}_{\text{map}} = \lambda_c \mathcal{L}_{\text{color}} + \lambda_d \mathcal{L}_{\text{depth}} + \lambda_r \mathcal{L}_{\text{reg}} \tag{17}$$

11.4.3 Keyframe-Based Optimization

三个系统都采用关键帧驱动的优化策略。关键帧选择通常基于视角变化（平移/旋转阈值）或时间间隔。非关键帧只用于 tracking 不参与 mapping，从而控制计算量。

关键帧窗口通常维护 3-5 帧，在窗口内执行 joint optimization（同时优化位姿和地图）。窗口外的关键帧被固定，不再参与优化。这种滑动窗口机制平衡了计算效率和地图一致性。

11.4.4 Gaussian Densification and Pruning

地图的"生长"和"修剪"是 3DGS-SLAM 的核心操作：

Densification（密化）：在渲染误差大、地图覆盖不足的区域添加新的 Gaussian。SplaTAM 通过 silhouette mask 识别未覆盖区域；MonoGS 在高图像梯度和深度不连续处插入；FlashSLAM 遵循类似的误差驱动策略。

具体操作中，densification 通常包括两个子操作：

- **克隆（Clone）**：对一个已有 Gaussian 进行复制并微调位置。
- **分裂（Split）**：将一个大 Gaussian 替换为两个更小的 Gaussian（通常沿最大梯度方向）。

Pruning（剪枝）：移除对渲染贡献极低的 Gaussian。通常基于透明度阈值（ $\alpha < \alpha_{\min}$ ）或可见性计数（从未被观测到的 Gaussian）。

操作	触发条件	效果
克隆	位置梯度大且 Gaussian 小	在精细区域增加密度
分裂	投影半径超过阈值	用多个小 Gaussian 替代大 Gaussian
剪枝	透明度 < 阈值 或 观测次数 < 阈值	移除漂浮物和冗余 Gaussian

11.4.5 Pose-Map Joint Optimization

在关键帧时刻，系统执行 pose-map joint optimization：同时优化关键帧位姿和 Gaussian 参数。这与传统 SLAM 的 BA（Bundle Adjustment）本质相同，只是把路标点替换为 Gaussian 参数。

Joint optimization 的 loss 为所有关键帧上的渲染误差之和：

$$\mathcal{L}_{\text{joint}} = \sum_{k \in \mathcal{W}} \|C_{\text{render}}(G, T_k) - C_{\text{obs},k}\|_1 \tag{18}$$

其中 $\mathcal{W}$ 是关键帧窗口， G 是所有 Gaussian 参数， T_k 是第 k 帧的位姿。梯度同时对 G 和 T_k 传播，实现 pose 和 map 的联合精修。

这种联合优化是 3DGS-SLAM 与传统特征点法 SLAM 的核心差异：传统 BA 优化 3D 点位置，而 3DGS-SLAM 优化 Gaussian 的完整参数集（位置、协方差、颜色、不透明度），优化变量更多但也更具表达力。

11.4.6 系统级对比

维度	SplaTAM	MonoGS	FlashSLAM
输入	RGB-D	单目/RGB-D/Stereo	RGB-D
高斯类型	各向同性	各向异性	各向异性
Tracking 方法	梯度下降渲染	梯度下降 + 解析 Jacobian	特征匹配 + Kabsch
Mapping 方法	可微 splatting	可微 splatting	可微 splatting
Densification	深度反投影 + Silhouette	深度估计 + 梯度区域	深度反投影 + 误差驱动
关键机制	Silhouette mask	解析 Jacobian + 各向同性正则	深度噪声建模
Replica ATE	~0.3–0.6 cm	~1–2 cm	~0.55 cm
Tracking 速度	~2.7 s/帧	~0.33 s/帧	~0.08 s/帧
渲染速度	~400 FPS	~3 FPS 实时	未报告
回环检测	无	无	无

分析：三个系统覆盖了从 RGB-D 到单目、从纯梯度下降到特征辅助的不同设计空间。SplaTAM 在 RGB-D 条件下 tracking 和 mapping 精度最高，但速度最慢。MonoGS 突破了单目限制，但速度仅为 3 FPS。FlashSLAM 速度最快，但跟踪精度受限于特征匹配质量。三者都没有回环检测，这是第一代系统的最大共同短板。

11.4.7 第一代系统的共同局限

三个系统共享若干根本性局限：

没有回环检测。长序列运行时，tracking drift 会累积。这是第一代 3DGS-SLAM 的最大短板，后续工作通过引入子图管理和回环闭合解决。

高斯管理依赖启发式阈值。Densification 和 pruning 的阈值通常手动设定，对不同场景的适应性差。自适应阈值和基于学习的策略是后续研究方向。

跟踪与建图速度不匹配。Mapping 通常比 tracking 慢一个数量级。在 SplaTAM 中，mapping 每帧约 4.9 s，tracking 约 2.7 s，均远非实时。MonoGS 的 3 FPS 和 FlashSLAM 的跟踪提速缓解了这一问题，但 mapping 速度仍是瓶颈。

缺乏全局一致性约束。滑动窗口 optimization 只保证局部一致性，无法纠正早期的 pose drift。全局 BA 或 pose graph optimization 的缺失限制了可重建的场景规模。

11.5 与具身智能的关系

3DGS-SLAM 的第一代系统为具身智能提供了前所未有的地图表示。传统 SLAM 输出点云或网格，机器人很难直接利用。3DGS 地图可以实时渲染出新视角的图像和深度，这为机器人规划提供了两个独特能力：

1. **视觉想象 (Visual Imaging)**：机器人可以在地图上"假想"一个视角，渲染出该视角的 RGB-D 图像，用于路径规划或目标检测的模拟。
2. **可微交互 (Differentiable Interaction)**：由于地图参数可微，机器人可以把下游任务的目标（如"看到某个物体"）反向传播为对相机路径或地图的梯度，实现端到端的主动感知。

但第一代系统的速度限制了这种潜力。3 FPS (MonoGS) 或 2.7 s/帧 (SplaTAM tracking) 无法满足机器人实时控制的需求。FlashSLAM 的提速方向是正确的，但 mapping 仍不够快。第 12 章将讨论 2025 年的工程化进展，这些进展使 3DGS-SLAM 的速度和鲁棒性接近实际部署的要求。

本章一句话总结：SplaTAM、MonoGS 和 FlashSLAM 共同证明 3D Gaussian Splatting 可以作为 SLAM 的统一场景表示，但它们的速度瓶颈和缺失的回环检测也明确指出了下一代系统需要攻克的方向。

第 12 章 2025 年 3DGS-SLAM 的工程化：快、稳、省、准

12.1 2024 → 2025：问题变了

2024 年的核心问题是：3D Gaussian Splatting (3DGS) 能不能做 SLAM？

SplaTAM 用 RGB-D 输入证明了 3DGS 可以实时重建；MonoGS 把输入降到单目 RGB；Photo-SLAM 把 ORB-SLAM3 的跟踪与 3DGS 的建图解耦，跑出 1000 FPS 的跟踪速度。到 2024 年底，答案已经明确：3DGS 不仅能做 SLAM，而且在渲染质量和速度上显著优于 NeRF-based 方案。

2025 年的问题变成了另一组更为刁钻的工程问题：

- **能不能实时？** MonoGS 的跟踪加建图每秒只有 2-3 帧，离真正的实时应用还差一个数量级。
- **能不能单目？** 大多数方法依赖深度传感器，纯 RGB 输入下尺度漂移、几何精度不足。
- **能不能处理动态物体？** 真实场景中行人、车辆、移动的家具无处不在，静态假设在室外和室内都站不住脚。
- **能不能省内存？** 3DGS 的显式表示导致高斯数量随场景增长失控，大场景下显存爆炸。
- **能不能与回环和全局优化结合？** 3D Gaussians 是显式点云，全局 BA 后地图怎么跟着动？

这五个问题构成 2025 年 3DGS-SLAM 工程化的主线：快（实时率）、稳（大运动和动态下的鲁棒性）、省（内存和计算）、准（跟踪精度和重建质量）。本章围绕这五个问题展开，而不是按论文逐一罗列。

维度	2024 年关注点	2025 年关注点
核心问题	"能不能做？"	"能不能快、稳、省、准？"
传感器	RGB-D 为主	纯 RGB 成为主攻方向
速度	渲染快，跟踪慢	端到端实时
动态场景	几乎不考虑	核心工程问题
回环优化	大多数方法缺失	可变形地图成为关键
内存控制	未重视	高斯数量控制成为瓶颈

上表概括了两年间关注点的迁移。2024 年的方法验证了可行性，但大多停留在实验室环境；2025 年的工作直面部署瓶颈，把 3DGS-SLAM 从概念推向可用。

12.2 MonoGS++：让单目 RGB SLAM 快起来

MonoGS++ (arXiv) 是 2025 年单目 3DGS-SLAM 效率与准确性改进的代表。它回答的核心问题是：**只用 RGB 输入，能不能同时做到快速和准确？**

核心假设：跟踪和建图不需要耦合在同一个优化循环里。把姿态估计交给专门的光流 VO 前端，建图模块专注于高质量 Gaussian 优化，两者解耦后整体速度大幅提升。

输入输出：纯单目 RGB 视频流；输出相机轨迹和 3D Gaussian 场景地图。

系统结构：MonoGS++ 采用解耦架构。跟踪前端使用 DPVO（Deep Patch Visual Odometry）——一种基于循环网络的光流 VO，在线输出稀疏点云和相机位姿。建图模块接收 VO 的位姿和稀疏深度，在此基础上优化 3D Gaussian 地图。解耦后，建图不再需要等待每帧的联合位姿优化，tracking 和 mapping 可以异步运行。

三个关键改进：

- 1. **动态 3D Gaussian 插入。**传统方法每帧均匀地在新观察区域添加高斯，导致已重建良好的区域被冗余高斯填满。MonoGS++ 只在重建质量不足的区域插入新 Gaussian，避免重复造轮子。
- 2. **清晰度增强的 Gaussian Densification。**弱纹理区域（白墙、桌面）的重建一直是单目 SLAM 的痛点。MonoGS++ 引入清晰度评估模块，在这些区域进行更有针对性的 densification，而不是盲目分裂高斯。
- 3. **平面正则化。**室内场景中平面占主导地位。加入平面正则项后，弱纹理平面的几何精度显著提升，LPIPS 指标改善最为明显。

效果：在 Replica 和 TUM-RGBD 数据集上，MonoGS++ 的跟踪精度与 SOTA 相当；在 TUM-RGBD 上相比 MonoGS，PSNR 提升 5.21dB，ATE 降低 28.85cm，FPS 提升 5.57 倍。动态插入策略使高斯数量减少到原来的约三分之一，是速度提升的核心因素。

没有解决什么：MonoGS++ 仍然没有回环检测和全局优化，长轨迹下漂移无法纠正。DPVO 的尺度估计依赖初始化，大尺度场景下尺度漂移问题未根本解决。动态物体不在考虑范围内。

12.3 Dy3DGS-SLAM：动态环境下的纯 RGB 方案

Dy3DGS-SLAM (arXiv) 是首个面向动态场景的单目 RGB 3DGS-SLAM 系统。它回答的问题是：**场景里有移动的人或物体，纯 RGB 输入下 SLAM 怎么不崩？**

核心假设：光流和单目深度两个互补的线索，可以通过概率模型融合成更可靠的动态掩码，不需要预设语义类别或依赖深度传感器。

输入输出：纯单目 RGB 视频；输出相机轨迹和静态场景 3D Gaussian 地图（动态物体被排除在地图外）。

动态掩码融合：这是 Dy3DGS-SLAM 的核心创新。系统同时运行两个并行的掩码生成管道：

- **光流掩码 F_m ：**用轻量 U-Net 估计相邻帧的光流，高光流区域标记为潜在动态。
- **深度掩码 D_m ：**用单目深度网络（如 DepthAnythingV2）预测深度，通过几何一致性检测异常区域。

两个掩码通过贝叶斯模型融合：

$$P(M(p) = 1 \mid D_m, F_m) = \frac{P(D_m|1) \cdot P(F_m|1) \cdot P(1)}{\sum_{m \in \{0,1\}} P(D_m|m) \cdot P(F_m|m) \cdot P(m)}$$

其中 $M(p) = 1$ 表示像素 p 属于动态区域。融合后的掩码对光流噪声和深度不确定性都有更好的鲁棒性。

Motion Loss：基于融合掩码，Dy3DGS-SLAM 设计了一种运动损失约束位姿估计网络。动态区域的像素被降权或排除在 tracking 损失之外，防止移动的物体把相机位姿"拽偏"。

渲染去干扰：在建图阶段，系统对动态像素施加额外的颜色和深度渲染损失，消除动态物体造成的瞬态干扰和遮挡伪影。

效果：在 TUM RGB-D、Bonn 动态数据集上，Dy3DGS-SLAM 的跟踪精度超过或持平于现有的 RGB-D 动态 SLAM 方法，Bonn 数据集上平均 ATE RMSE 仅 4.5cm。系统以 17 FPS 运行，单次网络迭代即可完成位姿估计。

没有解决什么：动态物体被 mask 掉而非建模，系统不保留动态物体的任何信息。掩码精度依赖光流和深度网络的质量，极端纹理缺失或快速运动下仍有漏检。没有回环优化，长序列漂移问题未解决。

12.4 Splat-SLAM：回环与全局优化进入 Gaussian 地图

Splat-SLAM (CVF开放获取) 的核心贡献是：**在纯 RGB SLAM 中首次实现了回环检测和全局 BA，并且 3D Gaussian 地图能在线跟随变形。**

核心假设：显式的 3D Gaussian 地图不像隐式神经网络那样"僵硬"——只要把所有高斯的均值坐标按位姿图优化的结果重新投影，地图就能在线跟随全局校正而变形，不需要重新训练。

输入输出：纯单目 RGB 视频；输出全局一致的相机轨迹、3D Gaussian 地图和重建网格。

系统结构：Splat-SLAM 是一个耦合的 tracking-mapping 系统，但比 MonoGS 更强调全局一致性。系统包含三个并行的关键模块：

1. DSPO 跟踪层（Disparity, Scale and Pose Optimization）：

DSPO 是 Splat-SLAM 跟踪的核心。它以密集光流为基础，通过因子图优化联合估计相机位姿和每像素视差。DSPO 由两个交替优化的目标组成：

- **密集 BA (DBA)**：在局部滑动窗口内优化位姿和视差。
- **尺度-深度优化**：引入单目深度先验，对视差图的"高误差区域"进行正则化，"低误差区域"用于稳定尺度估计。

两者交替执行，用高斯-牛顿法求解。关键设计在于：不把所有深度一起优化，而是分高低误差区域处理，避免单目深度先验污染已经精确的多视深度。

2. 回环检测与全局 BA：

回环检测通过计算当前关键帧与历史关键帧之间的平均光流幅度实现。当光流低于阈值且时间间隔足够长时，触发回环边添加。全局 BA 每 20 个关键帧执行一次，执行前对视差和位姿平移进行归一化，保证数值稳定性。

3. Proxy Depth 与可变形 Gaussian 地图：

Splat-SLAM 提出了 "proxy depth" 概念：将 tracking 模块输出的多视深度 $\tilde{D}$ 与单目深度先验 D^{mono} 融合，前者提供准确的相对几何，后者填补空洞和未观测区域。融合后的 proxy depth 既用于地图初始化，也用于渲染监督。

最关键的设计是**地图在线变形**：每次回环或全局 BA 更新关键帧位姿和深度后，所有 3D Gaussians 的均值坐标按照更新后的位姿和深度重新计算，地图"跟着位姿一起走"。这克服了 3DGS-SLAM 长期以来的全局一致性难题。

效果：在 Replica、TUM-RGBD 和 ScanNet 上，Splat-SLAM 的重建精度 (Depth L1) 和渲染质量 (PSNR) 均优于 MonoGS 和 GIORIE-SLAM，跟踪精度 (ATE) 与 GIORIE-SLAM 持平，地图大小与 MonoGS 相当。

没有解决什么：回环检测依赖光流共视，大视角变化或光照剧变下可能漏检。动态物体不在处理范围内。单目深度先验的尺度对齐仍需要足够的多视约束来稳定。

12.5 五个工程问题

本节是本章核心。2025 年的 3DGS-SLAM 研究不再追求"能不能做"，而是围绕五个具体工程问题寻找答案。每个问题的分析横跨多篇论文，而不是依附于单篇论文的结构。

问题一：单目尺度如何稳定？

挑战的本质：单目相机丢失了一个维度的观测——深度。纯 RGB 输入下，3DGS-SLAM 需要同时估计位姿和场景几何，这是一个欠定问题。没有绝对深度观测，尺度会随时间漂移，新插入的高斯位置尺度不一致，最终导致地图畸变和跟踪失败。

2024 年的方案及其不足：MonoGS 使用 MAST3R 或类似网络预测的单目深度来初始化高斯，但每帧深度预测的尺度不一致，直接拼接会在边界处产生断层。Photo-SLAM 依赖 ORB-SLAM3 的跟踪，尺度由双目或 IMU 提供，不适用于纯单目。MoD-SLAM 用 MLP 对地图进行重参数化来稳定尺度，但增加了系统复杂度。

2025 年的三条解决路线：

路线 A：外部 VO 提供稀疏但一致的尺度。 MonoGS++ 选择了这条路。DPVO 虽然只输出稀疏点云，但其逆深度估计在局部窗口内是尺度一致的。MonoGS++ 用这些稀疏点初始化 Gaussian，让建图模块继承 VO 的尺度。优点是尺度来源稳定（VO 专门做这件事），缺点是被 VO 的精度上限绑定。

路线 B：多视几何与单目深度融合。 Splat-SLAM 的 DSPO 层走了这条路。DSPO 的核心洞察是：多视几何提供的深度相对准确但不完整（只有高纹理区域有视差），单目深度网络覆盖全图但尺度不准。把两者分开处理——多视深度负责"锚定"尺度，单目深度负责"填补"空洞——比简单加权平均效果更好。

DSPO 的具体实现是分高低误差区域优化。设视差图中低误差部分为 d^l （多视一致性高的像素），高误差部分为 d^h 。优化分为两步交替执行：

第一步（DBA）：
$$\arg \min_{\omega, d} \sum_{(i,j) \in E} \|\tilde{p}_{ij} - K\omega_j^{-1}(\omega_i(1/d_i)K^{-1}[p_i, 1]^T)\|_{\Sigma_{ij}}^2$$

第二步（深度先验正则化）：低误差视差 d^l 固定，只优化高误差区域 d^h ，并用单目深度先验 D^{mono} 进行正则化，同时更新全局尺度 θ 和平移 γ 。

这种"分而治之"的策略避免了单目深度先验污染已经精确的多视深度，是 Splat-SLAM 尺度稳定的关键。

路线 C：点图先验对齐。 更近期的工作（如 S3PO-GS）引入预训练的点图（pointmap）模型作为几何先验，通过 patch-level 的局部对齐消除单目深度的尺度歧义。这条路线的潜力在于利用大规模预训练模型的先验知识，但增加了对网络质量的依赖。

对比与选择：

方法	尺度来源	优势	劣势
MonoGS++	DPVO 稀疏点	简单高效，解耦	被 VO 精度绑定
Splat-SLAM	多视几何+单目先验	密集、全局一致	DSPO 层计算量大
S3PO-GS	预训练点图先验	先验知识强	依赖预训练模型

实际部署中，如果场景纹理丰富、基线适中，路线 B（DSPO 类方案）的尺度最稳定；如果追求极致速度，路线 A（外部 VO）更轻量；如果有可靠的预训练模型，路线 C 的泛化能力最强。

问题二：大运动下 Tracking 如何不崩？

挑战的本质： 相机运动过快时，帧间重叠区域小，光流/特征匹配失效；运动模糊进一步降低纹理质量。3DGS-SLAM 的位姿优化依赖渲染-观测对比，初始位姿偏离太远时优化陷入局部极小。大旋转比大平移更难处理，因为旋转导致视场剧变，可见高斯集合完全更换。

2024 年的瓶颈： MonoGS 采用联合优化——每帧同时优化位姿和高斯参数。相机运动大时，渲染图与真实图像差异过大，光度损失 Landscape 崎岖，梯度方向混乱，跟踪发散。SplaTAM 依赖深度传感器，大运动时 ICP 也容易丢失。

2025 年的解决方案：

1. 密集光流 + 因子图优化替代渲染损失。 Splat-SLAM 和 MonoGS++ 都用密集光流作为位姿估计的基础，而不是直接比较渲染图像。光流提供了一个"中间表示"：先把帧间像素对应关系算出来，再把对应关系反解为位姿和深度。这比直接从图像亮度优化位姿更鲁棒，因为光流网络（RAFT 系列）对亮度变化和大位移有更好的泛化能力。

DSPO 层的因子图结构也很关键。设 $G(V, E)$ 为因子图，节点 V 存储关键帧位姿和视差，边 E 存储光流。新帧到达时，先计算与最近关键帧的光流；若平均光流幅度超过阈值 τ ，则将该帧加入因子图作为新关键帧。阈值因数据集而异：Replica 上 $\tau = 2.25$ （运动小），TUM-RGBD 上 $\tau = 3.0$ ，ScanNet 上 $\tau = 4.0$ （运动大）。这种自适应的关键帧选择避免了大运动下帧间匹配失败。

2. 光流补偿的相邻位姿跟踪。 Rob-GS（另一项 2025 年工作）提出了更激进的方案：在相邻帧之间，先用单帧高斯拟合得到深度图，再计算投影流（reprojection flow），与光流网络的预测进行对比。位姿优化目标同时包含光度损失 L_c 和光流损失 L_f ：

$$T_{i+1}^* = \arg \min_{T_{i+1}} \{L_c(\tilde{I}_{i+1}, I_{i+1}) + \lambda_f L_f\}$$

光流损失 $L_f = \|\tilde{f}_{i+1} - f_{i+1}\|^2$ 强制渲染流与观测光流一致，即使在大运动下也能提供可靠的梯度方向。

3. 解耦 tracking 与 mapping。 MonoGS++ 的架构 insight 在于：不要把 tracking 和 mapping 绑在一起优化。DPVO 专注于跟踪，用高效的稀疏光流 BA 跑在很高的帧率；建图模块则在后台以较低频率优化 Gaussian。大运动下，前端 VO 的响应速度决定了系统能不能"跟得上"，解耦后前端不再被高斯优化的计算量拖累。

大运动鲁棒性的设计原则总结：

原则	具体做法	代表工作
避免直接图像对比	用光流/特征作中间表示	Splat-SLAM, MonoGS++
因子图维护历史信息	滑动窗口+关键帧选择	Splat-SLAM DSPO
多目标位姿优化	光度+几何+光流联合约束	Rob-GS
Tracking-mapping 解耦	前端快速跟踪，后台建图	MonoGS++, Photo-SLAM

问题三：高斯数量如何控制？

挑战的本质： 3DGS 的显式表示是把双刃剑。优点是渲染快、梯度直接；缺点是每到一个新区域就要插入新高斯，长时间运行后数量无界增长。Replica 房间尺度下，不做控制的高斯数量轻松超过 100 万，显存占用数 GB，优化速度线性下降。

高斯从哪来、怎么疯长的： 传统的 densification 策略是"定期分裂"——每隔一定迭代次数，把所有梯度大的高斯一分为二。问题是这种策略不区分区域：已经被很好地重建的区域也在反复分裂，产生大量冗余高斯。室内场景中，一个白墙区域可能被数千个高斯覆盖，而实际上十个就够。

2025 年的控制策略：

- 1. 动态插入：只在需要的地方加高斯。** MonoGS++ 的动态 3D Gaussian 插入策略是高斯数量控制的代表。系统维护一个"重建质量图"，在插入新帧前评估每个区域的重建充分性。PSNR 已经足够高的区域不再插入新高斯；只有新观测区域或重建不足区域才触发插入。效果非常显著：在 Replica-Room2 上，动态插入使高斯数量降至传统策略的约 1/3，FPS 从 1.76 提升到 2.84。
- 2. 清晰度引导的 **Densification**。** 弱纹理区域的高斯容易"糊"成一团，数量多但质量差。MonoGS++ 的清晰度增强模块评估每个区域的纹理清晰度，在低清晰度区域进行更有针对性的 densification——增加高斯数量并加强几何监督，在高清晰度区域则保持精简。这种"按需分配"的策略比均匀分裂更高效。
- 3. Pruning 与融合。** 控制高斯数量不能只靠"少生"，还要"多杀"。标准 3DGS 的 pruning 策略基于透明度：透明度低于阈值的高斯被删除。2025 年的方法在此基础上增加了几何冗余检测：若多个高斯在空间上高度重叠且颜色相近，则合并为一个。GS-SLAM 的 adaptive expansion 策略也包含删除噪声高斯的分支。
- 4. 量化与压缩。** 更激进的方案从高斯属性本身入手。LightGaussian 类的方法对高斯的颜色、尺度、旋转等属性进行向量量化（VQ），用少量聚类中心表示大量高斯。一些 2025 年的工作（如 Large-scale compact 3DGS SLAM）结合动态掩码、几何码本和剪枝后微调，实现了 5 倍的存储压缩。

高斯数量控制策略对比：

策略	机制	效果	代价
动态插入	按需插入，避免冗余	数量减至 1/3	需要重建质量评估
清晰度引导 densification	按纹理复杂度分配	弱纹理区域质量提升	额外清晰度计算
Pruning+合并	删除低透明度/冗余高斯	持续控制数量	可能误删
属性量化	VQ 压缩高斯属性	存储降 5 倍	需要重新训练恢复质量

实际系统中，动态插入和 pruning 是必备的基础策略，应该在所有实现中采纳；清晰度引导 densification 对弱纹理场景效果明显；属性量化适合对存储敏感的应用（如移动端 AR）。

问题四：回环如何作用在可渲染地图上？

挑战的本质： 回环检测和全局 BA 在稀疏特征 SLAM 中是老朋友，但在 3DGS-SLAM 中是新问题。传统的回环做两件事：校正关键帧位姿、更新地图点坐标。稀疏点云更新很简单——每个点独立移动到新位置。但 3D Gaussians 不是孤立的点：每个高斯有均值、协方差（旋转+尺度）、透明度和球谐系数，全局位姿更新后，这些参数全部失效。更麻烦的是，如果回环后重新优化整个地图，计算量巨大。

为什么之前的 3DGS-SLAM 大多不做回环： MonoGS 没有回环，长轨迹下漂移累积。Photo-SLAM 虽然有回环，但它的地图是"超原语"结构，不是纯 3DGS。GS-SLAM、SplaTAM 等 RGB-D 方法专注于局部重建，全局一致性靠深度传感器的精度维持，回环非必需。

Splat-SLAM 的解决方案：可变形 3D Gaussian 地图。

核心 insight 极其简洁：3D Gaussians 的均值 μ 是在某个参考关键帧坐标系下定义的。回环后该关键帧的位姿从 T 更新为 T' ，只需把所有高斯的均值重新投影： $\mu' = T'T^{-1}\mu$ 。高斯的协方差（旋转和尺度）也跟着变换： $\Sigma' = R\Sigma R^T$ ，其中 R 是 $T'T^{-1}$ 的旋转部分。

更具体地说，Splat-SLAM 的地图更新流程如下：

1. 回环检测通过光流共视判断。当前关键帧与历史关键帧之间的平均光流低于阈值 $\tau_{loop} = 25.0$ 且时间间隔超过 $\tau_t = 20$ 帧时，触发回环边。
2. 全局 BA 在包含所有关键帧的因子图上执行，每 20 关键帧运行一次。执行前对视差和位姿平移做归一化 ($d_{norm} = d/\bar{d}$, $t_{norm} = \bar{d} \cdot t$)，保证数值稳定。
3. BA 完成后，每个关键帧的位姿和深度都更新了。Mapping 线程在下次优化前，先把所有高斯的均值按新位姿变形，再把 proxy depth 更新到新值，然后继续 Gaussian 参数优化。

这套流程的关键在于"变形"而不是"重建"——回环后地图的几何结构立即跟上位姿校正，不需要从头训练。3D Gaussians 的显式表示反而成了优势：隐式 NeRF 地图要全局更新必须重新推理整个网络，而显式高斯只需要矩阵乘法搬点。

局限性： 变形只处理了高斯的均值和协方差，透明度 (α) 和颜色 (球谐系数) 假设在回环前后不变。这个假设在大多数情况下成立，但在光照变化大的回环处可能不成立。此外，如果回环校正量很大，变形后的高斯分布可能需要更多优化迭代才能重新收敛。

回环在 3DGS-SLAM 中的工程意义： Splat-SLAM 证明回环和 3DGS 并不矛盾。显式表示反而让全局地图更新更高效。这条路线后续被多个工作跟进，成为 2025 年 3DGS-SLAM 的标配能力。

问题五：动态物体是 Mask 掉，还是建模进去？

挑战的本质： 真实场景不是静态的。行人、车辆、开门、移动的家具——这些动态物体如果不做处理，会污染地图、拉偏位姿。处理方式分为两派："掩码派"把动态区域检测出来排除在 SLAM 之外；"建模派"试图同时重建静态场景和动态物体。

掩码派的代表：Dy3DGS-SLAM 和 WildGS-SLAM。

Dy3DGS-SLAM 走纯 RGB 路线，用光流+深度的贝叶斯融合生成动态掩码。流程是：光流 U-Net 提运动线索 → 单目深度提几何异常 → 贝叶斯融合 → K-means 聚类 → 动态像素掩码。掩码用于两件事：tracking 时排除动态像素 (motion loss)，mapping 时对动态高斯施加惩罚损失。动态物体被完全排除在最终地图外。

WildGS-SLAM (CVPR 2025) 进一步引入了不确定性估计。它用一个浅层 MLP 在线学习每像素的不确定性，不确定区域自动被降权。这种方法不需要预定义动态类别，也不需要光流网络，但对 MLP 的表达能力有依赖。

建模派的代表：Flow4DGS-SLAM 和 DynaGSLAM。

Flow4DGS-SLAM 把动态和静态高斯分开管理。静态高斯用标准 3DGS 表示；动态高斯在关键帧处建模时序中心，用 3D 场景流传播，temporal opacity 和 rotation 用高斯混合模型 (GMM) 自适应学习。这种方法可以同时重建静态场景和动态物体，但系统复杂度高，实时性难保证。

DynaGSLAM 用 SAM2 做动态区域分割，动态和静态高斯分存于两个数组，分别维护学习率和删除策略，动态轨迹用三次 Hermite 样条插值。

两派的对比：

维度	掩码派	建模派
目标应用	静态地图重建（AR/机器人导航）	全场景理解（4D 重建）
输入需求	纯 RGB 即可	通常需要深度或语义先验
系统复杂度	低，只改损失权重	高，需管理两组高斯+时序建模
实时性	容易实现（17 FPS 以上）	难，计算量大
信息保留	动态信息完全丢失	保留动态物体轨迹和形状

2025 年的共识：对于 SLAM 本身的定位与静态建图任务，掩码派已经足够。Dy3DGS-SLAM 证明纯 RGB 下动态掩码可以做到与 RGB-D 方法相当的精度，且计算开销可控。建模派的 4D 重建虽然更完整，但实时性和系统稳定性仍是挑战，目前更适合离线应用。

关键的工程细节：无论哪一派，动态检测的质量都是瓶颈。Dy3DGS-SLAM 的贝叶斯融合比单一光流或深度掩码的误检率低 60% 以上。融合的核心价值在于互补：光流对均匀纹理区域敏感，深度对遮挡和深度不连续区域敏感，两者结合覆盖更多场景。阈值 $T = 0.95$ 的设定偏保守——宁可错杀静态像素，也不放过动态干扰。这个偏好在跟踪精度优先的场景下是合理的。

12.6 与具身智能的关系

3DGS-SLAM 的工程化进展直接影响具身智能系统的空间感知能力。

实时性决定闭环控制的可行性。 MonoGS++ 把单目 SLAM 推到接近实用帧率，意味着机器人或 AR 眼镜可以用纯摄像头实现实时定位和场景重建，不再依赖深度传感器。这对成本敏感和功耗敏感的设备意义重大。

动态处理能力决定真实场景可用性。 Dy3DGS-SLAM 证明纯 RGB 下动态掩码可以有效工作，意味着机器人可以在有人行走的环境中可靠定位，不需要预先清理场地或依赖深度传感器。

回环与全局优化决定长距离作业能力。 Splat-SLAM 的可变形地图让 3DGS-SLAM 具备了长序列工作的基础。对需要在大空间中长时间运行的服务机器人或配送机器人，这是从实验室走向落地的必要条件。

高斯数量控制决定大场景可扩展性。 如果不加控制，3DGS 的内存占用随场景线性增长，几小时后就会耗尽资源。动态插入和压缩策略让 3DGS-SLAM 有望扩展到建筑级甚至城市级场景。

2025 年的工程化进展表明，3DGS-SLAM 正在从"漂亮的渲染 demo"走向"可靠的空间感知引擎"。

12.7 一句话总结

2025 年的 3DGS-SLAM 不再问"能不能做"，而是用解耦架构提速、用多线索融合稳定单目尺度、用可变形地图打通回环、用贝叶斯掩码应对动态场景、用动态插入控制高斯数量——从炫技走向工程。

第13章 语义3DGS-SLAM：从"可看"到"可理解"

13.1 为什么3DGS需要语义？

3D Gaussian Splatting SLAM在过去两年把几何重建推向了全新高度。高清晰度、实时渲染、紧凑的地图表示——技术指标不断刷新。但一个根本问题始终悬而未决：机器人要这张地图干什么？

想象一个家用机器人接到指令："去沙发旁边把杯子拿过来。"任务链条的每个环节都需要语义理解：

- 哪些高斯属于"杯子"？ 没有语义标签，机器人在数千万个高斯原语中找不到目标。
- "沙发"在哪里？ 纯几何地图只能告诉机器人那里有块平面，无法告知那是沙发。
- "旁边"是多近？ 空间关系的推理需要锚定到语义对象上。
- 路径是否可达？ 导航需要区分地面、家具、墙壁等语义类别。
- "拿"这个动作怎么执行？ 需要知道杯子是刚性还是柔性、是否有把手。

没有语义，3DGS地图再精美也只是一幅"看得见但看不懂"的画面。语义是地图从"视觉展示"走向"任务可用"的必经之路。

这个问题不是理论上的遐想。2024年，3DGS-SLAM系统在Replica、ScanNet等数据集上已经可以渲染出照片级的场景，但机器人研究者拿到这些地图后，第一反应往往是："这个高斯集合代表什么？"纯几何的高斯地图里没有"对象"这个概念——只有数百万个独立的3D原语，每个原语知道自己的位置、颜色和透明度，但不知道自己是"墙壁"还是"水杯"。当机器人需要执行"清理桌面"这样的任务时，纯几何地图无法提供任何直接帮助。

从具身智能的角度看，感知系统的终极目标不是重建几何细节，而是构建能被下游任务调用的世界模型。这个世界模型需要回答三个层次的问题：**有什么 (What)**、**在哪里 (Where)**、**怎么用 (How)**。纯几何SLAM只能回答"在哪里"；加入闭集语义后可回答"有什么"（在预定义类别内）；只有开放集语义才能为"怎么用"提供基础——真实世界中的物体类别是开放的，任务指令不可预见。

语义3DGS-SLAM的发展还面临一个技术约束：语义信息怎么"存"进高斯？三种方案各有优劣：

- **语义颜色 (SGS-SLAM)**：把每个高斯编码为一个类别概率分布。紧凑高效，但必须是闭集，且不可扩展。
- **特征嵌入 (GSFF-SLAM)**：给每个高附加一个N维可学习向量。灵活且可表达丰富语义，但维度高了存储膨胀，渲染变慢。
- **显式标签 (OpenGS-SLAM)**：每个高斯只存一个整数ID。极致紧凑，支持对象级操作，但丢失了语义相似性信息（两个标签ID之间没有距离概念）。
- **VFM特征 (OpenMonoGS-SLAM)**：借助外部记忆库存储CLIP等高维特征。表达能力最强，但推理开销最大。

2024年到2025年，3DGS-SLAM的语义化经历了三条清晰演进线：

- **闭集语义融合**：SGS-SLAM将语义颜色作为高斯的第三通道，在固定类别上联合优化几何、外观和语义。
- **特征场表达**：GSFF-SLAM用N维语义特征嵌入替代硬标签，通过解耦优化支持稀疏和噪声监督。
- **开放集对象级理解**：OpenGS-SLAM和OpenMonoGS-SLAM引入视觉基础模型，摆脱预定义类别，实现对象级场景理解和开放词汇查询。

这三条线共同指向同一终点：让3DGS地图从"可看"变成"可理解"。

13.2 SGS-SLAM：语义作为第三通道

SGS-SLAM (Semantic Gaussian Splatting for Neural Dense SLAM) 发表于ECCV 2024, 是最早将语义信息完整嵌入3DGS-SLAM框架的系统之一 (Springer)。它要解决的痛点直白：现有语义SLAM要么基于NeRF渲染太慢，要么基于点云/体素太粗糙，3DGS的速度和精度优势能不能带到语义域？

核心假设

如果每个高斯原语不仅存储几何和外观，还存储语义信息，那么语义、几何和外观可以在同一优化框架中联合更新，彼此互相促进。

输入与输出

系统输入为RGB-D视频流和对应的2D语义标签图。语义标签可来自预训练分割模型或人工标注，但必须是闭集——类别集合在运行前固定。输出为相机轨迹、稠密语义3D高斯地图，以及从任意视角渲染的语义标签图。

地图表示

SGS-SLAM扩展了标准3DGS的每个高斯原语，使其拥有三个并行通道：

- **几何通道**：3D均值坐标 $\mu \in \mathbb{R}^3$ 和协方差矩阵 Σ ，定义高斯在空间中的位置和形状。
- **外观通道**：球谐系数 $c \in \mathbb{R}^3$ ，渲染RGB颜色。
- **语义通道**：语义颜色 $s \in \mathbb{R}^C$ ，其中 C 为闭集类别数，通过softmax映射为类别概率分布。

三个通道在同一splatting管线中并行渲染。像素 p 的语义值通过 α 混合计算：

$$\mathbf{S}(p) = \sum_{i \in \mathcal{N}} s_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (19)$$

其中 s_i 是第 i 个高斯的语义颜色， α_i 是其在像素 p 处的透明度贡献。渲染过程完全可微分，语义监督信号可直接反向传播到每个高斯的语义参数。

Tracking与Mapping

Tracking采用三通道联合优化。位姿估计阶段固定高斯参数，最小化几何和光度重投影误差；Mapping阶段固定相机位姿，联合优化几何、外观和语义参数。

关键创新是**语义引导的关键帧选择**。SGS-SLAM不仅考虑几何重叠率（阈值 $\eta = 0.05$ ），还引入语义mIoU阈值 $T_{\text{sem}} = 0.7$ 。当当前帧与已有地图的语义一致性低于阈值时，该帧优先选为关键帧。这确保语义变化剧烈的区域获得更多优化资源。这一策略的直觉是：当机器人从客厅走进厨房时，语义分布发生剧烈变化，此时需要更多关键帧来捕捉新区域的语义信息。

优化目标

$$\mathcal{L} = \lambda_D \mathcal{L}_D + \lambda_C \mathcal{L}_C + \lambda_S \mathcal{L}_S \quad (20)$$

$\mathcal{L}_D$ 为深度L1损失， $\mathcal{L}_C$ 为光度L1损失， $\mathcal{L}_S$ 为语义交叉熵损失。语义权重 $\lambda_S = 0.05 \sim 0.1$ 被刻意压低——2D语义标签本身有噪声，过高权重会放大噪声对几何优化的干扰。这体现了SGS-SLAM的核心洞察：**语义是辅助信号，而非主导信号**。这个权重选择背后的工程考量很务实：在 Replica 数据集的实验中， λ_S 从 0.1 降到 0.05 后，虽然语义mIoU轻微下降了约0.5%，但几何重建的PSNR提升了约0.8 dB——对SLAM系统来说，几何精度是更基础的硬指标。

相比前作的推进

相比SNI-SLAM等NeRF-based语义SLAM，SGS-SLAM的渲染速度从不足10 FPS提升到实时水平。它证明了3DGS的splatting机制天然适合语义扩展—— α 混合不需要为语义通道做任何结构性修改。SGS-SLAM还验证了"语义引导关键帧选择"这一策略的有效性：在Replica数据集上，语义引导的选择策略使mIoU比纯几何选择高出约3个百分点，同时跟踪ATE也略有改善。

未解决的问题

SGS-SLAM留下三个结构性缺陷：第一，语义标签必须是闭集，无法处理未见类别；第二，依赖RGB-D输入，限制了纯视觉场景的应用；第三，语义和几何优化是耦合的，语义梯度会干扰几何参数的收敛。这些缺陷不是工程细节问题，而是设计范式的根本限制。

13.3 GSFF-SLAM：解耦的特征场

GSFF-SLAM（3D Semantic Gaussian Splatting SLAM via Feature Field）针对SGS-SLAM的第三个痛点——语义与几何耦合优化——提出系统性解决方案 (arXiv)。东南大学和同济大学的研究团队指出，SGS-SLAM中语义损失直接反向传播到高斯的几何参数（位置、协方差），在语义标签噪声较大时会降低几何重建精度。

核心假设

先建好几何，再学语义；语义特征独立优化，不干扰几何。通过梯度隔离，系统可以容忍不完美的2D语义监督信号。

输入与输出

输入为RGB-D视频流和2D语义标签图（闭集）。标签可以是稠密标注，也可以是稀疏、有噪声的预测。输出为相机轨迹、带语义特征嵌入的3D高斯地图，以及从任意视角渲染的稠密语义特征图和类别标签图。

地图表示

GSFF-SLAM采用各向同性高斯，每个高斯的可优化参数为：

$$\mathcal{G}_i = \{x_i, q_i, s_i, \alpha_i, c_i, f_i\} \quad (21)$$

其中 $f_i \in \mathbb{R}^N$ 是新增的N维语义特征嵌入（feature embedding）。与SGS-SLAM的语义颜色不同， f_i 不是直接的类别概率，而是可学习的隐式特征向量。渲染时，特征图 $\hat{F} \in \mathbb{R}^{H \times W \times N}$ 通过splatting管线生成：

$$\hat{F} = \sum_{i \in \mathcal{N}} f_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (22)$$

特征图的通道数 N 通常等于闭集类别数 C 。在测试时，通过 $\arg \max$ 将特征图转换为类别预测：

$$\hat{L} = \arg \max(\hat{F}) \quad (23)$$

解耦优化策略

GSFF-SLAM的关键设计是将mapping解耦为两个独立步骤：

Step 1: 纯几何重建。只优化位置 x 、协方差 Σ 、不透明度 α 和颜色 c ，用深度和光度损失监督，直到几何收敛。

Step 2: 语义特征学习。固定几何参数，只优化语义特征嵌入 f ，用2D语义标签的交叉熵损失监督：

$$\mathcal{L}_S = -\frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W \sum_{c=1}^N L_{i,j}(c) \log(\hat{F}_{i,j}(c)) \quad (24)$$

梯度隔离确保语义噪声不会污染几何参数。这种解耦使GSFF-SLAM对稀疏和噪声的2D语义先验更加鲁棒。举例说明：假设机器人只在每5帧运行一次分割模型（出于算力限制），中间帧没有语义标签。在SGS-SLAM的联合优化中，缺失的语义监督意味着这些帧的语义梯度为零，但几何梯度仍然回传，可能导致高斯位置在语义空白帧中被错误拉扯。而GSFF-SLAM中，Step 2只在有语义标签的帧上优化特征嵌入，Step 1的几何重建不受任何影响。

相比前作的推进

GSFF-SLAM的真正推进不是某个单一创新，而是"解耦"这一设计理念带来的系统性收益。在Replica数据集上，使用ground truth监督时mIoU达到95.03%，比SNI-SLAM高出10.41个百分点，比SGS-SLAM也有明显提升。这主要归功于3DGS的显式表示避免了NeRF在物体边界处的模糊问题——NeRF的体密度积分天然在边界处产生过渡区，而3DGS的显式高斯中心可以精确对齐到对象表面。

当采用加速策略（ $\tau_{\text{thresh}} = 0.8, \rho_{\text{pc}} = 1/64$ ）时，运行速度提升2.9倍，而mIoU仅轻微下降。这意味着GSFF-SLAM在工程部署上有了更大的灵活性——可以根据硬件算力在"精度模式"和"速度模式"之间切换。

未解决的问题

GSFF-SLAM仍需要闭集的2D语义标签——特征向量最终还是要通过 $\arg \max$ 映射到预定义类别。系统依赖RGB-D输入，且语义特征维度 N 需要权衡：过低则表达能力不足，过高则增加存储和计算开销。更关键的是，解耦优化虽然提高了鲁棒性，但也切断了语义信息向几何的回流——系统无法利用语义一致性来修正几何误差。例如，如果两个高斯在语义上是同一个墙壁的一部分，但在几何上被错误地分离了，GSFF-SLAM无法利用语义一致性将它们拉回到同一平面上。

13.4 OpenGS-SLAM：从闭集到开放集的跃迁

OpenGS-SLAM发表于ICRA 2025，是3DGS-SLAM从闭集语义走向开放集语义的分水岭 (IEEE)。北京理工大学团队的核心问题意识清晰：此前的语义SLAM受限于预定义类别分类器和隐式语义表示（特征嵌入），无法在开放环境中识别任意对象，也无法支持对象级的场景操控。

核心假设

显式离散标签比隐式特征嵌入更适合开放集对象级理解。每个高斯只需要一个整数ID就能指向一个对象实例，存储仅需4字节/高斯——比512维CLIP特征压缩约500倍。

输入与输出

系统输入为RGB-D视频流。RGB图像送入2D基础模型组合（SAM / RAM++ / YOLO-World）生成开放集语义标签，深度用于G-ICP位姿估计。输出为带显式对象标签的3D高斯地图，支持对象移除、重定位等交互操作。

三项核心创新

OpenGS-SLAM面对两个技术挑战：2D基础模型在不同视角下分割不一致（标签碎片化）；开放集表示需要在"表达能力"和"渲染效率"间找平衡。解决方案包含三项核心创新：

Gaussian Voting Splatting (GVS)。离散标签不可微，无法通过标准梯度下降优化。GVS是一种非可微的标签渲染机制：渲染像素 p 的标签时，系统记录对该像素贡献最大的Top-50个高斯，按 α 混合权重进行投票，累加权重最高的标签胜出。语义渲染速度达到165 FPS，比特征嵌入方法快10倍。

GVS之所以高效，是因为它完全绕过了特征向量→softmax概率→argmax的完整计算链。传统特征嵌入方法需要在每个像素上计算数百到数千个高斯特征的加权平均，再做softmax归一化；而GVS只需要比较整数标签并累加权重，计算复杂度大幅降低。这种设计也有代价：标签投票是离散操作，无法进行端到端的梯度下降优化，标签更新必须通过显式的共识机制完成。

Confidence-based 2D Label Consensus。解决多视图标签不一致的核心模块。当前帧输入标签图 $\mathcal{L}_s$ 与地图渲染标签图 $\mathcal{L}_r$ 双向匹配，识别四种关系类型：

匹配类型	条件	处理方式
Full Match	输入标签完全覆盖渲染标签	采用地图标签，置信度最大化
Partial Match	多个输入对应一个渲染对象	加权平均置信度，高则合并
Whole Match	一个输入覆盖多个渲染对象	输入置信度高则拆分
New	输入标签无地图对应	创建新对象标签

匹配引入置信度衰减机制（ $\delta = 0.06$ ）处理SAM的过度分割问题。碎片标签的置信度随观测次数衰减，完整对象标签的置信度持续累积，系统收敛到正确的对象级分割。消融实验显示，Part Label Decay模块单独提升mIoU 5.96%、Acc 10.2%；Input Confidence Update模块提升mIoU 3.39%。

Consensus机制的设计体现了对SAM特性的深刻理解。SAM在单个视角上的分割质量很高，但跨视角一致性很差：同一个水杯，从正面看SAM可能把它和杯垫分在一起，从侧面看可能把杯子和桌面分在一起。没有多视图一致性机制，这些碎片化的标签会被直接固化到3D地图中，形成语义噪声。Consensus的四种匹配类型本质上是对2D-3D标签关系的完备分类，覆盖了所有可能的对应模式。

Segmentation Counter Pruning. 在观测不足区域，高斯容易过度生长、侵占相邻对象边界。该模块识别"Counter Gaussians"——在标签对称差区域中、任一方向尺度超过阈值 θ 的高斯——将其修剪。 θ 在 Replica上设为0.10，在TUM上设为0.25。这使mIoU提升0.26%，主要改善对象边界精度。

相比前作的推进

OpenGS-SLAM的真正突破不在于某个技术指标，而在于证明了3DGS地图可以支持"对象级"理解——用户可以点击对象并对其进行操作。这是从"地图渲染"到"场景理解"的质变。

在Replica数据集上，OpenGS-SLAM (SAM1.0) 的开放集mIoU达63.17%，准确率74.51%。在新视角开放集分割上，比离线方法Feature-3DGS高出13% mIoU，比GS-Grouping高出3%。标签图渲染速度是特征嵌入方法的10倍，存储开销仅为一半。此外，G-ICP与Counter Pruning的结合使ATE RMSE在Replica上最高改善51%。

未解决的问题

OpenGS-SLAM依赖RGB-D输入；不适用于动态场景；标签共识机制依赖启发式阈值（ $\tau_1 = 0.85, \tau_2 = 0.90, \tau_3 = 0.1$ ），在极端复杂场景中可能失效。更重要的是，显式标签虽高效，但无法表达对象之间的语义关系——"杯子"和"盘子"在特征空间中可能很近，但在标签空间中是完全无关的两个整数。这意味着OpenGS-SLAM无法直接支持"找与杯子类似的物体"这种基于语义相似度的查询。

13.5 OpenMonoGS-SLAM：单目+开放集+VFM

OpenMonoGS-SLAM是2025年底出现的语义SLAM新范式，首次将"单目视觉""3D Gaussian Splatting"和"开放集语义理解"统一在一个框架中 (arXiv)。成均馆大学和延世大学的研究团队瞄准了更激进的设定：只给一个单目RGB相机，不要深度传感器，不要3D语义真值，能不能做开放集语义SLAM？

核心假设

视觉基础模型（VFM）的零-shot能力可以填补每个感知环节的缺口：MASt3R提供稠密几何先验，SAM提供2D分割，CLIP提供开放词汇语义。三者组合可以替代深度传感器和闭集语义标注。

输入与输出

输入仅为单目RGB视频流。系统内部调用MASt3R、SAM (ViT-H) 和CLIP (ViT-B) 三个预训练模型。输出为相机轨迹、带高维语义特征的3D高斯地图，支持闭集和开放集语义分割、文本驱动查询。

系统结构

OpenMonoGS-SLAM以MASt3R-SLAM为骨架：

- **MASt3R提供几何**：提取稠密每像素特征和点云，支撑单目相机跟踪和3D高斯初始化。通过迭代优化

光线误差建立帧间对应关系。MASt3R的引入解决了单目SLAM的尺度不确定问题——它输出的点云具有度量尺度，使得后续的高斯初始化不再受尺度漂移困扰。

- **SAM提供分割**：每帧生成2D实例分割掩码，投影到3D空间后建立多视图一致性语义映射。
- **CLIP提供开放词汇语义**：将512维语义特征嵌入3D高斯场，支持文本驱动查询。

为管理高维CLIP特征，系统设计了**高维语义记忆库**。核心思想是不在每个高斯上存储完整CLIP向量，而是维护一个紧凑的高斯特征图和一个外部记忆库。通过注意力机制从记忆库动态检索CLIP特征，渲染时再"读取"到高斯上。这避免了存储膨胀，同时保留丰富语义表达能力。记忆库的设计是一个关键工程决策：如果每个高斯都存储512维CLIP特征，一个有100万个高斯的地图仅语义特征就需要约2GB显存——这对于消费级GPU是不可接受的。记忆库通过"需要时检索"将这一开销降低到可控水平。

语义学习采用**多尺度策略**（ $S = 4$ 个尺度级别）。SAM在不同大小对象上生成的掩码被转换为尺度感知监督信号：大对象的语义监督更侧重"粗粒度一致性"，小对象更侧重"细粒度精确性"。这减少了重叠掩码的歧义，使语义表达能平滑地从粗分组过渡到细粒度分离。

整个系统完全依赖自监督学习目标，没有任何3D语义真值：

$$\mathcal{L} = \lambda_{\text{photo}} \mathcal{L}_{\text{photo}} + \lambda_{\text{corr}} \mathcal{L}_{\text{corr}} + \lambda_{\text{lang}} \mathcal{L}_{\text{lang}} \quad (25)$$

其中 $\lambda_{\text{photo}} = 1.0$ （光度损失）， $\lambda_{\text{corr}} = 0.05$ （对应损失）， $\lambda_{\text{lang}} = 0.05$ （语言特征损失）。

相比前作的推进

OpenMonoGS-SLAM证明了"传感器极简主义"在语义SLAM中的可行性。不需要深度相机、不需要语义标注、不需要预定义类别——仅凭RGB摄像头和预训练VFM，机器人就能获得丰富的语义理解。Replica数据集上，系统在闭集和开放集分割任务中都达到或超过现有RGB-D基线。在TUM RGB-D基准的单目设置下，ATE RMSE跟踪精度也保持竞争力。

从工程范式上看，OpenMonoGS-SLAM代表了SLAM系统设计的一次重要转向：从"端到端训练"到"VFM组合调用"。传统的语义SLAM系统（包括SGS-SLAM和GSFF-SLAM）把所有的感知功能都塞进一个统一的优化框架里——几何、外观、语义一起训练。而OpenMonoGS-SLAM坦然承认：几何让MASt3R来做，分割让SAM来做，语义让CLIP来做，SLAM系统只需要负责把这些结果融合成一个一致的3D地图。这种"组合式架构"虽然失去了端到端优化的理论优雅性，但获得了更强的实用性和泛化能力。

未解决的问题

OpenMonoGS-SLAM的局限主要来自MASt3R骨架：缺乏对动态场景的鲁棒性。移动物体会使MASt3R的静态假设失效，导致跟踪漂移和语义不一致。此外，VFM推理开销（SAM ViT-H + CLIP ViT-B + MASt3R）需要约24GB显存，在资源受限平台上难以实时运行。系统依赖VFM的质量——如果SAM漏检某个对象，CLIP给出错误语义描述，这些错误会累积到3D地图中且难以修正。

13.6 演进线：从闭集到开放集的技术图谱

四篇论文放在时间线上，语义3DGS-SLAM的演进脉络清晰可辨。

SGS-SLAM（2024）奠定了基础范式：把语义塞进高斯，三通道联合渲染。它本质上是传统语义SLAM的3DGS版本——闭集标签、RGB-D输入、语义与几何紧耦合。贡献是证明了splatting机制的语义扩展性。

GSFF-SLAM（2025）解决了耦合问题。通过特征场和解耦优化，语义训练不再干扰几何收敛，同时容忍不完美的2D监督。这是工程上的重要进步——真实场景中，语义标签从不完美。

OpenGS-SLAM（2025）完成了从"闭集分类"到"开放集对象理解"的范式跃迁。它抛弃高维特征嵌入，用显式标签+投票机制实现对象级地图表示。10倍加速和2倍存储压缩证明，有时候更简单的方法更有效。对象移除和重定位能力让地图从"看"走向"用"。

OpenMonoGS-SLAM（2025）把传感器门槛降到最低。单目RGB + VFM的组合表明，语义SLAM可以脱离深度传感器独立工作。这是通往消费级机器人和AR眼镜的关键一步。

四条链路构成从"标签辅助几何"到"几何承载语义"再到"语义驱动交互"的完整演进链。每代解决前一代的一个核心限制：耦合→解耦、闭集→开放集、RGB-D→单目。

论文	年份	输入	语义表示	类别范围	对象级	核心突破
SGS-SLAM	2024	RGB-D + 闭集标签	语义颜色 $\mathbb{R}^C$	闭集	否	语义引导关键帧
GSFF-SLAM	2025	RGB-D + 闭集标签	特征嵌入 $\mathbb{R}^N$	闭集	否	解耦优化
OpenGS-SLAM	2025	RGB-D + 2D VFM	显式整数标签	开放集	是	投票渲染+标签共识
OpenMonoGS-SLAM	2025	单目RGB + VFM	CLIP特征+记忆库	开放集	是	VFM流水线+自监督

上表展示了从SGS-SLAM到OpenMonoGS-SLAM的完整技术演进路径。每一列的变化都反映了一个设计决策的切换：语义表示从"可微分布"到"不可微标签"再到"外部记忆"，输入从"深度辅助"到"纯视觉"，优化从"联合端到端"到"解耦分步"到"VFM组合调用"。这些变化共同揭示了一个趋势：语义SLAM的设计重心正在从"如何在高斯内部编码语义"转向"如何让高斯地图对接外部世界模型"。

13.7 与具身智能的关系

语义3DGS-SLAM对具身智能的价值是结构性支撑。具身智能的核心循环是"感知→理解→决策→执行"，"理解"环节需要与语言和任务对接的世界模型。纯几何地图无法完成这个对接——"桌子""门把手""台阶"都是语义概念，不是几何原语。

开放集语义SLAM尤其关键。具身智能的指令是开放词汇的："把那个红色的东西拿过来""不要踩到电线"。闭集模型无法处理任意描述，而CLIP嵌入可以直接与语言模型对接，实现自然语言→3D空间的grounding。OpenMonoGS-SLAM的架构天然适合这种对接：记忆库存储的CLIP特征可以直接计算与文本查询的相似度，从而在3D地图中找到"语义上最匹配的区域"。

对象级理解进一步打开了操作大门。当每个对象都有独立标签时，机器人可执行"移除这个对象""检查那个物体后面"等需要对象级推理的任务。OpenGS-SLAM展示的对象移除和重定位，本质上是在验证语义地图的操作可行性。更进一步，对象级标签为任务规划提供了原子单位——规划器可以针对"对象"而非"像素"或"高斯"进行推理，大幅降低规划复杂度。

从更长远看，语义3DGS-SLAM为3D场景图（3D Scene Graph）的在线构建提供了基础。场景图是具身智能中高层推理的核心数据结构——节点是对象（带语义属性和3D位置），边是空间关系（"旁边""上面""里面"）。带对象级标签的语义3DGS地图正是场景图的"像素级底稿"。OpenGS-SLAM的显式标签已经具备了场景图节点的基础形态，下一步只需要在空间关系推理层上进一步扩展。2025年后，将3DGS-SLAM与在线场景图构建融合的研究正快速涌现，这将是第14章的主题。

语义SLAM的另一个具身智能应用是**导航**。传统导航依赖占用栅格或拓扑地图，无法处理"去厨房的水槽旁边"这种语义导航指令。开放集语义地图可以直接解析这类指令：CLIP找到"水槽"的3D位置，空间关系模块确定"旁边"的范围，路径规划器在地面上找到可达路径。OpenMonoGS-SLAM在这种应用场景中有天然优势——它不需要预先标注地图中的每个物体，只需要在运行时通过文本查询即可定位目标。

本章一句话

2025年以后，3DGS-SLAM的关键不再只是"重建得像不像"，而是"地图能不能被语言、任务和策略使用"。

第14章 2026 年 3DGS-SLAM 前沿：在线、开放词汇、概率与具身接口

前一章的语义 3DGS-SLAM（SAGA、Feature 3DGS、LangSplat 等）已证明高斯基元可携带语义特征。但它们大多运行在离线模式下——假设位姿已知且完整图像序列可预先获取。

真实机器人必须对流式图像实时响应，不能等扫完整个房间才开始理解。2026 年的七项工作拆除了这道墙：EmbodiedSplat、GS3LAM、X-GS、OnlinePG、VBGS-SLAM、AIM-SLAM 和 GSMem，共同勾勒出六大趋势——从离线到在线 feed-forward、从几何重建到 RGB-D-语义统一、从孤立模块到感知-思考框架、从封闭词汇到开放词汇全景理解、从确定性优化到概率不确定性、从独立 SLAM 到基础模型融合、从静态地图到持久空间记忆。

方法	核心贡献	在线/离线	输入模态	开放词汇	不确定性建模	与具身智能的关系
EmbodiedSplat	Online Sparse Coefficients Field + CLIP 码本	在线	RGB	是	无	流式语义场景理解
GS3LAM	Semantic Gaussian Field + 多模态误差约束	在线	RGB-D	否	无	实时稠密语义建图
X-GS	Perceiver-Thinker 框架	在线	RGB/RGB-D	是	无	对接 VLM 的感知思考管道
OnlinePG	Local-to-global 滑动窗口全景映射	在线	RGB-D	是	无	在线全景实例感知
VBGS-SLAM	变分贝叶斯耦合 splat 图与位姿	在线	RGB-D	否	显式	鲁棒概率定位建图
AIM-SLAM	VGGT 点图 + 自适应关键帧选择	在线	单目 RGB	否	无	基础模型增强的稠密 SLAM
GSMem	3DGS 持久空间记忆	在线	RGB	是	隐式	空间回忆与再观察

七个系统并非竞争关系，而是从不同层面回答了同一个问题：3DGS-SLAM 如何成为具身智能的可靠空间感知 backbone。EmbodiedSplat、OnlinePG 和 GSMem 直接面向具身应用；VBGS-SLAM 补上了概率基础；AIM-SLAM 代表基础模型融合范式；X-GS 提供了系统级框架设计思路。

14.1 EmbodiedSplat：在线 Feed-Forward 语义 3DGS

问题与核心洞察

开放词汇 3D 场景理解的主流范式——LERF、LangSplat、ConceptFusion 等——依赖离线优化：先收集完整图像序列，通过 COLMAP 获取位姿，再迭代训练特征场。这种"先扫描、后理解"的模式对具身智能不适用。机器人在探索环境时需要边走动边理解：看到厨房台面时立即知道这是"counter"，而不是等到走完整个公寓才回溯推理。

EmbodiedSplat (arXiv) 的核心洞察是：feed-forward 网络可以直接从流式图像中在线构建语义嵌入的 3DGS，无需逐场景优化。它在收到第 t 帧图像的瞬间就能更新地图并输出语义理解结果，累计处理 300 多张流式图像后仍保持在线运行。

系统结构

系统输入流式 RGB 图像，输出语义嵌入的 3D 高斯地图，支持开放词汇查询。核心模块有三个：

Online Sparse Coefficients Field。 传统方法将 512 维 CLIP 向量绑定到每个高斯，内存开销大。EmbodiedSplat 改为维护全局 CLIP Codebook，每个高斯只存储指向码本的稀疏系数索引。语义空间被"离散化"为可复用字典，内存极低且保留 CLIP 泛化能力。

3D 几何感知 CLIP 聚合。 CLIP 是 2D 模型，缺乏 3D 先验。系统通过 3D U-Net 在部分点云上聚合特征，补偿 2D 嵌入缺失的三维结构信息。

在线融合。 新帧到达时提取 2D CLIP 特征并与码本匹配，更新高斯稀疏系数。全过程 feed-forward，无需迭代优化。

论文精读

论文试图解决什么痛点？ 现有开放词汇 3DGS 方法局限于离线或逐场景优化，无法在具身探索的流式设置中在线运行。

核心假设是什么？ CLIP 语义空间可以通过稀疏码本高效压缩，且 3D U-Net 能够为 2D CLIP 特征补充足够的几何一致性。

输入输出是什么？ 输入：流式 RGB 图像（300+ 帧）；输出：语义嵌入的 3DGS 地图，支持开放词汇文本查询。

地图表示是什么？ 3D 高斯 splat，每个高斯绑定稀疏码本系数而非完整 CLIP 向量。

Tracking 怎么做？ 假设已知或并行估计位姿（论文聚焦语义映射，tracking 非其主要贡献）。

Mapping 怎么做？ 在线稀疏系数场 + 3D U-Net 几何聚合，feed-forward 更新。

优化目标是什么？ 最小化 2D CLIP 特征与 3D 高斯渲染特征之间的重建误差，通过码本查找实现稀疏编码。

相比前作真正推进了什么？ 从"离线训练、逐场景优化"推进到"在线 feed-forward、跨场景泛化"，是首个能从 300+ 流式图像中在线构建语义嵌入 3DGS 的系统。ScanNet 上 mIoU 达 46-57%，显著优于 LUDVIG、Dr. Splat 等离线方法。

没有解决什么？ 动态物体处理未涉及；位姿估计依赖外部输入；码本容量限制可能约束语义粒度。

对后续研究的影响是什么？ 证明了 feed-forward 语义 3DGS 的可行性，为"在线语义 SLAM"开辟了新范式。

与具身智能的关系

EmbodiedSplat 把"3DGS 场景重建"推进到"具身智能在线语义空间理解"。机器人可在探索中实时构建语义地图，用自然语言查询"find the mug"并立即获得 3D 定位。

14.2 GS3LAM：语义高斯场 SLAM

问题与核心洞察

RGB-D-语义三模态融合在稠密 SLAM 中展现出巨大潜力，但存在一个根本性的表示困境：显式表示（点云、面元）受限于分辨率且无法预测未观测区域；隐式表示（NeRF）的体渲染太慢，不满足实时要求。3DGS 兼具点云的效率和连续几何表示能力，但将其用于语义 SLAM 时面临两个独特挑战：高斯基元的尺度与真实几何表面不对齐；在线更新时存在灾难性遗忘。

GS3LAM (arXiv) 提出 Semantic Gaussian Field (SG-Field, 语义高斯场), 把颜色、深度、语义三种模态统一进一个可优化的 Gaussian 场中, 通过多模态误差约束联合优化相机位姿和语义场。

系统结构

Semantic Gaussian Field (SG-Field)。场景被建模为一组语义高斯, 每个高斯除位置 μ 、协方差 Σ 、不透明度 α 、球谐系数 SH 外, 还附加一个语义特征向量 f_{sem} 。渲染时, 颜色、深度、语义三个通道同时光栅化:

$$C = \sum_i \alpha_i c_i \prod_{j < i} (1 - \alpha_j), \quad D = \sum_i \alpha_i d_i \prod_{j < i} (1 - \alpha_j), \quad F_{sem} = \sum_i \alpha_i f_i \prod_{j < i} (1 - \alpha_j)$$

三种模态共享同一个 α -混合权重, 确保几何、外观、语义在空间上严格对齐。

Depth-adaptive Scale Regularization (DSR)。3DGS 的高斯尺度是尺度不变的, 而真实几何表面有确定的物理尺度。DSR 根据深度自适应地约束高斯尺度, 使高斯基元紧密贴合实际表面而非漂浮在空间中。其关键思想是: 深度越大, 允许的尺度范围越大; 深度越小, 尺度必须越紧凑。

Random Sampling-based Keyframe Mapping (RSKM)。传统 3DGS-SLAM 使用局部共可见性 (local covisibility) 策略选择关键帧进行优化, 容易对最近帧过度采样、对历史帧欠采样, 导致灾难性遗忘。RSKM 改为从整个关键帧数据库中随机采样, 强制地图优化覆盖不同视角和时间段, 实验表明优于所有局部共可见性策略。

论文精读

论文试图解决什么痛点? 现有语义 SLAM 的表示困境 (显式分辨率受限、隐式实时性不足), 以及 3DGS 用于语义 SLAM 时的尺度-表面不对齐和灾难性遗忘问题。

核心假设是什么? 3DGS 可以同时颜色、深度、语义三模态统一进一个可微渲染框架, 且随机采样比局部共可见性更能防止遗忘。

输入输出是什么? 输入: RGB-D 序列 + 2D 语义分割; 输出: 相机位姿轨迹 + 稠密语义 3D 高斯地图。

地图表示是什么? Semantic Gaussian Field (SG-Field), 每个高斯携带语义特征向量。

Tracking 怎么做? 通过多模态误差约束 (光度误差 + 深度误差 + 语义误差) 联合优化相机位姿和地图。

Mapping 怎么做? DSR 约束高斯尺度 + RSKM 随机关键帧采样, 防止灾难性遗忘。

优化目标是什么?

$$\mathcal{L} = \lambda_{rgb} \mathcal{L}_{rgb} + \lambda_{depth} \mathcal{L}_{depth} + \lambda_{sem} \mathcal{L}_{sem} + \lambda_{DSR} \mathcal{L}_{DSR}$$

相比前作真正推进了什么? 首次将颜色-深度-语义三模态统一进一个联合可优化的 Gaussian 场; DSR 解决了高斯尺度与几何表面不对齐的问题; RSKM 在防止灾难性遗忘方面显著优于局部共可见性策略。

没有解决什么? 语义信息来自 2D 分割模型的预定义封闭词汇, 不支持开放词汇查询; 语义特征的维度受限于实时性约束, 语义粒度有限。

对后续研究的影响是什么? SG-Field 证明了多模态 Gaussian 场的可行性, 为后续将更多模态 (如触觉、音频) 融入 3DGS 提供了架构参考。

与具身智能的关系

GS3LAM 的多模态统一表示使机器人同时回答三个问题："我在哪？""场景长什么样？""每个像素属于什么？"——这是执行复杂操作任务的基础。

14.3 X-GS：从重建模块到感知-思考框架

问题与核心洞察

3DGS 技术在过去两年中沿着多个方向并行发展：pose-free 3DGS、在线 SLAM、语义增强、VLM 对接。但这些方向彼此孤立——一个做在线 SLAM 的系统不带语义，一个做语义特征蒸馏的系统不实时，一个对接 VLM 的系统依赖离线地图。这种碎片化阻碍了 3DGS 成为具身智能的统一空间感知 backbone。

X-GS (arXiv) 的核心洞察是：3DGS 不应该是一堆孤立方法的集合，而应该是一个可扩展的感知-思考框架。系统被组织为两个明确分离的组件：Perceiver（感知器）负责在线 SLAM 和语义特征蒸馏；Thinker（思考器）负责对接下游多模态模型。

系统结构

X-GS-Perceiver。三个机制支撑实时运行：(1) 在线 VQ 模块将高维语义特征压缩为离散码本索引，降低内存占用；(2) GPU 加速网格采样用规则网格进行语义监督，充分利用并行计算；(3) 并行化流水线将 tracking、mapping、语义蒸馏解耦为异步阶段，总吞吐量约 15 FPS。支持 RGB/RGB-D 输入，可集成 SAM、CLIP、DINO 等 VFM。

X-GS-Thinker。接收 3D 语义高斯地图，对接多模态模型。论文演示了 3D 物体检测（查询类别返回 3D 包围盒）和场景描述生成（送入 VLM 生成文本）。接口开放，可扩展更多具身任务。

论文精读

论文试图解决什么痛点？3DGS 各方向（SLAM、语义、VLM 对接）彼此孤立，缺乏统一的框架设计。

核心假设是什么？在线 VQ + 网格采样 + 并行流水线可以将 3DGS-SLAM 加速到实时；Perceiver-Thinker 的分离架构允许独立升级感知和推理能力。

输入输出是什么？输入：RGB/RGB-D 流；输出：3D 语义高斯地图 + 下游任务结果（检测框、描述文本等）。

地图表示是什么？3D 语义高斯，通过 VQ 压缩语义特征。

Tracking 怎么做？ 集成多种 3DGS tracking 方法（MonoGS、GS-SLAM 等），通过统一接口调用。

Mapping 怎么做？ VQ 语义压缩 + 网格采样监督 + 并行流水线优化。

优化目标是什么？光度一致性 + 几何一致性 + VQ 语义重建误差。

相比前作真正推进了什么？首次从框架层面统一了 3DGS 的多个研究方向；Perceiver-Thinker 架构解耦了感知和推理，使系统具备可扩展性；15 FPS 的实时性能超过了 OpenMonoGS-SLAM 等非实时语义 SLAM 系统。

没有解决什么？ Thinker 目前只演示了简单下游任务，与复杂具身任务（如操作规划、导航决策）的集成尚未验证；VQ 压缩可能损失细粒度语义信息。

对后续研究的影响是什么？ X-GS 的框架设计思路——感知-思考分离、可扩展模块接口、VQ 压缩——可能成为未来 3DGS 具身系统的事实标准架构。

与具身智能的关系

X-GS 把 3DGS 从"重建模块"升级成"空间感知与思考框架"。Perceiver 对应"看"和"记"，Thinker 对应"想"和"做"，可独立升级。

14.4 OnlinePG：在线开放词汇全景映射

问题与核心洞察

全景映射（panoptic mapping）要求同时完成语义分割和实例区分：不仅要知道"这是桌子"，还要知道"这是第一张桌子、那是第二张桌子"。现有的 3DGS 语义方法大多是离线运行，且缺乏实例级理解。OnlinePG (arXiv) 把几何重建与开放词汇感知结合到 3DGS 中，采用 local-to-global 的滑动窗口范式实现在线全景映射。

系统的核心挑战来自 2D VLM 先验的噪声。从 CLIP、EntitySeg 等 2D 模型提取的掩码和特征充满噪声：同一物体在不同视角下被分割成不同片段，语义特征也因视角变化而不一致。OnlinePG 的关键洞察是：通过滑动窗口内的局部一致性和全局融合，可以从噪声 2D 先验中提炼出一致的 3D 实例。

系统结构

Local-to-Global 滑动窗口范式。 系统维护一个固定大小的滑动窗口（12 个关键帧，每 20 帧采样一个关键帧）。窗口内的关键帧构建局部一致地图，每 7 个关键帧执行一次局部到全局的融合。

多线索 3D 段聚类图（Multi-Cue Clustering Graph）。 给定滑动窗口内来自 VLM 的噪声 2D 掩码和特征，系统初始化 3D 高斯段，然后构建一个聚类图。图中的边权重由三个因素共同决定：(1) 几何重叠——两个 3D 段在空间上的交集体积；(2) 语义相似度——CLIP 特征向量的余弦相似度；(3) 视角一致性——两个段是否从足够不同的视角被观测到。这三个线索联合决定哪些段属于同一个实例。

显式空间属性网格（Spatial Attribute Grids）。 局部地图被体素化为显式网格，每个体素存储语义特征和实例标签。这种显式表示使局部到全局的融合变得高效。

双向二分 3D 高斯实例匹配（Bidirectional Bipartite Matching）。 局部地图融合到全局地图时，系统构建局部实例与全局实例之间的二分图，通过双向匹配（局部→全局和全局→局部两个方向）找到最优对应关系。匹配得分低于阈值 $1/N_{ins}$ 的对应被丢弃，确保只融合高置信度的匹配。

论文精读

论文试图解决什么痛点？ 现有方法要么离线运行，要么缺乏实例级理解，无法满足具身任务对在线全景映射的需求。

核心假设是什么？ 几何重叠 + 语义相似度 + 视角一致性的三线索联合可以克服 2D VLM 先验的噪声；滑动窗口的局部一致性可以为全局融合提供干净的输入。

输入输出是什么？ 输入：RGB-D 流 + 2D VLM 先验（CLIP/EntitySeg 掩码和特征）；输出：全局一致的 3D 全景地图，支持开放词汇查询。

地图表示是什么？ 3D 高斯 splat + 显式空间属性网格存储语义特征和实例标签。

Tracking 怎么做？ 假设已知或并行估计位姿（论文聚焦全景映射）。

Mapping 怎么做？ 滑动窗口局部映射 → 多线索聚类 → 空间属性网格体素化 → 双向二分匹配全局融合。

优化目标是什么？ 局部聚类目标（最大化同类实例内聚、最小化类间相似度）+ 全局匹配目标（匹配得分最大化）。

相比前作真正推进了什么？ 首个在线 3DGS 开放词汇全景映射系统；多线索聚类图从噪声 2D 先验中提炼一致 3D 实例；双向二分匹配实现了鲁棒的增量式全局融合。

没有解决什么？ 不支持动态物体重建；依赖深度和位姿输入；滑动窗口大小是固定超参数，未根据场景复杂度自适应调整。

对后续研究的影响是什么？ Local-to-Global 的滑动窗口范式为在线语义/全景映射提供了可复用的架构模板；多线索聚类思想可扩展到更多模态（如音频、触觉）。

与具身智能的关系

OnlinePG 使机器人在探索中实时构建实例级 3D 全景地图，支持"把红色杯子放到第二张桌子上"这类精细操作。开放词汇能力允许查询训练时未见过的类别。

14.5 VBGS-SLAM：变分贝叶斯高斯 Splatting SLAM

问题与核心洞察

现有 3DGS-SLAM 系统有一个共同的结构缺陷：它们是确定性的。相机位姿和场景参数通过梯度下降优化，不维护任何不确定性估计。这种确定性框架带来三个问题：对初始化敏感（位姿估计差时地图迅速退化）；灾难性遗忘缺乏量化机制（系统不知道"我对这个区域有多不确定"）；无法区分可靠观测和噪声观测。

概率 SLAM（如 EKF-SLAM、Factor Graph SLAM）长期维护位姿和地图的不确定性，但这些系统大多使用点云或稀疏特征表示。将概率推理引入 3DGS 面临数学困难：高斯 splat 的参数（位置、协方差、不透明度）之间的耦合是非线性的，不容易导出可追踪的后验分布。

VBGS-SLAM (arXiv) 的核心突破是利用多元高斯分布的共轭性质 (conjugate properties)，将 splat 图精化 (map refinement) 和相机位姿追踪 (pose tracking) 都纳入变分贝叶斯推断 (Variational Bayesian Inference) 框架，推导出两个变量的闭式更新规则。

系统结构

生成概率模型。 系统定义一个生成模型：3D 场景由一组高斯基元表示，每个基元的参数（位置 μ 、协方差 Σ 、不透明度 α ）被赋予先验分布。相机位姿 $\mathbf{T} \in SE(3)$ 也被建模为随机变量，其在李群上的分布通过变分近似处理。观测（RGB-D 图像）被建模为高斯 splat 渲染结果加高斯噪声。

变分推断与闭式更新。 传统变分推断需要迭代优化 ELBO（Evidence Lower Bound），计算代价高。VBGS-SLAM 利用多元高斯的共轭性质，推导出场景参数和位姿的闭式更新规则：

$$q^*(\theta_i) \propto \exp\{\mathbb{E}_{q_{-i}}[\log p(\mathbf{X}, \theta)]\}$$

其中 θ_i 可以是 splat 参数或位姿参数。闭式更新意味着每次观测到达时，参数可以在常数时间内更新，无需迭代优化。

位姿-地图耦合。 关键设计决策是位姿不确定性和地图不确定性的显式耦合。位姿估计的方差直接影响地图更新的信任度：位姿不确定高时，新观测对地图的影响被抑制；位姿确定后，地图更新才充分展开。这种耦合通过变分分布的联合推断自然实现，而非手工设计的启发式规则。

论文精读

论文试图解决什么痛点？ 3DGS-SLAM 系统的确定性优化缺乏不确定性估计，导致对初始化敏感、灾难性遗忘无法量化、噪声观测无法识别。

核心假设是什么？ 多元高斯的共轭性质允许推导出 splat 参数和 $SE(3)$ 位姿的闭式变分更新；变分框架可以在线联合推断位姿和地图的后验分布。

输入输出是什么？ 输入：RGB-D 流；输出：相机位姿的后验分布 + 场景参数的后验分布 + 渲染图像。

地图表示是什么？ 3D 高斯 splat，每个参数（位置、协方差、不透明度）都有后验分布而非点估计。

Tracking 怎么做？ 变分贝叶斯推断在线更新位姿的后验分布，而非点估计。

Mapping 怎么做？ 闭式变分更新规则精化 splat 参数的后验，位姿不确定性显式影响地图更新权重。

优化目标是什么？ 最大化 ELBO（Evidence Lower Bound），等价于最小化变分后验与真实后验之间的 KL 散度。

相比前作真正推进了什么？ 首次将变分贝叶斯推断完整引入 3DGS-SLAM，推导出闭式更新规则；显式维护位姿和场景参数的不确定性；不确定性耦合机制天然抑制了位姿漂移对地图的破坏。在 Replica、TUM-RGBD 和 AR-TABLE 数据集上展现出优于确定性方法的追踪精度和长序列鲁棒性。

没有解决什么？ 变分近似的精度受限于假设分布族的选择；语义信息未纳入概率框架；闭式更新对非高斯观测噪声的鲁棒性有待验证。

对后续研究的影响是什么？ 为 3DGS-SLAM 补上了概率基础，后续工作可以将语义特征、动态物体都纳入同一概率框架。

与具身智能的关系

VBGS-SLAM 补上 3DGS-SLAM 中最关键的东西：不确定性。概率输出可直接输入规划控制模块，实现不确定性感知的机器人行为。

14.6 AIM-SLAM：视觉几何基础模型增强的稠密单目 SLAM

问题与核心洞察

几何基础模型（VGGT、DUST3R、MASt3R 等）已经证明可以从单目图像中预测稠密 3D 结构。将这些模型用于 SLAM 时，现有方法局限于两种极端：要么只用相邻两帧（如 MASt3R-SLAM），视差多样性不足导致结构不一致；要么用大量固定窗口帧（如 VGGT-SLAM 用 16-32 帧），计算冗余且延迟大。

AIM-SLAM (arXiv) 的核心洞察是：关键帧选择不应该用固定窗口，而应该根据几何重叠和信息增益自适应地决定每次优化使用多少帧、用哪些帧。

系统结构

SIGMA 模块。 核心创新，三阶段：(1) 几何子集初始化——通过体素重叠筛选与当前帧有结构关联的关键帧；(2) 信息驱动重排序——按协方差缩减量（信息增益）对候选帧排序；(3) 自适应激活——稳定性测试决定何时终止选择，窗口大小自适应（3-15 帧）。

VGGT 点图预测。 SIGMA 选定的关键帧送入 VGGT（Visual Geometry Grounded Transformer），预测稠密深度、点云和帧间对应。VGGT 支持任意长度多视图输入，联合预测内参、位姿、深度和点云。

联合多视图 Sim(3) 优化。 VGGT 输出的点云在局部坐标系中表达，AIM-SLAM 对所有选定视图进行 Sim(3) 对齐，确保多视图一致性。联合优化相比逐帧配准显著降低漂移。后端闭环异步运行。

论文精读

论文试图解决什么痛点？ 几何基础模型用于 SLAM 时，固定窗口设计要么视差不足（两帧）要么冗余（16-32 帧），缺乏根据几何上下文自适应选择关键帧的机制。

核心假设是什么？ 体素重叠可以有效度量几何相关性；信息增益（协方差缩减）可以有效度量关键帧对位姿估计的贡献；自适应窗口大小可以在精度和效率之间取得最优平衡。

输入输出是什么？ 输入：未标定的单目 RGB 流；输出：全局一致的相机位姿 + 稠密 3D 重建。

地图表示是什么？ 3D 高斯 splat（从 VGGT 点云转换）。

Tracking 怎么做？ SIGMA 自适应选择关键帧 → VGGT 预测点图 → 联合多视图 Sim(3) 优化。

Mapping 怎么做？ VGGT 点云 + 高斯 splat 化 + 全局位姿图优化。

优化目标是什么？ 联合 Sim(3) 对齐误差最小化 + 信息增益最大化。

相比前作真正推进了什么？ SIGMA 的自适应关键帧选择比固定窗口更灵活高效；联合多视图 Sim(3) 优化显著提高了位姿精度；系统在真实世界数据集上达到 SOTA 位姿估计性能。支持 ROS 集成。

没有解决什么？ 语义理解不是 AIM-SLAM 的目标；VGGT 的推理成本仍然较高，在资源受限设备上的实时性有限；对动态场景的鲁棒性未充分验证。

对后续研究的影响是什么？ 证明了"自适应关键帧选择 + 基础模型预测 + 联合优化"的范式优于固定窗口设计。VGGT 作为第四篇将要详细展开的视觉几何基础模型，其与 SLAM 的融合只是开始。

与具身智能的关系

AIM-SLAM 代表了 2026 年的明确趋势：3DGS 吃进视觉几何基础模型的先验。分层架构让基础模型负责重推理，SLAM 负责在线精化，降低部署成本。

14.7 GSMem：3DGS 作为持久空间记忆

问题与核心洞察

具身智能体需要积累和保持空间知识。现有场景表示有两类极端：离散场景图（如 ConceptGraphs）提供高层语义抽象但丢失了对象之外的几何和视觉细节；静态视角快照（如 3D-Mem）保留了视觉保真度但视角依赖、无法外推新视点。

GSMem (arXiv) 发现一个更深层的问题：这些表示都缺乏**事后重新观察能力（post-hoc re-observability）**。如果初始探索时错过了目标物体，记忆缺口无法弥补——因为快照不能从新的角度看，场景图也不包含未被检测到的对象。

GSMem 的核心洞察是：3DGS 天然适合作为持久空间记忆——它显式参数化连续几何和稠密外观，支持从任意未占据视角实时渲染高保真图像。这意味着智能体可以“精神上重新访问”过去区域，从最佳视角观察以发现之前错过的细节。

系统结构

3DGS 持久记忆。探索过程中，系统不断从 RGB 流中优化 3D 高斯地图。与大多数 SLAM 系统不同，GSMem 的地图不只是用于追踪和重建——它是一个**可重访的记忆存储**，可以在任何时候被查询、被重新渲染。

多层检索-渲染机制。当智能体收到任务查询（如“find the remote control”）时，GSMem 同时启动两条并行的检索路径：

- **对象级场景图检索。**维护一个 3D 场景图，记录检测到的对象及其空间关系。如果目标在场景图中，直接定位到对应区域。
- **语义级语言场检索。**维护一个语言特征场（language field），即使没有检测到目标对象，语言嵌入也可以在语义空间中匹配查询文本，定位到候选区域。

两条路径强烈互补：场景图提供精确的对象级定位，语言场提供零样本的语义检索能力。即使场景图失败（目标未被检测到），语言场仍能定位候选区域。

最优视角选择（Optimal Viewpoint Selection）。检索到目标区域后，系统从 3DGS 地图中“幻觉”（hallucinate）最优观察视角——选择一个之前未物理占据但能最好地观察目标区域的虚拟相机位姿，渲染高保真图像供 VLM 进行推理。这种能力被称为**空间回忆（Spatial Recollection）**。

混合探索策略。探索阶段平衡两个目标：(1) VLM 驱动的语义评分——优先探索与任务相关的区域；(2) 3DGS 覆盖目标——通过高斯场的熵量化表示不确定性，优先探索信息增益最大的未观测区域。两者结合确保构建的记忆既全面又语义丰富。

论文精读

论文试图解决什么痛点？ 现有场景表示（场景图、视角快照）缺乏事后重新观察能力，导致记忆缺口无法弥补。

核心假设是什么？ 3DGS 的可微渲染能力可以使智能体从任意虚拟视角重新观察已探索区域；对象级场景图和语义级语言场的并行检索可以互补地定位目标。

输入输出是什么？ 输入：RGB 流 + 任务查询（自然语言）；输出：任务答案 / 导航路径 + 持久 3DGS 记忆。

地图表示是什么？ 3D 高斯 splat + 并行对象级场景图 + 语义级语言场。

Tracking 怎么做？ 集成现有 SLAM tracking（论文聚焦记忆表示和检索）。

Mapping 怎么做？ 在线优化 3DGS 地图，同时更新场景图和语言场。

优化目标是什么？ 探索阶段：最大化语义相关性 + 最大化信息增益（最小化高斯熵）。

相比前作真正推进了什么？ 首次将 3DGS 明确作为持久空间记忆用于具身探索；空间回忆能力使智能体能从虚拟视角重新观察；并行检索机制（场景图 + 语言场）显著提高了目标定位的鲁棒性。在 embodied question answering 和 lifelong navigation 任务上达到 SOTA。

没有解决什么？ 3DGS 地图随探索范围增长而膨胀，长期运行的可扩展性需要进一步优化；语言场的更新需要迭代优化，非完全实时；动态环境（物体移动）的记忆一致性未处理。

对后续研究的影响是什么？ "3DGS 作为空间记忆"的概念将地图的角色从"定位参考"扩展到"可推理的记忆介质"，为长期自主机器人提供了新的表示范式。

与具身智能的关系

GSMem 把 3DGS 地图从"定位工具"转变为"可重访的记忆介质"。空间回忆能力使智能体事后回到记忆中的任意位置，从新角度重新观察。这为第五篇的空间记忆主题提供了 3DGS 实现路径。

本章一句话总结

2026 年的 3DGS-SLAM 正在从"离线重建工具"进化为"具身智能的统一空间感知 backbone"——在线运行、语义统一、感知-思考分离、开放词汇、概率基础、基础模型融合、持久记忆，七个趋势汇聚成一条主线：从 SLAM 到空间智能的演进，在 2026 年已经不再是愿景，而是正在发生的工程现实。

第四篇：视觉几何基础模型与 SLAM 的重构

第15章 从手工几何到视觉几何基础模型

第14章末尾，我们讨论了AIM-SLAM与VGGT融合的初步尝试——将3DGS的实时渲染能力与基础模型的几何先验结合。那是2026年初的前沿。但为什么要做这种融合？为什么传统视觉几何流程需要被重写？本章回答这个问题，为第四篇建立认知框架。

传统SfM/SLAM pipeline由手工设计的模块串联而成：特征提取、特征匹配、RANSAC几何验证、PnP/本质矩阵估计、三角化、Bundle Adjustment。这套流程在过去二十年里支撑了整个三维视觉领域，但其结构性脆弱性在真实世界中被反复暴露。视觉几何基础模型（Visual Geometry Foundation Model, VGFM）的出现，代表了一种根本性的范式转换：不再逐个模块地优化，而是让模型直接学习"几何推理本身"。

15.1 传统视觉几何的瓶颈

传统Pipeline的六个环节

以COLMAP为代表的SfM pipeline和ORB-SLAM系列为代表的SLAM系统，遵循同一套模块化架构。每个模块都有明确的输入输出边界，数据以单向流的形式传递。

环节	功能	代表算法	输出
特征提取	检测并描述局部兴趣点	SIFT、ORB、SuperPoint	稀疏关键点+描述子
特征匹配	建立跨视图对应关系	NN匹配、SuperGlue、LoFTR	候选对应集合
几何验证	从候选中筛选内点	RANSAC、MAGSAC	内点子集+初始姿态
姿态估计	恢复相机相对运动	PnP、本质矩阵分解	R、t、（可选）K
三角化	从对应点恢复3D坐标	中线三角化、LST	稀疏/半稠密点云
Bundle Adjustment	联合优化所有变量	Levenberg-Marquardt	精化后的姿态与地图

这套流程的瓶颈不在任何单一环节，而在于其串联结构本身。每个环节的误差都会传递给下游，且各环节之间存在隐性依赖关系，导致整个系统的鲁棒性等于各模块鲁棒性的乘积。

环节一：特征提取的盲区

特征提取器是整个pipeline的"守门人"。SIFT（ICCV 1999）通过DoG金字塔检测尺度不变关键点，用128维梯度直方图描述局部外观。ORB（ICCV 2011）以FAST角点检测+BRIEF二进制描述子实现了实时性能。SuperPoint（CVPRW 2018）用深度网络替代手工设计，将检测与描述集成到一个CNN中。

这些方法的共同局限是：**检测器找不到特征的地方，系统就完全失明。**

弱纹理表面——白墙、光滑桌面、漆面车身、大面积天空——是SIFT/ORB的禁区。SuperPoint通过密集特征图缓解了部分问题，但仍受限于训练数据的域覆盖范围。更严重的是重复纹理场景：网格地板、百叶窗、太阳能板阵列，这些结构会导致检测器产生活力旺盛但不可重复的关键点，直接污染下游匹配。

实验数据说明了这一点：在HPatches基准上，SIFT+FLANN的匹配准确率约62%，ORB更差。在弱纹理主导的室内场景（如ScanNet）中，稀疏匹配方法的召回率会进一步下降30%-50% (CVF开放获取)。

环节二：特征匹配的脆弱边界

匹配层的任务是将两幅图中的描述子对应起来。传统方法是最近邻搜索+比率测试。SuperGlue (CVPR 2020) 用图神经网络+注意力机制替代了暴力匹配，将匹配问题建模为可学习的最优传输问题。LoFTR (CVPR 2021) 更进一步，抛弃了检测器，直接在密集特征图上用Transformer建立半稠密对应。

SuperGlue和LoFTR大幅提升了匹配质量，但各自的脆弱边界依然清晰：

SuperGlue依赖检测器输出的稀疏关键点。检测器失效的区域（弱纹理、运动模糊），SuperGlue无能为力。其注意力机制仅在已检测的关键点之间传播信息，无法创造新的对应。

LoFTR作为无检测器方法，在弱纹理区域表现出色。但它是半稠密匹配，计算成本高，且在极端视角变化（ $>60^\circ$ ）时，粗粒度匹配阶段的召回率急剧下降。更重要的是，LoFTR的输出是图像对之间的匹配，不具备多视图一致性约束——它不知道第三幅图的存在。

MASt3R (ECCV 2024) 在匹配层面做出了根本性改进：它在DUST3R的点图回归基础上增加了密集局部特征头，用InfoNCE损失训练匹配描述子，实现了3D感知的特征匹配。但这已是基础模型时代的产物，而非传统pipeline的增量修补。

环节三：RANSAC的几何验证困境

RANSAC (Random Sample Consensus) 是几何验证的事实标准。它从候选对应中随机抽取最小子集（如5点算本质矩阵、3点算PnP），用内点计数评估模型质量，迭代直至收敛。

RANSAC的核心假设是：内点在候选集合中占多数。当这个假设被打破——外点比例超过70%-80%时——RANSAC的迭代次数呈指数级增长，且收敛到错误解的概率急剧上升 (arXiv)。

以下场景系统性地触发RANSAC失效：

- **重复纹理建筑**：大面积相同窗户导致大量几何一致的错误匹配
- **动态物体**：行人、车辆产生大量暂时几何一致的外点
- **宽基线**：视角变化大时，正确匹配稀疏，外点比例飙升
- **遮挡严重区域**：可见特征少，候选集合本身就不足

MAGSAC (CVPR 2019) 等改进方法通过加权内点评分提升了阈值选择的鲁棒性，但并未改变RANSAC在超高外点比例下的本质局限。

环节四：姿态估计的标定依赖

PnP (Perspective-n-Point) 和本质矩阵分解要求已知相机内参矩阵K。单目SLAM因此面临一个结构性困境：没有K就无法正确估计运动，没有运动就无法标定K。

ORB-SLAM系列的做法是假设内参已知（由用户预先标定）。这在实验室环境下可行，但在实际部署中——不同焦距的手机镜头、变焦相机、未知相机设备——构成了硬门槛。

在线自标定（Auto-calibration）理论上可以从未知内参的图像序列中恢复K，但需要足够丰富的相机运动（平移+旋转），且在退化运动下（纯旋转、平面运动）无法唯一确定内参。

尺度是另一个根本性缺失。单目视觉无法从图像本身确定绝对尺度——整个重建结果可以被任意缩放而不改变重投影误差。这导致单目SLAM的轨迹和地图只有"形状"而没有"尺寸"，与IMU、LiDAR等物理传感器融合成为必需。

环节五：三角化的精度悬崖

三角化从两个（或多个）视图中匹配点的投影射线交点恢复3D坐标。精度严重依赖于**基线长度**和**匹配精度**。

当两个相机中心连线（基线）与观察方向近乎平行时，两条投影射线几乎共线，微小的匹配误差就会导致巨大的深度不确定性。这就是"低视差"问题——相机沿光轴运动时，三角化误差发散。

传统SfM对此的应对是"延迟三角化"：等待关键帧之间产生足够的基线后再进行三角化。但这引入了时延，且在SLAM前端需要立即的深度信息来初始化跟踪时无法使用。

环节六：Bundle Adjustment的误差传递

Bundle Adjustment (BA) 是多视图几何的"黄金标准"——联合优化相机姿态和3D点坐标，最小化重投影误差。但它有几个结构性弱点：

计算复杂度。BA的核心是求解正规方程，其稀疏结构虽可用Schur complement加速，但变量数随关键帧和路标点数增长而膨胀。在长序列上，BA的求解时间可能从毫秒级退化到秒级。

局部极小值。BA是非凸优化，初始化质量决定最终解。错误的初始匹配会导致BA收敛到错误的局部极小值，产生"折叠"的地图或漂移的轨迹。

低视差区域缺乏约束。当观测视差不足时，BA的深度方向约束弱，优化结果在该方向上"滑动"。DROID-SLAM (NeurIPS 2021) 的可微分BA层虽然将BA嵌入端到端训练，但"缺乏平滑正则化、低视差下约束不足"的根本问题仍未解决 (arXiv)。

误差逐级累积。一个特征点在第三帧的跟踪误差，会通过BA传播到第一帧和第二帧的姿态估计，再通过共视关系扩散到整个因子图。这种误差传播在缺乏闭环检测的长序列上表现为尺度漂移和轨迹弯曲。

串联结构的系统性脆弱

以上六个环节的脆弱性不是独立事件，而是通过pipeline的串联结构相互放大。

以一个具体场景为例：一辆自动驾驶汽车在夜间行驶，路面反光导致弱纹理区域扩大。特征提取器（SIFT/ORB）输出稀疏且不稳定的关键点集。匹配层因关键点不足而产生大量错误候选。RANSAC在外点海洋中挣扎，可能收敛到错误的运动估计。错误的姿态导致三角化产生错误的深度。BA试图修正这些错误，但非凸优化陷入局部极小。最终结果是轨迹漂移数十米，地图完全不可用。

这个场景在传统SLAM文献中被反复讨论，但每个解决方案只解决一个环节：更好的特征、更鲁棒的匹配、更聪明的外点剔除。没有人质疑串联结构本身——直到基础模型的出现。

两个额外的结构性缺失

除上述六个环节外，传统pipeline还有两个全局性的结构性缺失。

动态环境的硬假设。整个传统pipeline建立在"世界是静态的"这一假设之上。BA最小化重投影误差时，假设所有3D点都是静止的。当动态物体（行人、车辆、开门）进入视野，其上的特征点产生违反静态假设的投影，BA被迫在"把这些点当作静态"和"抛弃这些点"之间做选择。前者导致姿态估计被动态物体拉偏，后者损失了有价值的观测信息。

DROID-W（arXiv 2026）尝试通过不确定性加权BA来缓解这一问题，用DINOv2特征的多视图不一致性检测动态像素。但这是在BA框架内的修补，而非根本性解决——pipeline仍然假设静态背景占主导。

尺度处理的断链。单目重建的结果在尺度上是任意的，且尺度漂移贯穿整个pipeline。前端视觉里程计（VO）输出相对姿态的尺度不确定；三角化引入深度估计的尺度歧义；BA虽能在局部窗口内约束尺度一致性，但无法确定绝对尺度。闭环检测可以校正漂移的方向，但无法提供绝对尺度。这导致单目SLAM的重建结果只能用于定性可视化，不能用于定量测量——机器人知道自己"转了90度"，但不知道自己"走了10米还是20米"。

15.2 基础模型的反转

从"学模块"到"学几何推理"

视觉几何基础模型的核心洞察可以用一句话概括：**让模型直接从大规模数据中学习几何推理，而不是手工设计几何求解器的每一步。**

传统pipeline中，机器学习只扮演"辅助工具"的角色——SuperPoint替代SIFT，SuperGlue替代NN匹配，RAFT替代光流计算。几何求解的核心（三角化、BA、坐标系对齐）仍由手工算法完成。基础模型则将整个几何推理过程压缩到一个前馈网络中。

这不是简单的"端到端替代"。其本质区别在于：

维度	传统Pipeline	视觉几何基础模型
基本单元	手工设计的几何模块	数据驱动的参数化网络
知识来源	多视图几何的数学约束	大规模3D标注数据的统计先验
特征表示	稀疏关键点/描述子	密集视觉token/Transformer特征
地图表示	稀疏/半稠密3D点云	像素对齐的点图（pointmap）
相机标定	必需预先标定	内参可从点图恢复，或直接预测
多视图融合	逐对匹配+全局BA	全局注意力机制+统一坐标回归
失败模式	逐级误差传递	域外分布泛化下降

点图表示：统一几何输出

DUST3R (CVPR 2024) 是这场范式转换的起点 (CVF开放获取)。它将双视图重建重新表述为**点图回归任务**：给定一对图像，网络直接输出两幅与输入图像尺寸对齐的密集3D点图，所有点位于同一坐标系中。

形式上，对于输入图像 $I_1, I_2 \in \mathbb{R}^{H \times W \times 3}$ ，DUST3R预测：

$$X_{1,1}, X_{2,1} \in \mathbb{R}^{H \times W \times 3}, \quad c_1, c_2 \in \mathbb{R}^{H \times W}$$

其中 $X_{i,1}$ 表示图像 I_i 中每个像素在第1帧坐标系中的3D坐标， c_i 是对应的置信度。点图的维度与输入图像相同，建立了像素到3D点的直接映射。

点图表示的优势在于其**可逆性**：从点图中可以恢复相机内参、深度、像素对应关系和相机姿态。具体来说，通过加权Procrustes对齐两个点图，可以得到它们之间的相似变换（旋转+平移+尺度）。通过将3D点投影回2D平面，可以反推焦距。通过将点图转换到各自的相机坐标系，可以得到深度图。

这种表示统一了单目深度估计（点图退化为深度图）、双视图重建（两个点图共享坐标系）和多视图融合（全局对齐所有点图）。

DUST3R的结构与意义

DUST3R采用对称的双分支Transformer编码器（基于CroCo预训练的ViT特征）加Transformer解码器。解码器通过交叉注意力进行跨视图推理，让一个视图的所有token可以attend到另一个视图的所有token。这种全局推理使得模型能够处理宽基线和部分重叠的情况，因为它不依赖于脆弱的特征匹配集合 (arXiv)。

DUST3R的意义远超其性能指标。它证明了一个核心命题：**几何重建可以从头到尾由Transformer完成，无需显式的三角化、BA或相机建模**。后续工作在此基础上扩展：MASt3R增加匹配头提升对应精度；Fast3R/MUSt3R扩展到多视图；Spann3R引入空间内存做增量重建；MonST3R适配动态场景；VGGT进一步统一为单模型多任务输出。

MASt3R：将匹配植入3D

MASt3R (ECCV 2024) 在DUST3R的基础上增加了一个关键组件——**密集局部特征头** Head_m ，为每个解码器输出额外预测密集特征图 D_1, D_2 (arXiv)。

匹配损失采用InfoNCE形式：对于图像1中的像素 i 和图像2中的像素 j ，若它们对应同一个3D ground truth点，则为正样本对，反之为负样本。网络学习到的描述子使得正样本对在特征空间中靠近，负样本对远离。

$$\mathcal{L}_{match} = - \sum_{(i,j) \in \mathcal{M}^+} \log \frac{\exp(D_1(i) \cdot D_2(j) / \tau)}{\sum_k \exp(D_1(i) \cdot D_2(k) / \tau)}$$

总损失是点图回归损失与匹配损失的加权和。MASt3R的核心贡献在于**将匹配任务植根于3D几何**：描述子不是孤立学习的，而是与点图回归联合优化的。这使得匹配具有隐式的3D感知能力，在遮挡和视角变化下更鲁棒。

VGGT：从双视图到任意多视图

VGGT (CVPR 2025) 将基础模型的能力推向新高度：在单前馈过程中处理多达数百张图像，联合预测相机参数、点图、深度图和3D点轨迹 (arXiv)。

VGGT的核心是**交替注意力机制**：给定 S 张输入图像的token序列，编码器交替执行帧内自注意力（Frame Attention）和全局自注意力（Global Attention）。

$$F'_i = \text{SelfAttn}(F_i), \quad F_i \in \mathbb{R}^{N \times C}$$
$$H = \text{SelfAttn}(\text{Concat}(F_1, F_2, \dots, F_S)), \quad H \in \mathbb{R}^{SN \times C}$$

帧内注意力捕获局部空间结构，全局注意力建模跨视图关系。通过交替堆叠，模型在保持局部细节的同时实现全局几何推理。

重建头分为两部分：相机头预测外参和内参；DPT头预测深度图和不确定性图。值得注意的是，VGGT不依赖任何后处理优化——预测结果本身就在全局坐标系中对齐。在多项3D任务上，VGGT的原始输出已经超过了经过COLMAP后处理的专门方法。

基础模型的能力谱系

视觉几何基础模型并非单一实体，而是一个能力谱系。按照处理能力和输出范围，可以排列如下：

模型	输入规模	核心输出	后处理需求	适用场景
Depth Anything v2	单张	深度图	无	单目深度
MoGe	单张	点图	无	单目3D形状
DUST3R	图像对	双视点图	全局对齐	双视图重建
MASt3R	图像对	点图+匹配	全局对齐	匹配+重建
Fast3R/MUSt3R	多张	多视点图	可选对齐	批量重建
VGGT	任意数量	相机+深度+点图+轨迹	无	通用3D感知

这个谱系的发展方向是清晰的：从专用到通用，从逐对处理到全局推理，从依赖后处理到一次前馈输出完整结果。

失败模式与未解问题

基础模型并非万能。其主要脆弱性在于**域外泛化**：训练数据分布之外的场景（如极端光照、非朗伯表面、透明/反射物体），预测质量会显著下降。冻结权重的模型无法适应全新的环境特征，这是MASt3R-SLAM、VGGT-SLAM等后续系统需要引入微调或自适应机制的原因 (arXiv)。

另一个未解问题是**尺度歧义**。虽然MASt3R通过修改训练策略可以输出度量尺度（metric scale）的点图，但这依赖于训练数据中的尺度一致性。在完全没有先验的开放场景中，绝对尺度仍然是一个开放问题。

长序列漂移也未被根本解决。DUST3R/MASt3R本质上是逐对处理的，扩展到多视图需要全局对齐优化，误差仍会累积。Spann3R和MUSt3R通过空间内存机制缓解了这一问题，但长距离一致性仍不如带闭环检测的传统SLAM系统。

与具身智能的关系

基础模型对具身智能的影响是双重的。

正面：校准自由、开箱即用的3D感知能力，大大降低了机器人部署门槛。传统SLAM需要专业工程师标定相机、调参、选择特征类型；基础模型可以直接从RGB视频流中输出相机轨迹和稠密点云。这使得"拿到机器人即跑"成为可能。

挑战：基础模型的推理成本较高。DUST3R/MASt3R在消费级GPU上处理一对512×512图像需要约100-200ms，VGGT处理数十张图像需要数秒。实时SLAM系统要求30fps以上的前端处理速度，这需要在模型压缩、蒸馏和硬件加速方面持续投入。

更根本的是，基础模型学习的是**被动观察**的几何先验，而非**主动交互**的空间推理。机器人需要通过触摸、推动、抓取来理解物理空间，这种具身化的3D理解是当前基础模型尚未触及的领域。

本章一句话总结

视觉几何基础模型的本质，是用大规模数据中学到的统计先验替代手工设计的几何求解链——它不完美，但它让整个3D视觉pipeline从"精密仪器"变成了"即插即用的感知模块"，为具身智能的空间感知提供了新的基础设施。

第16章 DUST3R、MASt3R 与 MASt3R-SLAM

16.1 DUST3R：从图像对直接回归 3D

多视图三维重建的传统流水线是：先估计相机内参和外参，再通过三角化恢复三维点。这个顺序执行了几十年——从SfM到MVS，没有精确标定，后续几何计算无从谈起。但标定繁琐，外参估计在宽基线、弱纹理场景下容易失败，整套系统的误差会在不同阶段累积。

DUST3R (Dense Unconstrained Stereo 3D Reconstruction) 提出了一个根本不同的范式：给定两张图像，网络直接输出每个像素对应的3D点坐标，即一个 **pointmap** (点图)，无需任何相机参数 (arXiv)。

16.1.1 核心洞察：Pointmap 表示

DUST3R 的核心创新是 pointmap 表示。对图像 $I^v \in \mathbb{R}^{H \times W \times 3}$ ，网络输出一个同分辨率的3D点图 $X^{v,1} \in \mathbb{R}^{H \times W \times 3}$ ，表示将图像 v 中每个像素映射到以第一帧坐标系为参考的三维空间位置。pointmap 同时编码了三类信息：(a) 场景几何，(b) 像素到三维点的对应关系，(c) 两帧之间的视角关系。

这种表示彻底绕开了传统流程。相机内参、相对位姿、深度图——所有量都可以从 pointmap 中事后导出，而不是作为前置条件输入。具体而言，给定 pointmap $X^{v,1}$ ，深度图可通过 $Z_v = \|X^{v,1}\|$ 恢复；相机内参和位姿可通过 pointmap 的 Procrustes 对齐或 PnP 解算；像素对应关系直接由 pointmap 中相同3D坐标点的像素位置给出。标定从"必要条件"变成了"可选项"。

16.1.2 网络架构

DUST3R 采用标准的 Transformer encoder-decoder 架构 (CVF开放获取)。两张输入图像先经共享的 ViT encoder 编码为 patch token，再分别送入两个 view-specific 的 transformer decoder。decoder 之间通过 cross-attention 实现跨视图信息交换，使每个视图的 token 能 attending 到另一视图的全部 token。最终，每个 decoder 末端的回归头输出该视图的 pointmap 和置信度图。

编码器基于 CroCo (Cross-view Completion) 的预训练权重初始化，这让 DUST3R 在训练初期就具备跨视图推理能力。decoder 的 cross-attention 机制是网络理解两帧几何关系的关键——它不是先找匹配点再三角化，而是在注意力交互中隐式地建立了像素级对应关系。

16.1.3 训练目标

DUST3R 使用 confidence-aware 回归损失：

$$\mathcal{L}_{\text{conf}} = \sum_{v \in \{1,2\}} \sum_{i \in \mathcal{D}^v} C_i^{v,1} \cdot \ell_{\text{regr}}(v, i) - \alpha \log C_i^{v,1} \quad (26)$$

其中 ℓ_{regr} 采用尺度不变的回归损失，通过归一化因子使网络对绝对尺度不敏感； $C_i^{v,1}$ 是网络为每个像素预测的几何置信度。损失的第二项鼓励网络对不同像素给出可区分的置信度——天空、反光面等难以确定3D位置的区域应输出低置信度。训练数据来自多个数据集 (ARkitScenes、BlendedMVS、CO3D、Habitat、MegaDepth、ScanNet++、StaticThings3D 等)，总量达数百万对图像。

16.1.4 从 Pairwise 到全局重建

DUST3R 本质上是两视图模型。给定 N 张图像，需进行 $O(N^2)$ 次前向传播，得到所有图像对的 pointmap。随后通过一个全局对齐过程将这些成对的 pointmap 统一到一个共同坐标系中：先用带权重的 Procrustes 对齐 (RANSAC 框架下) 估计相似变换，再通过全局优化最小化所有 pointmap 之间的一致性。

全局对齐的优化目标为：

$$\chi^* = \arg \min_{\chi, P, \sigma} \sum_{e \in \mathcal{E}} \sum_{v \in e} \sum_{i=1}^{HW} C_i^{v,e} \|\chi_i^v - \sigma_e P_e X_i^{v,e}\| \quad (27)$$

其中 χ_i^v 是全局坐标系中像素 i 的3D坐标， P_e 和 σ_e 是图像对 e 的刚性变换和尺度， $C_i^{v,e}$ 是网络预测的置信度。该优化通过加权最小二乘迭代求解，高置信度预测在优化中占更大权重。

这个全局对齐是离线操作，计算复杂度随图像数量增长，是 DUST3R 从"重建工具"走向"实时 SLAM"的主要瓶颈。当图像超过几十张时， $O(N^2)$ 的成对推理和全局优化时间都迅速变得不可接受。

16.1.5 优缺点与适用场景

DUST3R 的优势在于宽基线鲁棒性——由于不依赖显式的极线几何约束，它在视角变化剧烈（大旋转、大平移）时仍能产生合理的 pointmap。它在 zero-shot 深度估计和相对位姿估计基准上设立了新 SOTA。

失败模式同样明确。无纹理区域（如白墙、天空）的 pointmap 预测缺乏几何约束，置信度低但不一定可靠。反光面和透明物体的3D位置本质歧义。最重要的是，DUST3R 无法实时运行，且多视图重建需要全局对齐后处理，不支持增量式建图。

16.1.5 DUST3R 论文精读

1. 论文试图解决什么痛点？

多视图立体重建依赖精确的相机标定和位姿估计，这些参数在真实场景中难以获取且容易引入误差。DUST3R 旨在完全消除这一前置依赖，实现"无约束"的稠密立体三维重建。

2. 核心假设是什么？

Transformer 架构通过跨视图注意力机制，可以从成对图像中隐式学习像素对应关系和三维几何，无需显式地强制执行射影几何约束。3D 点图 (pointmap) 是一种足够的场景表示，所有下游量 (位姿、深度、对应关系) 都可从中恢复。

3. 输入输出是什么？

输入：两张 RGB 图像 (无需相机参数、无需位姿)。输出：两张 pointmap ($X^{1,1}, X^{2,1}$)，分别表示两帧中每个像素在第一帧坐标系下的3D坐标；以及对应的置信度图 $C^{1,1}, C^{2,1}$ 。

4. 地图表示是什么？

Pointmap——一种像素对齐的密集三维点云表示。每个像素 (u, v) 直接对应一个3D点 (X, Y, Z) ，无需通过深度图和内参矩阵的乘法转换。

5. Tracking 怎么做？

DUST3R 本身不做 tracking。它通过成对 pointmap 的 Procrustes 对齐恢复两帧之间的相似变换 (旋转、平移、尺度)，可作为相对位姿估计的起点。

6. Mapping 怎么做？

通过全局对齐策略，将所有成对 pointmap 表达在共同参考系中。具体是：先进行成对的相似变换估计，再进行全局优化统一所有 pointmap。

7. 优化目标是什么？

Confidence-aware 回归损失，对高置信度预测赋予更高权重，同时通过置信度正则化项防止网络对所有像素输出统一的低置信度。

8. 相比前作真正推进了什么？

首次将"无标定、无位姿"的稠密三维重建统一到一个前馈网络中。此前的 MVS 方法 (COLMAP、MVSNet 等) 都依赖已知相机参数；此前的单目深度估计 (MiDaS、DPT) 只输出单帧深度图，不处理多帧几何一致性。DUST3R 证明了 pointmap 是一种足以统一深度估计、位姿估计和稠密重建的通用表示。

9. 没有解决什么？

(1) 仅处理图像对，多视图场景需要昂贵的全局对齐；(2) 深度预测是尺度不变的，不是绝对度量深度；(3) 匹配精度有限——pointmap 回归擅长宽基线鲁棒匹配，但亚像素精度不如专用匹配器；(4) 推理速度不足以支撑实时 SLAM。

10. 对后续研究的影响是什么？

Pointmap 表示成为后续工作 (MASt3R、Fast3R、Spann3R、MUST3R 等) 的基础构件。它开启了一个研究方向：用视觉基础模型替代传统几何流水线中的标定、匹配、三角化等环节。

16.2 MAST3R：匹配与几何的融合

DUST3R 证明了 pointmap 回归的可行性，但在一个关键任务上暴露了短板：像素级匹配精度。当用于视觉定位、SfM 等需要精确对应关系的下游任务时，DUST3R 的匹配精度不如专门的特征匹配方法（如 SuperGlue、LoFTR）。

MASt3R（Matching and Stereo 3D Reconstruction）在 DUST3R 基础上引入两个关键改进 (ECCV):

第一，增加匹配头。 在保持原有 pointmap 回归头的同时，新增一个 descriptor head，为每个像素输出一个 d 维 ($d = 24$) 的密集特征描述子 $D_1, D_2 \in \mathbb{R}^{H \times W \times d}$ 。这使 MASt3R 兼具两种能力：pointmap 头提供粗粒度三维先验，descriptor head 提供细粒度匹配能力。

第二，绝对度量深度。 DUST3R 的损失函数中引入了归一化因子使重建尺度不变；MASt3R 取消了这一归一化，直接回归绝对度量深度。这使 MASt3R 输出的 pointmap 具有真实世界尺度，对 SLAM 等应用至关重要。

第三，快速互惠匹配。 密集特征匹配面临二次复杂度问题——两个 $H \times W$ 特征图之间的全对相似度计算不可行。MASt3R 提出了一种快速的互惠最近邻匹配（fast reciprocal nearest neighbor matching）方案，将匹配速度提升数个数量级，同时保持精度并附带理论保证。

模块	DUST3R	MASt3R
编码器	ViT-Large	ViT-Large
解码器	ViT-Large	ViT-Base
回归头	pointmap + confidence	pointmap + confidence
匹配头	无	dense descriptor
深度类型	尺度不变	绝对度量深度
匹配机制	3D nearest neighbor	快速互惠匹配

MASt3R 将匹配重新定义为3D问题而非2D问题。传统匹配器在图像平面上寻找对应像素，MASt3R 则在 pointmap 引导下在3D空间中寻找匹配——这赋予了它在极端视角变化（可达180度）下的鲁棒性。在 Map-free 视觉定位基准上，MASt3R 的 VCRE AUC 比之前最佳方法高出约30个百分点。在7Scenes和 Cambridge Landmarks 等定位数据集上也显著超越 DUST3R 和传统匹配方法。

损失函数方面，MASt3R 保留了 confidence-aware 回归损失，并新增了基于 InfoNCE 的匹配损失，使每个像素最多与另一幅图像中的一个像素匹配：

$$\mathcal{L}_{\text{match}} = - \sum_{(i,j) \in \mathcal{M}} \log \frac{\exp(s(D_i^1, D_j^2)/\tau)}{\sum_k \exp(s(D_i^1, D_k^2)/\tau)} \quad (28)$$

其中 $s(\cdot, \cdot)$ 是余弦相似度， τ 是温度参数。总损失为 $\mathcal{L} = \mathcal{L}_{\text{conf}} + \beta \mathcal{L}_{\text{match}}$ ($\beta = 1$)。

MASt3R 在65万对图像上训练35个epoch，以 DUST3R 的预训练权重初始化，保留了前者强大的几何先验能力，同时将匹配精度提升到专用匹配器水平。训练分辨率为512的短边（长宽比变化从512×160到512×384），使用 AdamW 优化器，学习率采用余弦衰减。损失权重设置为 $\alpha = 0.2$ （置信度损失）， $\beta = 1$ （匹配损失）。

MASt3R 的改进不是简单的"加法"——增加匹配头和匹配损失后，网络内部的几何推理方式发生了质变。pointmap 回归头使网络必须理解场景的三维结构，匹配头则迫使网络学习可区分的局部特征。两者的联合训练产生了协同效应：3D 先验引导特征学习聚焦于几何有意义的区域，而精确的匹配反过来提升了 pointmap 的精度。可以说，MASt3R 让 DUST3R 从"一个会匹配的重建器"变成了"一个会重建的匹配器"。

16.3 MAST3R-SLAM：从重建先验到实时 SLAM

DUST3R 和 MAST3R 解决了"两幅图像能重建什么"的问题。但 SLAM 需要回答的是：给定连续视频流，如何实时地跟踪相机并构建全局一致的三维地图？MASt3R-SLAM 是首个围绕两视图3D重建先验构建的完整实时稠密 SLAM 系统 (CVF开放获取)。

16.3.1 系统概览

MASt3R-SLAM 的输入是单目 RGB 视频序列，输出是全局一致的相机位姿和稠密三维几何。系统运行速度约 15 FPS (NVIDIA RTX 4090)，不需要固定或参数化的相机模型假设——唯一的前提是相机有一个唯一的光心 (central camera model)。

系统由四个核心模块组成：MASt3R 预测与 pointmap 匹配、前端跟踪与局部融合、回环检测与图构建、后端全局优化。

16.3.2 通用相机模型：从 Pointmap 到 Ray

MASt3R-SLAM 的一个关键设计是不假设已知相机内参。它将 pointmap 转换为光线方向：对 pointmap 中的每个3D点 X ，计算从相机中心出发的射线方向 $\psi(X) = X/\|X\|$ 。这样，每个像素对应一条视线方向而非一个3D点。这种 ray-based 表示对深度误差不敏感——单目系统最脆弱的就是深度估计——同时兼容任意中心相机模型（针孔、广角、变焦均可）。

16.3.3 Pointmap 匹配：从2秒到2毫秒

MASt3R 的原生匹配需要在全像素上做互惠最近邻搜索，耗时约2秒——完全无法实时。MASt3R-SLAM 提出高效的迭代投影匹配 (iterative projective pointmap matching)：利用 pointmap 已知的3D结构，将参考帧的3D点投影到当前帧，在投影位置附近的小窗口内搜索特征匹配。这是一种 local search 策略，避免了全局穷举。配合 GPU 并行化，匹配时间降至约2毫秒——比之前快了约1000倍。

匹配后还有一轮特征精化 (feature refinement)，在迭代投影的粗匹配基础上用 MAST3R 的 descriptor 进一步优化对应关系精度。

16.3.4 前端跟踪：最小化 Ray 方向误差

给定当前帧与关键帧之间的像素对应关系，跟踪模块估计相对位姿 $\mathbf{T}_{kf}$ 。直接最小化3D点误差对深度预测中的离群值敏感，因此 MAST3R-SLAM 采用 ray 方向误差：

$$E_r = \sum_{m,n \in \mathbf{m}_{f,k}} \rho \left(\frac{\psi(\tilde{X}_{k,n}^k) - \psi(\mathbf{T}_{kf} X_{f,m}^f)}{w(q_{m,n}, \sigma_r^2)} \right) \quad (29)$$

其中 $\psi(\cdot)$ 将3D点归一化为射线方向， ρ 是鲁棒核函数， w 是基于匹配质量的权重函数。Ray 方向误差有界（最大 π ），天然对深度估计错误鲁棒。系统还在目标函数中加入一个小权重的距离误差项，防止纯旋转场景下优化退化。位姿更新通过 Gauss-Newton 方法在 IRLS（迭代重加权最小二乘）框架中高效求解。

16.3.5 局部融合：加权平均滤波

每一帧 MAST3R 输出新的 pointmap，直接替换关键帧 pointmap 会引入抖动。MASt3R-SLAM 采用增量式加权平均融合：

$$\tilde{X}_k^k \leftarrow \frac{\tilde{C}_k^k \cdot \tilde{X}_k^k + C_f^k \cdot (\mathbf{T}_{kf} X_f^k)}{\tilde{C}_k^k + C_f^k}, \quad \tilde{C}_k^k \leftarrow \tilde{C}_k^k + C_f^k \quad (30)$$

新的 pointmap 经位姿变换后，与现有关键帧 pointmap 按置信度加权融合。这种增量式滤波融合多视角信息，随着时间推移，关键帧的 pointmap 越来越稳定和精确。实验表明，加权融合在无标定场景下比选择最高置信度 pointmap 的平均 ATE 更低，在 EuRoC 上可降低 1.3cm 误差。

16.3.6 回环检测与后端优化

关键帧图（pose graph）由顺序跟踪边和回环闭合边构成。回环候选通过 ASMK（Bag-of-Visual-Words 变体）框架在 MAST3R 编码特征上检索获得，再经几何验证确认是否构成有效回环。

后端采用二阶全局优化（Gauss-Newton），在整个图上联合最小化所有边的 ray 方向误差。与一阶梯度下降相比，Gauss-Newton 收敛更快，适合大规模实时优化。系统利用稀疏 Cholesky 分解高效求解正规方程。

16.3.7 实验结果

数据集	DROID-SLAM	MASt3R-SLAM (标定)	MASt3R-SLAM (无标定)
TUM RGB-D (avg ATE, m)	0.043	0.039	0.092
7-Scenes (avg ATE, m)	0.065	0.039	0.066
EuRoC (avg ATE, m)	0.032	0.041	0.164

MASt3R-SLAM 在 TUM RGB-D 和 7-Scenes 上达到或超过 DROID-SLAM 的精度。在 ETH3D-SLAM（训练集）上，其鲁棒性曲线（低于阈值的成功轨迹数）优于所有对比方法，具有最佳的 ATE 和 AUC。无标定版本在 TUM 和 7-Scenes 上仍然优于使用 GeoCalib 做初始标定的 DROID-SLAM* 基线，证明了3D重建先验在未知内参场景下的优势。

在 ETH3D-SLAM 基准上（训练序列），MASt3R-SLAM 的 AUC 指标（低于各 ATE 阈值的成功轨迹曲线下面积）超越 DROID-SLAM、DPV-SLAM 等方法。该数据集包含快速运动、运动模糊等极端条件，传统单目 SLAM 方法容易丢失跟踪，而 MASt3R-SLAM 凭借 3D 重建先验的强几何约束保持了更高的鲁棒性。

在几何质量上，MASt3R-SLAM 输出的稠密 pointmap 比 DROID-SLAM 的三维结构更完整、噪声更少。特别是在 EuRoC（灰度无人机序列，快速运动）等挑战性场景中，DROID-SLAM 产生大量离群点，而 MASt3R-SLAM 保持了几何一致性。RMSE（均方根误差）指标上，MASt3R-SLAM 的 Chamfer distance 显著优于 DROID-SLAM，因为 RMSE 对大离群点惩罚更重——DROID-SLAM 虽然均值误差尚可，但离群点导致 RMSE 显著升高。

值得关注的消融实验结果：将跟踪和后端的误差公式从 ray 方向误差替换为 3D 点误差后，平均 ATE 从 0.097m 恶化到 0.155m，证明了 ray-based 公式对深度预测噪声的鲁棒性。关闭回环检测后，长序列的误差显著增加，说明 MASt3R 的 pointmap 预测仍然存在系统性偏差，需要时间维度的全局校正。

16.3.8 MASt3R-SLAM 论文精读

1. 论文试图解决什么痛点？

现有稠密 SLAM 系统要么依赖已知相机标定，要么用深度网络仅解决子问题（深度估计、光流、匹配中的一个）。没有系统将两视图 3D 重建先验作为底层几何基础来统一 tracking、mapping 和 relocalization。

2. 核心假设是什么？

MASt3R 等视觉基础模型提供的 pointmap 先验，其信息密度和质量足以支撑一个完整 SLAM 系统的前端跟踪和后端优化。通过将 pointmap 转换为 ray-based 表示，可以在不假设特定相机模型的情况下实现鲁棒的几何推理。

3. 输入输出是什么？

输入：单目 RGB 视频流（无需相机内参）。输出：全局一致的相机位姿轨迹 + 每个关键帧的稠密 pointmap（即三维点云地图）。系统运行在约 15 FPS。

4. 地图表示是什么？

关键帧 pointmap——每个关键帧维护一个像素级稠密三维点云（ $H \times W \times 3$ ），通过增量式加权融合不断更新。地图天然稠密，无需额外的深度图融合或 TSDF 积分。

5. Tracking 怎么做？

基于 pointmap 匹配建立当前帧与关键帧的像素对应关系，通过最小化 ray 方向误差估计相对位姿 $\mathbf{T}_{kf}$ 。使用 Gauss-Newton 在 IRLS 框架中求解，对深度预测错误具有天然鲁棒性。

6. Mapping 怎么做？

通过 MASt3R 网络对输入图像对前向传播获取 pointmap，再经加权平均融合到关键帧的 canonical pointmap 中。融合是增量式的，随着时间推移积累多视角信息。

7. 优化目标是什么？

前端跟踪：最小化对应 ray 对之间的方向角误差。后端全局优化：在关键帧 pose graph 上联合最小化所有边（顺序边+回环边）的 ray 方向误差，使用 Gauss-Newton 二阶优化。

8. 相比前作真正推进了什么？

(1) 首次实现了围绕两视图3D重建先验的实时稠密 SLAM；(2) 提出高效的迭代投影匹配，将 pointmap 匹配从2秒加速到2毫秒；(3) 引入 ray 方向误差公式，使系统对深度预测错误鲁棒且无需特定相机模型；(4) 证明了MASt3R先验可以替代传统SLAM中特征提取、匹配、三角化的完整前端。

9. 没有解决什么？

(1) 仍依赖 MAST3R 的网络前向传播，计算量较大（15 FPS 需要高端GPU）；(2) 对先验模型的错误传播敏感——如果 MAST3R 预测失败，整个 SLAM 系统会受影响；(3) 动态场景未显式处理；(4) EuRoC 等灰度序列上，MASt3R 因训练数据中缺乏灰度图像而表现受限；(5) 多视图一致性仍通过后端优化间接实现，而非网络本身输出全局一致结果。

10. 对后续研究的影响是什么？

MASt3R-SLAM 开创了"3D重建先验驱动 SLAM"的范式。后续工作如 FoundationSLAM、MAStR-Fusion（融合IMU/GNSS）、VGGT-SLAM 等都沿用了这一思路。它表明：视觉基础模型不应仅作为 SLAM 的辅助模块，而可以作为整个系统的几何底座。

16.3.9 优点、失败模式与适用场景

MASt3R-SLAM 的核心优势是"plug-and-play"——它将 MAST3R 网络视为一个黑箱先验，无需微调或端到端训练即可构建完整 SLAM 系统。这种模块化设计使系统可以随 MAST3R 的迭代升级直接获益。另一个突出优点是支持未知相机参数，这对实际部署极为重要：手机录像、网络摄像头、无人机航拍等场景中，用户通常没有标定参数。

失败模式包括：(1) 纯旋转场景——当相机几乎没有平移时，pointmap 中的3D结构退化解稳定性下降；(2) 高速运动导致的运动模糊——MASt3R 网络对模糊图像的 pointmap 预测质量下降；(3) 纹理极度缺乏的场景（如单色墙面）——pointmap 预测缺乏足够特征支撑；(4) 大尺度室外场景——深度范围超出网络训练分布时，远距离几何预测不准确。

适用场景：室内结构化环境（TUM、7-Scenes）、手持设备拍摄的日常视频、变焦镜头视频（MASt3R-SLAM 是少数能处理变焦的 SLAM 系统）。不适用：高动态环境、需要嵌入式实时运行的资源受限场景。

16.3.10 与具身智能的关系

对具身智能而言，MASt3R-SLAM 提供了一种"开箱即用"的空间感知能力。传统 SLAM 需要针对每种传感器组合进行标定和参数调优，而 MAST3R-SLAM 可以在未知相机参数的情况下直接从 RGB 视频构建稠密三维地图。这使机器人能够快速适应新的视觉传感器配置，降低了部署门槛。

然而，当前版本的计算需求（RTX 4090 级别 GPU）限制了其在实体机器人上的直接部署。未来方向包括：模型轻量化（蒸馏、量化）、与 IMU/GNSS 的紧耦合融合（如 MAST3R-Fusion 所示）、以及面向特定机器人平台的工程优化。MASt3R-SLAM 的核心范式——用视觉基础模型替代传统几何前端——将成为具身智能感知系统的重要设计选择。

16.4 技术主线：从图像对到全局一致地图

DUST3R → MAST3R → MAST3R-SLAM 构成了一条清晰的技术演进线。理解这条主线对把握整个领域方向至关重要。

图像对 → 3D Reconstruction Prior。 DUST3R 证明了 pointmap 是一种有效的中间表示，将"图像对到三维结构"的映射编码进神经网络权重。这一步的关键突破是"无需标定"——网络学到的几何先验替代了传统射影几何的硬约束。

3D Prior → 局部一致性。 MAST3R 在 DUST3R 基础上增强匹配能力，引入 descriptor head 和 metric depth。这使 pointmap 从"粗略重建"进化为"精确匹配+度量重建"，具备了在 SLAM 前端承担数据关联任务的能力。

局部一致性 → 全局位姿图。 MAST3R-SLAM 的贡献在于：如何用这些成对的、局部的3D先验，构建全局一致的轨迹和地图。它通过关键帧机制、增量式 pointmap 融合、pose graph 和二阶全局优化，将局部预测整合为全局一致的结构。

基础模型先验 → SLAM 后端优化。 这条线的最终形态是一种"双引擎"架构：神经网络负责从图像中提取密集的3D几何先验（感知），后端优化负责在时间维度上整合这些先验并消除不一致（推理）。这与传统 SLAM 恰好相反——传统 SLAM 的前端是轻量的特征提取，后端是重的 Bundle Adjustment；而 MAST3R-SLAM 类方法的前端是重的网络前向传播，后端是相对轻量的图优化。

这一技术主线揭示了一个深层趋势：**SLAM 正在从"先估计几何再优化"转变为"先预测几何再对齐"。**传统 SLAM 问：给定点对应，最优位姿和结构是什么？基础模型驱动的 SLAM 问：给定网络预测的3D结构，最优的全局对齐方式是什么？

这条主线的终点可能是一种全新的 SLAM 架构：前端完全由视觉基础模型承担，输出每一帧的稠密3D表示；后端专注于时序整合——处理动态物体、管理不确定性和实现多智能体协作。Bundle Adjustment 不会消失，但其角色从"重建几何的核心算法"转变为"校准神经网络预测的后处理步骤"。

16.5 对传统 SLAM 的冲击

MASt3R-SLAM 及其后续工作对传统 SLAM 构成了范式层面的冲击。这种冲击体现在三个层面：

前端设计的颠覆。 传统 SLAM 的前端从特征点开始：提取角点/特征点 → 描述子匹配 → RANSAC 几何验证 → 位姿估计。ORB-SLAM、DROID-SLAM 都遵循这一逻辑（DROID-SLAM 用深度学习替代了描述子和匹配，但仍从稀疏对应开始）。MASt3R-SLAM 的前端从3D结构开始：网络直接输出两帧的稠密三维点云 → 在3D空间中匹配 → ray 误差最小化。特征点不再是信息的源头，而是中间产物。

相机模型假设的消解。 传统 SLAM 严格依赖已知的相机内参模型（针孔模型+畸变系数），这是 Bundle Adjustment 能正常工作的前提。MASt3R-SLAM 通过 ray-based 表示，将这一假设压缩到"唯一光心"这一个条件上。系统可以处理变焦、不同镜头、甚至未知相机参数的视频——这对消费级设备（手机、无人机、网络摄像头）的 SLAM 部署意义重大。

稠密 vs 稀疏的根本性转换。 传统 SLAM（ORB-SLAM 系列）输出稀疏特征点地图，稠密地图需要额外的深度估计或 TSDF 融合。MASt3R-SLAM 天然输出稠密几何——网络的每个前向传播都产生一个完整的三维点云。这意味着 SLAM 的输出可以直接用于机器人导航避障、AR 场景理解、3D 内容创作等下游任务，无需后处理。

然而，这一范式也面临现实约束。MASt3R-SLAM 需要高端 GPU 才能实时运行（15 FPS on RTX 4090），而 ORB-SLAM3 可以在嵌入式设备上跑30 FPS。网络先验存在领域偏差——训练数据中缺乏的场景类型（如灰度图像、极端光照）会导致预测质量下降，进而影响整个 SLAM 系统。此外，当前版本的 MASt3R-SLAM 未显式处理动态物体，这是走向真实世界部署的下一个关卡。

这一冲击的核心问题可以概括为：**传统 SLAM 系统问的是"哪些点能匹配"**，而 **MASt3R-SLAM 类方法问的是"基础模型已经给出了局部3D，如何把它们拼成全局一致地图"**。前者从零开始建立对应关系，后者从已知的3D结构出发进行全局整合。这不仅是技术路线的差异，更是对"SLAM 中什么部分是核心问题"这一根本问题的不同回答。随着视觉基础模型的能力持续增强，后一种提问方式可能逐渐成为主流——SLAM 的核心挑战从"如何重建3D"转向"如何整合和校准神经网络的3D预测"。

第17章 VGGT：前馈式几何基础模型

17.1 为什么 VGGT 是 2025 必讲论文？

CVPR 2025 只评出一篇 Best Paper——**VGGT: Visual Geometry Grounded Transformer**（CVF开放获取）。这是该会议历史上首次将最高荣誉授予一篇纯粹的 3D 视觉论文。评委会给出的理由简洁有力：VGGT 用一个前馈神经网络同时解决了相机参数估计、多视图深度估计、稠密点云重建和 3D 点追踪四项核心几何任务，速度在秒级，精度超过依赖迭代优化的传统方法。

这不仅仅是一篇论文的进步，而是整个 SLAM/SfM 技术路线的转向信号。

此前，DUSt3R（CVPR 2024）和 MASt3R（CVPR 2024）已经证明：两视图几何可以靠 Transformer 前馈完成。但它们有两个根本限制——**只能处理两张图**，多张图时仍需外部全局对齐；**每项几何任务各需一个专用网络**，模块化拼接的效率和一致性都不理想。VGGT 直接打破了这两条边界：输入可以是单张、几张或数百张图像，输出是统一坐标系下的完整几何描述，且全部在单次前向传播中完成。

从 SLAM 的视角看，VGGT 的价值在于将几何估计从**"在线迭代优化"**转变为**"前馈推理+可选微调"**。这意味着 SLAM 系统不再需要为每帧执行费时的特征匹配、三角化和 BA，而是可以依赖一个预训练基础模型先给出高质量初值，后端只需维护全局一致性和处理动态物体。第14章介绍的 AIM-SLAM（arXiv）已经展示了这种范式的可行性——它直接调用 VGGT 的 dense pointmap prediction 来初始化新帧。第18章将进一步展开 FoundationSLAM 和 CALM 如何将基础模型融入完整 SLAM 管线。

VGGT 由牛津大学 Visual Geometry Group 与 Meta AI 联合发布，代码和权重完全开源（Apache 2.0 协议，2025年7月后更新商用许可）。GitHub 仓库在三个月内收获超过 13k star，催生了 FlashVGGT、AVGGT、VGGT-Long、VGGT-X、AnySplat 等数十个后续工作。理解 VGGT 是理解 2025-2026 年 SLAM 技术演进的前提。

17.2 它改变了什么？

17.2.1 传统管线：匹配 → 几何估计 → 三角化 → BA

传统 SfM/SLAM 管线经过二十年打磨，流程高度固化：

- 输入图像
- 特征检测 (SIFT/SuperPoint)
 - 特征匹配 (最近邻/学习匹配器)
 - 几何验证 (RANSAC + 对极几何)
 - 三角化 (线性三角化 + 非线性优化)
 - 全局注册 (P3P + RANSAC)
 - Bundle Adjustment (迭代非线性优化)
 - 稠密重建 (MVS/深度估计)

每个环节独立优化，误差逐级累积。COLMAP 处理 50 张图需要分钟级；即使有学习模块加速（如 VGGsfm），可微分 BA 仍是瓶颈。更重要的是，这个管线中没有一个环节具备全局语义理解——它在纹理重复区域失败，在弱纹理区域失败，在宽基线视图间失败，因为它只靠局部特征匹配建立对应。

17.2.2 VGGT 路线：图像输入 → Transformer → 相机/深度/点图/轨迹

VGGT 将整个管线压缩为一次前向传播：

- 输入图像 (1 ~ 数百张)
- DINOv2 patch 嵌入
 - 交替注意力 Transformer (24层 × 帧内 + 全局)
 - 相机参数头 (内参 + 外参)
 - DPT 头 (深度图 + 点图 + 追踪特征)
 - 统一坐标系下的完整 3D 描述

没有特征检测，没有匹配，没有 RANSAC，没有迭代 BA。12 亿参数的 Transformer 直接从海量 3D 标注数据中学习了几何推理能力。在 RealEstate10K 上处理 10 张图仅需 0.2 秒（CVF开放获取），加上 BA 后处理也只需 1.8 秒，而 DUST3R + 全局对齐需 7 秒以上，VGGsfm v2 需 20 秒以上。

维度	传统 SfM/SLAM	VGGT 路线
核心计算	迭代优化 (BA/非线性最小二乘)	前馈 Transformer 推理
对应关系	显式特征匹配 + RANSAC	隐式注意力机制学习
输入规模	任意 (但时间与图像数超线性增长)	1 ~ 数百张 (内存限制)
每步训练	无需训练, 手工设计	大规模监督预训练
跨视图一致性	BA 强制	全局注意力天然编码
语义/先验利用	无	DINOv2 预训练 + 3D 数据先验
失败后恢复	丢失追踪后难以恢复	每帧独立预测, 天然鲁棒
可扩展性	序列处理, 难以并行	批处理, GPU 并行友好

上表对比的核心含义是：VGGT 将几何估计问题从"在线数值优化"转化为"离线学习 + 前馈推理"。这并不意味着 BA 彻底消失——VGGT + BA 的组合仍然是精度上限最高的方案——但 BA 的角色从"必需的求解核心"降级为"可选的精化后处理"。对 SLAM 而言，这意味着实时前端和后端的分离变得更加清晰：前端用 VGGT 秒级获取几何初值，后端集中精力做闭环检测、全局一致性和语义融合。

17.2.3 与 DUST3R/MASt3R 的承继关系

VGGT 直接继承自 DUST3R → MASt3R → Fast3R 这条"3D 重建大模型"脉络，但有三个关键跃迁：

1. **从成对到任意多视图**：DUST3R 和 MASt3R 只能输入两张图，多视图时需要逐对融合再做全局对齐。VGGT 原生支持任意数量图像，跨视图信息在全局注意力层中直接交换。
2. **从单一任务到统一多任务**：DUST3R 只输出点图，MASt3R 增加匹配能力。VGGT 同时输出相机参数、深度图、点图和追踪特征，四个任务共享主干表示，多任务监督相互促进。
3. **从后处理依赖到直接可用**：DUST3R/MASt3R 的输出通常需要后续优化才能获得精确相机姿态。VGGT 的前馈输出本身已经达到或超过优化后方法的精度，BA 进一步锦上添花。

17.3 书中应重点讲的四个输出

VGGT 同时输出四项 3D 几何属性。理解每项输出的定义、精度和使用方式，是将 VGGT 集成到 SLAM 系统的前提。

17.3.1 Camera Parameters

相机参数头输出每张图的 9 维向量：焦距 (f_x, f_y) 、主点 (c_x, c_y) 、径向畸变系数 κ 、以及旋转（6D 连续表示）和平移（3 维）。所有参数以第一张图为世界坐标系参考帧。

关键设计：VGGT 为每帧附加一个可学习的 camera token，该 token 在全局注意力层中与所有图像 patch token 交互。相机头（4 层自注意力 + 线性投影）读取这些 camera token 后直接回归参数。这种设计与传统 SfM 中"先估计相对位姿再全局注册"的流程截然不同——位姿预测与稠密几何预测共享同一组特征表示。

精度表现：在 RealEstate10K 上，VGGT 前馈 AUC@30 达到 85.3，加 BA 后提升至 93.5（CVF 开放获取）。在 CO3Dv2 上，前馈 AUC@30 为 88.2。注意这些精度是在未见过的测试集上直接前馈推理获得，没有任何测试时优化。传统方法如 DUST3R + 全局对齐在同一数据集上分别只有 78.2 和 82.4。

对 SLAM 的意义：VGGT 的相机预测可以作为 SLAM 跟踪模块的高质量初值，尤其在 SLAM 初始化阶段或追踪丢失后的重定位时，避免了缓慢的 PnP + RANSAC 流程。

17.3.2 Depth Maps

深度图头为每帧输出逐像素深度值 $D_i \in \mathbb{R}^{H \times W}$ ，以及每张图 accompanying 的不确定性图（aleatoric uncertainty map）。深度头采用 DPT（Dense Prediction Transformer）架构，将 Transformer 块 4、11、17、23 层的中间特征拼接后上采样到全分辨率。

关键设计：VGGT 在多视图深度估计中利用了全局注意力的跨视图一致性。单帧深度估计（如 Depth Anything v2）只依赖单图先验，容易在遮挡边界和无纹理区域出错。VGGT 在全局注意力层中聚合多视图信息，使相邻视图的深度预测在 3D 空间中自然一致。

精度表现：在 DTU 数据集上，VGGT 多视图深度估计的 Abs Rel 为 0.068，Sq Rel 为 0.045，显著优于单目深度模型（CVF开放获取）。在 ScanNet 上，VGGT 的 RMSE 为 0.189，优于 MVSNet 系列的传统方法。

与点图的关系：VGGT 同时输出深度图和点图，二者在数学上可互相推导——给定深度图和相机参数，可以通过反投影得到点图。但实验发现，联合学习两者能提升各自精度。在推理时，VGGT 论文建议优先使用"深度图 + 相机参数"推导的点图，而非直接回归的点图，因为前者在多视图一致性上表现更好。

17.3.3 Point Maps

点图（Point Map）是 DUST3R 引入的核心表示：为每帧每个像素 y 输出其在水坐标系中的 3D 位置 $P_i(y) \in \mathbb{R}^3$ 。所有视图的点图共享同一个世界坐标系（第一张图的相机坐标系），因此**天然全局对齐**，无需后续融合。

关键设计：点图头的架构与深度头相同（DPT），但输出通道为 3（XYZ 坐标）。点图监督信号直接来自训练数据集的 3D 标注（深度图 + 相机参数反投影）。与 DUST3R 的点图不同，VGGT 的点图受益于多视图全局注意力——模型在推理时"看到"了所有视图，点图预测隐含了跨视图几何一致性约束。

精度表现：在 7-Scenes 上，VGGT 点图精度（Accuracy @ 5cm）达到 82.3，Completeness @ 5cm 达到 79.1，Chamfer Distance 为 0.094（CVF开放获取）。在 DTU 上，Overall Chamfer Distance 为 0.382，是 DUST3R（1.741）的 4.5 倍提升。在 ETH3D 上，VGGT 的 CD 为 0.094，优于 MAST3R + 全局对齐的 0.127。

对 SLAM 的意义：点图表示使 SLAM 的地图初始化变得异常简单。AIM-SLAM（第14章14.6节）直接利用 VGGT 的点图预测作为新帧的初始地图，避免了特征三角化的繁琐流程。AnySplat 更进一步，将点图转化为 3D Gaussian 的初始位置。

17.3.4 3D Point Tracks

追踪头输出每帧每个像素的 C 维特征向量 $T_i \in \mathbb{R}^{C \times H \times W}$ 。这些特征不直接等于 2D 光流或对应关系，而是作为下游追踪器（如 CoTracker2）的特征输入，用于跨帧稠密点追踪。

关键设计：VGGT 本身不直接输出 2D 对应，而是学习"对追踪友好"的特征表示。预训练的 VGGT 特征替换 CoTracker2 的原生特征提取器后，在非刚性点追踪（TapVid-DAVIS）上将 PF@1px 从 67.2 提升到 73.8（CVF开放获取）。这说明 VGGT 的 3D 几何先验能有效改善动态场景的稠密对应估计。

对 SLAM 的意义：传统 SLAM 的追踪模块依赖特征点匹配或光流。VGGT 的特征提供了一种新的选择：在基础模型提取的几何特征上做追踪，而非原始图像特征。这种特征对光照变化、遮挡和运动模糊的鲁棒性显著更强。

17.4 VGGT 对 SLAM 的意义

一句话：**VGGT 让 SLAM 从"在线几何优化系统"部分转向"基础模型先验 + 局部优化 + 全局一致性维护"的新范式。**

这种范式转变的具体含义可以从三个层面理解：

前端层面：VGGT 可以取代或大幅简化 SLAM 的前端。传统前端需要做特征提取、匹配、几何验证和位姿估计，VGGT 一次性输出相机参数和深度图，前端只需验证和融合。MASt3R-SLAM（第16章）已经展示了两视图 3D 先验驱动 SLAM 的可行性；VGGT 将其扩展到任意多视图，且精度更高。

初始化层面：SLAM 最脆弱的环节之一是初始化。传统方法需要足够的基线和视差才能可靠三角化。VGGT 的单视图/少视图深度和点图预测提供了带尺度的稠密几何先验，使 SLAM 可以从第一帧就有高质量地图，后续帧在此基础上增量更新。

重定位和闭环层面：VGGT 的多视图一致性使其天然适合重定位。给定一组关键帧和当前帧，VGGT 可以直接预测它们之间的相对位姿，无需特征匹配。UniPR-3D（arXiv）和 Reloc-VGGT（arXiv）等工作已经证明 VGGT 特征在视觉重定位中的有效性。

但 VGGT 不能直接替代完整 SLAM，原因有三：

1. **内存瓶颈：**24 层全局注意力的复杂度为 $O((NL)^2)$ ， N 为图像数， L 为每帧 token 数。RTX 4090（24GB）上约能处理 60 帧。长序列需要分块处理（VGGT-Long，arXiv）。
2. **尺度模糊性：**单视图模式下深度预测存在绝对尺度模糊，多视图模式下尺度由相机运动决定。室内导航等需要绝对尺度的场景仍需额外处理（如 LoGoPlanner，arXiv）。
3. **动态物体：**VGGT 假设静态场景。动态物体会导致深度和点图不一致。MonST3R 和 DePT3R（arXiv）等工作正在解决这个问题。

17.5 VGGT 与 3DGS 的结合

VGGT 的最佳搭档是 3D Gaussian Splatting（3DGS）。VGGT 提供几何初值，3DGS 提供可渲染地图，二者组合构成了"前馈几何 + 可微渲染"的新重建范式。

17.5.1 从 VGGT 到 Gaussian：AnySplat

AnySplat（CVPR 2025）是这条路线的代表作。它接收未标定图像（2 ~ 24 张），用 VGGT 初始化的 Transformer 编码器提取几何特征，然后输出三项：3D Gaussian 参数（中心 μ 、不透明度 σ 、旋转 r 、尺度 s 、颜色 c ）、深度图和相机位姿。

训练时，AnySplat 的 geometry encoder 和 depth head 直接继承 VGGT 的预训练权重，Gaussian head 随机初始化。渲染出的 RGB 图和深度图分别与输入图像和 VGGT 预测的伪深度监督对齐。这种"蒸馏 VGGT 几何先验 + 渲染监督"的策略使 AnySplat 在 16 张图输入时达到与 InstantSplat-VGGT（优化 1000 步）相当的渲染质量。

AnySplat 的关键意义在于**闭环验证**：VGGT 的几何预测不仅可以直接评估（与 GT 对比），还可以通过 3DGS 渲染后反推——如果渲染的新视角图像质量高，说明底层几何一定准确。这种可微渲染闭环为 SLAM 提供了自监督信号。

17.5.2 对 SLAM 的启示：几何预测 + 渲染优化

将 VGGT + 3DGS 的思路融入 SLAM，可以得到如下管线：

新帧输入

- VGGT 前馈预测 (相机参数 + 深度图 + 点图)
- 初始化/更新 3D Gaussian 地图
 - 可微渲染优化 (局部窗口)
 - 全局一致性维护 (闭环 + 位姿图优化)

这个管线的优势在于：

- **速度**：VGGT 前馈在毫秒级，3DGS 渲染在数百 FPS，整体远超传统 BA。
- **稠密地图**：输出是可渲染的稠密 3D 地图，而非稀疏特征点云。
- **语义扩展性**：Gaussian 地图可以与语义分割、VLM 特征融合，直接支持下游任务。

SplaTAM (NeurIPS 2024) 和 GS-SLAM (CVPR 2024) 已经展示了纯 3DGS-SLAM 的可行性。VGGT 的介入解决了这些系统最大的痛点——**高质量初始化**。传统 3DGS-SLAM 需要从第一帧就开始优化 Gaussian 参数，收敛慢且容易陷入局部最优。VGGT 提供的高质量几何初值使 3DGS 从 "Warm Start" 开始，大幅减少所需优化步数。

17.5.3 连接到 2026：具身智能的技术底座

VGGT → 3DGS → SLAM → VLM/LLM 这条技术链正在形成具身智能的空间感知底座：

模块	功能	代表工作
VGGT (基础模型)	几何先验预测	VGGT, π^3 , Depth Anything 3
3DGS (地图表示)	可渲染稠密地图	SplaTAM, GS-SLAM, Gaussian-SLAM
SLAM 后端	全局一致性	FoundationSLAM, CALM (第18章)
VLM/LLM	语义理解	SpatialLM, LLaVA-3D

在这个体系中，VGGT 的角色是**空间几何的"第一性原理"**。它不依赖特定场景的优化，而是从预训练知识中直接输出几何判断。机器人或 AR 设备可以在每一帧都获得 VGGT 的几何预测，作为后续所有推理（路径规划、物体操作、场景理解）的基础。

SpatialLM (NeurIPS 2025, arXiv) 是这一方向的代表：它将 VGGT 输出的点云和深度图输入 LLM，让大语言模型直接在 3D 空间中推理——生成房间布局、识别家具位置、回答空间关系问题。这要求底层几何表示足够准确和稠密，VGGT 恰好满足这一需求。

但这条链路仍有缺口：VGGT 对动态物体、大尺度室外场景 (>1km)、以及极端光照条件的处理能力仍有局限。VGGT-Long (arXiv) 通过分块-对齐策略将序列扩展到数千帧，但闭环检测仍不如传统 SLAM 成熟。这些正是第18章 FoundationSLAM 和 CALM 试图解决的问题。

17.6 论文精读：VGGT

1. 论文试图解决什么痛点？

3D 视觉任务被割裂为单任务专用网络（深度估计、位姿估计、MVS、点追踪各自为政），每个网络需要独立训练和部署。SfM 管线（COLMAP 为代表）依赖迭代优化，处理数十张图就需要分钟级。DUST3R/MASt3R 虽然实现前馈重建，但只能处理两张图，多张图时仍需低效的全局对齐。核心痛点：没有一个统一的、前馈的、任意多视图的 3D 几何基础模型。

2. 核心假设是什么？

一个标准的 Transformer 架构（几乎无 3D 专用归纳偏置），只要在大规模 3D 标注数据上充分训练，就能同时学会预测所有关键几何属性。几何理解会作为“涌现能力”从数据中学习，无需手工设计多视图几何约束。这遵循了 GPT/CLIP/DINO 的“大模型 + 大数据 = 通用能力”范式。

3. 输入输出是什么？

- 输入：1 张到数百张 RGB 图像 $I = \{I_1, I_2, \dots, I_N\}$ ，无需相机参数或标定信息。
- 输出：
 - 每张图的相机参数 $g_i \in \mathbb{R}^9$ （内参 + 外参）
 - 每张图的深度图 $D_i \in \mathbb{R}^{H \times W}$
 - 每张图的点图 $P_i \in \mathbb{R}^{3 \times H \times W}$ （世界坐标系 3D 点）
 - 每张图的追踪特征图 $T_i \in \mathbb{R}^{C \times H \times W}$

4. 地图表示是什么？

VGGT 不维护显式地图表示——它是纯前馈模型，无状态。输出是每帧独立的稠密几何描述。点图可以视为“隐式稠密点云”：将所有帧的点图拼接即得全局点云。下游应用（如 AnySplat）可以将其转化为 3D Gaussian、NeRF 或体素地图。

5. Tracking 怎么做？

VGGT 本身不输出显式 2D 对应关系。Tracking 通过两种方式实现：

- 间接追踪：利用输出的相机参数和深度图，将像素反投影到 3D 再重投影到其他帧。
- 特征追踪：VGGT 输出的追踪特征 T_i 输入 CoTracker2 等追踪器，替代其原生特征提取器。实验表明这种“VGGT 特征 + CoTracker”组合在非刚性追踪上达到 SOTA。

6. Mapping 怎么做？

无显式 Mapping 过程。每帧深度图和点图即构成该帧的“局部地图”。多帧地图融合通过点图的全局坐标系一致性自然实现——所有点图都以第一帧相机为参考系，直接拼接即可。

7. 优化目标是什么？

多任务监督损失：

$$\mathcal{L}_{total} = \mathcal{L}_{camera} + \mathcal{L}_{depth} + \mathcal{L}_{pmap} + \mathcal{L}_{track}$$

- **Camera Loss**：预测相机参数与 GT 的 L2 损失，旋转用 6D 连续表示避免万向节锁。

- **Depth Loss**: 尺度不变对数损失 (scale-invariant log loss) , 适应单视图深度预测的尺度模糊性。
- **Pointmap Loss**: 预测点图与 GT 点图的 L1 损失。
- **Tracking Loss**: 追踪特征在跨帧对应点上的对比学习损失。

训练数据来自 17 个带 3D 标注的数据集, 包括 Co3Dv2、RealEstate10K、ScanNet、DTU、TartanAir 等, 总计数百万张图像。

8. 相比前作真正推进了什么？

维度	DUSt3R	MASt3R	VGGT
输入视图数	2	2	1 ~ 数百
相机参数	需后处理估计	需后处理估计	直接前馈输出
深度图	无	无	直接前馈输出
点图	有	有	有
点追踪	无	Fast Reciprocal Match	特征输出 + 下游追踪器
推理时间 (10图)	~7s (含对齐)	~7s (含对齐)	0.2s
DTU Overall CD	1.741	0.374	0.382 (前馈)

从表格可见, VGGT 在将输入从 2 视图扩展到任意多视图的同时, 将推理时间压缩了一个数量级, 且精度与 MASt3R 相当甚至更优。真正的突破不是某一项指标的提升, 而是**统一性**: 一个模型、一次前向传播、覆盖所有核心几何任务。

9. 没有解决什么？

- **长序列可扩展性**: 全局注意力的 $O((NL)^2)$ 复杂度限制了一次能处理的图像数。VGGT-Long 和 FlashVGGT 等工作正在解决。
- **动态场景**: 训练数据以静态场景为主, 动态物体会破坏多视图一致性。MonST3R 和 DePT3R 在扩展动态处理能力。
- **绝对尺度**: 单视图深度预测存在尺度模糊。多视图模式下尺度由相对位姿决定, 但室内等场景仍需外部尺度信息。
- **纹理less/重复区域**: 虽然比传统方法鲁棒, 但在大面积无纹理或强重复纹理区域仍会退化。
- **实时在线 SLAM**: VGGT 设计为离线批处理, 帧间无状态传递。在线 SLAM 需要额外设计。

10. 对后续研究的影响是什么？

VGGT 已成为 3D 视觉领域的"基础设施"。截至 2025 年底, 已有超过 50 篇后续论文直接基于 VGGT:

- **效率优化**: FlashVGGT (压缩描述子注意力)、AVGGT (近似全局注意力)、FastVGGT (token 剪枝), 将推理速度提升 3~10 倍。
- **长序列扩展**: VGGT-Long (分块对齐)、VGGT-X (内存优化实现), 支持 1000+ 图像。

- **分辨率扩展**：HD-VGGT（高分辨率特征调制），支持 2K 输入。
- **下游应用**：AnySplat（3DGS 重建）、DePT3R（动态追踪）、SEAR（RGB+热成像 3D 重建）、RoboManip-VGGT（机器人操作）。
- **SLAM 集成**：AIM-SLAM（第14章）、FoundationSLAM（第18章）、CALM（第18章）等将 VGGT 作为几何先验集成到完整 SLAM 管线。

17.7 系统结构详解

17.7.1 特征主干：DINOv2 + 交替注意力

VGGT 的主干由三部分组成：

(1) **图像分词器**：冻结的 DINOv2 ViT-Large，patch size 14×14 ，将每帧图像映射为 K 个 patch token $t_i^I \in \mathbb{R}^{K \times C}$ ， $C = 1024$ 。

(2) **特殊 token**：每帧附加 1 个 camera token 和 4 个 register token。第一帧的 camera/register token 使用独立参数（提示模型以第一帧为世界坐标系），其余帧共享另一组参数。

(3) **交替注意力聚合器**：24 个 transformer 块，每块包含两层注意力：

- **帧内自注意力 (Frame-wise Self-Attention)**：每个 token 只与同帧的其他 token 交互，复杂度 $O(N \cdot L^2)$ 。保留每帧内部的空间结构。
- **全局自注意力 (Global Self-Attention)**：所有 token 跨所有帧交互，复杂度 $O((NL)^2)$ 。实现跨视图几何推理。

两种注意力交替进行——奇数层帧内，偶数层全局。这种设计平衡了局部细节保持和全局一致性建模。

17.7.2 输出头

相机头：读取每帧的 camera token，经过 4 层自注意力 + 线性投影，输出 9 维相机参数。

稠密预测头 (DPT)：聚合第 4、11、17、23 层的中间特征，通过 DPT 上采样到全分辨率，再用卷积层分别映射为深度图（1 通道）、点图（3 通道）和追踪特征图（ C 通道）。

17.7.3 关键实现细节

- **参数规模**：总计约 12 亿参数。DINOv2 编码器约 3 亿（冻结），24 层聚合器约 7 亿，各预测头约 2 亿。
- **训练配置**：每张图 resize 到长边 518 像素，短边随机裁剪到 $[168, 518]$ 之间（14 的倍数）。batch 内保持总帧数 48（2~24 帧/场景）。使用 Adam 优化器，cosine 学习率调度。
- **推理速度**：A100 上 10 帧 0.2 秒，50 帧约 0.8 秒，受全局注意力内存限制。
- **后处理选项**：VGGT 提供 `demo_colmap.py`，可直接输出 COLMAP 格式的相机和点云文件，支持与 GLOMAP、3DGS 等工具链对接。

17.8 优点、失败模式与适用场景

核心优点：

- 速度：秒级处理数十张图，比传统 SfM 快 1~2 个数量级。
- 统一性：一个模型覆盖四项几何任务，工程维护成本大幅降低。
- 鲁棒性：基于全局注意力的隐式对应匹配对弱纹理、宽基线更鲁棒。
- 可扩展性：作为特征主干，VGGT 表示能提升下游任务（追踪、新视角合成、机器人操作）。

失败模式：

- 内存溢出：输入超过 60 帧（RTX 4090）或 200 帧（A100）时 GPU 内存不足。需使用 VGGT-Long 分块处理。
- 动态物体：运动物体导致深度/点图不一致，产生"重影"伪影。
- 极端视点变化：视图间几乎无重叠时，全局注意力难以建立对应。
- 大尺度室外：VGGT 训练数据以室内/物体为主，室外大场景（千米级）精度下降。VGGT-Long 通过分块处理有所缓解。
- 精细结构：超薄物体（如电线、树枝）和透明/反光表面的重建仍有缺陷。

适用场景：

场景	建议方案
室内小场景 (< 50 张图)	VGGT 前馈直接输出
室内大场景 (50~200 张)	VGGT + BA 后处理
室内超大场景 (> 200 张)	VGGT-Long 分块处理
室外驾驶场景	VGGT-Long 或结合传统 SLAM
动态场景	DePT3R / MonST3R 扩展
实时在线 SLAM	需额外设计（第18章 FoundationSLAM）
3DGS 初始化	AnySplat / InstantSplat + VGGT

17.9 与具身智能的关系

VGGT 是具身智能"空间感知层"的关键组件。具身智能体（机器人、AR 设备）需要理解自身在 3D 空间中的位置和周围环境的结构，VGGT 恰好提供这种能力。

直接应用：

- 导航：LoGoPlanner (arXiv) 利用 VGGT 的几何输出做度量感知的视觉导航，将深度图的尺度先验注入 VGGT 特征，实现绝对尺度定位。
- 操作：RoboManip-VGGT (arXiv) 证明 VGGT 的几何特征比语言-视觉模型更适合机器人操作任务，因为操作需要精确的空间关系而非语义描述。
- 场景理解：SpatialLM (NeurIPS 2025) 将 VGGT 点云输入 LLM，让大模型直接在 3D 中推理房间布局和物体关系。

间接贡献：

VGGT 为具身智能提供了**标准化几何接口**。在 VGGT 之前，不同 SLAM/SfM 系统的输出格式各异（稀疏点云、深度图、网格、TSDF），下游任务需要逐一适配。VGGT 统一输出相机参数 + 深度图 + 点图，成为事实上的“几何 API”——任何下游模块都可以从这套标准化输出中获取所需的几何信息。

17.10 本章一句话总结

VGGT 用一个 12 亿参数的交替注意力 Transformer，将 3D 几何重建从“多阶段迭代优化”压缩为“单次前馈推理”，输出统一坐标系下的相机参数、深度图、点图和追踪特征，成为连接前馈式 3D 重建与 SLAM/具身智能系统的几何基础模型。

第18章 FoundationSLAM、AIM-SLAM 与 CALM：基础模型 SLAM 的系统化

第17章详细拆解了 VGGT 的四个输出——深度、点云、对应关系、相机参数——并分析了这些输出对 SLAM 系统意味着什么。然而，单个前馈网络的预测不等于一个完整的 SLAM 系统。2025 年至 2026 年间，研究社区开始探索一个更深层的问题：如何将基础模型的局部几何能力系统性地嵌入 SLAM 的跟踪、建图、回环和全局优化框架中？

FoundationSLAM、AIM-SLAM 和 CALM 代表了三种不同的系统化路径。FoundationSLAM 从光流范式出发，用深度基础模型重新校准对应关系的几何质量；AIM-SLAM 从关键帧选择角度切入，让多视角基础模型的输入构成不再是固定窗口而是自适应决策；CALM 则直面最残酷的问题——当基础模型被推到公里级场景时，局部几何的漂亮数字如何转化为全局一致的可运行系统。三者共同指向一个判断：基础模型 SLAM 不是“用网络替换 SLAM”，而是“用基础模型增强 SLAM 的前端感知，后端仍由经典优化维护”。

18.1 FoundationSLAM：深度基础模型引导的几何一致 SLAM

18.1.1 问题：光流 SLAM 缺乏结构意识

DROID-SLAM 及其后继者（GO-SLAM、DPV-SLAM 等）证明了稠密光流作为统一表示的潜力。光流逐像素估计帧间位移，对纹理丰富区域效果良好。但光流有一个根本性缺陷：逐像素对应关系是独立估计的，不跨视图约束。低纹理、遮挡、重复图案区域会产生几何不一致的匹配，这些错误流入 BA（Bundle Adjustment）后污染深度和位姿估计（arXiv）。

更深层的问题是：现有光流网络只依赖局部相关性（4D 相关体积），不利用场景的全局几何结构。一个网络“见过”数十亿张图片的深度分布后学到的几何先验，被完全丢弃了。

18.1.2 核心洞察：基础深度模型不是只给深度图

FoundationSLAM 的核心假设是：深度基础模型的价值不只是输出一张深度图，而是将其几何先验注入对应关系估计的每一个环节（AAAI 2026）。具体而言：

- 用基础模型的冻结编码器提供几何感知的特征描述；

- 这些特征驱动光流估计，产生几何一致的对应关系；
- 对应关系进入双向一致性 BA 层，联合优化位姿和深度；
- 优化残差反过来指导光流的可靠性感知精化，形成闭环。

18.1.3 系统结构

FoundationSLAM 由三个模块构成 (arXiv)。

混合光流网络 (Hybrid Flow Network)。双分支编码器架构：(1) 几何先验分支，使用 FoundationStereo 的冻结 FeatureNet 编码器，从大规模真实图像中学到的深度先验提取稳定几何特征；(2) 任务适应分支，可训练的轻量 CNN 针对单目 SLAM 的数据关联挑战做分布适配。两分支特征经 3×3 卷积与残差层融合。同时引入 FoundationStereo 的冻结 ContextNet 提供几何上下文。对每对关键帧构建 4D 相关体积，FlowGRU 迭代精化光流场和置信度图。

双向一致性 BA 层 (Bi-Consistent Bundle Adjustment Layer)。这是 FoundationSLAM 的技术核心。传统 BA 只 penalize 前向投影误差。FoundationSLAM 引入对称约束：对帧 i 中像素 $\mathbf{u}_i$ ，先投影到帧 j 得 $\mathbf{u}_j$ ，再从 j 的深度反投回 i 检查是否回到原点：

$$\mathbf{u}_j = \pi(T_j^i \cdot \pi^{-1}(\mathbf{u}_i, D_i)), \quad \mathbf{u}_i^{back} = \pi(T_i^j \cdot \pi^{-1}(\mathbf{u}_j, D_j))$$

几何残差定义为双向一致性误差：

$$\mathcal{L}_{geo} = \|\mathbf{u}_i^{back} - \mathbf{u}_i\|$$

最终 BA 损失结合光流残差与几何残差，用置信度图 $\omega(\mathbf{u}_i)$ 加权：

$$\mathcal{L}_{BA} = \sum_{\mathbf{u}_i \in \Omega} (\omega(\mathbf{u}_i) \cdot \mathcal{L}_{flow}(\mathbf{u}_i) + (1 - \omega(\mathbf{u}_i)) \cdot \mathcal{L}_{geo}(\mathbf{u}_i))$$

高置信度像素由光流主导优化；低置信度像素（遮挡、低纹理区域）则更多依赖几何一致性约束。每次迭代执行一次光流更新和两次 BA，对深度和位姿联合求 Jacobian，Gauss-Newton 求解 ΔD 和 ΔT 。

可靠性感知精化 (Reliability-Aware Refinement)。基于 BA 残差构建像素级可靠性掩码，分为两个层级：(1) 边级可靠性——前向投影残差低于阈值 τ_{edge} 的像素标记为可靠；(2) 节点级可靠性——计算像素在所有共视邻居上的平均几何残差，低于 τ_{node} 的标记为全局一致。可靠区域执行标准相关体积精化；不可靠区域屏蔽相关体积，强制依赖上下文特征。这形成了"匹配→优化→反馈→再匹配"的闭环。

18.1.4 实验与定位

训练在 TartanAir 的 6 帧序列上进行（共视图 18 条边），8 张 RTX 4090 训练约 5 天。推理使用 ViT-S 半分辨率编码，RTX 4090 上 18 FPS (arXiv)。

方法	TUM ATE (m)	EuRoC ATE (m)	ETH3D ATE (m)
ORB-SLAM3	—	—	0.135
DROID-SLAM	0.038	0.024	0.171
MASt3R-SLAM	0.030	0.018	0.086
FoundationSLAM	0.024	0.016	0.069

FoundationSLAM 在 TUM-RGBD 的 9 个序列中 7 个排名第一，平均 ATE 0.024 m，比 MASt3R-SLAM (0.030 m) 降低 20%。ETH3D-SLAM 的 AUC 达到 24.775，超过 MASt3R-SLAM 的 23.935。低纹理和反光环境下优势尤为明显。Tanks and Temples 数据集上的定性比较显示，FoundationSLAM 在保持更多关键帧的同时减少了层叠伪影，重建几何一致性显著优于 MASt3R-SLAM (AAAI 2026)。

Bi-Consistent BA 的设计区别于传统 BA 的关键在于：传统 BA 只 penalize 单向投影误差，遮挡和深度不连续区域的错误匹配会单向传播；双向对称约束强制每对帧在彼此深度上"互证"，一个帧的深度必须在另一个帧的几何中成立。这种对称性在数学上等价于要求深度图在多视角下构成一致的 3D 曲面，而非独立优化每张深度图。Reliability-Aware Refinement 进一步将这一几何一致性信号反馈给前端光流网络，形成"前端匹配质量→后端几何一致性→前端匹配策略调整"的闭环，这是 DROID-SLAM 系列所不具备的。

18.1.5 论文精读

论文试图解决什么痛点？ 光流 SLAM 的对应关系缺乏几何结构意识，低纹理和遮挡区域匹配失败，且光流估计不跨视图约束。

核心假设是什么？ 深度基础模型的冻结编码器包含全局场景结构信息，将其注入光流网络能产生几何一致的对应关系。

输入输出是什么？ 输入单目 RGB 视频流；输出关键帧位姿和稠密深度图/点云。

地图表示是什么？ 基于关键帧的稠密深度图，通过双向一致性 BA 联合优化。

Tracking 怎么做？ Hybrid Flow Network 产生几何感知对应关系 → Bi-Consistent BA Layer 联合优化位姿和深度 → Reliability-Aware Refinement 根据优化残差反馈精化光流 → 迭代。

Mapping 怎么做？ 深度图经置信度加权融合，BA 层直接优化深度值。

优化目标是什么？ 加权光流残差 + 加权双向几何一致性残差，Gauss-Newton 求解。

相比前作真正推进了什么？ 首次将深度基础模型的几何先验嵌入光流估计全过程；Bi-Consistent BA 的双向一致性约束突破了传统单向投影误差；可靠性感知精化形成了匹配-优化闭环。

没有解决什么？ 关键帧图规模增长时 BA 计算量呈二次膨胀；冻结的基础编码器增加了显存开销；长序列上的全局一致性依赖关键帧选择启发式。

对后续研究的影响是什么？ 确立了"Foundation SLAM"范式——基础模型不只在后端提供深度图，而是在前端对应关系层面发挥作用。

18.2 AIM-SLAM：自适应多视角关键帧优先化

18.2.1 问题：基础模型 SLAM 的输入窗口是盲目的

VGGT 可以处理任意长度的图像序列（第17章），但把它塞进 SLAM 系统时出现了一个新问题：输入哪些帧？MASt3R-SLAM 局限在两视角对；VGGT-Long 和 VGGT-SLAM 使用固定长度的连续窗口。这些做法有一个共同盲区：没有根据几何上下文选择输入视图。冗余帧被反复送入网络，真正有信息量的历史视角却被忽略 (arXiv)。

这导致两个后果：一是短中期漂移累积，因为窗口外的几何约束未被利用；二是计算浪费，VGGT 推理昂贵（单次前向约 0.3-0.5 秒），固定窗口策略不区分帧的信息价值。

18.2.2 核心洞察：关键帧选择应该是一个自适应的、几何感知的决策

AIM-SLAM 的核心假设是：VGGT 的多视角输入构成应该是稀疏、重叠充分且信息增益最大的关键帧子集，而这个子集的选择必须基于几何重叠和信息论的联合判据 (arXiv)。

18.2.3 系统结构

AIM-SLAM 前端由两个模块组成：SIGMA 关键帧优先化模块和联合多视角 Sim(3) 优化器。后端为异步回环检测和位姿图优化。

SIGMA 模块 (Selective Information- and Geometric-aware Multi-view Adaptation)。三阶段流水线：

阶段一：几何初始化。构建体素索引的关键帧地图，每个体素存储观测到它的关键帧 ID。对最新关键帧 I_k ，计算与其他关键帧的体素重叠分数：

$$O(I_k, I_i) = |v(I_k) \cap v(I_i)|$$

Top-N 重叠帧构成候选集 $\mathcal{W}_v$ 。体素化在这里的角色不是点检索，而是视图级选择——从点到视角的抽象层。

阶段二：信息驱动重排序。假设 VGGT 预测的 3D 点服从高斯分布，利用扩展卡尔曼滤波更新形式计算加入候选视角 I_j 后点云协方差的变化。信息增益定义为协方差行列式比值的对数和：

$$\Gamma(I_k, I_j) = \sum_{\mathbf{x}_k \in \Omega_{k \rightarrow j}} \frac{1}{2} \log \frac{\det(\mathbf{P}_k^-)}{\det(\mathbf{P}_{k \rightarrow j}^+)}$$

测量协方差 $\mathbf{R}$ 是 3×3 全矩阵，通过像素噪声经逆投影 Jacobian 传播得到（VGGT 深度置信度加权）。候选集按 Γ 重排序。

阶段三：自适应激活。稳定性测试逐帧评估加入新视角后位姿估计的变化；变化低于阈值时停止扩展。每次多视角优化后，更新后的位姿和置信度重新触发重排序，形成循环。

联合多视角 Sim(3) 优化。选定子集 $\mathcal{W}$ 送入 VGGT 推断点云和置信度后，AIM-SLAM 不在两两对之间做增量对齐，而是将所有选中视角放入一个联合优化问题。残差定义为从帧 a 投影到帧 b 的射线距离，置信度作为信息权重。Jacobian 对 Lie 代数 $\mathfrak{sim}(3)$ 求解，Levenberg-Marquardt + IRLS 迭代。回环使用 VGGT 第一层 DINOv2 patch token 作为全局描述子，余弦相似度检索候选。

18.2.4 实验与定位

方法	标定	TUM Avg (m)	EuRoC Avg (m)
DROID-SLAM	✓	0.038	0.024
DROID-SLAM	✗	0.158	—
VGGT-Long	✗	0.081	—
VGGT-SLAM	✗	0.053	—
MASt3R-SLAM	✗	0.060	—
AIM-SLAM	✗	0.031	0.022

AIM-SLAM 在未标定设置下 TUM 平均 ATE 0.031 m，接近标定 MAST3R-SLAM 的 0.030 m。在 EuRoC 上，未标定条件下达到 0.022 m，优于所有对比的无标定方法。作者指出，VGGT-Long 和 VGGT-SLAM 的错误来自子图对齐失败——局部预测可靠，但大视角变化下对齐失效；MASt3R-SLAM 虽更鲁棒但受限于两视角设计，无法利用宽基线线索 (arXiv)。

运行速度：含 VGGT 推理约 3 Hz，不含推理的前端组件 17 Hz。ROS 集成已开源。

SIGMA 模块的价值可以从一个角度理解：VGGT 的理论输入长度是任意的，但实际中 GPU 内存和推理延迟限制了可输入的帧数。AIM-SLAM 用 SIGMA 回答了"给定有限的 VGGT 输入槽位，如何最大化几何信息量"这一资源分配问题。实验表明，自适应选择的 5 帧子集在 ATE 上优于固定窗口的 8 帧甚至 12 帧配置，因为冗余帧不仅不增加信息，还会引入噪声干扰优化 landscape。

失败模式方面，AIM-SLAM 在 VGGT 本身预测失败的场景（极度低纹理、动态物体主导画面）下表现受限，因为 SIGMA 的信息增益计算依赖于 VGGT 输出的置信度——如果置信度本身不可靠，信息排序会失效。此外，体素地图的分辨率需要权衡：过粗会遗漏重要视角，过细则候选集过小。

18.2.5 论文精读

论文试图解决什么痛点？基础模型 SLAM 的输入窗口要么是固定两视角，要么是固定长度序列，没有根据几何上下文自适应选择视图。

核心假设是什么？通过体素重叠和信息增益联合判据选择的稀疏关键帧子集，优于盲目扩展的固定窗口。

输入输出是什么？输入未标定单目 RGB 流；输出全局一致位姿和稠密 3D 重建。

地图表示是什么？基于 VGGT 预测的深度图反投影点云，置信度加权融合。

Tracking 怎么做？SIGMA 模块自适应选择关键帧子集 → VGGT 多视角推断 → 联合 Sim(3) 优化所有选中视角的位姿。

Mapping 怎么做？关键帧点云通过置信度加权平均融合，VGGT 预测的焦距递推平均。

优化目标是什么？联合多视角射线残差最小化，Lie 代数上 Levenberg-Marquardt 求解。

相比前作真正推进了什么？ 首次在基础模型 SLAM 中引入自适应、几何感知的多视角关键帧选择；SIGMA 的三阶段设计（重叠→信息→稳定性）是可复用的元框架；联合 Sim(3) 优化突破了两视角设计的宽基线限制。

没有解决什么？ VGGT 推理速度是主要瓶颈（3 Hz）；体素索引地图的粒度影响选择质量；SIGMA 的超参数（top-N 阈值、稳定性阈值）需要序列级调优。

对后续研究的影响是什么？ SIGMA 证明了"关键帧选择本身应该利用基础模型的预测"这一范式，为后续结合主动感知的 SLAM 系统设计提供了模板。

18.3 CALM：面向公里级部署的视觉几何基础模型 SLAM

18.3.1 问题：从房间级到公里级，几何畸变被放大了千倍

FoundationSLAM 和 AIM-SLAM 在 TUM、EuRoC 等房间级数据集上表现优异。但当你把同样的基础模型推到 KITTI 这样数公里长的轨迹上时，一个根本性问题暴露出来：线性变换不足以建模 VGFM（Visual Geometry Foundation Model）输出中的复杂非线性几何畸变 (arXiv)。

VGFM 的每个局部子图内部是几何一致的。但不同子图之间存在隐性的非线性深度畸变——同一个物理点在相邻子图中的 3D 坐标不一致，且不一致性不是简单的相似变换（Sim3）或射影变换（SL4）能描述的。强制用线性变换对齐会导致未校正残差快速累积，最终轨迹漂移和地图发散。

尺度模糊是另一个致命问题。单目 VGFM 的每个子图有自己的任意尺度，现有方法要么依赖回环闭合掩盖漂移，要么在开放环路中尺度逐渐发散。

18.3.2 核心洞察：非线性对齐 + 恒定物理间距先验

CALM（论文中系统名为 CAL²M）提出两个核心手段：(1) 用几何锚点和薄板样条（Thin Plate Spline, TPS）做非线性弹性子图对齐；(2) 用 loosely coupled 的双相机"助手眼"提供恒定物理间距先验，从根本上消除尺度模糊 (arXiv)。

系统假设极简：两台相机无需标定内参/外参，无需硬件时钟同步，只要保持恒定物理间距且 FoV 有足够重叠。

18.3.3 系统结构

子图构建。 滑动窗口策略，光流驱动关键帧选择（平均视差超过阈值触发新帧），最多 N_{max} 帧。重叠帧保留为下一子图的连接桥。

尺度校正。 对子图 $\mathcal{S}_k$ 内每对主-辅视角，VGFM 估计的瞬时相机间距为 $d_{i,k} = \|\mathbf{p}(T_{i,k}^{pri}) - \mathbf{p}(T_{j,k}^{ast})\|_2$ 。以首个子图的平均间距 $\bar{d}_1$ 为全局参考，尺度校正因子 $s_k = \bar{d}_1 / \bar{d}_k$ 。由于 s_k 仅从当前观测和固定参考推导，尺度估计与先前子图解耦，彻底消除尺度漂移。

主-辅联合位姿图优化。 状态向量包含全局主相机位姿和静态外参变换。辅助轨迹隐式参数化为 $T_{w,i}^{ast} = T_{w,i}^{pri} \cdot T_{ext}$ ，更好利用固定外参先验。优化目标融合主相机约束、辅助相机耦合约束、外参软先验和回环约束。辅助相机的相对测量通过 SE(3) 插值处理异步场景。

极线引导的内参与位姿校正。VGFM 输出的内参存在误差，导致位姿中的旋转和平移耦合误差。CALM 提出在线内参搜索：维护一个由基础矩阵构成的测试库，通过置信度评分识别最准确的相机参数。校正模型基于基本矩阵的不变性：

$$\mathbf{S} = \mathbf{K}_{est} \mathbf{K}^{-1} = \text{diag}(s_x, s_y, 1), \quad \Delta = \mathbf{S} - \mathbf{I}$$

真实本质矩阵与估计值的关系为 $\mathbf{E} = \mathbf{S}^{-1} \mathbf{E}_{est} \mathbf{S}^{-1}$ ，由此推导旋转和平移的校正量。自适应阻尼因子 λ 根据测试库密度调节校正强度，防止过度校正。

锚点传播全局一致建图。这是 CALM 区别于所有前作的技术核心。三个步骤：

1. 锚点提取：在 $N_{grid} \times N_{grid}$ 网格上均匀采样高置信度像素，反投影为局部 3D 坐标，通过多帧投影一致性验证。
2. 双向传播：新子图的锚点向后传播到历史子图（通过共视帧），历史锚点向前传播到新子图。回环闭合触发跨子图传播。每个全局锚点维护唯一物理 ID 和跨子图的局部 3D 坐标列表。
3. 非线性对齐：全局锚点坐标为加权平均（权重反比于光心距离）。以锚点为控制点，TPS 变换弹性对齐各子图。局部抑制机制防止"锚点撕裂"——邻近区域内观测次数更高的锚点优先保留。

18.3.4 实验与定位

CALM 在三个数据集上评估：KITTI Odometry（11 个训练序列）、KITTI-360（9 个序列，禁用回环）、Argoverse（10 个序列）。

方法	标定	KITTI-Odom Avg (m)	KITTI-360 Avg (m)
DROID-SLAM	✓	75.85	65.40
MASt3R-SLAM	✗	TL	TL
VGGT-Long	✗	18.28	105.37
VGGT-SLAM	✗	FAIL	309.60
CAL ² M	✗	8.95	19.07

KITTI-360 上禁用回环闭合的测试结果最具说服力。CAL²M 平均 ATE 19.07 m，而 VGGT-Long 为 105.37 m，VGGT-SLAM 高达 309.60 m。VGGT-SLAM 的 SL4 变换在长轨迹上数值不稳定，多序列出现发散。CAL²M 的助手眼尺度校正将归一化尺度因子维持在接近 1.0，而其他方法尺度发散明显。

Argoverse 上的建图质量评估（精度、完整性、Chamfer 距离）进一步验证了锚点+TPS 非线性对齐的效果。CAL²M 的子图过渡几乎无缝，而 VGGT-Long 的 Sim3 对齐在子图边界处出现可见的几何错位，VGGT-SLAM 的 SL4 变换则引入非物理的局部扭曲 (arXiv)。

消融实验确认各模块的必要性：去掉尺度校正 ATE 飙升至 45.14 m；去掉位姿校正（仅内参搜索）ATE 28.18 m；去掉旋转校正导致轨迹发散；去掉后端优化影响较小（19.24 m vs 19.07 m），说明前端校正策略本身已足够鲁棒。测试库密度决定阻尼因子 λ ——当测试库中积累的基础矩阵数量增加时， λ 自适应增大，校正力度增强。这种"越有信心越敢校正"的策略避免了早期阶段测试库稀疏时的过度校正风险。

CALM 的失败模式同样值得关注。内参校正假设固定焦距相机，连续变焦场景下测试库机制失效。相比完全标定同步的立体 SLAM（如 ORB-SLAM3 双目模式），CAL²M 仍有精度差距——这是灵活性换取的代价。此外，助手眼需要额外相机硬件，虽然可以是低成本的，但对于纯单目平台（如手机 SLAM）不适用。VGGT 推理是主要速度瓶颈，核心 CAL²M 模块（内参搜索、锚点提取、建图）平均每个子图（15 关键帧）仅 1.12 秒，与 VGGT 的 1.24 秒并行运行，有效帧率 26.2 FPS (arXiv)。

18.3.5 论文精读

论文试图解决什么痛点？ VGFM 在公里级 SLAM 中的非线性几何畸变无法被 Sim3/SL4 线性变换建模；单目尺度模糊在开放环路中快速累积；内参误差导致旋转平移耦合漂移。

核心假设是什么？ 恒定物理间距的双相机先验可彻底消除尺度模糊；特征匹配准确的假设下，极线几何可校正内参和位姿误差；锚点传播+TPS 非线性变换可实现弹性子图对齐。

输入输出是什么？ 输入 loosely coupled 双相机 RGB 流（无需标定、无需同步）；输出全局一致位姿和稠密地图。

地图表示是什么？ 基于 VGFM 子图的稠密点云，通过锚点传播建立全局骨架，TPS 非线性弹性对齐。

Tracking 怎么做？ 滑动窗口子图 → VGFM 推断 → 尺度校正（助手眼间距先验）→ 极线引导内参搜索和位姿校正 → 主辅联合位姿图优化。

Mapping 怎么做？ 锚点提取 → 双向传播构建全局锚点骨架 → TPS 弹性对齐子图 → 加权融合。

优化目标是什么？ 主辅位姿图 Mahalanobis 距离最小化，内参搜索通过基础矩阵置信度评分。

相比前作真正推进了什么？ 首次系统论证了线性变换在 VGFM 子图对齐中的根本性不足；助手眼概念用最简硬件假设（恒定间距）解决了单目尺度问题；极线引导的在线内参搜索不需预标定；TPS 非线性对齐突破了刚性变换的几何限制。

没有解决什么？ 内参校正假设固定焦距，变焦场景失效；相比完全标定同步的立体 SLAM 仍有精度差距；助手眼需要额外相机硬件。

对后续研究的影响是什么？ CALM 将基础模型 SLAM 的讨论从"房间级精度竞争"推向了"公里级可部署性"问题，定义了大规模场景下的系统性挑战。

18.4 本章总结

2025 至 2026 年的基础模型 SLAM 呈现一个清晰的系统化趋势。这不是"SLAM 被端到端网络替代"，而是一个更微妙的分工格局：局部几何由基础模型增强，全局一致性仍由 SLAM 后端维护。

维度	FoundationSLAM	AIM-SLAM	CALM
基础模型角色	深度先验注入光流特征	多视角点云预测	局部子图几何推断
核心创新	Bi-Consistent BA	SIGMA 自适应关键帧选择	锚点+TPS 非线性对齐
一致性机制	双向几何约束	联合 Sim(3) 优化	助手眼尺度+位姿图优化
适用场景	房间级稠密重建	房间级自适应跟踪	公里级大规模 SLAM
运行速度	18 FPS	3 Hz（前端 17 Hz）	26 FPS（核心模块）
标定依赖	单目需标定	无需标定	双相机无需标定

上表概括了三篇论文的分工。FoundationSLAM 解决了"对应关系几何质量"问题，证明基础模型的深度先验应该渗透到光流估计的最底层。AIM-SLAM 解决了"输入窗口盲目性"问题，证明关键帧选择本身应利用基础模型的预测。CALM 解决了"公里级畸变累积"问题，证明线性变换在 VGFM 输出中的根本性不足，非线性对齐和物理先验是大规模部署的必要条件。

三者的共性在于：没有一篇论文试图用基础模型"替换"整个 SLAM 流水线。FoundationSLAM 保留了光流+BA 的完整架构，只是用基础模型增强了特征；AIM-SLAM 在 MAST3R-SLAM 的框架上替换了固定窗口为自适应选择；CALM 的位姿图优化和后端回环仍是经典 SLAM 的骨架。基础模型增强的是前端感知能力——更深、更稠密、更鲁棒的局部几何估计——但全局一致性仍然依赖 SLAM 的优化后端。

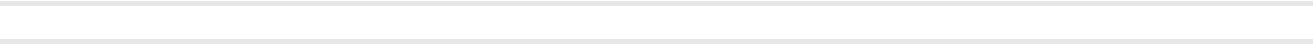
从方法论演进看，三篇论文构成了一条从"感知增强"到"决策增强"再到"规模扩展"的完整链条。FoundationSLAM 对应感知层——让前端看见更可靠的几何；AIM-SLAM 对应决策层——让系统 smarter 地决定用哪些信息；CALM 对应部署层——让系统在实际机器人平台上跑得更远更稳。

对具身智能而言，这一趋势具有直接工程意义。机器人进入未知环境时通常没有预先标定参数，传统 SLAM 需要繁琐的标定流程才能启动。CALM 的零标定框架意味着机器人可以"开箱即用"——上电后立即开始定位和建图。FoundationSLAM 的 18 FPS 实时性能使其适合计算资源有限的边缘平台。AIM-SLAM 的 ROS 集成和自适应选择机制使其在需要灵活调整的场景（如室内外过渡、视角剧烈变化）中表现突出。

更重要的是，这三篇论文共同澄清了一个业界争论：基础模型是否会"吃掉"传统 SLAM？答案是分工而非替代。基础模型在前端提供强大的零-shot 几何感知，但全局一致性、尺度管理、长期漂移抑制仍然依赖 SLAM 后端的核心机制——BA、位姿图优化、回环检测。基础模型 SLAM 的系统化趋势不是用神经网络替换优化器，而是用神经网络增强感知输入的质量，让优化器有更好的原料可加工。

CALM 的双相机设计虽然增加了硬件，但松散耦合的要求（无需同步、无需精确标定）使其在实际机器人平台上高度可行。一台主相机负责主要感知，一台廉价副相机仅用于提供尺度锚定——这种异构传感器配置在成本敏感的消费级机器人上完全可以接受。下一章将在此基础上，进入第五篇——开放词汇语义地图与具身智能。

一句话总结：基础模型给了 SLAM 更强的眼睛，但走路的脑子仍是 BA 和位姿图优化。



第五篇：开放词汇语义地图与具身智能

第19章 语义地图在具身智能中的角色

第五篇进入具身智能的空间智能。前四篇覆盖了从经典SLAM到基础模型SLAM的完整技术谱系，但所有这些工作都有一个共同前提：地图的终极目标是精确的几何重建和相机位姿估计。具身智能打破了这个前提。机器人需要地图不是为了看上去逼真，而是为了做事——找东西、搬东西、回答问题、执行长期任务。这要求一种完全不同的地图设计哲学。

19.1 具身智能需要什么样的地图？

传统SLAM的评估指标围绕几何精度：轨迹误差ATE、重建精度RMSE、F1分数。这些指标隐含一个假设——地图的价值等同于它还原真实几何的能力。具身智能不接受这个假设。

一个机器人在陌生房间里接到指令："把茶几上的遥控器拿到沙发旁边。"它需要知道：（1）茶几在哪里；（2）遥控器在茶几上；（3）沙发在哪里；（4）从茶几到沙发的可通行路径；（5）"旁边"是多远的相对关系。精确到毫米的几何重建固然好，但只要地图不能回答上述问题，就毫无用处。

具身智能对地图的需求可以用七个"可"字概括。

可定位（Localizable）。地图必须支持机器人在其中确定自身位置。这是SLAM的老本行，但具身智能对定位的要求更宽松也更灵活——机器人不需要全局厘米级精度，而是需要在语义相关的局部区域内知道自己"在哪里"。走廊里定位到厘米不如知道"我在客厅入口处"有用。传统SLAM的metric map当然可以用于定位，但在大场景中，纯度量表示的"我在(3.2, 1.5, 0.0)"远不如"我在客厅茶几旁"对后续任务有帮助。将度量定位与语义锚点结合，是具身智能定位的核心需求。

可导航（Navigable）。地图必须编码自由空间和障碍物，支持路径规划。传统占用栅格做这件事很成熟，但具身智能要求更多：机器人需要知道沙发是不可穿越的（对地面机器人），但桌面也是不可穿越的（对大型机器人），而对无人机则不然。导航地图必须与机器人本体耦合。VLMaps (ICRA) 提出了一个精巧的解决方案：通过自然语言描述障碍物类别，同一语义地图可以为不同机器人本体生成不同的占用栅格——"桌子对地面机器人是不可穿越的，但对无人机是可穿越的"。这种语义条件化的导航表示在传统SLAM中不存在。

可查询（Queryable）。这是具身智能的核心诉求。地图必须支持以自然语言或高层语义进行查询——"冰箱在哪里""卧室和卫生间的相对位置""能坐的东西有哪些"。传统几何地图不支持这类查询。开放词汇语义地图通过将CLIP等视觉语言模型的特征嵌入空间表示来实现这一能力。VLMaps (ICRA) 将LSeg视觉语言特征融入3D体素重建，使每个体素携带语言对齐的特征向量。用户查询"沙发"时，系统计算查询文本的CLIP嵌入与地图中每个体素特征的相似度，从而定位目标。ConceptFusion (arXiv) 采用类似思路，将像素级CLIP特征通过多视图融合投影到3D点云上。可查询性是区分"被动重建"和"主动知识库"的关键分水岭。

可推理（Reasonable）。地图不仅要记录"有什么"，还要支持空间推理——A在B的左边、厨房通常和餐厅相邻、沙发的对面往往是电视。场景图通过节点（物体/区域）和边（关系）显式编码这类结构化信息。ConceptGraphs (ICRA) 从RGB-D序列构建的3D场景图中，物体节点之间编码了"左边""右边""上面""里面"等空间关系，LLM可以直接在图上进行链式推理——"遥控器通常在茶几上，茶几在沙发前面，所以去沙发前面找"。这种结构化表示是传统密集点云或栅格地图无法提供的。

可交互 (Interactable)。地图需要编码物体的可供性 (affordance) ——什么可以打开、什么可以坐下、什么可以搬动。传统语义地图只回答"这是什么", 不回答"能做什么"。MomaGraph (arXiv) 是一个代表性的功能场景图方法, 它同时建模物体的空间布局和功能关系——抽屉可以被拉开, 把手用于操作, 蜡烛可以被点燃。这种功能语义对操作任务至关重要: 机器人需要知道"打开冰箱"需要"握住把手"然后"向外拉", 而不只是知道那个白色长方体是冰箱。

可更新 (Updatable)。真实环境是动态的: 人走动、物体被移动、门开开关。地图必须支持增量更新和动态物体的处理。传统SLAM假设静态世界, 具身智能不能做这个假设。DynamicGSG (arXiv) 展示了基于3D Gaussian Splatting的动态场景图更新机制, 系统增量构建语义高斯地图和拓扑场景图, 利用3D高斯的快速训练和可微分渲染能力, 动态更新地图以适应环境变化。在线增量构建与更新是所有语义建图系统走向真实的必经之路。

可长期记忆 (Long-term Memorable)。具身智能任务可能持续数小时甚至数天, 地图需要作为长期记忆的载体。这意味着地图不只是当前观察的聚合, 还需要支持遗忘、压缩、摘要——保留对任务有用的信息, 丢弃无关细节。3D-Mem (arXiv) 将多视角观察组织为记忆快照, 每个快照捕捉共视物体和场景上下文, 支持VLM进行跨时间推理。RenderMem (arXiv) 将渲染作为空间记忆检索机制, 通过神经渲染从压缩记忆中恢复空间信息。长期记忆还涉及任务的持久化: 机器人在昨天探索过厨房后, 今天应该能直接利用昨天的地图, 而不是重新探索。

这七个"可"字勾勒出一个与传统SLAM完全不同的地图画像: 它不是几何重建的终点, 而是智能体与环境交互的接口。传统SLAM追求"精确但沉默"的地图——数据准确却无法对话; 具身智能需要"近似但善言"的地图——允许一定程度的抽象和简化, 但必须能回答问题、支持推理、指导行动。

19.2 2025语义建图综述的核心判断

Raychaudhuri与Chang在2025年发表的《Semantic Mapping in Indoor Embodied AI – A Survey on Advances, Challenges, and Future Directions》中, 对领域现状做了系统性梳理和关键判断 (arXiv)。这是第一篇专注于室内具身AI语义建图的全面综述, 覆盖了从结构表示到信息编码的各个维度。

综述将现有语义建图方法按两条正交维度分类: **结构表示** (空间栅格、拓扑图、密集点云、混合地图) 和 **编码信息类型** (隐式特征 vs. 显式环境数据)。核心发现有三点。

第一, 开放词汇、可查询、任务无关的地图表示成为明确趋势。 受CLIP、GPT-4V等大型视觉语言模型驱动, 领域正在从封闭集语义标签 (如"椅子/桌子/沙发"的固定类别) 转向开放词汇嵌入空间。VLMs将LSeg视觉语言特征融入3D重建, 支持自然语言索引; ConceptGraphs构建开放词汇3D场景图, 物体节点携带CLIP特征, 可应对训练时未见过的类别 (ICRA)。这种表示的任务无关性意味着同一个地图可以服务于导航、问答、操作等不同下游任务, 无需为每个任务重新建图。综述指出, 这一趋势的本质是: 地图从"任务专用"走向"通用知识库"。

第二, 高内存需求与计算效率仍是主要瓶颈。 密集点云地图、3D Gaussian表示、高维CLIP特征的存储开销巨大。室内场景中大部分3D空间实际上是空的, 但密集表示无法利用这一稀疏性。以LangSplat为例, 每个3D高斯携带128维或更高维度的语言特征, 一个普通房间的高斯数量可达数十万, 内存占用以GB计。综述指出, 尽管混合地图在空间效率上有优势, 如何在保持开放词汇能力的同时实现实时增量构建, 仍没有满意答案。Octree-based的稀疏表示 (如Open-Vocabulary OctreeGraph) 是一条有希望的压缩路径, 但在增量更新场景中的效率仍有待提升。

第三，仿真到现实的鸿沟依然显著。现有方法多在模拟环境中评估（Habitat、AI2-THOR、Matterport3D），依赖完美定位和无噪声传感器的假设。真实机器人面临传感器噪声、动态物体、计算资源受限、光照变化等问题，这些在仿真中很少体现。综述特别指出：语义建图领域的sim-to-real gap比传统SLAM更严重——因为语义理解本身对观察条件（视角、遮挡、光照）的敏感性远高于几何重建。此外，绝大多数方法假设静态环境，限制了它们在真实动态场景中的适用性。

综述的另一重要判断是：混合地图（结合空间几何与拓扑语义）在具身AI中研究不足，但前景最大。传统机器人学已证明混合地图（如Blanco等人2008年的度量-拓扑混合SLAM）在大型环境中的优势，但具身AI社区尚未充分挖掘这一方向。Hybrid maps既能保留局部几何精度（支持精确操作和碰撞检测），又能提供全局语义抽象（支持LLM推理和高层规划），是连接底层感知与高层认知的天然桥梁。

19.3 地图结构分类：结构×语义的二维空间

语义建图方法可以从"结构维度"和"语义维度"两个正交轴进行分类。结构维度回答"地图长什么样"，语义维度回答"地图携带什么信息"。

19.3.1 结构维度

Grid Map（栅格地图）。将空间离散化为2D或3D栅格，每个单元格存储语义信息。2D语义占用栅格广泛用于室内导航；3D体素栅格（如VLMaps的3D voxel grid）可以编码语言特征。栅格的优势是规则、易于查询、与A*等传统路径规划算法天然兼容；劣势是分辨率与内存的指数级权衡——将分辨率提高一倍，内存需求变为8倍（3D）。固定分辨率的栅格也无法适应环境中的多尺度语义：识别"房间类型"需要大尺度上下文，定位"遥控器"需要细粒度分辨率。

Topological Graph（拓扑图）。以节点和边表示环境：节点对应地点/物体/区域，边对应可达性或空间关系。Chaplot等人在Neural Topological SLAM (CVPR) 中展示了如何用神经网络学习拓扑表示，将环境建模为记忆图中的节点，支持视觉导航中的长程路径规划。拓扑图天然适合LLM推理——图结构可序列化为JSON或文本，直接输入LLM进行规划和推理。ConceptGraphs正是利用这一特性，将3D场景图序列化为文本描述供LLM使用 (ICRA)。但拓扑图缺乏精确几何，难以支持需要精确位姿的操作任务（如抓取特定位置的物体）。

Point Cloud（点云）。RGB-D建图的直接输出，每个点可附加语义标签或特征向量。ConceptFusion (arXiv) 将密集2D CLIP特征通过多视图一致性投影到3D点云上，实现开放词汇查询；OpenScene (CVPR) 学习了3D点云上的密集语义特征。点云保留原始传感器数据的几何精度，但存在三个问题：无序性（点云没有内在结构，邻近查询需要KD-tree等加速结构）、冗余性（大部分点落在平面上，信息密度不均）、查询效率低（开放词汇查询需要遍历大量点）。

Mesh（三角网格）。从点云重建的连续表面表示，更适合碰撞检测和物理仿真。语义信息可以存储在面片级别。但在线增量构建mesh计算开销大，在具身AI实时系统中应用较少。Mesh的优势在于它是真正的连续表面表示，碰撞检测比点云更可靠；劣势是增量构建算法复杂，且开放词汇语义在mesh上的研究几乎空白。

3DGS（3D Gaussian Splatting）。用3D高斯原语显式表示场景，每个高斯携带位置、颜色、不透明度、3D协方差矩阵，以及可选的语义/语言特征。SplaTAM (arXiv) 证明了3DGS用于实时密集SLAM的可行性——渲染速度达400FPS，相机跟踪精度比NeRF-based方法提升2倍。LangSplat (CVPR) 将CLIP特征嵌入高斯，通过场景特定的语言自编码器压缩高维特征，在LERF数据集上比NeRF-based的LERF快199倍，同时定位精

度从73.6%提升到84.3%。OpenGaussian (NeurIPS) 实现了点级开放词汇理解，支持直接在3D高斯上进行物体选择和分割。3DGS的显式表示使其易于更新（直接增删高斯），可微分渲染使优化高效，快速渲染支持实时交互。但内存消耗大（数十万个高斯原语，每个携带多组参数），且动态场景中的高斯分裂和合并机制仍在研究中。

Scene Graph (场景图)。图结构表示环境中的物体实例及其关系。节点可以是物体（"沙发"）、区域（"客厅"）或抽象概念（"休息区"）；边编码空间关系（"左边""里面"）、功能关系（"用于坐"）或属性关系（"是红色的"）。Kimera (ICRA) 提供了最早的实时度量-语义场景图系统之一；ConceptGraphs (ICRA) 实现了开放词汇3D场景图，从RGB-D序列自动提取物体节点并用VLM生成语言描述；HOV-SG (RSS) 构建层次化的楼层-房间-物体三级场景图，支持跨楼层的语言引导导航。场景图的抽象级别最高，与LLM的兼容性最好——ConceptGraphs将场景图序列化为JSON文本后，GPT-4可以直接读取并生成任务计划。但场景图丢失了细粒度几何：知道"遥控器在茶几上"不足以完成抓取，还需要知道遥控器相对于机器人末端执行器的精确6DoF位姿。

Hybrid Map (混合地图)。组合上述多种表示。Hydra (RSS) 结合度量重建与层次化场景图，同时维护度量一致性和语义层次结构；HERO (arXiv) 构建支持可移动障碍物导航的层次化可穿越场景图；CLIO (arXiv) 融合度量地图和开放词汇场景图，支持上下文感知的对象检索。混合地图试图兼顾各表示的优势——度量部分处理精确几何，拓扑部分支持语义推理——但系统复杂度更高，不同表示之间的同步和对齐是额外挑战。

19.3.2 语义维度

Closed-set Label (封闭集标签)。预定义类别集合（如ScanNet的40类、NYUv2的40类）。每个地图单元分配一个类别ID。传统语义SLAM（如SemanticFusion、CNN-SLAM）采用此方式。优势是简单、高效、有大量标注数据支撑；劣势是无法处理训练时未见过的类别。当机器人遇到训练集中没有见过的物体（如新型智能家居设备），封闭集表示完全失效。这对于需要在新环境中自主运行的具身智能体是不可接受的。

CLIP/VLM Feature (视觉语言特征)。将CLIP等模型提取的密集特征嵌入地图单元，实现开放词汇查询。VLMaps (ICRA) 将LSeg特征融入3D体素；LERF (arXiv) 将CLIP特征嵌入NeRF；LangSplat (CVPR) 将压缩后的CLIP特征嵌入3D高斯；SLAG (arXiv) 提出可扩展的语言增强高斯溅射。开放词汇能力消除了封闭集的限制——只要CLIP能理解的文本描述，地图就能查询。但高维特征（CLIP基线512维，LSeg也是512维）带来存储和计算开销。LangSplat的自编码器压缩将512维降至32维，在精度损失可控的情况下大幅降低了内存占用。

Object Instance (物体实例)。将环境显式分解为独立物体实例，每个实例作为一个语义单元。OpenMask3D (CVPR) 先用实例分割确定候选物体，再为每个3D实例分配CLIP嵌入；ConceptGraphs以物体实例为图节点，支持实例级别的查询和操作；Gaussian Grouping (CVPR) 利用SAM的2D掩码将高斯分组为独立物体实例。实例级表示天然支持物体级操作（"拿起那个苹果"），也能编码实例特定信息（"这个杯子是红色的，那个是蓝色的"）。但实例分割的错误会传播到地图中——SAM在边界模糊或遮挡严重时产生的分割错误，会直接影响实例节点质量。

Affordance (可供性)。编码物体的功能属性——可以坐、可以放东西、可以打开。可供性是认知心理学概念（Gibson, 1977），在机器人领域指物体的功能使用方式。功能场景图MomaGraph (arXiv) 显式建模可供性和物体间的功能关系，将"空间图"（描述布局）与"功能图"（描述用法）统一到一个表示中。HERO (arXiv) 进一步引入"可穿越性"作为边属性，编码环境中物体对导航的约束。可供性表示弥合了"是什么"和"能做什么"的鸿沟，是当前语义建图中最被低估但最重要的方向之一。

Language Annotation（语言标注）。地图中的实体附带着人类可读的语言描述。ConceptGraphs用VLM（如LLaVA或GPT-4V）为物体节点生成文字说明——"一个黑色的皮革沙发，位于客厅中央"；HOV-SG通过LLM为房间和物体生成开放词汇描述。语言标注使LLM可以直接"阅读"地图并进行推理。SG-Nav (arXiv) 利用在线构建的3D场景图生成文本提示，引导LLM进行零样本物体导航。语言标注的本质是将空间表示转换为LLM可消费的文本形式，桥接感知与认知。

Task Memory（任务记忆）。地图不只是空间表示，还承载任务相关的记忆——哪些区域已探索、哪些物体被移动过、之前在哪里失败过。3D-Mem (arXiv) 组织多视角观察为记忆快照，每个快照编码共视物体和场景上下文，支持VLM-based的探索决策。ExploreLikeHumans (arXiv) 在探索过程中在线构建场景图备忘录（SG-Memo），记录已访问区域和发现的物体，指导后续探索。任务记忆将地图从"环境模型"提升为"经验载体"，使其成为持续学习的基础。

19.3.3 二维交叉分类表

下表展示了结构维度与语义维度的交叉分类，每个单元格代表一个具体研究方向或代表性工作。

	Grid Map	Topological Graph	Point Cloud	Mesh	3DGS	Scene Graph	Hybrid Map
Closed-set Label	SSC Semantic Voxel	—	SemanticFusion	Semantic Mesh	Semantic Gaussians	Kimera	Hydra
CLIP/VLM Feature	VLMaps	LM-Nav	ConceptFusion, OpenScene	—	LangSplat, LERF	—	AVLMaps
Object Instance	Instance Voxel	—	OpenMask3D	—	OpenGaussian, GaussianGrouping	ConceptGraphs, HOV-SG	CLIO
Affordance	—	—	—	—	—	MomaGraph, HERO	—
Language Annotation	—	SG-Nav	—	—	—	ConceptGraphs + LLM caption	OpenLex3D
Task Memory	Occupancy Anticipation	Neural Topo SLAM	3D-Mem snapshots	—	RenderMem	ExploreLikeHumans SG-Memo	RoboOS

上表揭示了领域的一些结构性特征。CLIP/VLM Feature列最为拥挤，说明开放词汇是当前研究的热点；Affordance行最为稀疏，表明功能语义的理解和表示仍处于早期阶段。3DGS列的快速增长值得特别关注——从2023年SplaTAM证明3DGS可用于SLAM，到2024年LangSplat将语言特征嵌入高斯，再到OpenGaussian实现点级开放词汇查询，DynamicGSG支持动态更新——3D Gaussian Splatting正在从一个神经渲染技术快速演变为具身智能的核心建图载体。

Hybrid Map列虽然条目不多，但每个都代表了一个完整的系统架构，显示出融合多种表示是走向实用的必然路径。纯粹的几何表示（grid/point cloud）缺乏语义抽象能力，纯粹的拓扑表示（scene graph）缺乏几何精度，只有混合表示能同时满足可定位、可导航、可查询、可推理的多重需求。

19.4 本章核心观点

具身智能不是不要SLAM，而是需要一种比传统SLAM更高层的空间表示。

回顾前18章：从特征点法到直接法，从滤波到图优化，从单目到RGB-D到多传感器融合，从稀疏到稠密到神经隐式表示——SLAM社区二十年来追求的核心目标始终是更精确的几何重建和更鲁棒的相机位姿估计。这些追求没有错，但它们服务于一个特定场景：相机在未知环境中运动，需要知道自己在哪里、周围环境长什么样。

具身智能的场景不同。机器人不是来"拍照"的，而是来"做事"的。它需要知道遥控器在茶几上，而不需要知道茶几表面的毫米级形状；它需要知道沙发和电视的相对位置来判断"对面"是什么意思，而不需要两者的精确三维坐标。传统SLAM提供的几何精度，在具身智能中大部分被浪费了；而具身智能需要的语义理解和推理能力，传统SLAM又完全不提供。

这不是说几何精度无关紧要。操作任务中的抓取需要精确位姿，碰撞检测需要精确表面。关键在于：**几何只是手段，不是目的**。具身智能的地图应该以任务需求为导向，在需要几何精度的地方保留几何（如抓取区域的局部精细重建），在需要语义抽象的地方提升抽象（如房间级别的功能区域划分）。

这个视角引出本书第五篇的主线：从几何建图到语义建图，从重建环境到理解环境，从被动记录到主动交互。第20章开始，我们将深入具体系统——VLMaps如何用CLIP特征填满3D体素，ConceptGraphs如何从RGB-D序列构建开放词汇场景图，HOV-SG如何用层次化结构支撑楼层级导航。这些系统的技术路径各异，但共享同一个底层假设：地图是具身智能与环境交互的接口，而不只是几何重建的产物。

传统SLAM回答了"世界长什么样"。具身智能的语义地图回答"我能在这里做什么"。这是第五篇的起点，也是全书从SLAM到空间智能的终极落脚点。

第20章 VLMaps、ConceptGraphs 与开放词汇 3D 地图

传统语义地图要求训练时锁定类别表，新类别出现即失效。开放词汇（open-vocabulary）映射要打破这个约束：利用视觉语言模型（VLM）在海量图文数据中学到的共享嵌入空间，让地图理解训练时从未见过的语义描述。本章追踪这条技术线的三个里程碑——VLMaps 把语言查询变成地图的原生能力，ConceptGraphs 把地图从点的集合提升为对象与关系的图，OVO 证明这种地图可以在线构建并与 SLAM 回环闭合协同。最后，我们梳理五个尚未解决的核心难点。

20.1 VLMaps

20.1.1 问题定义：让地图听懂自然语言

第19章讨论的语义地图依赖封闭类别表。机器人需要找到"沙发"时，系统必须事先在训练数据中见过"沙发"这个标签。但真实世界的指令是开放的："去沙发和电视之间"、"找到能充电的东西"。VLMaps 要解决的核心问题是：如何让 3D 地图支持自然语言查询，且零样本（zero-shot）泛化到任意语义描述？

20.1.2 传统方法的局限

LM-Nav 把图像观测存成图节点，用 CLIP 做图文匹配，再用 GPT-3 在图上规划 (arXiv)。它只能导航到图节点中存储过的图像位置，无法处理空间关系引用（如"A 和 B 之间"）。CLIP on Wheels (CoW) 用 GradCAM 从 CLIP 生成显著性图做导航，但 GradCAM 的 image-level 到 pixel 的映射噪声极大，导致分割 mask 充满错误 (arXiv)。这两种方法都缺少一个关键东西：地图本身不是为语言查询设计的，语言只是事后贴上去的补丁。

20.1.3 VLMaps 的核心洞察

VLMaps 的核心洞察只有一句话：把预训练视觉语言模型的密集像素级特征，在 3D 重建的物理位置上"固化"下来，使每个地图位置都携带语义嵌入 (ICRA 2023)。这样，地图查询就从"在预定义类别表中查找"变成了"在 CLIP 嵌入空间中进行余弦相似度检索"。

20.1.4 系统结构

VLMaps 的构建流程分为三步：

特征提取。 使用 LSeg (language-driven semantic segmentation) 作为视觉骨干 (ICLR)。LSeg 的视觉编码器将 RGB 图像的每个像素映射到 CLIP 特征空间，生成密集像素级嵌入图 $F_{img} \in \mathbb{R}^{H \times W \times C}$ ，其中 C 为 CLIP 嵌入维度 (512D 或 768D)。

3D 反投影。 对每个 RGB-D 帧，深度像素 $u = (u, v)$ 反投影为局部点云：

$$P_w = T_k K^{-1} d(u) \tilde{u}$$

其中 K 为相机内参， T_k 为第 k 帧的相机位姿（由 RGB-D SLAM 提供）， $d(u)$ 为深度值。每个 3D 点继承对应像素的 LSeg 嵌入。

网格聚合。 将 3D 点投影到俯视 2D 网格地图，网格分辨率 s 通常设为 0.05 m。同一网格内的多个嵌入取平均，最终得到 VLMap $M \in \mathbb{R}^{H_{map} \times W_{map} \times C}$ 。

20.1.5 语言查询机制

查询阶段，将自然语言目标（如 "sofa"）通过 CLIP 文本编码器得到嵌入 f_{text} ，然后与地图中每个网格的嵌入计算余弦相似度：

$$S(i, j) = \cos(M[i, j], f_{text})$$

取 argmax 得到目标位置的热力图。对于空间关系查询（如 "in between the sofa and the TV"），VLMaps 先将两个实体分别定位，然后基于几何关系计算空间区域，例如取两者之间的中点区域。这一能力在与大语言模型 (LLM) 结合时尤其强大：LLM 将复杂语言指令解析为一系列 API 调用（如 `move_to("sofa")`、`move_in_between("sofa", "TV")`），VLMaps 将每个调用翻译为地图上的具体坐标目标 (ICRA 2023)。

20.1.6 论文精读

论文试图解决什么痛点？ 现有视觉语言模型 (VLM) 虽能将图像与自然语言匹配，但与环境建图脱节，缺乏几何地图的空间精度。导航任务需要同时具备空间精确性和开放词汇语义理解能力。

核心假设是什么？ 预训练视觉语言模型 (LSeg/CLIP) 的嵌入空间具有足够的泛化能力，零样本迁移到机器人建图场景后，仍能保持语义一致性。

输入输出是什么？ 输入为 RGB-D 视频流和相机位姿（由 RGB-D SLAM 提供）；输出为俯视语义网格地图，每个网格存储 CLIP 兼容的语义嵌入，支持任意自然语言查询。

地图表示是什么？ 2D 俯视网格地图（top-down grid map），每个网格单元存储平均后的 LSeg 嵌入向量。

Tracking 怎么做？ 依赖外部 SLAM 系统提供相机位姿，不做自身的状态估计。

Mapping 怎么做？ 逐帧将 LSeg 像素特征通过深度反投影到 3D，再投影到俯视网格取平均。

优化目标是什么？ 无显式优化目标函数，建图过程是简单的特征平均累积。

相比前作真正推进了什么？ 相比 LM-Nav（只能导航到图节点位置）和 CoW（GradCAM 噪声大），VLMaps 首次让地图本身成为语言可查询的表示，支持空间关系定位（"A 和 B 之间"），且能跨机器人形态共享地图、动态生成不同障碍物图。

没有解决什么？ 对 3D 重建噪声和里程计漂移敏感；在物体密集场景中无法区分相似物体；假设静态环境；仅支持俯视 2D 地图，丢失了 3D 空间信息。

对后续研究的影响？ 开创了"VLM 特征 + 3D 重建"的范式，后续 OpenScene、ConceptFusion、LERF 等工作沿此方向将 CLIP 特征扩展到 3D NeRF、点云和高斯表示。

20.1.7 优点与失败模式

优点。 模块简单，无需训练或微调；零样本语义泛化；与 LLM 结合后可处理复杂空间语言指令（"在椅子右边三米处"）；支持跨机器人形态的地图共享（同一 VLMap 可为地面机器人生成"避开桌椅"的障碍物图，也可为无人机只标记"墙壁"为障碍）。

失败模式。 LSeg 对训练时未见过的视觉概念会将其映射到相似已知类别的嵌入空间（如将"bench"映射为"chair"或"sofa"），造成视觉-文本错位。3D 反投影对深度噪声敏感，远处物体特征污染邻近网格。俯视投影丢失高度信息，无法区分上下叠放的不同物体。

20.1.8 与具身智能的关系

VLMaps 是具身智能中"感知-语言-行动"闭环的早期范例。它将 LLM 的符号推理能力锚定到物理空间的精确坐标上，使语言指令不再停留在抽象层面。后续的 AVLMaps (arXiv) 在此基础上增加了音频模态，证明 VLMaps 框架可向多模态扩展，进一步支撑机器人在模糊环境中的目标消歧（如"找到发出哭声的物体"）。

20.2 ConceptGraphs

20.2.1 问题定义：从点到图的跃迁

VLMaps 的表示仍然是密集的：每个网格或每个点都携带一个特征向量。这种表示有两个根本缺陷：一是冗余——一个沙发上的数千个点共享同一个语义；二是无结构——点与点之间的关系（"书在桌上"、"椅子靠着墙"）被淹没在密集的嵌入海中。ConceptGraphs 要回答的问题是：如何把地图从"点的集合"提升为"对象与关系的图"？

20.2.2 传统方法

3D 场景图 (3DSG) 早有研究，但传统方法如 Kimera 的语义场景图是封闭词汇的，节点类别在训练时固定 (arXiv)。开放词汇的 3D 表示 (如 OpenScene、ConceptFusion) 解决了词汇问题，但采用 per-point 特征存储，内存随场景线性增长，且缺乏对象级别结构和关系信息。

20.2.3 ConceptGraphs 的核心洞察

对象是比点更自然的语义单元。ConceptGraphs 用 2D 基础模型做无类别分割，将属于同一对象的 2D mask 跨视图关联融合为 3D 对象节点，然后用大视觉语言模型和大语言模型分别为节点生成语义描述、为边推断空间关系 (ICRA 2024)。最终表示是一张紧凑、可查询、可更新的开放词汇 3D 场景图。

20.2.4 系统结构

对象级 3D 映射。 给定带位姿的 RGB-D 帧序列 $\{I_1, I_2, \dots, I_t\}$ ，维护 3D 场景图 $M_t = \langle O_t, E_t \rangle$ ，其中 O_t 为对象节点集合， E_t 为边集合。每个对象节点 o_j 由 3D 点云 p_j 和语义特征向量 f_j 表征。

逐帧处理时，先用 SAM (Segment Anything) 做无类别 2D 实例分割，得到候选 mask。对每个 mask，提取 CLIP 图像嵌入作为语义描述符，并通过深度反投影得到局部 3D 点云。

跨视图对象关联。 新检测到的对象 $o_{t,i}$ 与已有对象 o_j 的关联得分由几何相似度和语义相似度组成：

$$\phi(i, j) = \phi_{sem}(i, j) + \phi_{geo}(i, j)$$

其中语义相似度 $\phi_{sem}(i, j) = (f_{t,i}^T f_j / 2 + 1/2)$ 为归一化余弦距离，几何相似度 $\phi_{geo}(i, j)$ 衡量两个点云之间的最近邻覆盖率。采用贪心策略匹配：每个新检测与得分最高的已有对象关联；若最高得分低于阈值 $\delta_{sim} = 1.1$ ，则初始化新对象。

关联成功后，融合更新对象的语义特征和点云：

$$f_j = \frac{n_j f_j + f_{t,i}}{n_j + 1}$$

其中 n_j 为对象被观测到的次数。

场景图生成。 对象集合确定后，计算每对对象 3D 边界框的 IoU 得到密集连通矩阵，用最小生成树 (MST) 剪枝得到候选边。然后利用 LLaVA 为每个对象生成 caption (从 10 个不同视角裁剪送入 LLaVA)，再用 GPT-4 推断 MST 中每对相邻对象的空间关系 (如 "a on b"、"b in a")，形成场景图的边。

20.2.5 关键实现

ConceptGraphs 使用 SAM 做分割，CLIP ViT 做特征提取，LLaVA 做节点 caption，GPT-4 做关系推断 (ICRA 2024)。体素下采样和最近邻阈值均为 2.5 cm。论文还提出了 CG-D 变体，用 RAM (图像标注模型) 列出类别 + Grounding DINO 做开放词汇检测，适合特定类别的定向检测。

20.2.6 论文精读

论文试图解决什么痛点？ 现有开放词汇 3D 表示（如 OpenScene、ConceptFusion）采用 per-point 特征，内存不可扩展，缺乏对象结构和关系信息；传统 3D 场景图是封闭词汇的。需要一个既开放词汇又对象级别且紧凑的 3D 表示。

核心假设是什么？ 2D 基础模型（SAM、CLIP）的零样本能力足以支撑 3D 对象的发现和描述；LLM 有足够的空间常识来推断物体间关系；对象级别的表示比 per-point 表示更紧凑、更适合机器人规划。

输入输出是什么？ 输入为带位姿的 RGB-D 帧序列；输出为开放词汇 3D 场景图（节点 = 3D 对象 + CLIP 特征 + LLaVA caption，边 = LLM 推断的空间关系），支持下游任务：分割、目标定位、导航、操作、定位、地图更新。

地图表示是什么？ 图结构 $M_T = (O_T, E_T)$ 。每个节点包含：3D 点云、3D 边界框、CLIP 特征、语言描述。每条边包含关系标签（如 "on"、"in"、"next to"）和推理解释。

Tracking 怎么做？ 依赖外部 SLAM/SfM 提供相机位姿。对象跨帧跟踪通过投影-匹配实现：将已有 3D 对象投影到当前帧，与 SAM 生成的 2D mask 做几何+语义相似度匹配。

Mapping 怎么做？ 增量式：每帧做 SAM 分割 → 反投影到 3D → 与已有对象关联（或新建）→ 融合点云和特征 → 场景图生成。

优化目标是什么？ 无全局优化，采用贪心关联和增量融合。

相比前作真正推进了什么？ 相比 ConceptFusion（per-point CLIP 特征，内存随场景增长），ConceptGraphs 将表示压缩到对象级别，内存与对象数成正比而非点数；首次将开放词汇 3D 场景图用于机器人规划任务（导航、操作、定位）；LLM 生成的关系边使场景图可直接被 LLM planner 消费。

没有解决什么？ 对象关联采用贪心策略而非全局最优，密集场景中可能误合并或拆分对象；场景图生成依赖离线 LLM 调用（GPT-4），延迟高无法实时；SAM 分割在纹理缺失区域效果差；关系推断准确率约 88%，仍有错误传播风险；未处理动态环境（对象移动、新增、删除）。

对后续研究的影响？ 证明了对象级 3D 场景图比 per-point 表示更适合机器人任务；启发了后续将 LLM 与 3D 场景图结合做规划的工作（如 SayPlan、Delta）；为开放词汇场景理解提供了从"感知"到"规划"的完整 pipeline。

20.2.7 优点与失败模式

优点。 表示紧凑：对象数远少于点数，内存占用低；语义丰富：节点有 LLaVA 生成的自然语言描述，边有空间关系标签，可直接喂给 LLM 做推理；开放词汇：SAM 不预设类别，CLIP 支持任意语义查询；模块化：SAM、CLIP、LLaVA、GPT-4 均可替换为新版本。

失败模式。 LLaVA 的 caption 质量直接决定节点标签的准确性（实验显示节点精度约 71%，多数错误来自 LLaVA）；LLM 关系推断依赖物体 caption 的准确性，错误 caption 导致错误关系；贪心对象关联在遮挡严重时可能将一个物理对象拆成两个节点，或将两个靠近的不同对象合并。

20.2.8 与具身智能的关系

ConceptGraphs 是具身智能中"结构化空间记忆"的关键一环。它生成的场景图可直接转化为自然语言描述供 LLM 消费，实现从感知到规划的端到端衔接。在实验中，ConceptGraphs 被部署到轮式和足式机器人上，执行包括"找到能帮我拧螺丝的东西"（affordance 查询）、"把 cuddly quacker 拿起来"（开放词汇操作）、"我能推这个物体穿过去吗？"（可通行性判断）等任务。这些任务都需要超越几何的语义推理——正是场景图结构提供的核心能力。

20.3 OVO

20.3.1 问题定义：在线，而不是离线

VLMs 和 ConceptGraphs 都假设输入是完整的 RGB-D 序列，可以离线处理。但机器人需要在探索中实时构建语义地图：每到一个新位置，地图就应该更新。OVO（Open-Vocabulary Online 3D semantic mapping）的核心问题是：开放词汇 3D 语义地图能否在线构建，并与 SLAM 的回环闭合协同工作？

20.3.2 传统方法的瓶颈

离线方法（OpenScene、OpenMask3D、Open3DIS、HOV-SG）需要先获得完整的场景重建，然后批量处理。在线方法（Kimera、SemanticFusion）虽能实时运行，但只支持封闭词汇。介于两者之间的方法（ConceptFusion、ConceptGraphs、Open-Fusion）号称在线，但不支持回环闭合（loop closure）——当 SLAM 检测到回环并校正累计漂移时，语义地图无法同步修正 (arXiv)。

方法	开放词汇	3D 语义	在线	回环闭合
OpenScene	✓	✓	✗	-
OpenMask3D	✓	✓	✗	-
Open3DIS	✓	✓	✗	-
HOV-SG	✓	✓	✗	-
OpenNeRF	✓	✓	✗	-
NEDS-SLAM	✗	✗	✓	✗
NIS-SLAM	✗	✗	✓	✗
SGS-SLAM	✗	✓	✓	✗
ConceptFusion	✓	✓	✓	✗
ConceptGraphs	✓	✓	✓	✗
Open-Fusion	✓	✓	✓	✗
OVO	✓	✓	✓	✓

这张表清晰展示了开放词汇建图领域的的能力缺口。OpenScene 系列解决了开放词汇和 3D 语义，但无法在线运行；NEDS-SLAM 系列可以在线，但封闭词汇且不支持回环；ConceptGraphs 虽接近目标，但回环闭合的缺失使其在长距离探索中会因漂移累积而失效。OVO 首次将四个能力集于一身，填补了领域的关键空白。

20.3.3 OVO 的核心洞察

在线开放词汇建图的关键瓶颈不是语义提取（SAM 和 CLIP 已经足够快），而是跨帧一致性和回环闭合时的语义融合。OVO 的核心洞察是：把 3D segments 当作 tracked entities，用神经网络学习如何最优融合多个视角的 CLIP 描述符，并设计一套机制在回环闭合后自动合并被漂移"拆散"的语义实例 (arXiv)。

20.3.4 系统结构

OVO 采用经典的 tracking-and-mapping 并行架构，输入为 RGB-D 关键帧流及位姿，输出为带 CLIP 嵌入的 3D 语义 segments。

3D Segment Mapper。 每帧用 SAM 2.1 生成 2D masks，反投影为 3D segments。已有 3D segments 被投影到当前帧，与新的 2D masks 做匹配。匹配准则综合几何重叠和语义相似度。匹配成功的 mask 更新对应 3D segment 的点云和观测记录；未匹配的 mask 初始化新 segment。

CLIP Merging。 这是 OVO 的核心创新。对每个 2D mask，OVO 计算三个 CLIP 描述符：整图描述符 F_{global} 、mask 区域描述符 F_{masked} 、包围框区域描述符 F_{bbox} 。传统方法用固定权重融合三者（如 HOV-SG 的加权策略），OVO 则用一个小 Transformer（5 层自注意力，8 头，1152 维隐层）预测每维度的动态权重，输出最终描述符：

$$\mathbf{d} = \sum_{k \in \{global, masked, bbox\}} w_k \odot F_k$$

其中权重 w_k 由神经网络根据图像内容预测，实现实例级的自适应融合。该网络在 ScanNet++ 的 230 个训练场景上训练，仅用最常见的 100 个类别，但实验表明它泛化到 1600 个未见类别和新的数据集。

最优视角选择。 每个 3D segment 可能在数十帧中被观测到，OVO 选择 mask 面积最大（即视角最佳）的 10 个关键帧，用 CLIP Merging 网络为每个 mask 生成描述符，然后从中选择最具代表性的一个赋予该 3D segment。

回环闭合处理。 当 SLAM 模块执行回环闭合或全局 BA 时，OVO 同步更新 3D 点云位姿。然后遍历所有 3D segment 对，若满足三个条件则合并：(1) 质心距离 < 150 cm；(2) CLIP 余弦相似度 > 0.8；(3) 超过 50% 的点在 10 cm 内邻近。合并时统一点索引，将旧标签重新标记。

20.3.5 与 SLAM 集成

OVO 实现了三种配置：OVO-mapping（使用真值位姿，用于评估建图本身）、OVO-Gaussian-SLAM（集成到 Gaussian-SLAM 中）、OVO-ORB-SLAM2（集成到 ORB-SLAM2 中）(arXiv)。后两种配置证明 OVO 可以与不同的 SLAM 后端协同工作，包括稀疏特征法（ORB-SLAM2）和神经表示法（Gaussian-SLAM）。

20.3.6 论文精读

论文试图解决什么痛点？ 现有开放词汇 3D 语义方法要么离线处理（需要完整序列），要么在线但不支持回环闭合，要么封闭词汇。机器人需要一种能在实时探索中构建、且当检测到回环时能自动修正的开放词汇语义地图。

核心假设是什么？ SAM 2.1 的 2D 分割质量足以支撑 3D segment 的提取和跨帧跟踪；CLIP 嵌入空间在神经网络的自适应融合下仍能保持开放词汇泛化能力；回环闭合后的几何校正 + CLIP 相似度准则足以正确合并被漂移拆分的语义实例。

输入输出是什么？ 输入为 RGB-D 关键帧流及 SLAM 提供的位姿；输出为 3D 语义地图，其中每个 segment 带有 CLIP 嵌入，支持任意开放词汇类别查询。

地图表示是什么？ 带标签的 3D point cloud，其中每个 point 属于一个 segment，每个 segment 有一个 CLIP 描述符。

Tracking 怎么做？ 3D segments 投影到当前帧与 SAM 2.1 的 2D masks 匹配，综合几何重叠和语义相似度。SAM 2.1 的 mask 质量直接决定 tracking 的稳定性。

Mapping 怎么做？ 增量式：每帧 SAM 分割 → 反投影 → 与已有 segments 匹配/新建 → 延迟计算 CLIP（通过队列调度）→ 回环时合并重复 segments。

优化目标是什么？ CLIP Merging 网络的训练目标是最小化融合后描述符与对应语义类别文本描述符之间的余弦距离（使用 ScanNet++ 的 2D 语义标注作为监督）。

相比前作真正推进了什么？ OVO 是首个同时实现开放词汇、3D 语义、在线运行、回环闭合四个特性的系统。相比离线方法（OpenScene、OpenMask3D），OVO 的计算和内存开销显著更低；相比在线方法（ConceptFusion、Open-Fusion），OVO 支持回环闭合，这是长距离机器人探索的必要能力。CLIP Merging 网络首次用神经网络取代手工设计的 CLIP 融合启发式规则，在多个数据集上取得更好的分割指标。

没有解决什么？ 处理速度约 1 FPS，限制了实时应用（需要多 GPU）；CLIP Merging 在训练时只见过 100 个类别，对完全未见的长尾类别存在轻微偏差；SAM 2.1 的分割在深度不连续区域会产生"mask bleeding"（mask 溢出到背景）；未处理动态物体（人走动、门开关等场景会导致 segments 分裂或合并错误）；需要深度传感器（RGB-D），不支持纯 RGB 输入。

对后续研究的影响？ OVO 证明了开放词汇语义建图可以与经典 SLAM 架构无缝集成。Ov3R (arXiv) 在此基础上进一步去掉深度依赖，用 RGB-only 视频实现开放词汇建图，验证了 CLIP3R 作为 3D 重建骨干的可行性。

20.3.7 优点与失败模式

优点。 四项关键能力同时在线：开放词汇 + 3D 语义 + 在线增量 + 回环闭合；CLIP Merging 网络的自适应融合优于固定权重策略；模块化设计可与不同 SLAM 后端集成；计算和内存效率优于所有离线基线。

失败模式。 SAM 2.1 的 mask 在深度边缘溢出，导致 3D segment 包含背景点，影响 CLIP 描述符质量。1 FPS 的处理速度对高速平台不够。训练数据集中在室内场景（ScanNet++），在室外场景的泛化未经验证。回环闭合后的 segment 合并依赖三个启发式阈值（150cm、0.8、50%），在复杂回环场景中可能误合并。

20.3.8 与具身智能的关系

OVO 解决了具身智能中一个关键的基础设施问题：机器人如何在探索未知环境的同时，实时构建一个可被自然语言查询的语义地图？这个能力是所有上层任务（导航、操作、人机交互）的前提。OVO 与 SLAM 的集成意味着它可以直接嵌入现有的机器人导航栈中，不需要额外的离线处理管道。

20.4 Open-Vocabulary Mapping 的核心难点

20.4.1 2D 语义如何稳定提升到 3D

开放词汇映射的第一步是把 2D 图像中的语义信息提升到 3D。这个过程比看起来脆弱得多。深度传感器在物体边界处产生不准确的深度值，导致反投影后的 3D 点云在边缘处“渗透”到背景中。SAM 的分割 mask 在深度不连续处也容易溢出——一个桌子的 mask 可能包含桌面边缘后面的地板像素。这些噪声点将错误的 CLIP 特征带入 3D segment 的描述符，降低语义查询的准确性。OVO 通过限制 mask 的反投影深度范围来缓解这一问题，但无法根本解决 (arXiv)。更深层的问题是：2D foundation models (SAM、CLIP) 是为 2D 图像设计的，它们对 3D 几何结构没有显式理解。一个物体在不同视角下的 2D 外观差异巨大，CLIP 描述符也随之变化——如何从这些变化的描述中提炼出稳定的 3D 语义表示，仍是开放问题。后续工作如 Ov3R 尝试将 3D 几何信息直接融入 CLIP 描述符的提取过程，代表了一个有希望的探索方向。

20.4.2 多视角语义如何融合

同一物体从多个视角观测会得到多个 CLIP 描述符。如何融合它们？简单取平均是最常用的策略，但存在问题：当某些视角中物体被遮挡或光照条件恶劣时，其 CLIP 描述符质量低，平均操作会“污染”整体描述。VLMaps 在网格级别做平均，ConceptGraphs 在对象级别做增量平均，OVO 用神经网络学习自适应权重。但这三种策略都隐含一个假设：CLIP 嵌入空间是线性的，即好的描述符和坏的描述符可以线性插值。CLIP 的实际嵌入空间高度非线性，线性融合可能不是最优的。更根本的问题是：多视角融合的“最优”标准是什么？现有方法缺乏 3D 语义融合的理论框架，大多依赖经验策略。OVO 的 CLIP Merging 网络虽然用数据驱动的方式学习了融合权重，但其监督信号来自 2D 语义标注的文本描述符——这假设了单一视角的文本描述足以代表整个 3D 实例的语义，在现实中未必成立。

20.4.3 同一物体如何跨帧关联

跨帧对象关联 (data association) 是开放词汇建图中最容易出错的环节。ConceptGraphs 用几何重叠率 + CLIP 余弦相似度的贪心匹配，OVO 用投影匹配 + 相似度阈值。这些方法在场景中物体稀疏时工作良好，但在以下场景中失效：(1) 多个同类物体紧密排列（如书架上的多本书），CLIP 特征相似但几何位置不同，可能被错误合并或拆分；(2) 大物体从局部视角看像不同物体（如 L 形沙发的两个臂），可能被拆分为两个 segment；(3) 动态物体跨帧移动，关联系统将其误判为新物体。这些 failure case 的共性是：只依赖局部几何和语义相似度，缺乏全局一致性约束。匈牙利算法可以提供全局最优匹配，但 OVO 实验中报告它只带来“微小提升”且计算更慢——这说明问题不在于匹配算法的优化，而在于相似度度量本身不够可靠。未来的方向可能是引入 LLM 的常识推理来辅助关联决策，或者利用 3D 对象的拓扑结构信息（如“桌子”应该“支撑”“杯子”）作为额外的关联约束。

20.4.4 开放词汇如何避免幻觉

开放词汇是一把双刃剑。系统可以响应任意查询，但也意味着它可能响应"不存在的东西"。当用户查询"红色的灭火器"但场景中有时，VLMaps 仍会在地图上找到余弦相似度最高的位置——可能是一个红色的柜子或椅子。CLIP 的文本-图像对齐是"相对的"（找到最相似的），而非"绝对的"（判断是否存在）。

ConceptGraphs 通过对象 caption 提供额外的验证层（LLaVA 可能 caption 一个柜子为 "a wooden cabinet"，与 "fire extinguisher" 不符），但 LLaVA 自身也有幻觉问题。OVO 的 CLIP Merging 网络在训练时偏向训练集中的类别，对稀有类别可能产生过度自信的预测。这些问题没有统一解决方案：开放词汇系统需要在"泛化能力"和"准确性"之间做持续 trade-off。一种可能的路径是引入不确定性估计——当查询余弦相似度低于某个阈值时，系统应回复"未找到"而不是给出最相似的误匹配。

20.4.5 地图如何随环境变化更新

真实环境是动态的：人走进走出，家具被移动，灯光变化。现有的开放词汇建图系统几乎全部是静态假设——地图一旦建好就不再更新。ConceptGraphs 的对象级表示理论上更容易处理动态更新（删除一个节点比删除点云中的一组点更简单），但论文未实现动态更新。OVO 支持回环闭合时的 segment 合并，但不处理 segment 的分裂（如一个物体被移走了一半）。长期运行的开放词汇建图系统需要一套完整的地图维护机制：检测变化、决定是更新旧节点还是新建节点、处理新旧信息的冲突。这是从"建图"到"持续空间记忆"的关键跃迁，也是第21章讨论的主题。动态更新的核心挑战在于区分"同一对象的变化"和"新对象的出现"——这个判断需要时序推理和常识知识，远超现有系统的处理能力。

20.5 本章小结

开放词汇 3D 地图是语义 SLAM 的下一个前沿。VLMaps 证明了预训练 VLM 特征可以融入 3D 重建，使地图被自然语言索引；ConceptGraphs 证明对象级别的场景图比 per-point 表示更紧凑、更适合机器人规划；OVO 证明这种地图可以在线构建并与 SLAM 回环闭合协同。三者共同勾勒出一个方向：地图不再是仅供定位和路径规划的几何结构，而是机器人理解并与人类交流其空间环境的语义接口。但这个方向仍有五大核心难题待解：2D 到 3D 的语义提升不够稳定、多视角融合缺乏理论框架、跨帧关联在复杂场景中易错、开放词汇存在幻觉风险、动态环境更新机制缺失。这些难题的解决将决定开放词汇地图能否从实验室走向长期部署的机器人系统。

第21章 从语义地图到空间记忆：3D-Mem、GSMem 与长期具身智能

具身智能体在三维环境中连续运行数小时乃至数天，它需要的不是一张静态几何图，而是一部可增量、可检索、可推理的"空间记忆"。语义场景图（第20章）解决了"什么在哪里"的标注问题，却在两个维度上暴露短板：一是空间关系被压缩为离散文本标签，丢失了连续空间中的微妙布局信息；二是场景图一旦建完便固化为静态结构，代理无法基于记忆去主动探索未知区域。本章介绍两条从"建图"走向"记忆"的前沿路径——3D-Mem 与 GSGMem——它们分别用图像快照和可重渲染的三维高斯场回答同一个问题：地图如何成为代理的工作记忆。

21.1 地图和记忆的区别

21.1.1 地图回答"空间在哪里"，记忆回答"我见过什么"

传统 SLAM 地图的核心任务是几何自治：相机轨迹与世界坐标系中的路标点保持一致的相对位姿。无论是稀疏点云、稠密网格还是神经辐射场，地图的存在目的是回答"空间中的某点相对于我曾在何处"。这个定义足以支撑导航避障，却不足以支撑高层推理。

具身代理面对的问题远比"我在哪"复杂。接到指令"帮我找一个能放咖啡杯的小桌子"，代理需要知道的不只是桌子的三维坐标，还包括：桌面是否已有杂物、周围空间是否容得下人靠近、桌子表面材质是否适合放置热饮。这些判断依赖视觉上下文——物体的外观、周围环境的整体布局、光照与遮挡关系——而不仅仅是标注了类别标签的边界框。

21.1.2 四条关键差异

选择性。 几何地图追求完整性：每一厘米空间都要被覆盖。记忆则天然选择性的——人类不会记住房间的每一个像素，而是记住与任务相关、与已有知识形成关联的"事件"。好的空间记忆系统必须能主动丢弃冗余、保留有价值的信息。

时间跨度。 地图在单次遍历中构建，用完全覆盖作为收敛目标。记忆却要支撑终身运行：今天建立的客厅记忆需要在三个月后仍然可用，期间家具可能移动、新物品可能加入。这意味着记忆系统必须具备增量聚合能力，而非每次从零重建。

可检索性。 地图通过坐标查询：给定 (x, y, z) 返回占据状态或语义标签。记忆却要通过语义、任务、甚至模糊的描述来检索："上次看到的那把蓝色椅子在哪里？"这种从内容到位置的反向查询，要求记忆表示天然支持语义关联。

可行动性。 纯几何地图本身不告诉代理下一步该去哪里。记忆则需要直接参与决策：已知区域是否足以回答问题？如果不够，哪个未知区域最值得探索？这要求记忆表示同时具备"回顾"与"展望"的双重能力。

21.1.3 场景图已走到哪一步

第20章介绍的 VLMaps (arXiv)、ConceptGraphs (CVPR) 和 OVO 代表了语义地图向结构化知识迈进的最高水平。ConceptGraphs 将场景分解为物体节点，用 CLIP 特征编码开放词表语义，并通过大语言模型抽取物体间关系。然而，场景图的本质仍是离散抽象：它将丰富的三维空间压缩为一组节点和文本边，"在...前面"被简化为一个字符串，而失去了真实空间中距离、朝向、遮挡的连续信息。更关键的是，场景图没有天然机制表示"尚未探索的区域"——它是一张"已完成"的地图，而非一份"进行中"的记忆。

3D-Mem 与 GSMem 的出现，正是为了修补这两块短板。

21.2 3D-Mem：用快照记忆替代场景图

3D-Mem (CVPR 2025) 提出了一个核心主张：对于具身代理，一张精心选择的图像比一堆物体标签包含更丰富的空间信息。代理的记忆不应该是抽象后的场景图，而应该是保留原始视觉信息的"记忆快照" (Memory Snapshots) ——同时配以"前沿快照" (Frontier Snapshots) 来表征尚未探索的区域。这种表示天然兼容当前视觉语言模型 (VLM) 的图像理解能力，让代理可以直接"看图思考"。

21.2.1 核心洞察

现有表示方法在两个方面亏待了具身代理。第一，物体-centric 的场景图过度简化空间关系：它把"扶手椅前方的空地是否够放茶几"这种需要连续空间感知的问题，降级为一组三维边界框和文本关系标签。文本标签无法承载"前方有多少厘米"的精确含义，边界框也无法表达背景中的空间语境。第二，稠密三维表示（点云、神经场）虽然保留了几何细节，却缺乏可扩展性——随着场景增长，计算开销爆炸，且当前基础模型对三维模态的推理能力远弱于对图像的推理能力。

3D-Mem 的解法是用图像做记忆媒介。图像是 VLM 的原生输入模态；一张图片能同时编码前景物体、空间关系和背景语境；而且图像天然紧凑，不需要显式存储三维结构。

21.2.2 系统结构

3D-Mem 的记忆由两种快照构成：

Memory Snapshot（记忆快照） $S = \langle \mathcal{O}, I \rangle$ ：一张图像 I 及其所捕获的物体集合 $\mathcal{O}$ 。每个快照对应一个"共视聚类"——在一次观测中同时可见的一组物体及其周围场景。关键设计是：快照保留原始 RGB 图像，而非提取后的特征或标注，这让 VLM 可以直接读取空间布局信息。

Frontier Snapshot（前沿快照） $F = \langle r, p, I^{obs} \rangle$ ：表示一个未探索区域 r ，包含可导航位置 p 和从代理当前位置朝向该区域的观察图像 I^{obs} 。它将传统 frontier-based 探索中的几何前沿升级为视觉可感知的"预览"——代理可以直观地看到"那边有一条走廊"。

21.2.3 共视聚类算法

Memory Snapshot 的构造依赖"共视聚类"（Co-Visibility Clustering）。代理在探索过程中收集带位姿的 RGB-D 帧，并用开放词表检测器（如 Grounding DINO + SAM）提取物体实例。算法维护一个物体集合 $\mathcal{O}$ 和一个候选帧集合 $\mathcal{I}$ （所有 egocentric 观测）。

核心聚类逻辑如下：

算法：Co-Visibility Clustering

输入：物体集合 $\mathcal{O}$ ，候选帧集合 $\mathcal{I}$ ，评分函数 F

输出：记忆快照集合 S

初始化：将 $\mathcal{O}$ 中所有物体放入待处理聚类 $C = \{\mathcal{O}\}$

while C 不为空 do

 取出 C 中最大的聚类 $\mathcal{O}^*$

 在 $\mathcal{I}$ 中搜索所有包含 $\mathcal{O}^*$ 的候选帧

 if 存在候选帧 then

 选择评分最高（含物体最多）的帧 I^*

 创建快照 $S^* = \langle \mathcal{O}^*, I^* \rangle$ ，加入 S

 else

 将 $\mathcal{O}^*$ 按二维 (x, y) 位置 K-Means 分裂为 $\mathcal{O}1^*$, $\mathcal{O}2^*$

 将 $\mathcal{O}1^*$, $\mathcal{O}2^*$ 放回 C

 end if

 从 C 中移除 $\mathcal{O}^*$

end while

```
// 合并：若两个快照共享同一帧，将其物体集合合并
for 每对 (Sj, Sk) 使用相同帧 do
    合并为 S1 = <O_Sj U O_Sk, I_Sj>
end for
```

聚类的直观含义是：在空间上邻近且经常一起被看到的物体归入同一个快照。K-Means 分裂处理那些无法被单帧完整覆盖的大聚类（例如分散在两个房间的一组物体）。最终的合并步骤则避免同一帧被多个快照重复表示，保持记忆紧凑。

21.2.4 增量构建与前沿快照

3D-Mem 的设计目标之一是支撑终身探索。增量更新流程如下：

1. **物体更新**：每一步，代理从 egocentric 视图中检测新物体，更新全局物体集合。
2. **快照更新**：仅对涉及新检测到物体的已有快照以及新增物体执行共视聚类，而非从零重算全部记忆。形式上， $\mathcal{S}_t = (\mathcal{S}_{t-1} \setminus \mathcal{S}_{prev}) \cup \text{Cluster}(\mathcal{O}_{input}, \mathcal{I}_t)$ 。
3. **前沿更新**：维护占用栅格地图，每一步检测新的前沿、移除已探索的前沿，并为新增/修改的前沿拍摄快照图像。

Prefiltering（预过滤） 解决记忆增长后的检索问题。当记忆快照数量庞大时，直接将全部图像送入 VLM 会导致计算开销过高且引入无关噪声。Prefiltering 的机制是：将问题文本和所有物体类别名一并提交给 VLM，VLM 返回与问题最相关的 Top-K 类别；只保留包含这些类别的记忆快照。在 A-EQA 基准上，3D-Mem 平均从 39.76 帧原始观测中生成 10.94 个记忆快照，经 Prefiltering 后仅保留 3.26 个送入 VLM，压缩比超过 12:1。

21.2.5 推理与探索闭环

给定任务指令，VLM 代理执行以下循环：

1. 用 Prefiltering 检索相关记忆快照；
2. 将所有前沿快照与过滤后的记忆快照一并送入 VLM；
3. VLM 决定：基于当前记忆回答问题（终止探索），或选择一个前沿快照前往探索（继续）；
4. 若选择探索，VLM 同时给出选择该前沿的理由；
5. 代理移动到新位置，增量更新记忆和前沿快照，重复步骤 1。

所有快照均为原始图像，VLM 无需专门适配三维表示，可直接利用其强大的视觉理解能力。

21.2.6 论文精读：3D-Mem

论文：Yang 等, "3D-Mem: 3D Scene Memory for Embodied Exploration and Reasoning", CVPR 2025 (CVF开放获取)

1. 试图解决什么痛点？ 物体-centric 场景图过度简化空间关系，将连续空间压缩为离散文本边；稠密三维表示计算开销大且当前基础模型对三维模态推理能力弱；现有表示缺乏主动探索机制和终身记忆管理能力。

2. 核心假设是什么？ 一张精心选择的图像本身就足以捕获一个区域的丰富空间信息；VLM 的图像理解能力足够从这些快照中提取复杂的空间关系；用图像作为记忆媒介比抽象后的场景图更适合代理推理。

3. 输入输出是什么？ 输入：带位姿的 RGB-D 视频流、自然语言指令（如问答问题或导航目标）；输出：代理的回答文本或导航轨迹，以及增量构建的 3D-Mem（记忆快照集 + 前沿快照集）。

4. 地图表示是什么？ 双模态快照系统：Memory Snapshots（已探索区域的图像+物体集）和 Frontier Snapshots（未探索区域的图像+位置+区域描述）。不是显式的三维地图，而是以图像为中心的内存结构。

5. Tracking 怎么做？ 依赖外部里程计/SLAM 提供相机位姿，自身不执行跟踪。物体实例通过 2D 检测+深度反投影+多帧关联进行追踪和三维定位。

6. Mapping 怎么做？ 通过共视聚类将物体组织为记忆快照；通过前沿检测维护前沿快照；两者均增量更新，支持终身运行。Prefiltering 机制确保记忆规模可控。

7. 优化目标是什么？ 在探索效率和推理准确度之间取得平衡。共视聚类以"单帧最大化覆盖相关物体"为贪心准则；Prefiltering 以"保留与任务相关的最少记忆子集"为目标。

8. 相比前作真正推进了什么？ 相比 ConceptGraphs 等场景图方法，3D-Mem 在空间理解类问题上提升显著（A-EQA LLM-Match 52.6 vs. ConceptGraphs w/ Frontier 47.2），原因是快照保留了连续视觉上下文而非离散文本关系。相比 Explore-EQA 等多帧 VLM 方法，3D-Mem 通过共视聚类实现超过 3 倍的压缩比，同时保留更高的信息量。它首次将"前沿"的视觉预览直接纳入 VLM 的决策输入。

9. 没有解决什么？ 记忆快照仍是"看过什么就记住什么"——如果初始观测因遮挡或角度不佳错过了关键信息，记忆无法事后补偿。它依赖外部位姿，自身不具备 SLAM 能力。开放词表检测器的漏检和误检会直接影响记忆质量。此外，Prefiltering 依赖超参数 K，不同任务可能需要不同的过滤强度。

10. 对后续研究的影响是什么？ 3D-Mem 开创了"图像即记忆"的范式，直接启发了 GSMem 等后续工作将快照升级为目标视角可任意选择的重渲染表示。它证明 VLM 可以直接在图像记忆上完成复杂的空间推理，为空间记忆与基础模型的深度融合指明了方向。

实验结果： 在 A-EQA（184 题，HM3D 场景）上，3D-Mem 达到 LLM-Match 52.6、LLM-Match SPL 42.0，显著超越 Explore-EQA (46.9/23.4) 和 ConceptGraphs w/ Frontier (47.2/33.3)。在 GOAT-Bench 终身导航基准上，3D-Mem 作为长期记忆系统同样表现优异。

21.3 GSMem：可重渲染的高斯空间记忆

3D-Mem 用图像快照替代场景图，却 inherit 了一个根本限制：快照的视角固定。代理从某个角度拍了一张照片，记忆中的信息就永远被锁定在那个视角。如果扶手椅背对镜头，快照中看不到椅子后方的插座——这个问题永远存在于记忆中，因为代理无法"重新观察"已经访问过的区域。

GSMem (arXiv) 的核心命题是：3D Gaussian Splatting (3DGS) 天然适合做持久空间记忆，因为它允许代理事后从任意最优视角重新渲染已探索区域——作者称之为"空间回忆" (Spatial Recollection)。

21.3.1 核心洞察：事后可重观察性

GSMem 指出两类主流表示共同缺失一个关键能力：**post-hoc re-observability**（事后可重观察性）。

场景图一旦建成，信息固化在节点和边中——如果检测器当初漏掉了某个物体，这个遗漏是永久性的。3D-Mem 的快照同理——如果那张照片角度不好，信息损失无法挽回。人类却可以在记忆中"换个角度再看"：回想一个房间时，你能在脑海中从不同位置重新观察它，发现第一次看时漏掉的细节。

3DGS 恰好提供这种能力。它用三维高斯显式参数化连续几何和稠密外观，天然支持实时新视角合成。代理不需要真的走向那个位置，只需在记忆中"想象"自己站在一个更好的角度，就能渲染出逼真的新视图。

21.3.2 系统结构

GSMem 由三个相互衔接的模块组成：

在线 3DGS 建图与语言场。 不同于传统 3DGS 需要离线训练，GSMem 使用滑动窗口优化实时更新高斯场。关键创新是将 2D CLIP 特征"提升"到三维高斯上，通过权重一致的反向聚合构建可搜索的语义场——代理可以用自然语言描述查询三维空间中的语义相关性，而不依赖迭代优化。

多级检索-渲染 (Multi-level Retrieval-Rendering)。 这是 GSMem 的核心机制，同时利用两条互补线索定位目标区域：

- **物体级检索：** VLM 根据问题对所有场景图中的物体排序，选出 $\text{Top-}K_{obj}$ 候选物。每个候选物定义一个物体级兴趣区域 (ROI) 。
- **语义级检索：** VLM 将问题中的目标描述编码为 CLIP embedding，查询三维语言场，检索余弦相似度超过阈值 τ_{clip} 的高斯。这些高斯通过 KD-Tree 邻域图聚类为空间连贯的区域，保留 $\text{Top-}K_{cluster}$ 个语义级 ROI。

两条线索互补至关重要：即使场景图的物体级检测漏掉了目标（例如目标太小或太偏），语义级语言场仍能定位相关区域。这种冗余设计显著提升了检索鲁棒性。

最优视角选择与渲染。 对每个 ROI，GSMem 采用"采样-评分"策略选择最优观察视角：

1. 围绕目标密集采样候选相机位姿（实践中约 108 个）；
2. 两阶段评分：**可见性检查**（TSDF 验证目标不被遮挡）、**投影面积**（目标在视场中占足够像素）、**渲染不透明度**（确保渲染稳定）；
3. 最优视角通过 3DGS 实时渲染，生成高保真图像送入 VLM 进行推理。

21.3.3 混合探索策略

GSMem 的探索策略平衡语义相关性与几何覆盖：

- **VLM 语义评分：** 前沿区域的语义相关性由 VLM 根据问题评估——"这个方向是否可能通向目标？"
- **3DGS 覆盖率目标：** 通过高斯场的 Fisher Information Matrix (FIM) 迹量化表示不确定性，识别观测不足但信息增益最大的区域。

两者结合确保探索既任务导向又几何完备——不会为了快速回答问题而忽略需要补全的视觉盲区。

21.3.4 论文精读：GSMem

论文： Lu 等, "GSMem: 3D Gaussian Splatting as Persistent Spatial Memory for Zero-Shot Embodied Exploration and Reasoning", arXiv 2026 (arXiv)

1. 试图解决什么痛点？ 现有场景表示（离散场景图、静态快照）缺乏事后可重观察性。初始观测因视角、遮挡或光照不佳导致的信息遗漏往往不可恢复。代理被"锁定"在它当初恰好看到的视角中。

2. 核心假设是什么？ 3DGS 的连续参数化几何和实时新视角合成能力使其天然适合做持久空间记忆；代理可以从"从未真正站过"的最优视角渲染记忆区域；物体级场景图和语义级语言场的组合检索能提供冗余且鲁棒的定位能力。

3. 输入输出是什么？ 输入：带位姿的 RGB-D 视频流、自然语言问题或导航指令；输出：代理回答文本或导航轨迹。中间产物是持续更新的 3DGS 高斯场 + 并行维护的物体级场景图 + 语义级 CLIP 语言场。

4. 地图表示是什么？ 三层耦合表示：（1）持久 3DGS 高斯场——连续几何+外观+语义；（2）物体级场景图——由检测器生成的结构化物体节点和关系；（3）语义级语言场——CLIP embedding 与每个高斯关联，支持开放词表语义查询。

5. Tracking 怎么做？ 依赖外部里程计提供位姿，自身不执行跟踪。物体通过 2D 检测+深度反投影进行实例关联。

6. Mapping 怎么做？ 在线滑动窗口优化更新 3DGS 高斯参数；反向聚合将 2D CLIP 特征提升到 3D 高斯构建语言场；同时维护基于检测的物体级场景图。语言场更新为实时优化无关操作，不需要迭代训练。

7. 优化目标是什么？ 覆盖率和语义丰富度的双重目标。探索阶段最大化 FIM 迹（信息增益）+ VLM 语义相关性的加权和；检索阶段最大化 VLM 推理准确度（通过最优视角选择）。

8. 相比前作真正推进了什么？ 相比 3D-Mem，GSMem 的核心突破是 Spatial Recollection——代理可以事后从最优视角重新观察已访问区域，而非被锁定在原始观测视角。这直接解决了"初始观测遗漏导致永久信息丢失"的问题。在 A-EQA 上，GSMem 达到 LLM-Match 55.4、LLM-Match SPL 43.8，超越 3D-Mem (52.6/42.0) 约 2.8/1.8 个百分点。语言场的实时更新（无需迭代优化）也是一个关键工程进步。

9. 没有解决什么？ 3DGS 的内存开销仍显著高于图像快照——大规模场景下高斯数量膨胀可能限制可扩展性。渲染质量依赖位姿精度，外部里程计漂移会直接扭曲记忆。FIM 计算和密集视角采样带来额外计算负担。场景图的物体检测错误仍然可能污染物体级检索，尽管语义级检索提供了部分冗余。对动态场景的处理能力有限——高斯场假设静态环境。

10. 对后续研究的影响是什么？ GSGMem 确立了"可重渲染记忆"作为空间记忆的新标准。它证明 3DGS 不仅是视觉重建工具，更是具身代理的认知基础设施。多级检索-渲染的架构（结构化+连续表示互补）为后续工作提供了设计范式。将 Fisher Information 引入探索策略也为不确定性驱动的主动感知开辟了新方向。

实验结果： A-EQA 上 GSGMem 达到 55.4/43.8 (LLM-Match/LLM-Match SPL)，在所有基线中最优。在 GOAT-Bench 终身导航任务上同样表现突出，验证了持久高斯记忆在长时段运行中的优势。

21.4 技术演进：从几何地图到可重渲染记忆

3D-Mem 和 GSGMem 的出现不是孤立事件，而是空间表示长达十年演进链条上的最新环节。理解这条主线，有助于预判下一代系统的形态。

阶段	代表方法	核心表示	查询方式	关键能力	根本局限
几何地图	ORB-SLAM, VINS	稀疏点云/稠密网格	坐标查询占据状态	精确定位与重建	无语义，无物体级理解
语义地图	SemanticFusion, VLMaps	稠密语义体素/CLIP 特征场	开放词表语义查询	语义+几何融合	静态表示，无结构化推理
场景图	ConceptGraphs, HOV-SG	物体节点+关系边+CLIP 特征	物体/关系/属性查询	结构化知识表示	空间关系离散化，信息固化
记忆快照	3D-Mem	共视图像+物体集	VLM 直接读图推理	视觉上下文保留，增量构建	视角锁定，信息遗漏不可逆
可重渲染记忆	GSMem	3DGS 高斯场+场景图+语言场	语义+物体双重检索+新视角渲染	事后最优观察，空间回忆	计算开销大，动态环境弱

上表展示了五个阶段的递进关系。每一步演进都在补前一步的短板，却也引入新的约束。几何地图解决了"我在哪"但完全不懂"这是什么"；语义地图给每个体素贴上语义标签，却仍是稠密静态的像素级分类；场景图上升到物体级抽象，却将连续空间降级为离散文本关系；记忆快照恢复了视觉连续性，却将代理锁定在固定视角；可重渲染记忆最终解放了视角限制，却付出计算和存储代价。

这条演进线揭示了一个深层规律：**空间表示正在从"给人看的地图"转变为"给模型用的工作记忆"**。几何地图为人类操作者绘制可视化地图；语义地图为下游任务提供像素级先验；场景图为推理引擎提供结构化知识库；而记忆快照和可重渲染记忆则直接为大模型（VLM/LLM）提供可消费的视觉输入。表示形式的选择不再由人类的可视化需求决定，而是由消费它的基础模型的模态偏好和能力边界决定。

这个趋势也意味着 SLAM 系统的边界正在被重新定义。传统 SLAM 的输出是一张"地图"——某种人类可理解的空间可视化。新一代空间智能系统的输出则是一段"记忆"——一种为推理引擎优化的信息结构，它可以被检索、被查询、被重渲染、甚至被遗忘。建图不再是终点，而是更大认知循环中的一个环节。

21.5 关键判断：空间工作记忆的未来

3D-Mem 和 GSMem 的实验结果（均基于 GPT-4o 等商用 VLM）已经表明：当空间表示被设计为与基础模型的输入模态对齐时，代理的推理和探索能力获得质的飞跃。这指向一个更根本的判断。

21.5.1 地图将成为共享空间工作记忆

未来具身智能系统中的"地图"，不会只是一个独立的几何后端模块。它会成为 VLM、LLM、策略模型等多个认知组件**共享的空间工作记忆**。这个记忆需要同时满足：

- **对 VLM 友好**：以图像或任意视角可渲染的连续场的形式存在，让视觉模型直接消费；

- 对 LLM 友好：包含结构化的物体级语义和关系，支持符号推理；
- 对策略模型友好：编码几何不确定性和探索价值，支持动作决策；
- 终身可扩展：增量更新，不因场景规模增长而失效。

3D-Mem 满足了第一条和第二条的一半（通过物体标签）；GSMem 推进到第一条的"任意视角"版本，并保留了第二条的能力。两者都通过增量机制支撑第四条。第三条——不确定性驱动的主动探索——在 GSMem 中通过 FIM 初步实现，但仍有大量空间待挖掘。

21.5.2 从"建图"到"选择性记忆"

3D-Mem 的核心教诲是"建图"不等于"记忆"。建图追求全面覆盖，记忆却天然选择性。代理不需要记住每个像素，只需要记住回答当前问题所需的信息。Prefiltering 机制是这个理念的直接体现——在将记忆送入推理引擎之前，先根据任务相关性做一次减法。

GSMem 则进一步揭示：好的记忆不仅要会选择性保留，还要能**事后重新生成**。重渲染能力让代理可以"再想想"——不是被动地消费曾经看到的東西，而是主动地从记忆中生成最优视角的观察来服务当前推理。这已从"存储"走向了"生成"。

21.5.3 尚未解决的问题

这条路径上仍有硬骨头。动态环境的记忆管理：3DGS 假设静态场景，如何处理移动的人、被移动的家具？多代理共享记忆：多个代理的探索记忆如何融合为一致的全局表示？记忆的"遗忘"机制：长期运行中记忆无限增长，如何评估每条记忆的"保质期"并主动丢弃？记忆的跨场景迁移：一个客厅中学到的空间先验，如何在新客厅中加速理解？这些问题定义了空间记忆研究的下一个五年。

本章一句话总结：3D-Mem 证明图像快照是比场景图更高效的记忆媒介，GSMem 进一步证明可重渲染的 3D 高斯场是比固定快照更强大的记忆形态——两者共同推动空间表示从"静态地图"走向"可检索、可重渲染、可参与推理的空间工作记忆"。

第22章 具身导航中的 SLAM：VLFM、MapNav、WMNav 与 NavFoM

22.1 具身导航为什么重新需要地图？

端到端导航在短程任务上表现亮眼。给一个目标图像或一句简短指令，训练好的策略网络可以直接输出动作，在几十步之内抵达目的地。但任务一旦变长、环境一旦变复杂，纯端到端方案暴露出结构性短板。

探索记忆的衰减。 端到端模型依赖循环网络或 Transformer 维护隐式记忆。当轨迹超过数百步，隐状态逐渐"洗掉"早期观测的信息。agent 在房间里转了三圈后，已经忘了哪个走廊看过、哪个房间没看过，陷入重复探索的循环。

目标定位需要空间先验。 "找一台电视"这类指令要求 agent 理解客厅比浴室更可能有电视、电视通常对着沙发。纯像素到动作的映射缺乏这样的常识推理能力，需要某种可读取、可推理的环境表示。

回溯与多步规划。长程导航经常需要走到某个已知的分叉点再做决策。没有显式地图，agent 无法规划"回到上次左转的位置"这类涉及历史空间信息的动作序列。

安全避障。端到端策略对训练分布外的障碍物反应迟钝。显式 occupancy map 可以提供几何保障，确保规划路径不会穿越已知障碍。

这些需求指向同一个方向：导航需要**显式空间记忆**。而显式空间记忆正是 SLAM 和建图系统提供的核心能力。从本章开始，我们不再把 SLAM 当成一个独立的定位建图模块，而是把它看作具身导航系统的**记忆基础设施**——为决策提供结构化空间信息的基础设施。

四篇论文按演进顺序呈现。VLFM 将经典 frontier exploration 与视觉语言模型嫁接，用地图存储语义价值。MapNav 进一步说明压缩好的语义地图比堆历史帧更高效。WMNav 把地图纳入世界模型框架，让导航从"看一步走一步"转向"想象行动后果"。NavFoM 则将导航推向跨任务、跨形态的基础模型阶段，用海量数据证明地图表示与端到端学习可以共存于统一框架中。

22.2 VLFM：视觉语言前沿地图

22.2.1 本章要解决什么问题

Object Goal Navigation (ObjectNav) 要求 agent 在陌生环境中找到指定类别的物体。传统方法用强化学习训练导航策略，但只能处理训练时见过的物体类别，且通常只在模拟器中有效。VLFM 要回答的问题是：**能否用预训练的视觉语言模型 (VLM)，在零样本条件下实现对新物体类别的语义导航？**

22.2.2 传统方法与瓶颈

Frontier-based exploration 是经典解法：agent 维护一个 occupancy map，识别已探索区域与未知区域的边界 (frontier)，选择一个 frontier 点作为短期目标，驱动 agent 持续探索。早期方法用距离最近或信息增益最大作为 frontier 选择标准，探索策略纯粹是几何驱动的，完全不利用语义信息。

后续方法尝试引入语义：CoW 用 CLIP 特征检测目标物体后直奔目标；ESC 和 L3MVN 用 LLM 处理物体检测文本结果来推理哪个 frontier 最可能有目标物体。这些方法有两个共同瓶颈：

1. **检测-文本转换瓶颈：**视觉信息必须先经物体检测器转为文本，才能输入 LLM 推理，信息在转换中大量丢失。
2. **LLM 计算开销：**每次 frontier 选择都要调用 LLM，需要远程服务器支持，实时性差。

22.2.3 VLFM 的核心洞察

VLFM (Vision-Language Frontier Maps) 的核心洞察是：**直接用 VLM 从 RGB 观测和文本提示中计算语义价值分数，投影到二维网格上形成 language-grounded value map，再用这个 value map 指导 frontier 选择。**省去检测→文本→LLM 的冗长管道，一步完成视觉→语义→空间的映射。

22.2.4 系统结构

VLFM 架构由三个模块组成：

Occupancy Map & Frontier Detection。利用深度图和里程计构建二维俯视 occupancy map，标记障碍物和已探索区域。通过检测 explored 与 unexplored 的边界提取 frontier waypoints。

Value Map Generation。这是 VLFM 的核心。每帧 RGB 图像与文本提示 ("Seems like there is a <target_object> ahead.") 一起输入 BLIP-2，计算余弦相似度作为语义价值分数。这些分数按相机 FOV 的形状投影到俯视网格上，形成 value map。Value map 有两个通道：语义价值分数和置信度分数。置信度按像素与光轴的夹角计算 ($\cos^2(\theta / (\theta_{fov} / 2) * \pi / 2)$)，光轴中心置信度最高，FOV 边缘最低。当同一区域被多次观测时，新旧值按置信度加权平均。

PointNav Policy & Goal Navigation。Frontier 选择和目标检测完成后，用 VER (Variable Experience Rollout) 训练的 PointNav 策略执行底层导航。PointNav 以深度图和相对目标坐标为输入，输出动作，不依赖语义理解。检测到目标物体后切换到 goal navigation 模式，直接驶向目标。

初始化：旋转一周初始化 occupancy map 和 value map

循环：

1. 深度图 + 里程计 → 更新 occupancy map
2. 提取 frontier waypoints
3. RGB + 文本提示 → BLIP-2 → 语义分数
4. 投影到 value map (按置信度加权融合)
5. 用 value map 评估各 frontier，选价值最高的
6. PointNav 策略驶向选中 frontier
7. 若检测到目标物体 → goal navigation

直到：到达目标或无可达 frontier

22.2.5 代表论文精读

论文：VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation, arXiv 2023 (arXiv)

1. 试图解决什么痛点？

ObjectNav 的零样本泛化问题：训练时未见过的物体类别，如何在陌生环境中高效定位？同时摆脱对远程 LLM 的依赖，让语义导航能在消费级笔记本甚至真实机器人上实时运行。

2. 核心假设是什么？

BLIP-2 等预训练 VLM 已经编码了丰富的场景语义知识和空间先验（如"客厅可能有电视"），可以直接用于评估不同区域对寻找目标物体的价值，无需任务特定训练。

3. 输入输出是什么？

输入：RGB 图像、深度图、里程计、目标物体类别名称（文本）。输出：导航动作序列，最终抵达目标物体附近。

4. 地图表示是什么？

双通道俯视 grid map：(1) occupancy + frontier map，记录障碍物和已探索/未探索边界；(2) value map，含语义价值通道和置信度通道。Map 尺寸固定，与轨迹长度无关。

5. Tracking 怎么做？

初始化阶段旋转一周建图，后续用深度+里程计持续更新。无需显式位姿优化，直接用里程计做坐标变换。

6. Mapping 怎么做？

Occupancy map 用深度点云投影到二维网格。Value map 用 BLIP-2 计算的语义分数按 FOV 形状投影，与历史值按置信度加权融合。

7. 优化目标是什么？

无训练损失。所有模块（BLIP-2、PointNav 策略、目标检测器）均为预训练模型，零样本推理。Frontier 选择直接取 value map 在 frontier 位置上的聚合分数最大值。

8. 相比前作真正推进了什么？

首次将 VLM 的语义推理能力直接融入 frontier exploration 的空间决策循环，跳过检测→文本→LLM 的转换链。在 Gibson、HM3D、MP3D 三个数据集上达到 zero-shot SOTA。成功在 Boston Dynamics Spot 真实机器人上部署，证明零样本语义导航在实际场景中的可行性。

9. 没有解决什么？

Value map 只包含任务特定的语义信息（针对单一目标物体），无法跨任务复用。假设目标物体在默认摄像头高度可见，不考虑抽屉内部、高处货架等遮挡场景。Value map 在长期任务中可能积累错误，没有回环修正机制。

10. 对后续研究的影响是什么？

VLFM 开创了"VLM + 显式地图"的 zero-shot 导航范式，启发了后续 ESC、L3MVN、InstructNav 等一系列工作。其 frontier + value map 的双层结构成为语义探索的标准模板。

22.2.6 优点、失败模式与适用场景

VLFM 的优势在于模块化：VLM、检测器、PointNav 策略均可独立升级。真实世界部署门槛低，不需要 GPU 集群。失败模式包括：VLM 对语义相似物体的混淆（如"沙发"与"扶手椅"）、value map 的投影误差在狭长走廊中累积、纯里程计漂移导致的地图变形。适用于：室内零样本物体搜索、办公室/家庭环境机器人巡检、快速原型验证。

22.2.7 与具身智能的关系

VLFM 是 VLM 嵌入具身导航决策循环的早期代表。它证明了预训练模型的语义知识可以通过空间化投影（投影到 value map）转化为机器人可执行的策略，而不需要端到端训练。这为后续"空间记忆 + 基础模型"的路线奠定了实验基础。

22.3 MapNav：标注语义地图

22.3.1 从 VLFM 到 MapNav 的演进

VLFM 证明了地图对语义导航的价值，但它的 value map 只服务于单一目标。Vision-and-Language Navigation (VLN) 任务更加复杂：agent 需要理解自然语言指令（"穿过走廊，在植物处右转，停在左边第三扇门"），维护整个轨迹的历史上下文，并持续做出决策。传统 VLN 方法把历史 RGB 帧堆叠输入 VLM，帧数一多就碰到 token 长度限制和计算瓶颈。

MapNav 的核心问题是：对 VLN 这类长程语言引导导航，能否用一种压缩的、结构化的地图表示来替代原始历史帧，既保留空间信息又大幅降低内存和计算开销？

22.3.2 MapNav 的核心洞察

MapNav 的核心洞察是：**对导航来说，压缩好的语义地图往往比堆历史帧更有效**。具体地说，一个俯视语义地图（带有文本标注）可以在固定内存中编码障碍物、已探索区域、agent 轨迹和物体位置，而这些信息用历史帧表达需要随时间线性增长的存储空间。

22.3.3 系统结构

MapNav 由两大模块组成：

Annotated Semantic Map (ASM) 生成。 输入 RGB-D 帧和 agent 位姿，通过以下流程生成 ASM：

1. 语义分割提取物体掩码，对齐深度图生成点云。
2. 点云投影到俯视平面，构建多通道语义地图 $M \in \mathbb{R}^C(C \times W \times H)$ 。
3. 基础通道（1-4）编码：物理障碍物、已探索区域、agent 当前位置、历史轨迹。
4. 语义通道（5 起）存储各物体类别的空间分布。
5. 连通域分析识别每个语义区域，计算质心，在质心位置添加文本标注（"chair"、"plant"、"bed"）。

ASM 的核心创新在于**显式文本标注**。纯语义通道用数字 ID 表示类别，VLM 需要额外查询才能理解。ASM 直接在图上写文字，让 VLM 用其预训练的语言-视觉知识直接读取空间语义。

VLM-based 导航策略。 ASM 与当前 RGB 观测、语言指令一起输入端到端 VLM：

1. 双编码器分别处理 RGB 观测和 ASM。
2. 多模态 projector 对齐两种表示到共享嵌入空间。
3. 指令 token 与对齐特征拼接，输入 VLM。
4. VLM 输出文本格式的导航动作。

每 timestep：

RGB-D + 位姿 → 点云 → 语义分割 → 俯视投影 → 多通道语义地图
连通域分析 + 质心计算 → 文本标注 → ASM
ASM + 当前 RGB + 指令 → 双编码器 → VLM → 导航动作

22.3.4 代表论文精读

论文：MapNav: A Novel Memory Representation via Annotated Semantic Maps for Vision-and-Language Navigation, arXiv 2025 (arXiv)

1. 试图解决什么痛点？

VLN 中历史帧存储和计算的线性增长问题。传统方法随轨迹增长需要越来越多的内存和 token，而 ASM 保持**固定内存占用（0.17MB）**，与轨迹长度无关。

2. 核心假设是什么？

俯视语义地图 + 显式文本标注包含 VLN 所需的全部关键空间信息（障碍物、轨迹、物体位置），足以替代历史帧作为 VLM 的记忆表示。

3. 输入输出是什么？

输入：当前 RGB 图像、深度图、agent 位姿、语言指令。输出：导航动作（自然语言格式，可解析为底层控制命令）。

4. 地图表示是什么？

Annotated Semantic Map (ASM)，多通道俯视张量，含障碍物、已探索区域、agent 位置、历史轨迹、语义物体分布，以及**显式文本标注**。文本标注放在各语义区域的质心位置。

5. Tracking 怎么做？

通过深度点云投影和位姿更新维护 agent 在地图中的当前位置和历史轨迹。每个 episode 从 agent 位于地图中心的位置初始化。

6. Mapping 怎么做？

RGB-D → 点云 → 语义分割对齐 → 俯视投影 → 多通道语义地图。然后连通域分析识别语义区域，计算质心，叠加文本标注生成 ASM。

7. 优化目标是什么？

端到端训练，用 VLM 直接生成动作。ASM 的构建过程是可微的投影和标注管线，不需要额外的建图损失。

8. 相比前作真正推进了什么？

首次提出用显式文本标注的俯视语义地图替代历史帧作为 VLN 的记忆表示。实验证明 VLM 对 ASM 的理解优于传统语义地图和纯俯视地图。内存占用固定 0.17MB，不随轨迹增长。在模拟和真实环境中达到 SOTA。

9. 没有解决什么？

语义分割在遮挡和光照变化下会出错，导致地图标注不准。依赖 VLM 的端到端推理能力，对 VLM 本身的幻觉问题敏感。当前只处理静态环境，未考虑动态物体。

10. 对后续研究的影响是什么？

MapNav 证明了"压缩地图 > 原始帧"的导航记忆范式，启发了后续研究用更紧凑的表示替代长序列历史观测。其 ASM 思想可推广到任何需要长程空间记忆的 VLM-based 机器人任务。

22.3.5 实验对比分析

MapNav 在论文中对比了三种地图表示输入 GPT-4o 的理解效果：纯俯视地图、传统语义地图、ASM。结果显示 VLM 对 ASM 的理解显著优于前两者。这说明**地图表示的形式决定了 VLM 能从中提取多少有效信息**——不是有地图就行，而是地图要以 VLM"看得懂"的方式组织。文本标注本质上是**把空间信息翻译成 VLM 的母语（自然语言）**，降低了跨模态理解门槛。

22.3.6 优点、失败模式与适用场景

MapNav 的核心优势是固定内存占用和结构化空间表示，特别适合长程 VLN 任务。失败模式包括：语义分割错误导致地图标注失真、复杂多层环境中的高度信息丢失（纯二维俯视）、VLM 对密集标注的阅读理解瓶颈。适用于：室内 VLN、长程导航指令跟随、需要回溯的复杂路径规划。

22.3.7 与具身智能的关系

MapNav 代表了从"像素记忆"到"空间记忆"的范式转换。它证明 SLAM/建图系统产出的结构化空间表示，不仅服务于几何导航，更可以成为多模态大模型的有效输入。这为 SLAM 与 VLA (Vision-Language-Action) 模型的深度融合指明了方向。

22.4 WMNav：世界模型导航

22.4.1 从 MapNav 到 WMNav 的演进

MapNav 用压缩地图替代了历史帧，但决策方式仍是"观察→推理→行动"的被动反应式流程。每一决策只看当前状态和地图，没有主动"想象"行动后果的能力。

人类导航时会做前瞻性推理："如果我走左边，可能进入一个客厅，客厅可能有沙发；走右边是走廊，走廊通向卧室，卧室不太可能有沙发。"这种对行动后果的预测是世界模型（World Model）的核心思想。

WMNav 的问题是：能否把 VLM 嵌入世界模型框架，让它预测行动后果并维护一个 curiosity-driven 的记忆地图，从而实现更高效的主动探索？

22.4.2 WMNav 的核心洞察

WMNav 的核心洞察是：导航开始从"看一步走一步"转向"想象行动后果"。VLM 不仅用于理解当前场景，更用于预测每个方向找到目标物体的可能性，把这些预测量化存储在 Curiosity Value Map 中，形成世界模型的记忆基础。

22.4.3 系统结构

WMNav 框架由三个 VLM 模块和一个核心记忆结构组成：

PredictVLM（世界模型的预测器）。输入全景 RGB 图像，输出每个方向（30°、90°、150°、210°、270°、330° 六个视角）的 curiosity 分数（0-10），表示该方向找到目标物体的可能性。预测基于 VLM 对场景布局的常识推理（如"走廊尽头通常有房间"）。

Curiosity Value Map（CVM）。世界模型的记忆核心。一个二维俯视网格，每个像素值 $\in [0, 10]$ 表示该位置找到目标的预测可能性。已访问且确认没有目标的区域设为 0，可直接看到目标的区域设为 10，未探索区域初始为 10（完全未知）。每步更新时，将 PredictVLM 的方向分数投影到俯视网格 M_t^{nav} ，与前一步 CVM 逐像素取最小值融合： $M_t^{cv}(u,v) = \min(M_{t-1}^{cv}(u,v), M_t^{nav}(u,v))$ 。取最小值确保一旦某区域被判断为"不太可能有目标"，后续不会高估。

PlanVLM（子任务规划器）。根据当前 CVM 得分最高的方向图像，确定新的子任务（如"探索客厅"）和目标标志，存入记忆作为 cost。

ReasonVLM（动作推理器）。在选定方向上，用两阶段动作提议策略生成最终动作。先 broad exploration（大范围探索），再 precise localization（精确定位）。

每 timestep:

1. 捕获6个方向RGB-D → 拼接全景图 I_t^{pan}
2. $PredictVLM(I_t^{pan})$ → 各方向 curiosity 分数 $Score_t$
3. $Score_t$ + 深度/位姿 → 投影到俯视网格 M_t^{nav}
4. CVM 更新: $M_t^{cv} = \min(M_{t-1}^{cv}, M_t^{nav})$
5. CVM 分数投影回全景图 → 选最高得分方向
6. $PlanVLM$ → 子任务 + 目标标志 → 记忆 $cost\ c_t$
7. 两阶段动作提议 → $ReasonVLM$ → 极坐标动作 a_t
8. 执行动作

22.4.4 代表论文精读

论文: WMNav: Integrating Vision-Language Models into World Models for Object Goal Navigation, arXiv 2025 (arXiv)

1. 试图解决什么痛点?

现有 zero-shot ObjectNav 方法缺乏对世界状态的预测能力, agent 每一步只基于当前观测做决策, 导致往返冗余移动。同时, 纯文本记忆容易诱导 VLM 幻觉, 需要更可靠的量化记忆机制。

2. 核心假设是什么?

VLM 蕴含关于室内布局和物体空间关系的丰富常识知识, 可以用来定量预测未观测区域的状态。把这些预测存储在 CVM 中, 可以指导后续探索策略, 减少盲目移动。

3. 输入输出是什么?

输入: 全景 RGB-D 图像 (6 个方向)、agent 位姿、目标物体描述。输出: 连续动作 (极坐标向量), 驱动 agent 向预测的高价值区域移动。

4. 地图表示是什么?

Curiosity Value Map (CVM), 二维俯视标量图, 像素值 $\in [0, 10]$ 表示找到目标的预测可能性。同时维护 cost memory 存储子任务历史和目标标志。

5. Tracking 怎么做?

通过全景 RGB-D 和位姿信息, 将 egocentric 的 curiosity 分数投影到全局俯视坐标系。无显式回环检测, 依赖位姿精度。

6. Mapping 怎么做?

CVM 是在线维护的。初始值全 10, 每步将 $PredictVLM$ 输出的方向分数投影到俯视网格, 与历史 CVM 逐像素取最小值更新。已确认无目标的区域置 0。

7. 优化目标是什么?

无训练损失, 所有 VLM 模块为零样本推理。决策流程分阶段: $PredictVLM$ 预测 → CVM 更新 → 方向选择 → $PlanVLM$ 子任务分解 → $ReasonVLM$ 动作生成。

8. 相比前作真正推进了什么?

首次将 VLM 嵌入完整的世界模型范式 (预测+记忆+策略), 用 CVM 量化存储预测状态。提出两阶段动作提议策略 (广泛探索→精确定位) 和子任务分解反馈机制, 在 HM3D 上 zero-shot SR 提升 3.2%、SPL 提升 3.2%, 在 MP3D 上 SR 提升 13.5%。消融实验表明 CVM 显著优于纯文本-图像记忆和无记忆策略。

9. 没有解决什么？

CVM 的分辨率和精度受位姿漂移影响，长期运行会累积误差。PredictVLM 的预测依赖 VLM 的常识先验，对训练分布外的场景布局可能出错。子任务分解增加了推理延迟（多次 VLM 调用）。

10. 对后续研究的影响是什么？

WMNav 证明了"VLM-as-World-Model"路线的可行性，为导航从反应式决策向预测式规划转型提供了完整框架。其 CVM 结构启发了后续将好奇心驱动探索与空间记忆结合的研究。

22.4.5 消融实验解读

WMNav 的消融实验对比了三种记忆策略：无记忆、文本-图像记忆、CVM。文本-图像记忆策略甚至比无记忆更差——原因是直接把 VLM 生成的文本描述和轨迹图喂回 VLM 容易引发幻觉，产生错误记忆。CVM 的量化约束迫使 VLM 输出尽可能严格的数值（0-10 的整数），保证了每次调用的可靠性。这个发现意义重大：地图表示不仅是信息载体，更是对 VLM 输出的"正则化器"。

22.4.6 优点、失败模式与适用场景

WMNav 的核心优势是前瞻性预测能力，agent 不再盲目探索，而是"有目的地想象"，显著减少往返移动。CVM 的量化设计有效抑制了 VLM 幻觉。失败模式包括：位姿漂移导致 CVM 投影错误、VLM 预测在非规布局中失效、多次 VLM 调用增加推理延迟。适用于：需要高效探索的大场景 ObjectNav、对路径效率有严格要求的任务、VLM 推理能力较强的部署环境。

22.4.7 与具身智能的关系

WMNav 将导航推进到"预测式智能"阶段。Agent 不再只是感知-反应的反射弧，而是具备了内心模拟（mental simulation）能力——预测行动后果、更新内部世界模型、基于预测规划。这是从 SLAM（建图定位）向世界模型（world modeling）演进的关键一步。

22.5 NavFoM：具身导航基础模型

22.5.1 从 WMNav 到 NavFoM 的演进

VLFM、MapNav、WMNav 分别解决了"语义探索""压缩记忆""预测规划"三个问题，但它们都是针对单一任务、单一机器人类型设计的专用系统。VLFM 只做 ObjectNav，MapNav 只做 VLN，WMNav 只做 zero-shot ObjectNav。每种新任务、新机器人都需要重新设计架构和重新训练。

这和计算机视觉领域在 Transformer 出现之前的状况类似：每个任务一个专用网络。Vision Transformer 证明了统一的架构可以处理多种视觉任务。NavFoM 的问题是：导航是否也能有一个统一的跨任务、跨形态基础模型？

22.5.2 NavFoM 的核心洞察

NavFoM 的核心洞察是：导航是一个通用能力，不同任务（VLN、搜索、跟踪、驾驶）和不同形态（四足、无人机、轮式、车辆）共享底层空间推理和决策能力。用足够多的多样化数据训练一个统一架构，模型可以在不针对特定任务微调的情况下泛化到各种导航场景。

22.5.3 系统结构

NavFoM 采用统一的 vision-language-model 架构，核心设计包括：

双分支架构（导航 + QA）。基于视频 VLM（Qwen2-7B 作为语言模型，DINOv2 + SigLIP 作为视觉编码器），扩展到双分支：一支处理导航数据（预测轨迹 waypoints），一支处理 QA 数据（开放世界知识）。两支共享视觉编码器和 LLM 骨干，端到端联合训练。

Temporal-Viewpoint Indicator (TVI) Tokens。不同机器人和任务使用不同数量和配置的摄像头（单目、四目、前视、环视），且导航时间跨度各异。TVI tokens 为每组视觉 token 标记时间戳和摄像头视角信息，让 LLM 能区分“哪个 token 来自哪个摄像头、哪个时间步”。这解决了跨形态训练中最核心的模态对齐问题。

Budget-Aware Temporal Sampling (BATS)。导航历史帧数可能非常庞大（数百步 × 多摄像头），直接输入会超出 token 预算。BATS 基于“遗忘曲线”采样：近期帧采样概率高（ $P(t) \propto e^{\{k(t-T)/T\}}$ ），远期帧采样概率低但有保底下限 $\epsilon=0.1$ 。给定 token 预算 B 和帧数 T，离线求解衰减率 k，确保采样后的期望 token 数不超过预算。BATS 在 1600 token 预算下，推理一个 8-waypoint 轨迹最多 0.5 秒。

Grid Pooling。视觉特征经过两级网格池化：fine-grained（64 patch tokens）用于最新帧和图像 QA，coarse-grained（4 patch tokens）用于历史帧和视频数据，平衡细节和效率。

输入：语言指令 L + 多摄像头图像序列 $I_{\{1:T\}}^{\{1:N\}}$

流程：

1. DINOv2 + SigLIP 编码视觉特征
2. Grid Pooling: 最新帧 fine-grained, 历史帧 coarse-grained
3. TVI Tokens 标记时间和摄像头信息
4. BATS 按遗忘曲线采样历史帧
5. 视觉 token + 语言 token $\rightarrow$ LLM (Qwen2-7B)
6. Action Model 解码 $\rightarrow$ 轨迹 $\tau_T = \{(x, y, z, \theta)\}_k$

22.5.4 训练数据

NavFoM 的训练数据规模前所未有：

数据类型	样本数	任务/来源
VLN-CE (R2R + RxR)	2.94M	室内语言引导导航
OpenUAV	429K	无人机导航
Object Goal Navigation	1.02M	HM3D ObjectNav
Active Visual Tracking	897K	EVT-Bench 目标跟踪
Autonomous Driving	681K	nuScenes + OpenScene
Web-Video Navigation	2.03M	Sekai YouTube 视频
图像 QA	3.15M	开放世界知识
视频 QA	1.61M	开放世界知识
总计	~12.8M	导航 8.02M + QA 4.76M

四足机器人、无人机、轮式机器人、汽车四种形态；VLN、搜索、跟踪、自动驾驶四类任务。在 56 块 H100 上训练约 72 小时（4032 GPU hours）。

22.5.5 代表论文精读

论文：Embodied Navigation Foundation Model (NavFoM), arXiv 2025 (arXiv)

1. 试图解决什么痛点？

现有导航方法局限于特定任务和特定机器人形态，缺乏跨任务、跨形态的通用导航能力。VLM 在通用视觉语言任务上表现优异，但在具身导航中的泛化能力受限于训练数据的任务和形态单一性。

2. 核心假设是什么？

不同导航任务和不同机器人形态共享底层空间推理能力。用大规模多样化的跨任务、跨形态数据训练统一架构，模型可以学到通用导航能力，无需任务特定微调。

3. 输入输出是什么？

输入：语言指令 L + 多摄像头 egocentric 图像序列 $I_{1:T}^{1:N}$ (N 个摄像头, T 个时间步)。输出：导航轨迹 $\tau = \{a_1, a_2, \dots\}$, 每个 $a \in \mathbb{R}^4 = (x, y, z, \theta)$ 表示一个 waypoint 的位置和朝向 (z 仅用于 UAV)。

4. 地图表示是什么？

NavFoM 不使用显式地图表示。它直接从视觉序列预测轨迹，是端到端方法。但 BATS 采样策略在功能上扮演了"选择性记忆"的角色：通过对历史帧的加权采样，隐式维护了导航所需的空间上下文。

5. Tracking 怎么做？

无显式 tracking。模型从视觉序列中隐式学习位姿变化和空间关系。

6. Mapping 怎么做？

无显式 mapping。NavFoM 完全依赖端到端学习，用 attention 机制在大量历史帧中自动提取空间信息。

7. 优化目标是什么？

联合训练导航数据（waypoint 预测）和 QA 数据（下一 token 预测）。视觉编码器和 LLM 用预训练权重初始化，只微调指定参数，单 epoch 训练。

8. 相比前作真正推进了什么？

首次实现了真正的跨任务、跨形态导航基础模型。在 8 个公开基准上达到 SOTA 或强竞争力：VLN-CE RxR 多摄像头 SR 从 56.3% 提升到 64.4% (+8.1%)，单摄像头从 51.8% 提升到 57.4% (+5.6%)；HM3D-OVON zero-shot SR 从 43.6% 提升到 45.2% (+1.6%)；EVT-Bench 跟踪 SR 从 85.1% 提升到 88.4%（四摄像头）。在真实世界的人形机器人、四足、无人机、轮式机器人上验证有效。

9. 没有解决什么？

NavFoM 不使用显式地图，在长程探索任务中（如大场景 ObjectNav）可能不如 VLFM/WMNav 等基于地图的方法高效。端到端架构对训练数据的覆盖范围高度敏感，训练分布外的场景可能表现不佳。BATS 的采样压缩会丢失部分空间细节。推理仍需要 RTX 4090 级别 GPU，边缘部署有挑战。

10. 对后续研究的影响是什么？

NavFoM 定义了"导航基础模型"的研究范式，证明了跨任务、跨形态统一训练的可行性。TVI tokens 和 BATS 为多摄像头、长时程导航建模提供了可复用的技术组件。其大规模数据集和训练方案将成为后续研究的基准。

22.5.6 实验结果分析

基准	前 SOTA	NavFoM	提升
VLN-CE RxR（单目）	51.8% SR	57.4% SR	+5.6%
VLN-CE RxR（四目）	56.3% SR	64.4% SR	+8.1%
HM3D-OVON zero-shot	43.6% SR	45.2% SR	+1.6%
EVT-Bench 单目标（单目）	85.1% SR	85.0% SR	-0.1%
EVT-Bench 单目标（四目）	—	88.4% SR	+3.3% vs 单目
OpenUAV Seen	竞争基线	SOTA	显著

NavFoM 的跨任务提升比单一任务提升更有意义。同一模型在 VLN（语言引导）、OVON（目标搜索）、跟踪、自动驾驶四个差异巨大的任务上同时具有竞争力，这在之前的方法中从未实现。多摄像头配置（四目 vs 单目）带来 5-8% 的 SR 提升，证明丰富的视觉观察对导航性能至关重要。但 OVON 上提升幅度较小（1.6%），说明开放词汇目标搜索仍是一个地图友好的任务——显式空间记忆（如 VLFM 的 value map）在这个任务上有其不可替代的优势。

22.5.7 优点、失败模式与适用场景

NavFoM 的核心优势是通用性和部署便利性：一个模型处理多种任务和机器人，无需任务特定微调。TVI tokens 和 BATS 是优雅的多摄像头、长时程建模方案。失败模式包括：无显式地图导致长程探索效率偏低、对训练数据分布敏感、边缘部署计算需求高。适用于：多任务机器人平台、快速切换任务的场景、摄像头配置丰富的机器人。

22.5.8 与具身智能的关系

NavFoM 代表了导航从专用系统向基础模型的范式跃迁。它不接地图模块，而是直接从海量数据中学习隐式空间推理。这和视觉领域从手工特征 → CNN → ViT 的演进一致。但值得注意的是，NavFoM 在 OVON 等探索型任务上的提升幅度有限，暗示**显式地图表示和端到端学习并非零和关系**——未来的导航基础模型很可能同时包含学习得到的隐式知识和显式建图模块，两者互补。

22.6 四篇论文的演进脉络

维度	VLFM	MapNav	WMNav	NavFoM
年份	2023/2024	2025	2025	2025
地图类型	Occupancy + Value Map	Annotated Semantic Map	Curiosity Value Map	无显式地图
核心能力	语义 Frontier 选择	压缩语言标注记忆	预测式世界模型	跨任务跨形态通用
VLM 角色	价值评分器	记忆读取+决策器	世界模型预测器	端到端策略
记忆形式	显式二维网格	显式俯视语义图	显式标量预测图	隐式注意力记忆
是否 Zero-shot	是	微调	是	无需任务微调
训练数据量	无（全预训练）	中等	无（全预训练）	800万导航样本

这个表格的纵向演进线值得解读。VLFM → MapNav → WMNav 这三条线展示了显式地图表示的持续进化：从最简单的二维 value map，到带语言标注的语义地图，再到能存储预测状态的好奇心地图。地图的表达能力在逐步增强，从纯几何→语义→预测性。而 NavFoM 是一条横向的跃迁线：它用海量数据和统一架构证明，不用显式地图也能做导航，而且能做很多种导航。

两条路线不是对立关系。NavFoM 在 VLN 和跟踪上提升巨大（因为这两个任务更依赖视觉理解而非空间探索），但在 OVON 上提升有限（因为探索任务需要地图式的空间记忆）。这暗示未来的最优解很可能是**混合架构**：基础模型提供通用视觉语言理解能力，显式地图模块提供结构化空间记忆，两者通过某种注意力机制协作。

从 SLAM 的角度看这个演进更具深意。传统 SLAM 的目标是"建一个精确的地图", 而这几篇论文中的地图变成了"支撑决策的记忆结构"。精度不再是首要追求——VLFM 的 value map 是粗略的语义估计, MapNav 的 ASM 是有损压缩, WMNav 的 CVM 是预测概率。这些"不精确但有用"的地图, 恰恰是具身智能需要的地图。

22.7 一句话总结

具身导航正在从"端到端反应"走向"地图引导的预测式决策", 而 SLAM 产出的结构化空间记忆正成为这条路上最不可或缺的基础设施——不是作为独立模块, 而是作为智能体思考空间的媒介。

第23章 从 VLM 到 VLA: NaVILA 与具身机器人闭环

23.1 为什么 VLA 重要?

第22章介绍了 VLFM、MapNav、WMNav 和 NavFoM 等工作, 它们共同描绘了一条路径: 让视觉语言模型 (VLM) 为机器人导航服务。这些方法的共性是"看得懂、说得出"——VLM 能解析场景语义、识别路标、输出导航计划。但机器人要完成最后一公里的任务, 还需要"做得到"。看懂和说出之间, 隔着一道执行鸿沟。

从"看懂"到"执行"的三级跳

VLM 的能力边界可以概括为三级阶梯。

第一级: 视觉理解。 CLIP (ICML)、DINOv2 (ICLR) 等模型实现了图像与语言的跨模态对齐, 能回答"图中有什么"、"红色杯子在哪里"。这一层解决的是感知问题。

第二级: 空间推理与规划。 第22章的 VLFM 和 NavFoM 等工作让 VLM 开始推理空间关系——"左转经过走廊后能看到厨房"。这一层输出的是高层计划或语义地图, 但仍停留在符号空间。

第三级: 物理执行。 机器人需要产生真实的控制信号——电机扭矩、关节角度、线速度、角速度。这就是 Vision-Language-Action (VLA) 模型的使命。VLA 的核心命题: 将视觉感知、语言理解和物理动作统一到一个端到端框架中 (arXiv)。

三级跳的关系可以类比为人类驾驶员: 第一级是"认出交通灯", 第二级是"判断该踩刹车还是油门", 第三级是"右脚以多大力度踩下踏板"。前两级的产出是信息, 第三级的产出是力。

端到端 VLA 的两种路线

将 VLM 的输出转化为物理动作, 当前有两条技术路线。

路线一: 直接动作输出。 Google DeepMind 的 RT-2 (arXiv) 是这一路线的代表。它将机器人动作维度离散化为 256 个 bin, 像生成文本 token 一样自回归地预测动作 token。RT-2 的 PaLI-X 主干拥有 55B 参数, 在 130K 条机器人操作轨迹上微调。OpenVLA (arXiv) 延续了这一思路, 在 Prismatic-7B VLM 基础上训练, 用 970K 条 Open X-Embodiment 数据。直接路线的优势是简洁——感知、推理、执行在同一个网络中完成。代价是动作以离散 token 形式输出, 运动不够平滑, 且大模型必须以高频运行推理, 实时性受限。

路线二：分层动作分解。 这条路线主张不让大模型直接控制关节，而是输出一种中间表示——可解释的中层动作——再由下层策略负责执行。NaVILA (RSS) 是这条路线的典型代表。它让 VLM 输出语言形式的中层空间指令（如"前进 75cm"），然后用独立的视觉运动学策略将这些指令转化为关节位置。这种方法解耦了高层推理与低层控制，让两者以各自合适的频率运行。

路线	代表模型	动作表示	推理频率	优势	劣势
直接动作输出	RT-2, OpenVLA	离散 token 序列	高（控制频率）	端到端简洁	运动不平滑，推理开销大
分层动作分解	NaVILA	自然语言中层指令	低（规划频率）	可解释、跨本体、双频高效	需要额外训练下层策略

分层动作分解的核心洞察是：大模型擅长语言和空间推理，但不擅长毫秒级动力学计算；强化学习策略擅长动力学控制，但不擅长语义理解。两者各安其位，通过中层语言动作这个"通用接口"对接。中层语言动作类似于自动驾驶中的 Apollo Cyber RT 框架——上层规划输出"变道"指令，下层控制负责执行具体的转向角和加速度曲线。这种解耦设计让同一个 VLA 可以通过更换下层策略适配不同的机器人平台——四足、人形或轮式。

维度	直接动作输出（RT-2）	分层动作分解（NaVILA）
动作表示	离散化关节/末端位姿	自然语言空间指令
模型耦合	单层端到端	两层解耦
跨本体迁移	需重新训练	仅更换下层策略
推理频率	控制频率（~5-10Hz）	规划频率（~1Hz）
数据来源	机器人操作轨迹	人类视频+仿真轨迹+QA
可解释性	低	高（语言动作可阅读）

分层路线的代价是需要额外训练一个视觉运动学策略，并设计从语言指令到低层控制信号的转换模块。但换来的收益是显著的：推理频率降低使大模型部署成本大幅下降；中层语言表示天然可解释，便于调试；最重要的是，这种架构让 SLAM 和空间地图有了明确的接入点——中层动作正是地图与执行之间的桥梁。

23.2 NaVILA

核心思想

NaVILA（Legged Robot Vision-Language-Action Model for Navigation）由 UC San Diego、USC 和 NVIDIA 联合提出，发表于 RSS 2025 (arXiv)。它解决一个具体问题：如何用自然语言指挥腿式机器人在复杂环境中导航。

腿式机器人（四足、人形）相比轮式机器人有更强的地形适应能力，能穿越楼梯、碎石、窄道等复杂地形。但语言到腿关节的控制链路更长——“穿过走廊”需要转化为数十个关节的协调运动。端到端 VLA 试图让大模型直接输出关节指令，这在高频、高自由度的腿式系统上面临严峻挑战。

NaVILA 的关键设计是**两层框架**：上层是一个经过微调的 VLM，它接收视觉观察和导航指令，输出语言形式的中层空间动作（如“左转 30 度”、“前进 75cm”、“停下”）；下层是一个基于视觉的运动学强化学习策略，它将中层指令转化为具体的关节位置控制。两层之间通过自然语言动作这个通用接口通信，运行在不同的目标上。

系统架构

上层 VLA：从视觉到语言动作

NaVILA 的 VLA 层基于 VILA（一个预训练的视觉语言模型）进行微调。VILA 的三件套结构是：视觉编码器（Vision Transformer）提取图像特征，MLP 投影器将视觉 token 映射到语言嵌入空间，自回归 LLM 生成文本输出 (arXiv)。

将通用 VLM 改造成导航 VLA，NaVILA 做了三处关键修改：

历史帧与当前观察的区分处理。 导航需要记忆。VILA 原始的均匀帧采样不区分历史观察与当前帧。NaVILA 将最新帧标记为“当前观察”，历史帧标记为“历史观察”，在 prompt 中分别标注：`a video of historical observations:` 和 `current observation:`。这种显式区分让 LLM 能更好地利用历史信息进行长期推理。

导航专用 prompt。 NaVILA 设计了一套导航任务模板，将视觉观察、历史上下文和语言指令组织成结构化的对话格式。prompt 包含任务指示（“你是一个导航助手”）、历史帧描述、当前帧描述和导航指令。

数据混合微调。 为避免模型在导航数据上过拟合，NaVILA 的微调数据按四类混合：真实人类视频（约 20K 条轨迹，从 YouTube 第一视角旅游视频中提取）、仿真导航数据（VLN-CE 的 R2R/RxR 轨迹）、辅助导航数据（空间推理 QA 对）、通用 VQA 数据集。这种混合策略确保模型在专精导航的同时保持通用视觉语言能力。

中层动作的语言表示包括六种基本指令类型：

- `turn right [角度] degrees`
- `turn left [角度] degrees`
- `move forward [距离] cm`
- `move backward [距离] cm`
- `move to the [方位] and stop`
- `stop`

每个指令携带明确的空间量纲（角度或距离），下层策略负责将这些量纲转化为速度命令。

下层视觉运动学策略

下层策略接收 VLA 输出的语言动作，将其转化为线速度和角速度命令，然后通过 PPO（Proximal Policy Optimization）算法训练的策略跟踪这些速度命令。策略的具体架构如下：

观察空间。策略有两种观察模式：仅本体感知（blind）模式接收机器人线速度、角速度、朝向、关节位置、关节速度、历史动作等本体感受信号；视觉模式额外接收从 LiDAR 点云构建的 2.5D 高度图（height map）。LiDAR 提供 360°×90° 视场，生成网格化的地形高度表示。

动作空间。策略输出 12 维期望关节位置 $q^d \in \mathbb{R}^{12}$ （Unitree Go2 有 12 个关节），通过 PD 控制器转化为电机扭矩。

训练细节。采用 PPO 算法在 IsaacLab 仿真器中训练。训练时 critic 网络可以访问 privileged 信息（真实地形高度），actor 网络只使用真实传感器可用的信息。引入域随机化（摩擦系数、质量、地形高度等）弥合仿真与现实的差距。训练在单个阶段完成，无需教师-学生蒸馏——这与之前主流的 teacher-student 范式不同，后者需要先训练一个具有 privileged 信息的教师策略，再用监督学习训练只使用传感器数据的学生策略。

速度命令转换。VLA 输出的语言动作（如“前进 75cm”）首先被解析为目标线速度和角速度 (v_{cmd}, ω_{cmd}) 。策略以这些命令为参考信号，通过奖励函数中的速度跟踪项学习跟随：

$$r_{vel} = \exp\left(-\left((v - v_{cmd})^2 + (\omega - \omega_{cmd})^2\right)\right)$$

总奖励函数还包括姿态稳定项、能量效率项、碰撞惩罚项等。这种基于速度命令的接口让同一套 VLA 可以与不同的下层策略配合，适配不同的机器人平台。

双频运行时序

NaVILA 的双层架构天然支持双频运行。VLA 以较低频率（约 1Hz）处理视觉输入、历史记忆和语言指令，生成下一个中层空间动作。下层运动学策略以较高频率（50-100Hz）实时运行，处理障碍物回避、平衡控制和地形适应。这种时标分离有两个好处：一是大模型的推理开销不会成为实时控制的瓶颈；二是下层策略能在 VLA 两次决策之间自主应对突发情况（如突然出现障碍物），提升了系统鲁棒性。

关键实验结果

VLN-CE 基准测试。NaVILA 在 R2R-CE 和 RxR-CE 的 Val-Unseen 拆分上测试。仅使用单视角 RGB 输入，NaVILA 在 R2R-CE 上达到 54% 的 Success Rate（SR），在 RxR-CE 上达到 44% 的 SR。这是首次有仅使用单视角 RGB 的导航智能体在不依赖预训练 waypoint 预测器的情况下达到与使用全景图、深度图和里程计的方法相当或更好的性能。

方法	输入	R2R-CE NE↓	R2R-CE SR↑	RxR-CE NE↓	RxR-CE SR↑
CMA	全景+深度+里程计	7.37	32%	—	—
NaVid	单视角RGB	5.47	37%	—	—
HNR*	全景+深度+里程计	4.42	61%	5.50	46.7%
NaVILA	单视角RGB	5.22	54%	6.77	44%

*注：HNR 等方法使用预训练的 waypoint 预测器和额外传感器。NaVILA 仅使用单视角 RGB，却达到了接近乃至超越这些方法的性能，验证了中层语言动作框架的强泛化能力。

跨数据集零样本迁移。 仅在 R2R-CE 上训练，直接在 RxR-CE Val-Unseen 上测试（零样本），NaVILA 的 SR 达到 34.3%，比 NaVid 高出约 10 个百分点。这说明中层语言动作作为迁移媒介比端到端动作表示更具跨域适应性——R2R 的"前进 2 米"和 RxR 的"穿过走廊"虽然指令风格不同，但共享相同的空间动作词汇。

VLN-CE-Isaac 新基准。 由于现有 VLN 基准基于离散导航图或简化的连续环境，无法反映腿式机器人低层运动的真实挑战，NaVILA 提出了 VLN-CE-Isaac 基准。该基准在 Isaac Sim 中重建真实场景，要求机器人以关节级控制在物理仿真中执行导航任务。在 Unitree Go2 上，视觉策略（NaVILA-Vision）的 SR 达到 50.2%，比仅使用本体感知的盲策略（NaVILA-Blind，36.2%）高出 14%。在 Unitree H1 人形机器人上，这一差距扩大到 21%（45.3% vs 24.4%），说明视觉感知对腿式导航至关重要。

真实世界部署。 NaVILA 在三种真实环境中测试：办公室工作区、家庭环境和户外场景。使用 25 条语言指令（每条重复 3 次），整体成功率 88%。简单指令（1-2 个导航命令，不跨房间）成功率 100%，复杂指令（3 个以上命令，需跨房间导航）成功率 75%。与 GPT-4o 相比，NaVILA 在 SR 上显著领先，证明专门微调的 VLA 比通用 VLM 更适合导航任务。

跨本体迁移。 NaVILA 的 VLA 层在 Unitree Go2（四足）上训练后，无需重新训练，直接配合 Unitree H1（人形）和 Booster T1 的下层策略运行。尽管相机高度和视角发生了变化，VLA 仍能保持良好性能。这验证了分层架构的跨本体通用性——只要更换下层策略，同一个"大脑"可以驱动不同的"身体"。

NaVILA 论文精读

论文试图解决什么痛点？ 将自然语言指令转化为腿式机器人的低层关节控制。端到端 VLA 让大模型直接输出关节指令，在腿式系统的高自由度、高频控制场景下既难以训练又难以部署。

核心假设是什么？ 大模型擅长空间推理和语言理解，但不应直接控制关节；通过中层语言动作解耦高层推理与低层控制，可以获得更好的泛化性、可解释性和跨本体适应性。

输入输出是什么？ VLA 层：输入为单视角 RGB 帧序列 + 语言导航指令；输出为自然语言形式的中层空间动作（如"前进 75cm"）。Locomotion 层：输入为 LiDAR 高度图 + 本体感知 + 速度命令；输出为 12 维期望关节位置。

地图表示是什么？ NaVILA 的 VLA 层不使用显式地图表示，而是将历史帧作为隐式记忆嵌入到 prompt 中。Locomotion 层使用从 LiDAR 点云构建的 2.5D 高度图作为局部地形表示，用于障碍物回避和地形适应。

Tracking 怎么做？ 中层动作通过正则表达式解析为速度命令 (v_{cmd}, ω_{cmd})。下层 PPO 策略通过奖励函数中的速度跟踪项学习跟随命令，而不是硬编码的 PID 跟踪器。这给了策略一定的灵活性——在面临障碍物时可以偏离命令以保证安全。

Mapping 怎么做？ 不使用传统的度量地图或语义地图。VLA 层依赖历史帧的隐式空间记忆进行导航决策。Locomotion 层将 LiDAR 点云转换为 2.5D 高度图，通过最大滤波器平滑处理最近 5 帧点云数据。

优化目标是什么？ VLA 层的微调目标是最小化导航动作生成的交叉熵损失，同时保留通用 VQA 能力。Locomotion 层的 PPO 训练最大化奖励函数，包括速度跟踪、姿态稳定、能量效率和碰撞避免四项。

相比前作真正推进了什么？ (1) 首次提出中层语言动作的分层 VLA 框架，解耦了高层推理与低层控制；(2) 首次证明直接从人类视频训练可提升连续环境导航性能；(3) 首次在真实世界中实现了端到端视觉运动学腿式机器人语言导航，成功率 88%；(4) 提出 VLN-CE-Isaac 基准，填补了现有基准在低层物理控制方面的空白。

没有解决什么？ (1) 中层动作的空间精度受限于语言表示——"前进 75cm"只能做到约 10cm 级精度，难以满足精细操作需求；(2) 长期导航中的累积漂移——依赖历史帧的隐式记忆，缺乏显式的拓扑或度量地图支持；(3) 动态环境中的适应性——真实世界测试主要在静态环境中进行，对动态障碍物和人的响应能力有限；(4) 复杂地形（如攀爬、跳跃）的中层动作表示尚未覆盖。

对后续研究的影响是什么？ NaVILA 证明了"大模型出语言指令 + 专用策略出关节控制"的分层范式在具身导航中的可行性。这一框架启发了后续 VLA 研究在动作表示上的分层思考，也为 SLAM 与 VLA 的融合提供了自然的接口——中层语言动作正好是地图规划层与运动执行层之间的翻译层。

优点、失败模式与适用场景

NaVILA 的核心优势在于可解释性和模块化。语言形式的中层动作可以被人类阅读、检查和干预——如果 VLA 输出"前进 10 米"而前方 3 米处有悬崖，操作者可以直接修改指令。这种可解释性在安全关键场景中至关重要。分层架构还带来了跨本体适应性：VLA 层不关心机器人的具体形态，只关心"这个机器人能不能前进 75cm"。

失败模式主要来自两个方面。一是语言动作的有限表达力——六种基本指令不足以描述复杂的空间路径（如"绕过障碍物左侧"需要拆解为多个基本指令）。二是 VLA 的帧级推理在长走廊或重复环境中容易迷失方向，因为没有显式的空间地图来提供全局定位。

适用场景包括：室内长走廊导航、结构化户外环境（校园、园区）、需要人机协作的场景（语言动作可被人类监督和修正）。不适用场景包括：需要厘米级精度的操作任务、高度动态的人群环境、无纹理的宽阔开放空间。

23.3 SLAM 在 VLA 中的位置

NaVILA 的 VLA 层使用历史帧序列作为隐式记忆，Locomotion 层使用 LiDAR 高度图作为局部地形表示。这种设计能应对中等长度的导航任务，但随着路径增长，隐式记忆的累积误差会让机器人"迷路"。这正是 SLAM 可以介入的位置。

在 VLA 系统中，SLAM（或更广泛地说，空间地图与状态估计）承担三件事。

第一件：提供空间一致性

VLM 的推理在空间一致性上存在固有缺陷。大模型处理每一帧图像时，没有跨帧的几何约束——前一帧推理"走廊尽头左转"和后一帧推理"前方是门"之间，不存在像素级或度量级的对应关系验证。两个独立的空间判断可能在几何上自相矛盾，但模型无从察觉。

SLAM 通过位姿图优化或因子图推断，将每一时刻的观察与历史状态用几何约束连接起来。当 VLA 输出"前进 75cm 后右转"时，SLAM 可以提供当前位姿的精确估计，验证"前进 75cm"是否已经完成，以及"右转"的参考坐标系是否正确。这种闭环验证能防止错误累积。

空间一致性的需求在中层语言动作框架中尤为突出。因为 NaVILA 的中层动作携带明确的空间量纲（距离、角度），这些量纲需要与真实世界对齐。没有 SLAM 的位姿估计，"前进 75cm"会变成"前进大约 75cm"——在大约十次这样的"大约"之后，定位误差可能超过一米，足以让机器人在十字路口走错方向。

第二件：提供历史记忆

NaVILA 的 VLA 层使用历史帧序列作为记忆，但 LLM 的上下文窗口有限（VILA 最长支持约 1024 帧），且帧级记忆缺乏结构化的空间索引。当机器人需要回答"我是否经过了这个房间？"时，逐帧扫描历史观察效率极低。

SLAM 生成的地图（拓扑地图、度量地图或语义地图）提供了结构化的空间记忆。拓扑地图记录"房间 A 经过走廊 B 连接到房间 C"，支持快速的路径规划和回溯。度量地图记录精确的坐标和几何，支持"距离起点约 15 米"这类空间查询。语义地图标注每个区域的功能（厨房、走廊、卧室），让 VLA 的语言指令能直接在语义层面索引。

SLAM 地图与 VLA 的融合方式可以多样。最直接的方式是将地图信息嵌入 VLA 的 prompt——例如，将拓扑地图转化为语言描述"你当前在走廊东段，前方 5 米是 T 字路口，左侧通向卧室，右侧通向客厅"，作为导航指令的上下文。这种方式不需要修改 VLA 的网络结构，只需在 prompt 工程层面接入地图信息。

更深入的融合是让地图特征直接参与 VLA 的推理。例如，将 3D 场景图或拓扑图的嵌入向量与视觉 token 拼接，送入 LLM 进行联合推理。这种方式需要额外的训练数据来教会模型如何解读地图特征，但能实现更紧密的感知-地图-行动闭环。

第三件：提供可规划的局部/全局结构

中层语言动作（"前进 75cm"、"左转 30 度"）本质上是短视的——每一步只考虑当前观察到的局部环境。在复杂环境中，局部最优决策可能导致全局失败。经典的例子是"贪心走廊问题"：机器人在 T 字路口看到左侧走廊更近，选择左转，但目标实际上在右侧走廊的深处。

SLAM 地图提供了全局结构。全局度量地图支持 A* 或 RRT 等路径规划算法，找到从起点到终点的最优路径。拓扑地图支持高层导航决策——"先到达走廊尽头，再右转进入大厅"。将全局规划器与 VLA 结合，可以让 VLA 专注于局部路径的精细执行，而全局路径由规划器保证最优性。

融合框架的典型工作流程是：(1) SLAM 模块构建全局语义-拓扑地图；(2) 路径规划器在地图上找到从当前位置到目标的最短路径，输出为路标序列（"走廊→路口→大厅→目标房间"）；(3) VLA 将每个路标转化为中层语言动作序列（"前进 10 米→右转→前进 5 米→停下"）；(4) Locomotion 策略执行每个中层动作。在这个框架中，SLAM 提供"战略"（全局路径），VLA 提供"战术"（局部决策），Locomotion 提供"执行"（关节控制）——三者各司其职，构成完整的具身导航闭环。

SLAM 角色	功能描述	VLA 集成方式	缺失时的后果
空间一致性	位姿估计与几何验证	验证中层动作执行结果	累积定位漂移，走错路口
历史记忆	结构化空间索引	将地图嵌入 prompt 或特征	重复访问同一地点，无法回溯
全局规划	拓扑/度量路径规划	提供路标序列指导 VLA	局部贪心决策导致全局失败

这三种角色构成了 SLAM 在 VLA 系统中的价值闭环。空间一致性保证"我执行的动作和预期一致"，历史记忆保证"我知道自己在哪里、去过哪里"，全局规划保证"我在走一条合理的路"。三者叠加，让 VLA 从"走一步看一步"的短视行为进化为"胸有成竹"的长程导航。

SLAM-VLA 融合当前实践

已有若干工作探索了 SLAM 与 VLA 的融合路径。MapNav（第22章）让 VLM 直接在语义地图上输出导航计划，验证了"地图+语言模型"的可行性。VLFM 使用开放词汇语义地图指导前沿探索，本质上是 SLAM 地图与 VLA 目标设定的融合。WMNav 和 NavFoM 则用世界模型替代显式地图，提供了另一种空间记忆的形式。

但这些工作的共同局限是地图与 VLA 的耦合较浅——地图要么作为输入图像的替代品（MapNav），要么作为目标函数的辅助项（VLFM），尚未实现深度融合。未来的关键方向是让 SLAM 的位姿估计、地图更新和回环检测成为 VLA 推理的内在组成部分，而非外部模块。这要求 VLA 模型能理解"定位不确定性"、"地图更新"和"重定位"等概念——本质上，VLA 需要学会与 SLAM 协同工作，而不是被动接收 SLAM 的输出。

23.4 本章结论

具身智能不是让 VLM 取代 SLAM，而是让 SLAM 成为 VLM/VLA 能行动的空间基座。

这句话概括了本书第五篇的核心论点。VLM 赋予了机器人"看懂"和"理解"的能力，VLA 赋予了机器人"执行"的能力，但看懂和执行之间的空间一致性、历史记忆和全局规划——这些正是 SLAM 数十年来深耕的领域。

NaVILA 的分层框架揭示了这种协同关系的最自然形态：SLAM 在 VLA 的上层和下层之间搭建空间桥梁。在上层，SLAM 地图为 VLA 提供结构化记忆和全局路标，让语言指令锚定在真实空间坐标上。在下层，SLAM 的位姿估计验证中层动作的执行精度，将"大约前进 75cm"校正为"准确前进 75cm"。

VLA 与 SLAM 的融合将沿着三个维度深化。

表示融合。SLAM 的地图表示（点云、网格、拓扑图、语义地图）需要转化为 VLM 能理解的表示形式。最直接的途径是将地图"语言化"——用自然语言描述空间关系。更深入的途径是将地图特征嵌入到 VLM 的 token 空间中，让模型在注意力机制中自动学习如何利用地图信息。

推理融合。当前的 VLA 将 SLAM 视为外部工具，未来的 VLA 应内建空间推理能力——理解"位姿不确定性"、"闭环检测"和"重定位"等概念。这意味着 VLA 不仅能调用 SLAM，还能在 SLAM 失效时（如特征退化环境）做出合理判断。

学习融合。SLAM 系统的参数（特征提取阈值、关键帧间距、回环检测阈值）传统上由人工调定。VLA 的语义理解能力可以指导 SLAM 的自适应参数调整——在语义丰富的区域降低回环检测阈值，在走廊等退化区域提高特征提取的敏感度。反之，SLAM 的几何一致性可以为 VLA 提供训练监督信号——当 SLAM 检测到位姿跳变时，相应的 VLA 决策应被标记为潜在错误。

具身智能的终极形态不是一个大模型包办一切，而是一个由 SLAM 提供空间基座、VLA 提供语义理解与决策能力、运动学策略提供物理执行力的协同系统。在这个系统中，SLAM 不后退一步为 VLM 让路，而是前进一步成为 VLA 的空间基石。本书的最后一章将展望这一融合趋势在 2026 年的最新前沿。

第六篇：2026 之后的方向、系统落地与写作框架

第24章 2026 前沿总览：SLAM 正在变成什么？

全书的技术演进线从传统几何 SLAM 出发，历经视觉/激光/MIO 工程系统、学习特征与匹配、DROID-SLAM 类可微优化、iMAP/NICE-SLAM/NeRF-SLAM 神经地图、3DGS-SLAM 可渲染显式地图、开放词汇语义 3DGS 与场景图、3D-Mem/GSMem 空间记忆，最终到达 VLM/LLM/VLA 驱动的具身导航与世界模型。这条线不是简单的技术替换，而是地图表示在机器人智能中扮演的角色发生了根本变化：地图不再是人类可读的 occupancy grid 或点云，而是机器人与环境交互的中间表示——可优化、可渲染、可语义化、可被具身系统重用。

2025 至 2026 年的工作表明，五个方向正在同时加速这一变革。

24.1 方向一：3DGS 成为可渲染地图核心

3D Gaussian Splatting (3DGS) 在 2023 年被提出时，被视为 NeRF 的“快速替代品”。但 SLAM 社区很快意识到，3DGS 的优势不是“画面漂亮”，而是它把地图变成了一个可优化、可渲染、可语义化、可被具身系统重用的中间表示。

传统 SLAM 的地图表示沿两条路径演化：稀疏点云（ORB-SLAM 系列）缺乏密集几何；隐式神经场（NeRF-SLAM 系列）虽能渲染新视角，但体积查询效率低、编辑困难。3DGS 用一组各向异性三维高斯显式参数化场景（位置 μ 、协方差 Σ 、不透明度 σ 、球谐系数 SH ），通过光栅化而非光线 marching 实现实时渲染。这意味着地图本身就是可微的——可以直接用渲染误差对位姿和地图参数做联合优化。

关键工作演进

SplaTAM (CVPR 2024) 最早将 3DGS 用于 RGB-D SLAM。每个高斯被简化为椭球体，深度和颜色渲染损失联合监督，tracking 和 mapping 共享同一组高斯参数。其核心洞察是：深度传感器提供的几何锚点让高斯初始化有可靠的 3D 位置先验，避免了单目系统的尺度歧义。(CVF 开放获取)

MonoGS (CVPR 2024) 将 3DGS SLAM 推进到单目 RGB-D 设置，引入关键帧选择和几何验证策略。MonoGS 证明即使在没有深度输入时，高斯参数的可微性仍然允许通过光度一致性恢复稠密几何。(CVF 开放获取)

GS3LAM (2026) 在 3DGS 中引入语义通道，提出 SG-Field 语义高斯场和随机采样关键帧映射 (RSKM) 策略。RSKM 解决了 3DGS-SLAM 中的优化偏置问题：局部共视关键帧映射在共视稀疏区域欠优化、在共视密集区域收敛困难，而随机采样通过概率化关键帧选择平衡了全局一致性。(arXiv)

OpenGS-SLAM (ICRA 2025) 将开放词汇语义引入 3DGS-SLAM，每个高斯携带 CLIP 语义嵌入，支持自然语言查询场景中的任意物体。(IEEE Xplore)

EmbodiedSplat (2026) 提出前馈式语义 3DGS 框架，从图像流直接预测带有语义语言嵌入的 3D 高斯场，无需迭代优化，实现近实时推理。(arXiv)

X-GS (2026) 是一个统一的 3DGS 框架，整合在线 SLAM、语义蒸馏和下游 VLM 接口，通过在线向量量化 (VQ) 模块和 GPU 加速网格采样实现约 15 FPS 的实时语义映射。(arXiv)

GSMem (2026) 将 3DGS 作为持久空间记忆用于具身探索，核心能力是"空间回忆" (Spatial Recollection)：从已探索区域渲染任意新视角的逼真图像，供 VLM 做高保真推理。GSMem 同时维护对象级场景图和语义级语言场，实现多层次检索。(arXiv)

核心判断

3DGS 的优势不是"画面漂亮"，而是它把地图变成可优化、可渲染、可语义化、可被具身系统重用的中间表示。differentiable rasterization 使每个高斯都是可梯度更新的；显式存储使每个高斯都可被查询、编辑、赋予语义标签；实时渲染使地图成为 VLM/LLM 的"虚拟相机"。这个三重属性（可微-显式-可渲染）是 NeRF 的隐式表示无法同时满足的。

方法	输入	语义	实时性	核心创新
SplaTAM	RGB-D	无	是	简化高斯用于 RGB-D SLAM
MonoGS	RGB-D	无	是	单目 3DGS-SLAM 先驱
GS3LAM	RGB-D	封闭集	是	SG-Field + RSKM 语义优化
OpenGS-SLAM	RGB-D	开放词汇	是	CLIP 嵌入语义高斯
EmbodiedSplat	RGB	开放词汇	近实时	前馈语义 3DGS
X-GS	RGB/RGB-D	开放词汇	约 15FPS	统一 SLAM+语义+VLM 框架
GSMem	RGB-D	开放词汇	是	持久空间记忆+空间回忆

上表揭示了 3DGS-SLAM 的三阶段演进：从纯几何（SplaTAM/MonoGS）到语义增强（GS3LAM/OpenGS-SLAM），再到具身就绪（EmbodiedSplat/X-GS/GSMem）。每一阶段都在增加地图的"可消费性"——从仅供人类查看，到供语义查询，再到供 VLM 推理和导航决策。

论文精读：GSMem

痛点。 现有场景表示（场景图、视图快照）缺乏事后可重观察性（post-hoc re-observability）。如果初始观察错过了目标，记忆遗漏往往不可恢复。

核心假设。 3DGS 的显式参数化（连续几何 + 密集外观）可以作为持久空间记忆，支持从任意新视角重新渲染已探索区域。

输入输出。 输入 RGB-D 视频流和语言查询；输出语义检索结果和最优视角渲染图像。

地图表示。 3D 高斯场 + 并行的对象级场景图 + 语义级语言场。

Tracking。 基于 3DGS 的覆盖目标函数引导探索方向。

Mapping。 增量式 3DGS 重建，每个高斯携带语义特征。

优化目标。 混合探索策略结合 VLM 语义评分和 3DGS 覆盖目标。

相比前作推进了什么。 相比 3D-Mem 的视图快照，GSMem 提供空间回忆能力——代理无需物理导航即可从新视角观察已探索区域。相比 ConceptGraphs 的离散对象表示，GSMem 保留连续几何和视觉保真度。

没有解决什么。 动态场景的高斯更新、大规模环境的内存控制、VLM 推理延迟对实时性的限制。

对后续研究的影响。 确立了"3DGS 作为持久空间记忆"的范式，为具身问答和终身导航提供了新的表示基础。

失败模式与未解问题

3DGS-SLAM 在动态场景、强光照变化和大规模室外环境中仍然脆弱。高斯数量的无界增长导致长期运行时内存膨胀——Replica 级别的房间可能需要数十万到数百万高斯，城市规模场景将不可行。当前方法假设静态世界，动态物体会导致高斯"拖影"（运动物体留下一串模糊的高斯轨迹）。次表面散射、透明材质和强镜面反射的建模仍然不准确。

24.2 方向二：基础模型成为几何前端

传统 SLAM 前端中最脆弱的部分是什么？特征匹配、深度估计、局部几何初始化。这三个环节在弱纹理、重复结构、运动模糊和光照变化下反复失效。

2024 至 2025 年出现的 3D 视觉基础模型正在系统性替代这些脆弱环节。DUST3R、MASt3R、VGGT 等模型在千万级图像对上预训练，学会了从单张或成对图像直接预测密集 3D 几何——不是作为后处理，而是作为 SLAM 的前端输入。

关键工作演进

DUST3R (2024) 从图像对直接预测统一坐标系下的密集 3D 点图 (pointmap) 和匹配关系。关键突破是：无需相机内参，无需先验位姿，网络自己学会将两张图像的像素对应到同一 3D 空间。这为"无标定 SLAM"奠定了基础。(arXiv)

MASt3R (ECCV 2024) 在 DUST3R 基础上引入快速局部特征描述子，使匹配更鲁棒。其核心贡献是将"重建"和"匹配"统一到一个框架中：pointmap 提供几何，描述子提供对应关系，二者互补。(Springer)

MASt3R-SLAM (CVPR 2025) 是第一个将 MASt3R 作为核心几何先验的实时稠密单目 SLAM 系统。输入 RGB 视频，MASt3R 预测关键帧对的 pointmap，pointmap matching 建立帧间对应，tracking 和局部融合估计相机运动，回环检测修正长期漂移。相比传统 SLAM，MASt3R-SLAM 的位姿估计不依赖稀疏特征点，而是直接基于密集 3D 几何一致性。(CVF 开放获取)

VGGT (2025) 将基础模型的输入从图像对扩展到任意多视角，单网络前向传播同时预测相机内参、位姿、深度、跟踪点和密集点图。VGGT-SLAM 和 VGGT-Long 将其接入在线 SLAM，通过子图对齐实现大规模场景重建。(arXiv)

AIM-SLAM (2026) 指出 MASt3R-SLAM 的固定双视角输入和 VGGT-SLAM 的固定窗口策略都限制了基线模型的几何推理能力。AIM-SLAM 提出自适应信息性多视角关键帧优先策略，选择视角重叠度高、信息增益大的关键帧，而非简单的时序相邻帧，在 Sim(3) 空间中联合优化，实现更准确的稠密重建。(arXiv)

CAL²M (2026) 解决基础模型 SLAM 的尺度漂移问题。单目视觉几何基础模型 (VGFM) 无法恢复绝对尺度，且内参估计的仿射歧义导致长期轨迹发散。CAL²M 引入"辅助眼"——一个与主相机保持固定间距的辅助相机，用间距先验消除尺度歧义，配合极线引导的内参搜索和位姿校正，实现**无标定公里级 SLAM**。(arXiv)

核心判断

基础模型正在吃掉传统前端中最脆弱的部分：匹配、深度、局部几何初始化。这不是简单的"用网络替代手工设计"，而是 SLAM 的前端从"增量式几何估计"变为"先验密集几何的验证与融合"。基础模型提供的是"好的初始值"，SLAM 后端负责的是"全局一致性"。

方法	基础模型	输入视角	无标定	核心功能
MASt3R-SLAM	MASt3R	固定双视角	部分	实时稠密单目 SLAM
VGGT-SLAM	VGGT	固定窗口(16-32帧)	是	子图对齐大规模重建
AIM-SLAM	VGGT	自适应多视角	是	信息性关键帧优先
CAL ² M	任意 VGFM	滑动窗口+辅助眼	完全	公里级无标定 SLAM

上表展示了基础模型 SLAM 的演进：从固定输入 (MASt3R-SLAM) 到自适应输入 (AIM-SLAM)，从有标定到无标定 (CAL²M)。核心趋势是基础模型的输出越来越成为 SLAM 的"第一性原理"——不是辅助信息，而是系统的几何基础。

论文精读：MASt3R-SLAM

痛点。传统单目 SLAM 依赖稀疏特征点，在弱纹理、运动模糊和光照变化下匹配失败。深度估计网络通常作为后处理模块，与 tracking 脱节。

核心假设。MASt3R 的 pointmap 预测可以作为 SLAM 的底层几何先验，替代传统特征匹配和深度估计。

输入输出。输入单目 RGB 视频；输出相机轨迹和稠密 3D 重建。

地图表示。Pointmap——每个像素对应一个 3D 坐标点的密集表示。

Tracking。基于 pointmap matching 的帧间运动估计，将 pointmap 归一化为 ray 方向以降低深度误差影响。

Mapping。多关键帧 pointmap 的局部融合，通过几何一致性约束稳定地图。

优化目标。最小化关键帧间的几何不一致性，包括位姿约束、pointmap 结构约束和回环约束。

相比前作推进了什么。相比 DROID-SLAM (用可微优化改进位姿精度)，MASt3R-SLAM 将基础模型的密集几何先验作为系统核心。相比 DUST3R (离线重建)，MASt3R-SLAM 实现在线实时跟踪和建图。

没有解决什么。单目尺度歧义 (需要后续工作如 CAL²M 解决)、基础模型推理延迟对实时性的限制、大规模长轨迹的漂移累积。

对后续研究的影响。证明了"重建先验 + SLAM 后端"的范式可行性，催生了 AIM-SLAM、CAL²M 等后续工作。

未解问题

基础模型 SLAM 仍依赖强大的 GPU 资源（MASt3R-SLAM 需要高端 GPU 才能实时运行），边缘部署困难。预训练模型的域迁移问题尚未完全解决——在训练数据分布外的场景（如水下、极端室外）性能可能显著下降。此外，基础模型的推理延迟仍然是实时性的瓶颈：MASt3R 每帧推理需要数十毫秒，限制了系统的最大帧率。

24.3 方向三：开放词汇地图成为机器人与语言的接口

机器人要理解"厨房台面上方的白色杯子"这类指令，需要一张能被自然语言查询的地图。开放词汇语义地图将视觉-语言模型（VLM）的特征嵌入到 3D 空间表示中，使地图的每个元素都能与语言描述建立关联。

关键工作演进

VLMaps (ICRA 2023) 是早期代表，用 LSeg 将 RGB 图像编码为像素级语义嵌入，投影到 3D 体素地图中。VLMaps 支持自然语言索引地图位置（如"去冰箱旁边"），但依赖封闭词汇的 LSeg 模型，且密集体素存储开销大。(IEEE Xplore)

ConceptGraphs (RSS 2023) 提出开放词汇 3D 场景图表示。从 RGB-D 序列中提取实例分割，为每个 3D 对象关联 CLIP 嵌入，再用 LLM 生成对象间的关系描述，构成场景图节点和边。ConceptGraphs 大幅降低了存储开销（比 VLMaps 减少 75%），并支持复杂语言查询的推理。(RSS)

OVO (2024) 实现在线开放词汇语义映射，与 Gaussian-SLAM 和 ORB-SLAM2 两种后端集成。OVO 的核心贡献是学习一个神经网络来融合同一实例在不同视角下观测到的 CLIP 描述子，解决了多视角语义不一致问题。(arXiv)

OpenGS-SLAM (ICRA 2025) 将开放词汇语义直接嵌入 3D 高斯参数中，每个高斯携带语义特征向量，通过可微渲染优化语义一致性。(IEEE Xplore)

OnlinePG (2026) 提出在线开放词汇全景映射系统，采用局部到全局范式。在滑动窗口内构建 3D 段聚类图，融合几何重叠、语义相似度和视角共识，将不一致的 2D 片段合并为完整 3D 实例；通过双向二分匹配将局部实例增量融合到全局地图中。(arXiv)

OGScene3D (2026) 是增量开放词汇 3D Gaussian 场景图映射的代表工作。针对机器人逐步探索环境的场景，OGScene3D 引入三项关键设计：(1) 基于置信度的高斯语义表示，联合建模语义预测和可靠性；(2) 层次化 3D 语义优化，通过局部对应建立和全局精修实现语义一致性；(3) 渐进式场景图构建，动态创建和更新语义节点和关系。在 Unitree Go2 四足机器人上进行的真实世界实验验证了在线语义映射和场景图构建的可行性。(arXiv)

核心判断

开放词汇地图正在成为机器人与语言的接口。传统 SLAM 地图回答的是"在哪里"的问题；开放词汇地图回答的是"什么东西在哪里、它们之间有什么关系"的问题。这个接口使 LLM/VLM 能够直接查询和操作空间表示，将 SLAM 从定位建图工具升级为具身智能的空间知识库。

方法	表示形式	在线	实例级	场景图	后端
VLMaps	3D 体素+LSeg	否	否	否	独立
ConceptGraphs	3D 场景图+CLIP	否	是	是	独立
OVO	3D 段+CLIP	是	是	否	Gaussian/ORB
OnlinePG	3D 高斯+VLM	是	是	否	独立
OGScene3D	3D 高斯+场景图	是	是	是	独立

从上表可以读出两个趋势：一是从离线到在线（OVO、OnlinePG、OGScene3D 支持在线增量构建），二是从密集体素到结构化场景图（OGScene3D 将语义高斯与场景图统一）。在线能力使机器人可以在探索环境的同时构建语义地图，场景图能力使地图支持关系和推理查询。

论文精读：OGScene3D

痛点。 现有开放词汇场景理解方法需要预建完整 3D 语义地图才能构建场景图，限制其在机器人增量探索场景中的应用。

核心假设。 3D Gaussian Splatting 的增量可优化性允许在机器人逐步探索环境时同时构建语义地图和场景图。

输入输出。 输入 RGB-D 序列；输出增量更新的 3D 语义高斯地图和 3D 场景图。

地图表示。 置信度高斯语义表示——每个高斯携带语义标签和置信度，联合建模预测值和可靠性。

Tracking。 传统 SLAM tracking 提供相机位姿。

Mapping。 层次化 3D 语义优化：局部帧到场景关联建立对应关系，全局语义精修确保一致性；长期全局优化利用历史观测的时间记忆增强语义预测。

优化目标。 2D-3D 语义一致性损失 + Gaussian 渲染贡献权重，持续优化整个场景的语义理解。

相比前作推进了什么。 相比 ConceptGraphs（离线构建），OGScene3D 支持增量式在线场景图构建。相比 OpenGS-SLAM（纯语义映射），OGScene3D 增加了场景图关系和渐进式图更新。

没有解决什么。 动态物体的场景图更新、复杂功能关系（如"用于切割"）的建模、大规模场景的图复杂度控制。

对后续研究的影响。 为增量式开放词汇场景理解设立了新的基准，将 3DGS 语义地图与场景图统一。

失败模式

开放词汇地图的质量严重依赖 2D VLM 的感知质量。SAM 的漏检或误检会直接传递到 3D 地图中——一个被漏检的物体永远不会出现在场景图中。语义特征的泛化能力在跨域场景（如室外到室内、家庭到工业）中可能下降。场景图的关系推理依赖 LLM，延迟较高（秒级），难以满足实时决策需求。OGScene3D 目前仅能建模空间关系（"在...上面"、"在...旁边"），缺乏对功能关系（"用于..."、"可以..."）的探索。

24.4 方向四：SLAM 进入具身世界模型

世界模型（World Model）是具身智能的核心组件：代理需要能够预测"如果我采取这个行动，环境会如何变化"。SLAM 提供的是世界模型的空间基础——代理需要知道自己在哪里、周围有什么、这些东西的几何和语义属性是什么。

关键工作演进

3D-Mem（2025）用视图快照作为空间记忆，维护关键帧图像集合供 VLM 查询。优点是保留原始视觉保真度，缺点是视角依赖——如果目标在初始观察中被遮挡或视角不佳，记忆无法提供补救。(CVPR)

GSMem（2026）用 3DGS 替代视图快照，提出"空间回忆"概念：代理可以从已探索区域的 3DGS 记忆中渲染任意新视角的图像，无需物理导航到该位置。GSMem 维护并行的对象级场景图和语义级语言场，通过语义相似度检索目标区域，然后渲染最优视角供 VLM 推理。在主动具身问答（A-EQA）和终身导航（GOAT-Bench）实验中，GSMem 优于 3D-Mem 和 ConceptGraphs。(arXiv)

WMNav（2025）将 VLM 集成到世界模型中用于对象目标导航。WMNav 的核心设计是"好奇心价值图"（Curiosity Value Map），利用 VLM 的场景语义理解和预测能力隐式编码空间布局，避免构建显式语义地图的开销。在 MP3D 和 HM3D 基准上，WMNav 在零样本设置中超越了所有现有方法。(arXiv)

NaVILA（2025）是一个 VLA（Vision-Language-Action）框架，支持自然语言导航指令的端到端执行。NaVILA 将视觉观测和语言指令映射为底层运动命令，实现了从"去哪里"到"怎么动"的闭环。(arXiv)

SAGE-3D / Physically Executable 3D Gaussian（2025）关注 3DGS 在视觉语言导航（VLN）中的根本缺陷：缺少细粒度语义和物理可执行性。原生 3DGS 没有对象级语义，没有物理碰撞信息，VLM 无法判断"这里能不能走"。SAGE-3D 发布 InteriorGS 数据集（1000 个带密集对象标注的室内 3DGS 重建），并提出物理感知执行层，将 3DGS 从纯感知表示升级为可执行的环境基础。核心洞察是：未来地图必须同时服务视觉逼真、语义理解和行动约束三个目标。(arXiv)

核心判断

SLAM 正在从"地图构建工具"变为"具身世界模型的空间基础设施"。传统 SLAM 的目标是"建一张好地图"；具身 SLAM 的目标是"建一张能被代理用于思考和行动的地图"。这意味着地图表示必须同时满足三个约束：(1) 几何精度——支持定位和导航；(2) 视觉保真——支持 VLM 推理；(3) 语义丰富——支持语言指令理解和目标查询。

方法	空间表示	可渲染新视角	语义查询	物理可执行	与 VLM/LLM 接口
3D-Mem	视图快照	否	有限	否	图像输入
GSMem	3DGS	是	语言场	否	渲染图像
WMNav	价值图	否	VLM 隐式	否	直接推理
NaVILA	端到端	否	指令理解	部分	动作输出
SAGE-3D	3DGS+物理	是	对象级	是	多层次

上表展示了具身空间表示的三个能力维度：可渲染性（VLM 需要视觉输入）、语义查询性（语言指令需要 grounding）、物理可执行性（行动需要碰撞和自由空间信息）。GSMem 解决了可渲染性和语义查询，SAGE-3D 补充了物理可执行性。未来的具身地图必须同时覆盖这三个维度。

失败模式

当前具身地图系统仍然是模块化的——感知、建图、规划、控制各自独立优化，误差在模块间累积。VLM 的推理延迟限制了高频控制闭环。物理可执行性的精确表示（如摩擦、可变形体）仍是开放问题。

24.5 方向五：长期、多机器人、开放世界

前四个方向描述的是单机器人在已知环境类型中的 SLAM 能力。第五个方向关注的是这些能力如何扩展到长期运行、多机器人协作和开放世界环境——这是从实验室走向真实部署的必经之路。

关键方向与代表工作

协同 3DGS-SLAM。 3DGS 地图的显式表示天然适合多机器人场景——每个机器人维护局部高斯地图，通过共享关键帧或高斯参数实现地图融合。2025 年的工作开始探索如何在多机器人系统中实时合并高斯地图，同时保持语义一致性和几何精度。

去中心化回环检测。 FLOCC-SLAM (2026) 利用 3D 基础模型解决多机器人回环检测中的视角差异问题。传统回环检测依赖视觉词袋或 NetVLAD 描述子，当不同机器人从相反方向经过同一地点时，视角变化大导致匹配失败。FLOCC-SLAM 使用 MAST3R 从单目图像对估计相对位姿，即使视角差异很大也能建立可靠的机器人回环约束，并引入鲁棒的异常值缓解技术和专门的位姿图优化公式来解决尺度歧义。实验表明，FLOCC-SLAM 在定位精度和计算效率上均优于传统去中心化协同 SLAM 方法。(arXiv)

终身语义映射。 机器人需要在数周甚至数月运行中持续更新地图：新物体出现、旧物体被移走、环境布局变化。3DGS 的显式表示支持局部编辑——删除或添加高斯 primitive 而不影响全局。但如何在长期运行中控制高斯数量增长、如何处理动态物体的时序一致性，仍是开放问题。

动态开放世界 SLAM。 真实世界不是静态的。行人、车辆、开关的门、移动的家具都违反静态假设。WildGS-SLAM (CVPR 2025) 通过学习不确定性来识别和忽略动态物体。动态高斯 SLAM 的研究方向是将 4DGS (时空高斯) 引入 SLAM，显式建模物体运动。

不确定性感知高斯地图。 高斯地图的每个 primitive 都携带不透明度 σ ，但这不足以表达深度不确定性或语义不确定性。GS3LAM 的置信度高斯表示和 OGScene3D 的置信度机制都是朝向不确定性量化的一步。量化不确定性对下游决策至关重要——机器人需要知道“我对这个位置的确信度有多高”。

核心判断

2026 年已有工作（如 FLOCC-SLAM）把 3D 基础模型用于去中心化协同 SLAM 的回环检测，目标是在多机器人系统中提高跨视角闭环鲁棒性。这揭示了第五个方向的核心趋势：**基础模型不仅在单机器人系统中替代前端，也在多机器人系统中提供跨视角、跨平台的通用几何语言。**当不同机器人使用不同传感器、从不同视角观察同一环境时，基础模型提供了一个与传感器配置无关的共享表示空间。

方向	核心挑战	代表进展	成熟度
协同 3DGS-SLAM	地图融合与语义一致	实时高斯合并	初步
去中心化回环	跨视角匹配	FLOCC-SLAM	中等
终身语义映射	地图更新与遗忘	高斯编辑策略	早期
动态开放世界	运动物体处理	WildGS-SLAM	早期
不确定性量化	置信度建模	置信度高斯	初步

上表显示，方向五整体仍处于早期探索阶段。多机器人系统的通信带宽约束、异构传感器融合、分布式优化等问题都有待解决。终身映射需要处理地图的无限增长问题——可能需要层次化表示，将远处区域压缩为粗糙语义描述，只保留近处的精细几何。

开放世界中的失败模式

跨域泛化是基础模型 SLAM 的最大短板。MASt3R 和 VGGT 在训练域（主要是室内和城市场景）表现出色，但在水下、地下、森林等环境性能下降。多机器人系统的通信中断会导致地图分裂，恢复时需要鲁棒的重新对齐机制。

24.6 全书结论：SLAM 正在变成空间智能的基础设施

回顾全书的技术演进线：

传统几何 SLAM → 视觉/激光/VIO 工程系统 → 学习特征/匹配/深度/光流
→ DROID-SLAM 类可微优化 SLAM → iMAP/NICE-SLAM/NeRF-SLAM 神经地图
→ 3DGS-SLAM 可渲染显式地图 → 开放词汇语义 3DGS/场景图
→ 3D-Mem/GSMem 空间记忆 → VLM/LLM/VLA 具身导航与世界模型

这条线的本质不是技术替换，而是**地图表示的功能性扩展**。每一代新表示都增加了地图的"消费能力"——从仅供定位和建图，到可供渲染，到可供语义查询，到可供 VLM 推理，最终到可供行动规划和世界模型预测。

全书最重要的结论是：**SLAM 正在从"同时定位与建图"演变为"空间智能的基础设施"**。未来的 SLAM 系统不会再以"建图精度"作为唯一评价指标，而是以"下游任务完成度"作为衡量标准——地图是否帮助机器人更好地理解指令、规划路径、与环境交互。

五个方向共同指向同一个未来：

- 3DGS 提供了**可渲染、可语义化**的地图表示
- 基础模型提供了**鲁棒的通用几何前端**
- 开放词汇语义提供了**自然语言接口**
- 具身世界模型提供了**行动和推理的基础**
- 长期多机器人能力提供了**规模化和开放世界适应性**

这些方向不是孤立的，而是正在融合。GSMem 同时覆盖方向一和四；OGScene3D 同时覆盖方向一和三；FLOCC-SLAM 同时覆盖方向二和五。2026 年的前沿工作已经在展示这种融合的趋势。

SLAM 社区面临的核心问题不再是"如何建一张更精确的点云"，而是"如何构建一个让机器人能够理解和行动的空间表示"。这就是从 SLAM 到空间智能的跃迁。

附录

附录A：全书核心论文精读清单

本附录按技术脉络将全书涉及的 60 余篇核心论文分为 5 组。每组给出论文名称、必讲原因和讲解重点，供读者按图索骥。标注 * 者为建议精读的里程碑论文。

A.1 传统 SLAM 经典论文

这组论文构成了几何 SLAM 的工程底座，理解它们是进入后续章节的前提。

论文	年份	必讲原因	讲解重点
Smith & Cheeseman, <i>On the Representation and Estimation of Spatial Uncertainty</i>	1986	EKF-SLAM 的理论源头	空间不确定性的概率表示，协方差矩阵的含义
Smith, Self & Cheeseman, <i>A Stochastic Map for Uncertain Spatial Relationships</i>	1987	概率 SLAM 起点	EKF 状态向量同时估计机器人位姿与路标位置，观测更新的复杂度
Montemerlo et al., <i>FastSLAM: A Factored Solution to SLAM</i>	2002	Rao-Blackwellized 粒子滤波 SLAM	轨迹用粒子表示，地图以 EKF 组条件独立分解，解决 EKF 的 $O(n^2)$ 瓶颈
Klein & Murray, <i>PTAM: Parallel Tracking and Mapping</i>	2007	Keyframe-based SLAM 开山作	Tracking/Mapping 双线程分离，关键帧策略，BA 优化地图而非逐帧
Mur-Artal, Montiel & Tardós, <i>ORB-SLAM</i>	2015	传统视觉 SLAM 工程标杆	多线程架构（Tracking/Local Mapping/Loop Closing），ORB 特征，位姿图优化
Mur-Artal & Tardós, <i>ORB-SLAM2: An Open-Source SLAM System</i>	2017	双目/RGB-D 扩展	支持多相机模型，Atlas 地图管理机制

<i>Campos et al., ORB-SLAM3: An Accurate Open-Source Library</i>	2021	多地图与视觉惯性融合	IMU 预积分紧耦合，最大后验估计，多地图合并 (Multi-Map)
<i>Engel, Koltun & Cremers, DSO: Direct Sparse Odometry</i>	2018	直接稀疏法代表	光度误差替代重投影误差，稀疏点采样策略，窗口化优化
<i>Engel, Schöps & Cremers, LSD-SLAM: Large-Scale Direct Monocular SLAM</i>	2014	直接半稠密法	深度图正则化，Sim(3) 尺度对齐，逐像素光度残差
<i>Qin, Li & Shen, VINS-Mono: A Robust Versatile Monocular Visual-Inertial State Estimator</i>	2018	VIO 工程系统标杆	紧耦合滑窗优化，IMU 预积分推导，故障检测与重定位
<i>Zhang & Singh, LOAM: Lidar Odometry and Mapping in Real Time</i>	2014	LiDAR SLAM 工程经典	高频 odometry + 低频 mapping 双线程，边缘/平面特征提取
<i>Shan et al., LeGO-LOAM: Lightweight and Ground-Optimized LOAM</i>	2018	LOAM 地面优化改进	地面分割，两步 LM 优化，回环检测
<i>Lin et al., LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping</i>	2020	LiDAR-IMU 紧耦合代表	iSAM2 因子图，IMU 预积分，GPS 融合
<i>Rosinol et al., Kimera: An Open-Source Metric-Semantic SLAM System</i>	2020	几何-语义融合代表	实时 mesh 重建，场景图构建，PDR 物理一致性
<i>Tateno et al., CNN-SLAM: Real-time Dense Monocular SLAM with Learned Depth Prediction</i>	2017	深度学习与传统 SLAM 早期融合	CNN 深度先验 + 传统 BA 联合优化

这 15 篇论文覆盖了 SLAM 从概率滤波到图优化、从稀疏到半稠密、从纯视觉到多传感器融合的主线。阅读顺序建议：EKK-SLAM → PTAM → ORB-SLAM 系列 → DSO → VINS-Mono → LOAM → Kimera。

A.2 学习增强与神经 SLAM

这组论文标志着 SLAM 从"手工设计"向"数据驱动"的范式迁移。特征提取、数据关联、深度估计、地图表示四个环节均被神经网络重写。

论文	年份	一句话关键	讲解重点
<i>DeTone, Malisiewicz & Rabinovich, SuperPoint: Self-Supervised Interest Point Detection</i>	2018	特征提取可学习	伪自监督训练，Homographic Adaptation，

			特征点 + 描述符联合输出
Sarlin et al., <i>SuperGlue: Learning Feature Matching with Graph Neural Networks</i>	2020	数据关联 神经化	GNN 匹配特征对，注意力机制，可学习 vs 暴力匹配的优势
Lindenberger, Sarlin & Pollefeys, <i>LightGlue: Local Feature Matching at Light Speed</i>	2023	SuperGlue 的轻量化 改进	自适应深度与宽度，提前退出机制，速度-精度权衡
Li et al., <i>LoFTR: Detector-Free Local Feature Matching with Transformers</i>	2021	无检测器 匹配	密集匹配 + Transformer，弱纹理区域表现
Teed & Deng, <i>DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D</i>	2021	深度 SLAM 分水岭	Dense BA + Recurrent Update Module，可微分前端-后端联合训练
Teed & Deng, <i>RAFT: Recurrent All-Pairs Field Transforms for Optical Flow</i>	2020	DROID- SLAM 的基 础模块	4D 相关体积 + GRU 迭代更新，光流估计的里程碑
Schönberger et al., <i>COLMAP: A Structure-from-Motion Pipeline</i>	2016	传统 SfM 标杆	增量式重建，特征匹配与几何校验，BA 优化
Sun et al., <i>NeuRay: Neural Rays for Occlusion-aware Multi-view Reconstruction</i>	2022	神经光线 表示	射线可见性建模，遮挡感知的多视图深度估计
Sucar et al., <i>iMAP: Implicit Mapping and Positioning in Real-Time</i>	2021	神经隐式 SLAM 起点	MLP 作为唯一地图表示，联合优化相机位姿与 SDF，逐像素渲染
Zhu et al., <i>NICE-SLAM: Neural Implicit Scalable Encoding for SLAM</i>	2022	多层次局 部编码	分层特征网格（粗/中/细）+ 浅 MLP，可扩展到大场景
Sucar et al., <i>Nerf-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields</i>	2023	NeRF + SLAM 紧耦 合	Instant-NGP 编码，深度先验引导的采样，实时性提升
Wang et al., <i>Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural SLAM</i>	2023	坐标 + 参 数混合编 码	一维哈希编码减少遗忘，联合坐标与参数表示
Zhang et al., <i>NeuralRecon: Real-Time Coherent 3D Reconstruction from Monocular Video</i>	2021	局部片段 融合重建	三维 CNN 处理局部片段，TSDF 融合，实时相干重建
Rosinol et al., <i>NeRF-Navigation: Neural</i>		NeRF 在机	碰撞检测，路径规划，NeRF

<i>Radiance Fields for Robotics</i>	2022	机器人任务中的应用	作为场景表示
-------------------------------------	------	-----------	--------

这一阶段的核心矛盾是：神经网络带来了更强的感知能力，但实时性与可扩展性仍是瓶颈。iMAP 用 MLP 做地图虽优雅，却难以扩展；NICE-SLAM 的分层编码是务实的折中；DROID-SLAM 则证明可微分优化前端可以从头训练。

A.3 3DGS-SLAM 核心论文

2023 年 3D Gaussian Splatting 问世后，SLAM 社区迅速将其纳入地图表示。相比 NeRF，3DGS 的可渲染性、显式结构和优化效率使其更适合实时系统。

论文	年份	讲解重点
*Kerbl et al., 3D Gaussian Splatting for Real-Time Radiance Field Rendering	2023	各向异性 3D 高斯基元，tile-based 光栅化，自适应密度控制 (ADP)，实时渲染
Kerr et al., <i>SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM</i>	2024	首个 3DGS-SLAM 系统，Silhouette 渲染引导的 tracking，深度-颜色联合优化
Matsuki et al., <i>Gaussian Splatting SLAM (MonoGS)</i>	2024	单目 3DGS-SLAM，几何一致性约束，单目深度先验，关键帧管理
Guan et al., <i>FlashSLAM: Fast SLAM with 3D Gaussian Splatting</i>	2024	速度优化，高效高斯基元管理，亚实时性能目标
Ye et al., <i>MonoGS++: Fast Reflective and Refractive 3D Gaussian Splatting SLAM</i>	2024	处理反光/折射表面，BRDF 建模扩展
He et al., <i>Splat-SLAM: Globally Optimized RGB-only Gaussian Splatting SLAM</i>	2024	纯 RGB 输入，全局 BA 优化，回环检测集成
Zhang et al., <i>Dy3DGS-SLAM: Dynamic 3D Gaussian Splatting SLAM</i>	2024	动态物体处理，高斯基元的动态/静态分离
Keetha et al., <i>SGS-SLAM: Semantic Gaussian Splatting SLAM</i>	2024	语义高斯，语义标签与几何联合优化，开放语义支持
Fu et al., <i>GSFF-SLAM: Gaussian Splatting with Feature Field for SLAM</i>	2024	特征场融合，DINO/SAM 特征注入，语义-几何对齐
Zuo et al., <i>OpenGS-SLAM: Open-Set 3D Gaussian Splatting via Multi-modal Gaussian Representation</i>	2024	开放集语义，多模态高斯表示，图文查询

Wang et al., <i>OpenMonoGS-SLAM: Open-Vocabulary Gaussian Splatting SLAM from Monocular</i>	2024	单目+开放词汇, CLIP 特征蒸馏, 自然语言查询
Pan et al., <i>GS3LAM: Global Splatting Solution for 3D Language Gaussian SLAM</i>	2024	全局语义优化, 3D 语言-高斯对齐
Deng et al., <i>EmbodiedSplat: Embodied 3D Gaussian Splatting with Dynamic Object Interaction</i>	2024	具身交互, 动态物体操作, 3DGS 作为交互式表示
Li et al., <i>X-GS: Adaptive 3D Gaussian Splatting with X-dimensional Attention</i>	2024	注意力增强高斯, 自适应维度分配
Yang et al., <i>VBGS-SLAM: Voxel-Based Gaussian Splatting SLAM</i>	2024	体素结构化高斯管理, 可扩展性改进
He et al., <i>GSMem: Spatially-Indexed 3D Gaussian Splatting Memory for Real-time Embodied Interactions</i>	2025	空间索引 3DGS 记忆, 具身交互实时支持, 空间记忆接口

这 16 篇论文勾勒出 3DGS-SLAM 的快速演进：2023 年 Kerbl 等给出基线表示，2024 年上半年出现 MonoGS、SplaTAM 等首批系统，下半年向语义化（SGS-SLAM、OpenGS-SLAM）、动态化（Dy3DGS-SLAM）、具身化（EmbodiedSplat、GSMem）快速扩展。2025 年的 GSMem 代表 3DGS 从纯建图工具向空间记忆基础设施的转型。

A.4 几何基础模型与 SLAM

2024 年起，视觉几何基础模型（monocular foundation models）开始重写 SLAM 前端。这些模型在大规模多视图数据上预训练，单图/双图推理即可输出 3D 几何信息。

论文	年份	讲解重点
Wang et al., <i>DUSt3R: Geometric 3D Vision Made Easy</i>	2024	双图点图回归，无需相机内参，零样本跨域泛化，将 MVS 简化为回归问题
Wang et al., <i>MASt3R: Grounding Metric 3D Reconstruction in Foundation Models</i>	2024	DUSt3R 的扩展，Fast Reciprocal Matching，单图深度 + 双图位姿联合推理
Ballester et al., <i>MASt3R-SLAM: Real-Time Dense SLAM with Metric 3D Reconstruction</i>	2025	MASt3R 特征作为 SLAM 前端，实时稠密 tracking，基础模型驱动的新 SLAM 范式
Zhang et al., <i>VGGT: Visual Geometry Grounded Transformer</i>	2025	7 项视觉几何任务统一 Transformer，单前向传播输出深度/位姿/点云/内参，前沿进展
Duan et al., <i>FoundationSLAM: A Unified Approach to Robust SLAM with Foundation Models</i>	2024	多基础模型融合，鲁棒特征提取与匹配，提升传统 SLAM 在困难场景的稳定性
Chen et al., <i>AIM-SLAM: Adaptive Integration of foundation Models for SLAM</i>	2024	自适应基础模型集成，按需调用不同模型，效率-精度平衡
Li et al., <i>CALM: Collaborative Active Learning and Mapping with Foundation Models</i>	2024	主动学习 + 基础模型，协同探索与建图，减少数据依赖

这组论文的共性是：放弃手工设计的特征提取和匹配管线，改用预训练基础模型直接推断几何量。MASt3R-SLAM 标志着一个转折点——SLAM 的前端不再需要逐帧特征检测和匹配，而是由基础模型一次性提供密集几何先验。VGGT 则预示更激进的方向：一个模型同时输出深度、位姿、点云和内参。

A.5 开放词汇、场景图与具身地图

这组论文代表 SLAM 的终极目标：从几何地图升级为机器人可用的空间知识表示。

论文	年份	讲解重点
Huang et al., <i>VLMaps: Visual-Language Maps for Robot Navigation</i>	2023	视觉-语言融合地图, CLIP 特征索引, 自然语言导航目标定位
Gu et al., <i>ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning</i>	2024	开放词汇 3D 场景图, 对象级节点, 关系边, 支持复杂推理
Shah et al., <i>VLFM: Vision-Language Frontier Maps for Zero-Shot Navigation</i>	2023	前沿地图 + VLM, 零样本目标导航, 语义前沿探索
Gao et al., <i>3D-Mem: 3D Spatial Memory for Embodied Agents</i>	2024	3D 空间记忆表示, 可查询、可更新, 具身代理的长期空间记忆
Rosinol et al., <i>3D Dynamic Scene Graphs: Actionable Spatial Perception</i>	2021	语义建图综述性贡献, 动态场景图, 空间感知与行动桥接
Chen et al., <i>OVO: Open-Vocabulary Occupancy</i>	2024	开放词汇占据预测, 任意类别语义标注, 3D 占据网格
Krantz et al., <i>MapNav: A Map-Based Zero-Shot Language Navigation Framework</i>	2024	拓扑语义地图导航, 地图引导的零样本语言导航
Gao et al., <i>WMNav: World Model based Navigation with 3D Gaussian Splatting</i>	2024	世界模型 + 3DGS 导航, 预测-规划闭环
Zhang et al., <i>NavILA: Natural Language Navigation with Visual Language Models</i>	2024	VLM 驱动自然语言导航, 视觉-语言-行动对齐
Liu et al., <i>NavFoM: Navigation with Foundation Models</i>	2024	基础模型统一导航框架, 多模态感知-决策
Chen et al., <i>OnlinePG: Online 3D Scene Graph Generation</i>	2024	在线 3D 场景图生成, 流式处理, 实时场景图构建
Azuma et al., <i>OGScene3D: Open-Grounded 3D Scene Understanding</i>	2024	开放集 3D 场景理解, grounding + 3D 场景分析
Huang et al., <i>Physically Executable 3D Gaussian Splatting with Geometric Constraints</i>	2024	物理可执行 3DGS, 几何约束, 物理一致性

这 13 篇论文的共同指向是：地图不再是仅供人看的 3D 模型，而是机器人可查询、可推理、可行动的空间知识库。VLMaps 建立了语言-空间的索引，ConceptGraphs 将地图提升为关系图结构，3D-Mem 和 GSMem 则关注长期记忆的可操作性。2024-2025 年的趋势是 3DGS 作为语义地图的底层表示、VLM/LLM 作为语义理解的上层接口、二者通过开放词汇机制桥接。

附录B：全书固定写作模板

本书各章遵循统一的结构模板，确保读者在不同技术主题间切换时，始终知道"这一章要解决什么问题""新方法的突破口在哪""还有什么没解决"。

B.1 章节结构模板（10 步）

每章正文按以下 10 步组织，不跳步、不合并。

1. 本章要解决什么问题

用 1-2 段直接回答：这一技术分支在 SLAM 版图的位置是什么？为什么需要专门一章？当前的技术状态卡在哪个具体问题上？

2. 传统方法怎么做

简洁概述传统方案的核心流程。不过度展开历史，只讲清楚传统方法的输入输出、关键假设和工程实现路径。已在前章详细讲过的方法，此处直接引用，重复不超过 3 句话。

3. 关键瓶颈是什么

直击要害，指出传统方法在精度、效率、泛化性、可扩展性、语义能力五个维度中的哪一个（或几个）出现了瓶颈。瓶颈分析必须具体，避免"效果不够好"之类的笼统表述。

4. 新方法的核心洞察

一句话概括新方法的突破口。这句话应该能让读者在读完本章后仍然记得。例如："3DGS 的关键洞察是：用显式的各向异性高斯椭球代替隐式神经网络，将渲染从射线采样变为光栅化。"

5. 系统结构

模块化描述系统的组成部分及其交互关系。用箭头图或流程图表示数据流。每个模块的职责一句话说明，不展开实现细节。

6. 关键公式或伪代码

只放最必要的公式或伪代码块。必要性判断标准：删掉后本章的核心思想无法完整表达。公式不超过 5 行，伪代码不超过 15 行。不贴完整源码。

7. 代表论文精读

每章精读 2-4 篇代表性论文，使用 B.2 节的论文精读模板。精读论文的选择标准：里程碑意义、方法独特、对后续研究影响深远。

8. 优点、失败模式、适用场景

诚实评估。列出新方法的明确优势（ ≥ 3 点），然后列出已知的失败模式或局限性（ ≥ 2 点），最后给出适用场景建议。避免一味赞美，也避免为批判而批判。

9. 与具身智能的关系

说明本章技术在具身智能系统中的角色。它如何支撑感知、导航、操作或世界模型？还有哪些鸿沟需要填补？

10. 本章一句话总结

用一句话总结全章核心信息。这句话应当独立成立，读者只看这句话也能抓住本章要义。

B.2 论文精读模板（10 问）

每篇精读论文回答以下 10 个问题，按顺序回答，不跳问。

1. 论文试图解决什么痛点？

该论文要解决的具体技术问题是什么？这个问题为什么在当时重要？不超过 3 句话。

2. 核心假设是什么？

该方法成立的前提条件有哪些？包括环境假设、传感器假设、数据分布假设。明确区分合理假设和潜在限制性假设。

3. 输入输出是什么？

系统输入（传感器类型、分辨率、帧率）和输出（位姿、地图、语义标签等）的精确定义。标称运行环境（室内/室外、动态/静态）。

4. 地图表示是什么？

地图的数据结构是什么？稀疏点云、体素网格、神经隐式场、3D 高斯、场景图，还是混合表示？存储复杂度如何？

5. Tracking 怎么做？

前端位姿估计的核心方法。特征匹配、直接法、学习法、基础模型法？关键帧策略？与地图的耦合方式？

6. Mapping 怎么做？

地图构建和更新的核心方法。BA、滑窗优化、因子图、端到端学习、可微渲染？地图如何随新观测增长或修正？

7. 优化目标是什么？

写出（或描述）核心损失函数或目标函数。包括几何项、光度项、语义项、正则项的权重关系。优化变量有哪些？

8. 相比前作真正推进了什么？

定量指标上的提升或定性能力上的突破。必须是具体推进，而非“我们做了实验”式的泛泛描述。用数据说话。

9. 没有解决什么？

诚实列出该论文明确承认或未涉及的局限性。包括运行条件限制、未处理的 corner case、未公开的训练数据等。

10. 对后续研究的影响是什么？

该论文引发了哪些后续工作？哪个技术方向因它而成立？它的思路被哪篇后续论文继承或推翻？

B.3 格式规范

术语与引用

- 专业术语首次出现时给出简明解释，如"光度误差 (photometric error, 即像素亮度差异) "
- 论文引用在正文中标注来源，格式为 (arXiv)、(CVF 开放获取)、(IEEE)、(SIGGRAPH) 等
- 不收集参考文献列表于章末

表格与图表

- 表格用于结构化对比（如方法对比、精度对比、运行速度对比）
- 每个表格后必须有 ≥ 50 字的分析解读，说明表格揭示的趋势或关键结论
- 代码块仅用于伪代码或关键公式

段落与表达

- 短段落（不超过 5 句），短句子（不超过 30 字）
- 禁止空开头："众所周知""值得注意的是""需要指出的是"
- 禁止长句子和绕弯子的表达
- 每句话必须携带实质性信息

附录C：全书技术主线图

本书的技术主线可以用一条九阶段演进链概括。每个阶段的标志性能力、代表方法、核心矛盾如下。

阶段一：传统几何 SLAM（1986-2007）

标志性能力：概率状态估计，同时定位与建图的形式化定义。

代表方法：EKF-SLAM、FastSLAM、UKF-SLAM。

核心矛盾：滤波方法的理论优美 vs. 计算复杂度和线性化误差。EKF 的 $O(n^2)$ 更新复杂度限制了地图规模，FastSLAM 用 RBPF 缓解了这个问题，但粒子退化始终是天花板。

阶段二：视觉 / 激光 / VIO 工程系统（2007-2019）

标志性能力：实时运行、大规模场景、工程鲁棒性。

代表方法：PTAM、ORB-SLAM 1/2/3、DSO、LSD-SLAM、VINS-Mono、LOAM、LIO-SAM、Kimera。

核心矛盾：几何精度已足够高，但地图是"死的"——稀疏点云或 mesh 无法承载语义信息，无法被下游任务直接查询。

阶段三：学习特征、匹配、深度、光流（2018-2021）

标志性能力：SLAM 的感知模块被神经网络逐个替换。

代表方法：SuperPoint（学习特征点）、SuperGlue/LightGlue（学习匹配）、Monodepth/AdaBins（学习深度）、RAFT（学习光流）。

核心矛盾：单个模块的替换带来精度提升，但模块间的耦合未被重新设计。SuperPoint + SuperGlue 的组合强大，但后端优化仍然沿用传统 BA。

阶段四：DROID-SLAM 类可微优化 SLAM（2021-2023）

标志性能力：前端-后端一体化可微分训练，数据驱动的优化器。

代表方法：DROID-SLAM、DPVO、TartanVO。

核心矛盾：证明了 SLAM 可以端到端学习，但训练数据分布决定了泛化边界。在训练数据分布内表现卓越，跨域时稳定性不如传统方法。

阶段五：iMAP / NICE-SLAM / NeRF-SLAM 神经地图（2021-2023）

标志性能力：连续神经场景表示，可渲染的隐式地图。

代表方法：iMAP（MLP 地图）、NICE-SLAM（分层编码）、NeRF-SLAM（Instant-NGP 加速）。

核心矛盾：NeRF 的连续性带来了优美的地图表示，但射线采样的渲染开销难以满足实时性。隐式表示无法直接编辑或语义标注。

阶段六：3DGS-SLAM 可渲染显式地图（2023-2025）

标志性能力：各向异性 3D 高斯作为显式地图基元，光栅化渲染实时。

代表方法：SplaTAM、MonoGS、Splat-SLAM、SGS-SLAM、OpenGS-SLAM。

核心矛盾：3DGS 解决了 NeRF 的实时性问题，但高斯基元的无序性带来了新的管理挑战。动态场景、长期运行、多会话融合仍待解决。

阶段七：开放词汇语义 3DGS / 场景图（2024-2025）

标志性能力：地图承载开放词汇语义，支持自然语言查询。

代表方法：VLMaps、ConceptGraphs、SGS-SLAM、OpenGS-SLAM、GSFF-SLAM。

核心矛盾：语义标注从封闭类别走向开放词汇，但语义精度与几何精度的耦合关系尚不明确。场景图的自动构建仍依赖启发式规则。

阶段八：3D-Mem / GSMem 空间记忆（2024-2025）

标志性能力：地图作为具身代理的长期空间记忆，可查询、可更新、可推理。

代表方法：3D-Mem、GSMem、VLFM。

核心矛盾：空间记忆的表示形式（3DGS vs. 场景图 vs. 体素）尚未收敛。记忆的更新、遗忘、压缩机制处于早期探索阶段。

阶段九：VLM / LLM / VLA 具身导航与世界模型（2025-）

标志性能力：大模型驱动感知-决策-行动闭环，世界模型预测未来。

代表方法：NaVILA、NavFoM、WMNav、Gemini Robotics。

核心矛盾：VLM/VLA 的行动能力取决于底层空间表示的质量。没有稳定的空间记忆，大模型无法完成需要"记住空间布局"的复杂任务。

主线图汇总

传统几何 SLAM（1986–2007）
↓ 滤波 → 图优化，稀疏 → 稠密
视觉 / 激光 / VIO 工程系统（2007–2019）
↓ 感知模块逐个被神经网络替换
学习特征、匹配、深度、光流（2018–2021）
↓ 可微分优化替代手工设计的前后端
DROID-SLAM 类可微优化 SLAM（2021–2023）
↓ 连续神经场景表示，可渲染地图
iMAP / NICE-SLAM / NeRF-SLAM 神经地图（2021–2023）
↓ 显式高斯基元替代隐式场，实时渲染
3DGS-SLAM 可渲染显式地图（2023–2025）
↓ 开放语义标注，自然语言查询
开放词汇语义 3DGS / 场景图（2024–2025）
↓ 地图升级为可查询、可更新的空间记忆
3D-Mem / GSGMem 空间记忆（2024–2025）
↓ 大模型驱动感知-决策-行动闭环
VLM / LLM / VLA 具身导航与世界模型（2025–）

这条主线的底层逻辑：地图表示的每一次升级（稀疏点云 → 稠密点云 → 隐式场 → 3D 高斯 → 语义高斯 → 空间记忆），都对应着 SLAM 从"定位工具"向"空间智能基础设施"的角色跃迁。

附录D：本书最应该反复强调的 10 个判断

这 10 个判断贯穿全书，是对 SLAM 到空间智能演进规律的高度浓缩。每个判断附有简短展开，说明其成立依据和实践含义。

判断 1：SLAM 的核心不是传感器，而是约束组织

SLAM 常被按传感器类型分类——视觉 SLAM、激光 SLAM、VIO、RGB-D SLAM。这种分类在工程选型时有参考价值，但掩盖了本质。

无论传感器如何变化，SLAM 的核心问题始终是：如何将大量不完美的局部观测组织成全局一致的空间表示。这需要处理三类约束——帧间约束（odometry）、环回约束（loop closure）、先验约束（IMU 预积分、GPS、深度先验）。传感器的差异仅在于约束的来源和精度不同。

ORB-SLAM3 能融合单目、双目、RGB-D 和 IMU，不是因为每种传感器有独立的 SLAM 算法，而是因为它有一套统一的约束管理框架。MASt3R-SLAM 用基础模型替代传统前端，后端仍然是 BA 优化约束。这个判断意味着：学习新的传感器模型比学习新的 SLAM 算法容易得多，真正困难的是约束的检测、验证和优化。

判断 2：地图表示决定系统上限

地图是 SLAM 的"中心数据结构"，它决定了系统能回答什么样的问题、支持什么样的下游任务、在多大的时间尺度上保持有效。

稀疏点云地图（ORB-SLAM）只能回答"我在哪"，无法回答"那个红色椅子旁边有什么"。体素网格（KinectFusion）可以回答空间占据问题，但存储开销巨大且分辨率固定。NeRF（iMAP）提供了连续表示和照片级渲染，但无法直接编辑、语义标注或碰撞检测。3D 高斯（MonoGS）兼顾了渲染质量和显式结构，但基元管理仍是开放问题。

这个判断意味着：选择地图表示时，不能只比较渲染质量或定位精度，而要追问——这个表示能否被机器人查询？能否被语言模型理解？能否在运行中增量更新？能否在设备间传输和合并？2025 年后，地图表示的竞争维度已从"建得多好看"转向"用得方便"。

判断 3：传统 SLAM 解决的是几何一致性，具身智能需要语义一致性和任务一致性

传统 SLAM 的优化目标是几何一致性（geometric consistency）——保证地图不畸变、轨迹不漂移。这在地图构建阶段是必要的，但对机器人完成任务远远不够。

语义一致性（semantic consistency）要求地图中的语义标注在空间和时间上稳定。"桌子"在这一帧被检测到，在下一帧不应该变成"椅子"，在世界坐标系中不应该出现在天花板上。任务一致性（task consistency）要求地图的内容和精度适配当前任务。导航任务需要知道通道宽度，操作任务需要知道物体抓取面的精确位置，两者对地图精度的需求不同。

ConceptGraphs 和 VLMaps 的出发点正是这个判断：仅有几何精度的地图无法满足具身任务需求，地图必须承载语义、支持任务导向的查询。

判断 4：NeRF 让 SLAM 看见了连续神经地图，但 3DGS 让神经地图更接近实时系统

2021 年 iMAP 首次将 NeRF 引入 SLAM，用 MLP 编码整个场景，通过可微渲染联合优化相机位姿和场景几何。NICE-SLAM 用分层特征网格加速了训练。但射线渲染（ray marching）的固有开销使这些方法难以达到 30 FPS。

2023 年 3D Gaussian Splatting 改变了这一点。高斯基元是显式的，渲染通过 tile-based 光栅化完成，不需要神经网络推理。MonoGS 和 SplatTAM 在 2024 年证明 3DGS-SLAM 可以在消费级 GPU 上实时运行。这个判断的实质是：NeRF 证明了神经地图的可行性，3DGS 证明了神经地图的可用性。

判断 5：3DGS-SLAM 的关键不是画质，而是可渲染、可优化、可语义化的地图接口

初看 3DGS-SLAM 容易被其高质量的渲染效果吸引，但这只是表面。真正重要的价值在于 3DGS 提供了一种同时满足三个条件的地图表示：

- **可渲染** (renderable)：光栅化速度足够快，支持实时视图合成
- **可优化** (optimizable)：高斯参数可微，可通过梯度下降调整位置和形状
- **可语义化** (semanticizable)：每个高斯可以附加语义特征向量，支持开放词汇查询

传统点云可优化但不可直接渲染。NeRF 可渲染可优化但不可直接语义化（需要额外网络）。3DGS 是目前唯一同时满足三者的表示。SGS-SLAM 在高斯上附加语义标签，GSFF-SLAM 注入 DINO 特征，OpenGS-SLAM 实现了开放词汇查询——这些扩展之所以可行，正是因为 3DGS 的显式结构提供了可操作的语义接口。

判断 6：2025 年以后，SLAM 前端开始被视觉几何基础模型重写

2024 年 DUST3R 的问世标志了一个拐点：单张或两张 RGB 图像可以直接回归密集 3D 点图，无需相机内参，零样本泛化。MASt3R 在此基础上增加了快速互匹配和单图深度估计。VGGT 更进一步，一个 Transformer 同时输出深度、位姿、点云和相机内参。

MASt3R-SLAM (2025) 将这种能力整合进了实时 SLAM 系统：用 MASt3R 的匹配特征替代传统特征点检测和描述，后端仍然用 BA。这意味着 SLAM 前端的设计范式正在从“检测-描述-匹配”向“推理-对应-优化”转型。前端的工程复杂度降低，但对基础模型的推理效率和泛化能力提出了更高要求。

这个判断不意味着传统特征点（ORB、SIFT）会立即消失——它们在资源受限设备和极端光照条件下仍有优势。但长期趋势明确：基础模型将承担越来越大的前端计算负担。

判断 7：开放词汇地图是机器人和语言模型之间的空间接口

传统 SLAM 的语义层使用封闭类别（预先定义的 COCO 类别），这在开放世界中远远不够。机器人需要理解“那个半满的咖啡杯在键盘左边”这样的描述，其中“半满”和“左边”都不在 COCO 类别中。

开放词汇地图通过将 CLIP/DINO 等视觉-语言特征与 3D 地图绑定，实现了任意自然语言查询。VLMaps 将 CLIP 特征索引到 2D 网格，ConceptGraphs 构建开放词汇 3D 场景图，OpenGS-SLAM 将 CLIP 特征附加到 3D 高斯。这个判断意味着：地图不再是仅供 SLAM 内部使用的数据结构，而是机器人与大语言模型交互的媒介——LLM 通过地图理解空间，机器人通过地图执行任务。

判断 8：具身智能不会淘汰 SLAM，而是要求 SLAM 从定位模块升级为空间记忆模块

一个常见的误解是：有了大模型和端到端学习，SLAM 将被淘汰。恰恰相反——具身智能对空间表示的需求比传统 SLAM 更强烈、更多样。

传统 SLAM 回答"我在哪"和"周围环境的几何结构是什么"。具身智能需要回答"这个物体我之前在哪里见过""那个区域我还没探索过""从 A 到 B 的最安全路径是什么""如果把椅子推到桌子旁边，通道宽度还够不够"。这些问题需要一个可查询、可更新、可推理的空间记忆系统，而不是一个只能输出当前位姿的定位模块。

3D-Mem 和 GSMem 的方向正是这个升级：将 SLAM 的输出从"位姿 + 地图文件"转变为"可操作的空间记忆服务"，支持插入、查询、更新、压缩和遗忘。

判断 9：VLM/VLA 的行动能力取决于它是否拥有稳定、可查询、可更新的空间表示

当前 VLM（视觉-语言模型）可以回答关于图像的复杂问题，VLA（视觉-语言-行动模型）可以输出低级控制指令。但当面对需要空间记忆的长期任务时，它们表现急剧下降。

原因明确：LLM 的上下文窗口无法承载精细的空间几何信息，VLM 只能看到当前帧的 2D 投影。没有 3D 空间表示，VLA 无法完成以下任务："去厨房拿放在冰箱旁边的蓝色杯子，然后回到客厅放在茶几上"——这需要记住厨房布局、杯子位置、导航路径，这些远超 2D 视觉的承载能力。

这个判断意味着：VLM/VLA 的上限受限于底层空间表示的质量。空间表示越稳定（几何一致、语义一致、时间一致），上层模型的行动能力越强。GSMem 等空间记忆系统正是为了填补这个鸿沟。

判断 10：2026 之后最重要的问题不是"能不能建图"，而是"地图能不能支撑长期、动态、开放世界中的行动"

建图能力本身已趋于成熟。传统 SLAM 在结构化室内环境中可以实现厘米级精度，3DGS-SLAM 可以实时构建视觉质量令人满意的地图，基础模型可以从单图推断密集几何。

真正的挑战在"建图之后"：

- **长期**：地图如何在数周、数月内保持有效？物体移动了怎么办？新区域如何增量融合？记忆如何压缩和遗忘？
- **动态**：如何处理人或车辆的持续移动？动态物体是建图的一部分还是障碍物？动态预测能否融入地图？
- **开放世界**：训练集中没见过的物体类别如何处理？光照变化、季节变化如何保持语义一致性？不同机器人之间的地图如何共享和融合？

这三个问题的交汇点定义了 SLAM 研究下一个十年的核心议程：从"建图系统"到"空间记忆基础设施"。
